TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN ỨNG DỤNG PHÂN TÍCH DỮ LIỆU LỚN TRONG KINH DOANH (BIG DATA)



LƯU THỊ THƠ: 24126221

TRẦN QUANG HUY: 24126088

MAI KHÁNH THUẬN: 24126222

TRẦN THỊ BÍCH NGỌC: 24126154

TRẦN NGUYỄN THẢO NGUYÊN: 24126157

Đề Tài :

PHÂN TÍCH DỮ LIỆU TRÊN SÀN THƯƠNG MẠI ĐIỆN TỬ TIKTOK SHOP

ĐỒ ÁN CUỐI KỲ

GIÁO VIÊN HƯỚNG DẪN

TS. Trần Quang Khải

Tháng 11, Thành phố Hồ Chí Minh

BẢNG TỰ CHẤM

STT	NỘI DUNG		ĐIỂM		CÓ (✓)
1	Dữ liệu	Đa dạng về nguồn dữ liệu thu thập		0.5	
		Cài đặt chương trình thu thập		0.5	
		Lớn (> 5000 records)		0.5	
		Làm sạch, chuẩn hóa dữ liệu		1.0	
2	Lưu trữ	Lưu trữ dữ liệu thu thập vào các DBMS		0.5	
3	Chức năng	Thiết kế dashboard		1.0	
		Cài đặt mô hình học không giám sát		1.0	
		Cài đặt mô hình học giám sát		1.0	
		Cài đặt mô hình ngoài chương trình		1.0	
4	Điểm cộng	Giao diện tương tác (GUI)		2.0	
		Thiết lập phân quyền người dùng		0.5	
		Tìm kiếm, lọc dữ liệu bằng truy vấn		0.5	
		Hỗ trợ tương tác với dữ liệu (CRUD)		1.0	
		Kết xuất báo cáo thống kê		1.0	
		Sao lưu, phục hồi dữ liệu		0.5	

5	Báo cáo	Bảng phân công công việc, chức năng giữa các thành viên		-1.0	
		Báo cáo trình bày đúng định dạng (theo mẫu)		2.0	
		Có soạn slides thuyết trình		-1.0	
6	Video	Theo đúng yêu cầu		1.0	
7	Vấn đáp	Trả lời các yêu cầu từ GV (đồ án, lý thuyết)		-4.0	
8	Đánh giá	Độ khó và độ hoàn thiện của đề tài (1-2-3-4) * 0.25		1.0	
TỔNG					

Tp. Hồ Chí Minh, ngày 01 tháng 11 năm 2026



Trần Nguyễn Thảo Nguyên

BẢNG PHÂN CÔNG CÔNG VIỆC

#	THÀNH VIÊN	CÔNG VIỆC	% DONE	GHI CHÚ	KÝ TÊN
1	*Trần Quang Huy (24126088)*	*Phân tích dữ liệu: Thu thập Dataset CSV/JSON; khởi tạo CSDL và Collection trên MongoDB Atlas.* *Thiết kế và phát triển Website: Thiết lập chức năng đăng nhập và phân quyền người dùng theo vai trò.*	*100*		
2	*Lưu Thị Thơ (24126221)*	*Phân tích dữ liệu: Khảo sát và làm sạch dữ liệu bằng Pandas, xử lý dữ liệu rác, giá trị Null và trùng lặp.* *Thiết kế và phát triển Website: Xây dựng chức năng tìm kiếm và lọc dữ liệu bằng truy vấn.*	*100*		
3	*Trần Thị Bích Ngọc (24126154)*	*Phân tích dữ liệu: Chuẩn hóa chuỗi và kiểu dữ liệu, xử lý Outliers, xây dựng Star Schema cho Power BI và chuẩn bị dữ liệu phục vụ mô hình.* *Thiết kế và phát triển Website: Xây dựng chức năng CRUD cho các dữ liệu quan trọng.*	*100*		
4	*Trần Nguyễn Thảo Nguyên (24126157)*	*Phân tích dữ liệu: Xây dựng các công thức DAX Measures cốt lõi và thiết kế Dashboard tương tác gồm KPI, biểu đồ và Slicers.* *Thiết kế và phát triển Website: Xây dựng chức năng kết xuất báo cáo thống kê ra tập tin.*	*100*		
5	*Mai Khánh Thuận (24126222)*	*Phân tích dữ liệu: Phân cụm khách hàng bằng K-Means (RFM), huấn luyện và đánh giá mô hình học máy*	*100*		

		dự đoán, phát hiện giao dịch bất thường. *Thiết kế và phát triển Website: Xây dựng chức năng sao lưu và phục hồi dữ liệu.*			

Tôi xin cam đoan bảng phân công công việc trên đã được thống nhất thông qua giữa các thành viên và phân bổ khối lượng chính xác như bảng trên, đúng với tiến độ được đề cập trong tỷ lệ hoàn thành công việc.

Tp. Hồ Chí Minh, ngày 01 tháng 11 năm 2026



Trần Nguyễn Thảo Nguyên

MỤC LỤC

DANH MỤC HÌNH ẢNH

DANH MỤC BẢNG

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI VÀ QUẢN LÝ DỰ ÁN 1

1.1. BỐI CẢNH VÀ LÝ DO CHỌN ĐỀ TÀI.....1

1.2. MỤC TIÊU NGHIÊN CỨU 1

1.3. QUY TRÌNH QUẢN LÝ MÃ NGUỒN VÀ LÀM VIỆC NHÓM TRÊN GITHUB.....2

CHƯƠNG 2: MÔ HÌNH HÓA VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU NOSQL (MONGODB)..... 4

2.1. PHÂN TÍCH NGUỒN DỮ LIỆU TIKTOK SHOP VÀ PHÂN LOẠI DỮ LIỆU 4

2.1.1. Bối cảnh và Nguồn gốc Tập dữ liệu TikTok Shop.....4

2.1.2. Tích hợp các Thực thể trong Hệ sinh thái Social Commerce.....4

2.1.3. Bảng đặc tả thuộc tính và Quy tắc nghiệp vụ 5

2.2. PHÂN TÍCH MỐI QUAN HỆ DỮ LIỆU (DATA RELATIONSHIPS ANALYSIS) 8

2.2.1. Sơ đồ và Phân tích Mối quan hệ giữa các đối tượng trong hệ sinh thái TikTok Shop.....8

2.2.2. Lý do lựa chọn NoSQL (MongoDB Atlas) thay thế cho RDBMS truyền thống đối với Big Data TMĐT.....9

2.2.3. Chiến lược thiết kế Document: Mô hình Phi chuẩn hóa (Bảng phẳng - Flat Document Structure) 11

2.3. THIẾT KẾ CÁC COLLECTION TRONG HỆ THỐNG DATABASE 12

2.3.1 Tổng quan về Collection transactions và Collection users 12

2.3.2. Bảng phân rã thuộc tính của Collection users 12

2.3.3. Ma trận Phân quyền người dùng (RBAC Matrix) 14

2.4. THU THẬP VÀ IMPORT DỮ LIỆU BÁN HÀNG TIKTOK SHOP VÀO MONGODB 14

2.4.1. Quy trình Thu thập và Nạp Dữ liệu Tự động bằng Python 14

2.4.2. Quy trình Quản trị và Import Dữ liệu qua MongoDB Compass & Mongo Shell.....18

2.5. KHỞI TẠO VÀ CẤU HÌNH MONGODB ATLAS (CLOUD DATABASE) 19

2.5.1. Quy trình Khởi tạo Cụm Lưu Trữ Đám Mây (Cluster0 - Free Deployment) 19
2.5.2. Thiết lập Bảo Mật Mạng (Network Access & IP Whitelist) 20
2.5.3. Khởi tạo và Phân Quyền Người Dùng CSDL (Database User Authentication) 21
2.5.4. Lấy và Bảo Mật Chuỗi Kết Nối (Connection String - URI) qua Môi Trường .env.local 22
CHƯƠNG 3: TRÍCH XUẤT, LÀM SẠCH VÀ CHUẨN HÓA DỮ LIỆU (ETL PIPELINE WITH PANDAS) **23**
3.1. KẾT NỐI CLOUD MONGODB ATLAS VÀ KHẢO SÁT DỮ LIỆU THÔ (DATA PROFILING) **23**
3.1.1. Quy trình kết nối và trích xuất dữ liệu từ Cloud Database: 23
3.1.2. Khảo sát cấu trúc bảng và nhận diện các dữ liệu tiềm ẩn của dữ liệu thô: 23
3.2. QUY TRÌNH LÀM SẠCH DỮ LIỆU CHI TIẾT (DATA CLEANING TRÊN GOOGLE COLAB) **27**
3.3. QUY TRÌNH CHUẨN HÓA DỮ LIỆU (DATA STANDARDIZATION & NORMALIZATION) **30**
3.3.1. Xác định vấn đề dữ liệu (Data Profiling) 30
3.3.2. Tiến hành xử lý và giải thích chi tiết các lệnh lập trình 31
3.3.3. Kết quả đạt được sau khi chuẩn hóa 32
3.4. PHÂN TÍCH VÀ XỬ LÝ ĐIỂM DỮ LIỆU BẤT THƯỜNG (OUTLIER ANALYSIS) **32**
3.4.1. Xác định vấn đề và Khảo sát trực quan ban đầu 32
3.4.2. Áp dụng phương pháp khoảng tứ phân vị (IQR) để tính toán kiểm chứng 34
3.4.3. Đánh giá nghiệp vụ và Quyết định xử lý dữ liệu 35
3.5. XUẤT TẬP DỮ LIỆU SẠCH (CLEANED DATASET) PHỤC VỤ DASHBOARD VÀ MACHINE LEARNING **35**
CHƯƠNG 4: MÔ HÌNH HÓA VÀ TRỰC QUAN HÓA DỮ LIỆU (POWER BI DASHBOARD) **37**
4.1. NẠP VÀ BIẾN ĐỔI DỮ LIỆU TRÊN POWER QUERY EDITOR **37**
4.1.1. Mục đích và Quy trình trích xuất dữ liệu (Extract & Load) 37
4.1.2. Kiểm tra tính toàn vẹn của dữ liệu (Data Profiling) 38
4.1.3. Chuẩn hóa định dạng kiểu dữ liệu (Data Types) 39

4.2. XÂY DỰNG MÔ HÌNH DỮ LIỆU ĐA CHIỀU (DATA MODELING - STAR SCHEMA) 41
4.2.1. Phương pháp luận và Kiến trúc Lược đồ hình sao (Star Schema) 41
4.2.2. Chi tiết các bước phân tách bảng trên Power Query Editor 42
4.2.3. Thiết lập mối quan hệ trong mô hình (Relationships - Model View) 45
4.3. XÂY DỰNG BỘ CÔNG THỨC DAX MEASURES CỐT LÕI 48
4.4. TỔNG HỢP THỐNG KÊ SỐ LIỆU 53
4.5. THIẾT KẾ BÁO CÁO QUẢN TRỊ KINH DOANH TIKTOK SHOP TƯƠNG TÁC (DASHBOARD DESIGN) 56
4.5.1 Dashboard Tổng quan Tổng quan Doanh thu & Quy mô Kinh doanh 56
4.5.2 Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi 66
4.5.3 Dashboard Phân tích Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường 78
CHƯƠNG 5: TRIỂN KHAI CÁC MÔ HÌNH HỌC MÁY (MACHINE LEARNING) 89
5.1 MÔ HÌNH HỌC MÁY KHÔNG GIÁM SÁT 89
5.1.1 Thiết lập môi trường thực nghiệm 89
5.1.2. Nạp dữ liệu đầu vào 90
5.1.3. Vai trò của bước chuẩn bị dữ liệu 91
5.1.4. Khảo sát và đánh giá chất lượng dữ liệu đầu vào 91
5.1.5. Xây dựng bộ chỉ số RFM 95
5.1.6. Tính toán bộ chỉ số RFM theo khách hàng 98
5.1.7. Chuẩn hóa dữ liệu và phân cụm bằng thuật toán K-Means 99
5.1.8. Trực quan hóa và đánh giá kết quả phân cụm 104
5.2. MÔ HÌNH HỌC MÁY CÓ GIÁM SÁT (SUPERVISED LEARNING) 105
5.2.1. Xác định bài toán phân loại 106
5.2.2. Xác định biến đầu vào 107
5.2.3. Chia tập dữ liệu huấn luyện và kiểm tra 108
5.2.4. Tiền xử lý dữ liệu trước khi huấn luyện mô hình 109
5.2.5. Huấn luyện mô hình Random Forest Classifier 110
5.2.6. Đánh giá mô hình Random Forest bằng Confusion Matrix 111
5.2.7. Phát hiện giao dịch bất thường bằng Isolation Forest 115
5.2.8. Bài toán hồi quy (Regression): Dự báo giá trị đơn hàng (Total_Amount) 119

5.3. MÔ HÌNH KHẢO SÁT VÀ PHÁT HIỆN ĐIỂM BẤT THƯỜNG (ANOMALY DETECTION) 125
5.3.1. Xây dựng mô hình Isolation Forest 126
5.3.2. Trực quan hóa các giao dịch bất thường 127
5.3.3. Đối chiếu kết quả phát hiện với nhãn bất thường ban đầu 128
5.4. HOÀN THIỆN DỮ LIỆU VÀ XUẤT KẾT QUẢ PHÂN CỤM KHÁCH HÀNG RFM 129
5.4.1. Ánh xạ nhãn phân cụm vào dữ liệu giao dịch 129
5.4.2. Kiểm tra và xử lý giao dịch chưa có nhãn phân cụm 131
5.4.3. Kiểm tra và xuất dữ liệu kết quả 133
CHƯƠNG 6: XÂY DỰNG WEBSITE TÍCH HỢP HỆ THỐNG VÀ TÍNH NĂNG MỞ RỘNG (NEXT.JS) 135
6.1. KIẾN TRÚC HỆ THỐNG WEBSITE VÀ CẤU TRÚC THƯ MỤC DỰ ÁN (SOURCE-CODE/WEB-APP) 135
6.2. THIẾT LẬP ĐĂNG NHẬP VÀ PHÂN QUYỀN NGƯỜI DÙNG (ROLE-BASED AUTHORIZATION) 137
6.2.1. Kiến trúc Xác thực Người dùng (Authentication Flow với NextAuth.js) 137
6.2.2. Hiện thực hóa Ma trận Phân quyền RBAC (Role-Based Access Control) 138
6.2.3. Thiết kế Giao diện Đăng nhập, Thanh Điều hướng Nhận diện Quyền hạn và Bảo vệ Tuyến đường 140
6.3. TÍNH NĂNG TÌM KIẾM VÀ LỌC DỮ LIỆU NÂNG CAO BẰNG TRUY VẤN MONGODB 146
6.3.1. Thiết kế API Backend và Bóc tách Tham số Truy vấn động 146
6.3.2. Xây dựng logic truy vấn động cho Tìm kiếm từ khóa và Lọc theo Danh mục 147
6.3.3. Xây dựng Giao diện Người dùng và Quản lý Trạng thái Bộ lọc 152
6.3.4. Kết quả thực nghiệm và đánh giá chức năng trên website 160
6.4. XÂY DỰNG BỘ CHỨC NĂNG QUẢN LÝ TƯƠNG TÁC DỮ LIỆU CRUD (CREATE, READ, UPDATE, DELETE) 161
6.4.1. Tổng quan Kiến trúc và Mô hình Dữ liệu CRUD 161
6.4.2. Chức năng READ (Truy vấn, Lọc đa điều kiện & Phân trang) 161
6.4.3. Chức năng CREATE (Thêm mới Đơn hàng) 163
6.4.4. Chức năng UPDATE (Xem Chi tiết & Cập nhật Đơn hàng) 167

6.4.5. Chức năng DELETE (Xóa Đơn hàng & Cảnh báo An toàn) 170
6.4.6. Kết quả trang order.....173
6.5. TÍNH NĂNG KẾT XUẤT BÁO CÁO THỐNG KÊ RA TẬP TIN (EXPORT EXCEL/CSV) 177
6.5.1. Tổng quan chức năng và luồng xử lý 177
6.5.2. Xây dựng API thống kê dữ liệu (/api/reports) 179
6.5.3. Xây dựng giao diện Dashboard thống kê (/reports) 187
6.5.4. Xây dựng API kết xuất báo cáo Excel/CSV (/api/export).....194
6.5.5. Kiểm thử và đánh giá kết quả 198
6.6. TÍNH NĂNG SAO LƯU VÀ PHỤC HỒI DỮ LIỆU MONGODB (BACKUP & RESTORE).....203
6.6.1. Tổng quan chức năng Sao lưu và Phục hồi dữ liệu (Backup & Restore) 203
6.6.2. Xây dựng API sao lưu dữ liệu MongoDB (GET /api/backup) 204
6.6.3. Xây dựng API phục hồi dữ liệu MongoDB (POST /api/backup).....207
6.6.4. Xây dựng giao diện chức năng Backup & Restore (/backup-restore).215
6.6.5. Kiểm thử và đánh giá kết quả 220
CHƯƠNG 7: ĐÁNH GIÁ, NHẬN ĐỊNH DỮ LIỆU VÀ ĐỀ XUẤT GIẢI PHÁP KINH DOANH 225
7.1. NHẬN ĐỊNH VỀ DỮ LIỆU TIKTOK SHOP (DATA INSIGHTS) 225
7.2. ĐỀ XUẤT ĐÁNH GIÁ VÀ GIẢI PHÁP HÀNH ĐỘNG KINH DOANH (BUSINESS RECOMMENDATIONS).....226
7.3. ĐÁNH GIÁ HẠN CHẾ CỦA ĐỀ TÀI VÀ HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI.....227

DANH MỤC HÌNH ẢNH

Hình 2.1. Mô hình Mối quan hệ Thực thể Nghiệp vụ (ERD) trong Hệ sinh thái TikTok Shop.....9

Hình 2.2. Hình minh họa Collection “transactions” & Collection “users”.....12

Hình 2.3. Dữ liệu mẫu các tài khoản trong Collection "users" trên MongoDB Compass.....14

Hình 2.4. Đoạn mã Thu nhập Nguồn dữ liệu Bổ trợ từ REST API trực tiếp 15

Hình 2.5. Đoạn mã Đọc tập tin dữ liệu và Ép kiểu Dữ liệu Nghiệp vụ 16

Hình 2.6. Kết quả chạy chương trình Python data_collector.py trên Terminal ...18

Hình 2.7. Import dữ liệu bằng tính năng Add Data của MongoDB Compass.....18

Hình 2.8. Gỡ bỏ các trường trùng lặp bằng toán tử $unset 19

Hình 2.9. Tổng quan Cụm Cluster0 trên giao diện web MongoDB Atlas.....20

Hình 2.10. Cấu hình Network Access (IP Whitelist) trên MongoDB Atlas 21

Hình 2.11. Cấu hình Database Access (User) trên MongoDB Atlas 21

Hình 2.12. Mã nguồn của file .env.local.....22

Hình 3.1. Kết nối thành công Cloud MongoDB Atlas và nạp 8.500 dòng giao dịch vào Pandas.....23

Hình 3.2. Cấu trúc tổng quan và kiểu dữ liệu ban đầu qua lệnh df.info() 24

Hình 3.3. Kết quả kiểm tra giá trị khuyết thiếu và dòng trùng lặp 25

Hình 3.4. Kết quả khảo sát giá trị khuyết thiếu ngầm trên cột Creator_Name 25

Hình 3.5. Bảng thống kê mô tả số liệu toàn diện bằng hàm df.describe().....26

Hình 3.6. Bảng liệt kê danh sách Min - Max.....26

Hình 3.7. Quy trình triển khai làm sạch dữ liệu trên Colab 28

Hình 3.8. Lệnh kiểm tra các giá trị khác nhau của phương thức thanh toán và thành phố trước khi chuẩn hóa.....30

Hình 3.9. Kết quả sau khi kiểm tra.....30

Hình 3.10. Lệnh làm sạch dữ liệu.....31

Hình 3.11.Kết quả sau khi chạy lệnh làm sạch dữ liệu 32

Hình 3.12.Lệnh kiểm tra sơ bộ các giá trị ngoại lai 33

Hình 3.13.Kết quả sau khi kiểm tra các giá trị ngoại lai 33

Hình 3.14. Biểu đồ hộp để trực quan hóa các giá trị ngoại lai 33

Hình 3.15. Lệnh phân tích các giá trị ngoại lai bằng IQR.....34

Hình 3.16. Kết quả sau khi chạy lệnh phân tích Outlier 34

Hình 3.17.Lệnh kiểm tra lần cuối và trích xuất file 'tiktok_shop_sales_cleaned.csv' 35
Hình 3.18.Kết quả sau khi chạy lệnh trích xuất file tiktok_shop_sales_cleaned.csv 36
Hình 3.19. Kết quả file tiktok_shop_sales_cleaned.csv 36
Hình 4.1.Thao tác kết nối nguồn dữ liệu CSV (Get Data) trong Power BI 37
Hình 4.2. Thao tác chọn file 'tiktok_shop_sales_cleaned.csv' để nhập vào Power BI 38
Hình 4.3. Giao diện sau khi nhấn nhập dữ liệu vào Power BI 38
Hình 4.4.Kiểm tra tính toàn vẹn và chất lượng dữ liệu (Data Profiling) trên Power Query Editor 39
Hình 4.5.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột định danh trong Power Query....40
Hình 4.6.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột thời gian 40
Hình 4.7.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột về định lượng và tài chính....41
Hình 4.8.Giao diện xây dựng bảng sự kiện trung tâm (Fact_Orders) 43
Hình 4.9.Giao diện xây dựng bảng chiều khách hàng (Dim_Customer) trên Power Query Editor 43
Hình 4.10.Thao tác remove duplicates 44
Hình 4.11.Giao diện xây dựng bảng chiều sản phẩm (Dim_Product) trên Power Query Editor 44
Hình 4.12.Giao diện xây dựng bảng chiều thời gian (Dim_Date) trên Power Query Editor 44
Hình 4.13.Thao tác thêm các thuộc tính phân tích thời gian 45
Hình 4.14. Thao tác xây dựng mối quan hệ mô hình trong Model View 45
Hình 4.15. Thao tác liên kết bảng Dim_Customer với bảng Fact_Orders 46
Hình 4.16.Thao tác liên kết bảng Dim_Product với bảng Fact_Orders....46
Hình 4.17.Thao tác liên kết bảng Dim_Date với bảng Fact_Orders 47
Hình 4.18. Kết quả sau khi xây dựng mô hình Star Schema....47
Hình 4.19. Khởi tạo New measure 48
Hình 4.20. Nhập mã công thức DAX Measures của Total Revenue 48
Hình 4.21. Nhập mã công thức DAX Measures của Total Orders 48
Hình 4.22. Nhập mã công thức DAX Measures của AOV 49

Hình 4.23. Nhập mã công thức DAX Measures của Total Quantity 49
Hình 4.24. Nhập mã công thức DAX Measures của Total Review Count 49
Hình 4.25. Nhập mã công thức DAX Measures của Average Product Rating 50
Hình 4.26. Nhập mã công thức DAX Measures của Total Video Views 50
Hình 4.27. Nhập mã công thức DAX Measures của Total Customers 50
Hình 4.28. Nhập mã công thức DAX Measures của Total Discount Amount 51
Hình 4.29. Nhập mã công thức DAX Measures của Total Shipping Fee 51
Hình 4.30. Nhập mã công thức DAX Measures của Cancel Rate 52
Hình 4.31. Nhập mã công thức DAX Measures của Return Rate.....53
Hình 4.32. Nhập mã công thức DAX Measures của Anomaly Rate 53
Hình 4.33. Bộ thẻ KPI Cốt lõi.....57
Hình 4.34. Bộ lọc Dữ liệu Slicer.....58
Hình 4.35. Biểu đồ Top 10 Thành phố Dẫn Đầu Doanh Thu.....59
Hình 4.36. Biểu đồ Top 5 Sản Phẩm Đạt Doanh Thu Cao Nhất.....60
Hình 4.37. Biểu đồ Cơ Cấu Doanh Thu Theo Kênh Bán.....61
Hình 4.38. Biểu đồ Biến Động Doanh Thu & Số Lượng Đơn Hàng Theo Thời Gian.....62
Hình 4.39. Biểu đồ Cơ Cấu Phương Thức Thanh Toán Theo Trạng Thái Đơn Hàng.....63
Hình 4.40. Màn hình giao diện Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh hoàn chỉnh 63
Hình 4.41. Màn hình giao diện Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh lọc theo khu vực Đà Nẵng và thời gian từ tháng 1 đến tháng 6/2024 65
Hình 4.42. Biểu đồ Bộ thẻ KPI Banner Sản phẩm 67
Hình 4.43. Biểu đồ Bộ lọc Dữ liệu Slicer Ngành Hàng & Shop.....68
Hình 4.44. Biểu đồ Xếp Hạng Doanh Thu Theo KOL/KOC.....69
Hình 4.45. Biểu đồ Tương Quan Lượt Xem Video & Sản Lượng Bán 71
Hình 4.46. Biểu đồ Giá Trị Đơn Trung Bình (AOV) Theo Kênh Bán & Phương Thức Thanh Toán.....72
Hình 4.47. Biểu đồ Quy Mô Doanh Thu Theo Ngành Hàng & Danh Mục.....73
Hình 4.48. Bảng Tra Cứu Chi Tiết Sản Phẩm & Đánh Giá.....74
Hình 4.49. Màn hình giao diện Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi hoàn chỉnh 75

Hình 4.50. Màn hình giao diện Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi lọc theo ngành hàng Thời trang nữ 76
Hình 4.51. Biểu đồ Bộ thẻ KPI Banner Rủi Ro & Khách Hàng 79
Hình 4.52. Biểu đồ Đồng hồ đo Tỷ Lệ Giao Dịch Bất Thường 79
Hình 4.53. Biểu đồ Bộ lọc Dữ liệu Slicer Nhân khẩu học & Thiết bị 80
Hình 4.54. Biểu đồ Ribbon Chart Tương Quan Doanh Thu & Chi Phí Chiết Khấu Theo Kênh Bán 81
Hình 4.55. Biểu đồ Tương Quan Doanh Thu & Chi Phí Vận Chuyển Theo Tỉnh/Thành Phố 82
Hình 4.56. Biểu đồ Area Chart Biến Động Chi Phí Chiết Khấu & Phí Ship Theo Thời Gian 84
Hình 4.57. Biểu đồ Cơ Cấu Số Lượng Đơn Hàng Theo Trạng Thái 85
Hình 4.58. Màn hình giao diện Dashboard Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường hoàn chỉnh 85
Hình 4.59. Màn hình giao diện Dashboard Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường lọc theo khách hàng nam 18 tuổi 87
Hình 5.1. Giao diện notebook Machine Learning trên Google Colab 89
Hình 5.2. Giao diện Pull Request cập nhật chức năng Backup/Restore và hệ thống thông báo trên GitHub. 90
Hình 5.3. Kết quả nạp và kiểm tra cấu trúc dữ liệu đầu vào 90
Hình 5.4. Code khảo sát chất lượng dữ liệu 92
Hình 5.5. Kết quả khảo sát chất lượng dữ liệu 94
Hình 5.6. Xác định các trường dữ liệu phục vụ xây dựng RFM 95
Hình 5.7. Đoạn code thống kê giá trị đơn hàng và kiểm tra dữ liệu bằng Pandas 96
Hình 5.8. Thống kê giá trị đơn hàng theo trạng thái 96
Hình 5.9. Kết quả lọc và kiểm tra dữ liệu đơn hàng hoàn thành 97
Hình 5.10. Đoạn code tính toán các chỉ số RFM và kiểm tra chất lượng dữ liệu khách hàng. 99
Hình 5.11. Kết quả tổng hợp bộ chỉ số RFM theo khách hàng. 99
Hình 5.12. Chuẩn hóa dữ liệu RFM bằng StandardScaler. 100
Hình 5.13. Biểu đồ Elbow khảo sát số cụm K. 100
Hình 5.14. Đoạn code sử dụng K-Means và Silhouette Score để xác định số cụm tối ưu 101
Hình 5.15. Kết quả đánh giá số cụm bằng Silhouette Score 102
Hình 5.16. Kết quả phân cụm K-Means và thống kê RFM trung bình 103

Hình 5.17. Đoạn mã trực quan hóa kết quả phân cụm khách hàng bằng thuật toán K-Means. 104
Hình 5.18. Biểu đồ phân cụm khách hàng bằng thuật toán K-Means. 104
Hình 5.19. Phân bố trạng thái đơn hàng ban đầu 106
Hình 5.20. Mã hóa biến mục tiêu cho bài toán phân loại 107
Hình 5.21. Xác định và kiểm tra các biến đầu vào 108
Hình 5.22. Kết quả chia dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20 109
Hình 5.23. Mã hóa các biến phân loại bằng One-Hot Encoding 110
Hình 5.24. Huấn luyện mô hình Random Forest Classifier 111
Hình 5.25. Đoạn mã đánh giá mô hình Random Forest bằng Confusion Matrix và các chỉ số phân loại. 112
Hình 5.26. Ma trận nhầm lẫn và các chỉ số đánh giá mô hình Random Forest .. 112
Hình 5.27. Đoạn mã thử nghiệm mô hình Random Forest với tham số class_weight="balanced" nhằm cải thiện khả năng phân loại dữ liệu mất cân bằng. 114
Hình 5.28. Kết quả thử nghiệm Random Forest với trọng số lớp cân bằng 114
Hình 5.29. Phân bố nhãn bất thường trong dữ liệu ban đầu 115
Hình 5.30. Đoạn mã trực quan hóa các giao dịch bất thường bằng thuật toán Isolation Forest. 116
Hình 5.31. Trực quan hóa giao dịch bất thường bằng Isolation Forest 117
Hình 5.32. Đối chiếu kết quả Isolation Forest với nhãn bất thường gốc 118
Hình 5.33. Đoạn mã kiểm tra các biến đầu vào, biến mục tiêu và chất lượng dữ liệu cho bài toán hồi quy. 120
Hình 5.34. Kiểm tra các biến đầu vào và biến mục tiêu cho bài toán hồi quy 120
Hình 5.35. Đoạn mã chia tập dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20 cho mô hình hồi quy. 121
Hình 5.36. Kết quả chia tập dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20 121
Hình 5.37. Đoạn mã huấn luyện mô hình Random Forest Regressor 122
Hình 5.38. Đoạn mã tính toán các chỉ số đánh giá mô hình hồi quy 123
Hình 5.39. Đoạn mã kiểm tra công thức tính Total_Amount và đối chiếu giá trị thực tế với giá trị ước tính. 124
Hình 5.40. Kết quả kiểm tra công thức tính Total_Amount và đối chiếu giá trị thực tế với giá trị ước tính. 125
Hình 5.41. Kết quả phân loại giao dịch bằng thuật toán Isolation Forest 126

Hình 5.42. Đoạn mã trực quan hóa các giao dịch bất thường bằng thuật toán Isolation Forest 127

Hình 5.43. Biểu đồ phân tán giữa giá trị giảm giá và tổng giá trị đơn hàng, thể hiện kết quả phát hiện bất thường bằng Isolation Forest..... 127

Hình 5.44. Ma trận nhầm lẫn và báo cáo phân loại khi đối chiếu kết quả Isolation Forest với nhãn bất thường ban đầu. 128

Hình 5.45. Mã nguồn ánh xạ nhãn phân cụm RFM vào dữ liệu giao dịch. 130

Hình 5.46. Kết quả kiểm tra dữ liệu sau khi gắn nhãn phân cụm. 131

Hình 5.47. Kết quả thống kê trạng thái của các giao dịch chưa có nhãn. 131

Hình 5.48. Thông tin chi tiết các giao dịch chưa được gán nhãn phân cụm. 132

Hình 5.49. Kết quả xử lý các giao dịch chưa được gán nhãn RFM. 133

Hình 5.50. Kết quả kiểm tra và xuất dữ liệu sau khi tích hợp nhãn phân cụm. .134

Hình 5.51. Mã lệnh tải tệp kết quả CSV từ Google Colab..... 134

Hình 6.1 Cấu trúc thư mục dự án 135

Hình 6.2. Sơ đồ Kiến trúc Xác thực NextAuth 137

Hình 6.3. Đoạn mã truy vấn người dùng bằng username hoặc email 137

Hình 6.4. Đoạn mã kiểm tra trạng thái tài khoản và xử lý khi tài khoản bị khóa 137

Hình 6.5. Đoạn mã triển khai cơ chế xác thực mật khẩu kép 138

Hình 6.6. Đoạn mã Ký số và đính kèm đặc quyền vào JWT Token 138

Hình 6.7 Giao diện tự động chuyển hướng người dùng về trang login khi chưa xác thực tài khoản..... 140

Hình 6.8 Giao diện khi tài khoản không phải ADMIN truy cập đường dẫn /backup-restore 140

Hình 6.9. Đoạn mã triển khai quy trình xử lý đăng nhập bất đồng bộ 141

Hình 6.10. Đoạn mã triển khai cơ chế tự động nạp dữ liệu mẫu 141

Hình 6.11. Đoạn mã triển khai tính năng điền nhanh tài khoản kiểm thử (handleFillDemo) cho 3 vai trò người dùng 142

Hình 6.12. Đoạn mã triển khai cơ chế ẩn/hiện Navbar thông minh theo tuyến đường và phiên làm việc 142

Hình 6.13. Đoạn mã render huy hiệu vai trò động (Dynamic Role Badge) dựa trên quyền hạn người dùng..... 143

Hình 6.14. Đoạn mã triển khai nút Đăng xuất và xử lý thu hồi phiên đăng nhập (signOut) 143

Hình 6.15. Khai báo ProtectionProps và xử lý màn hình chờ xác thực phiên làm việc (status === "loading") trong Protection.tsx.....144

Hình 6.16. Đoạn mã xử lý giao diện cảnh báo "Yêu cầu đăng nhập" khi người dùng chưa xác thực tài khoản 144

Hình 6.17 Đoạn mã triển khai logic kiểm tra vai trò (Role Guard) và hiển thị thông báo "Không đủ quyền truy cập".....145

Hình 6.18. Áp dụng component Protection bọc giao diện trang Quản trị /backup-restore giới hạn quyền ADMIN.....145

Hình 6.19. Đoạn mã xử lý điều hướng về trang /orders và thẻ đóng </Protection> hoàn tất phạm vi bảo vệ 146

Hình 6.20 Khởi tạo kết nối MongoDB Atlas và bóc tách các tham số truy vấn URL (searchParams).....147

Hình 6.21. Đoạn mã xây dựng truy vấn tìm kiếm từ khóa linh hoạt bằng toán tử $or và Regex 148

Hình 6.22 Đoạn mã lọc dữ liệu theo Trạng thái đơn, Phương thức thanh toán, Danh mục và KOC 149

Hình 6.23. Đoạn mã lọc dữ liệu theo Khu vực tỉnh thành, Gian hàng và Đơn bất thường.....150

Hình 6.24. Đoạn mã lọc dữ liệu theo khoảng giá trị tiền (Total_Amount) và khoảng ngày (Order_Date).....151

Hình 6.25. Đoạn mã thực thi truy vấn MongoDB, phân trang và phản hồi kết quả 152

Hình 6.26. Khai báo các State quản lý tiêu chí tìm kiếm và bộ lọc.....153

Hình 6.27. Đoạn mã phục hồi trạng thái bộ lọc từ sessionStorage khi quay lại trang.....154

Hình 6.28. Hàm fetchOrders đóng gói tham số truy vấn URLSearchParams và gọi API 155

Hình 6.29. Đoạn mã tự động kích hoạt truy vấn khi thay đổi bộ lọc và đồng bộ lưu vào sessionStorage.....156

Hình 6.30. Đoạn mã hàm xóa trắng bộ lọc handleResetFilters và thuật toán đếm tiêu chí lọc.....157

Hình 6.31. Giao diện tiêu đề bộ lọc, nút đặt lại và ô tìm kiếm từ khóa theo 3 trường 158

Hình 6.32. Đoạn mã thiết kế giao diện bộ lọc nâng cao đa tiêu chí 160

Hình 6.33. Giao diện tổng quan tính năng tìm kiếm và bảng điều khiển bộ lọc nâng cao trên website 160

Hình 6.34. Kết quả thực thi truy vấn lọc dữ liệu đa điều kiện kết hợp và các chip hiển thị trực quan 160
Hình 6.35. Đoạn code trích xuất tham số từ URL Query String (/api/orders/route.ts).....162
Hình 6.36. Đoạn code xây dựng truy vấn Tìm kiếm Toàn cục (Global Search Query) 162
Hình 6.37. Đoạn code Thực thi Phân trang và Sắp xếp Dữ liệu (Pagination & Sorting) 163
Hình 6.38. Đoạn code Khởi tạo Badges Trực quan hóa Điều kiện Lọc đang Áp dụng.....163
Hình 6.39. Đoạn code khởi tạo đối tượng initialOrderState (orders/page.tsx)....164
Hình 6.40. Đoạn code tự động tính Total_Amount (orders/page.tsx) 165
Hình 6.41. Đoạn code kiểm tra Regex và điều kiện logic (handleCreateOrder)\ 165
Hình 6.42. Đoạn code xác thực và chống trùng lặp tại API Route (POST /api/orders/route.ts) 166
Hình 6.43. Đoạn code lấy dữ liệu chi tiết và khóa Order_ID (orders/[id]/page.tsx) 167
Hình 6.44. Đoạn code xử lý sự thay đổi chỉ số tài chính handleFinancialChange (orders/[id]/page.tsx) 168
Hình 6.45. Đoạn code xác thực định dạng dữ liệu 169
Hình 6.46. Đoạn code xử lý Cập nhật Linh hoạt và Chống Trùng mã.....169
Hình 6.47. Đoạn code hiển thị Toast cảnh báo xác nhận xóa (confirmAndDelete trong DataTable.tsx).....171
Hình 6.48. Đoạn code kiểm tra vai trò người dùng trước khi hiển thị nút Xóa (DataTable.tsx).....171
Hình 6.49. Đoạn code xử lý xóa dữ liệu theo ID (DELETE /api/orders/[id]/route.ts).....172
Hình 6.50. Giao diện thêm mới sản phẩm 174
Hình 6.51. Các thông báo yêu cầu nhập dữ liệu và đúng định dạng khi nhập sai hay thiếu 174
Hình 6.52. Giao diện và thông báo thêm đơn hàng thành công 175
Hình 6.53. Giao diện kết quả truy vấn lọc theo các điều kiện và bộ lọc hiển thị số dòng.....175
Hình 6.54. Giao diện các đơn hàng bất thường 175
Hình 6.55. Giao diện chi tiết đơn hàng 176
Hình 6.56. Thông báo khi cập nhật đơn hàng thành công 176

Hình 6.57. Thông báo xác nhận xóa khi nhấn nút xóa đơn hàng 176

Hình 6.58. Giao diện xóa đơn hàng thành công.....177

Hình 6.59. Giao diện của người xem.....177

Hình 6.60. Đoạn mã xây dựng tham số lọc và điều hướng đến trang báo cáo.....178

Hình 6.61. Luồng xử lý chức năng thống kê và kết xuất báo cáo 179

Hình 6.62. Đoạn mã khởi tạo API GET, kết nối MongoDB Atlas và trích xuất các tham số bộ lọc từ URL 180

Hình 6.63. Đoạn mã xây dựng bộ lọc tìm kiếm từ khóa trên mã đơn hàng, tên khách hàng và tên sản phẩm 180

Hình 6.64. Đoạn mã áp dụng bộ lọc động theo trạng thái đơn hàng, phương thức thanh toán, danh mục, nhà sáng tạo, thành phố và cửa hàng.....181

Hình 6.65. Đoạn mã xử lý bộ lọc giao dịch bất thường, khoảng giá trị đơn hàng và khoảng ngày đặt hàng 182

Hình 6.66. Đoạn mã khởi tạo Aggregation Pipeline và phân chia các nhánh thống kê bằng $facet.....182

Hình 6.67. Đoạn mã tổng hợp KPI doanh thu và tổng số đơn hàng trong tập dữ liệu đã lọc.....183

Hình 6.68. Đoạn mã đếm số đơn hàng được đánh dấu bất thường 183

Hình 6.69. Đoạn mã tính lượt xem video trung bình bằng toán tử $avg.....183

Hình 6.70. Đoạn mã tổng hợp dữ liệu phân bổ nguồn Traffic và phân bố trạng thái đơn hàng 184

Hình 6.71. Đoạn mã tổng hợp số lượng, doanh thu và lựa chọn Top 5 sản phẩm 185

Hình 6.72. Đoạn mã tổng hợp xu hướng doanh thu theo tháng từ trường Order_Date.....185

Hình 6.73. Đoạn mã chuẩn hóa dữ liệu thống kê và trả phản hồi JSON về phía Client.....186

Hình 6.74. Đoạn mã ghi nhận lỗi và trả phản hồi HTTP 500 khi xử lý API thống kê thất bại.....187

Hình 6.75. Đoạn mã bọc giao diện Dashboard bằng Protection và Suspense.....187

Hình 6.76. Đoạn mã trích xuất các điều kiện lọc và xác định trạng thái bộ lọc trên Dashboard 188

Hình 6.77. Đoạn mã gọi API thống kê và tự động tải lại dữ liệu khi tham số URL thay đổi 189

Hình 6.78. Đoạn mã bố trí hệ thống thẻ KPI tổng quan bằng CSS Grid.....189

Hình 6.79. Đoạn mã hiển thị KPI doanh thu và định dạng số theo quy ước Việt Nam 189

Hình 6.80. Đoạn mã xử lý dữ liệu và cấu hình biểu đồ Doughnut phân bổ nguồn Traffic 190

Hình 6.81. Đoạn mã chuyển đổi dữ liệu trạng thái đơn hàng thành các tập dữ liệu cho biểu đồ Bar 190

Hình 6.82. Đoạn mã cấu hình dữ liệu và giao diện biểu đồ Line doanh thu theo tháng 191

Hình 6.83. Đoạn mã render bảng Top 5 sản phẩm với thứ hạng, sản lượng và doanh thu 192

Hình 6.84. Đoạn mã kiểm tra quyền và gửi yêu cầu kết xuất Excel/CSV theo bộ lọc hiện hành 192

Hình 6.85. Đoạn mã lấy tên tập tin từ phản hồi API và tạo tên dự phòng theo ngày 193

Hình 6.86. Đoạn mã tạo liên kết tải xuống tập tin báo cáo từ dữ liệu Blob 193

Hình 6.87. Đoạn mã xóa trạng thái bộ lọc đã lưu và đưa Dashboard về trang báo cáo mặc định 194

Hình 6.88. Xây dựng bộ lọc động cho API kết xuất báo cáo. 195

Hình 6.89. Tổng hợp các chỉ số KPI bằng MongoDB Aggregation Pipeline. 196

Hình 6.90. Tạo worksheet và workbook cho báo cáo TikTok Shop. 197

Hình 6.91. Tạo tên tệp báo cáo theo ngày xuất. 197

Hình 6.92. Kết xuất dữ liệu thành tệp CSV hoặc Excel và trả về trình duyệt. 198

Hình 6.93. Xử lý lỗi trong API kết xuất báo cáo. 198

Hình 6.94. Giao diện Dashboard thống kê trước khi áp dụng bộ lọc. 199

Hình 6.95. Kết quả Dashboard sau khi áp dụng bộ lọc dữ liệu. 200

Hình 6.96. Giao diện tệp Excel báo cáo khi mở bằng phần mềm bảng tính. 202

Hình 6.97. Giao diện tệp CSV báo cáo khi mở bằng phần mềm bảng tính. 203

Hình 6.98. Đoạn mã kiểm tra phiên đăng nhập và phân quyền ADMIN trong API Backup & Restore. 204

Hình 6.99. Đoạn mã truy vấn toàn bộ dữ liệu giao dịch và sắp xếp bản ghi trước khi sao lưu. 205

Hình 6.100. Đoạn mã chuyển đổi dữ liệu thành JSON và thiết lập HTTP Response. 207

Hình 6.101. Đoạn mã tiếp nhận tệp JSON và kiểm tra phần mở rộng, dung lượng đầu vào. 209

Hình 6.102. Đoạn mã phân tích JSON và kiểm tra cấu trúc mảng dữ liệu đầu vào. 211
Hình 6.103. Đoạn mã kiểm tra các trường dữ liệu bắt buộc trong từng bản ghi giao dịch. 212
Hình 6.104. Đoạn mã khởi tạo React State 216
Hình 6.105. Giao diện tổng thể chức năng Sao lưu và Phục hồi dữ liệu MongoDB. 217
Hình 6.106. Đoạn mã xử lý yêu cầu Backup và tải tệp JSON về thiết bị. 218
Hình 6.107. Hộp thoại xác nhận phục hồi dữ liệu từ tệp JSON. 219
Hình 6.108. Giao diện chức năng Backup & Restore sau khi đăng nhập bằng tài khoản ADMIN.....221
Hình 6.109. Kết quả tải tệp Backup JSON thành công trên trình duyệt. 221
Hình 6.110. Cấu trúc một bản ghi dữ liệu giao dịch trong tệp Backup JSON. ...222
Hình 6.111. Hộp thoại xác nhận phục hồi dữ liệu từ tệp JSON đã sao lưu. 223
Hình 6.112. Thông báo kết quả phục hồi với 8.500 bản ghi được cập nhật. 223

DANH MỤC BẢNG

Bảng 2.1. Từ điển dữ liệu của 27 trường thông tin trong tập dữ liệu.....5

Bảng 2.2. Bảng so sánh CSDL quan hệ truyền thống & CSDL phi quan hệ.....10

Bảng 2.3. Phân rã thuộc tính của Collection users.....12

Bảng 3.1. Tổng hợp kỹ thuật làm sạch dữ liệu áp dụng cho từng trường thuộc tính 29

Bảng 4.1. Tổng hợp các trường thông tin trong mô hình dữ liệu 42

Bảng 4.2. Thống kê Min–Max–Mean của các biến số liệu.....53

Bảng 4.3. Tổng hợp giá trị các chỉ số kinh doanh.....55

Bảng 5.1.Kết quả khảo sát dữ liệu trên Google Colab của nhóm. 92

Bảng 5.2. Thống kê các chỉ số RFM trung bình theo cụm khách hàng.....103

Bảng 5.3. Ma trận nhầm lẫn cho Random Forest 113

Bảng 5.4. Ma trận nhầm lẫn cho Isolation Forest 118

Bảng 5.5. Phân bố nhãn trên toàn bộ dữ liệu.....133

Bảng 6.1. Chức năng các trang giao diện chính 136

Bảng 6.2. Ma trận Phân quyền RBAC 139

Bảng 6.3. Các chỉ số KPI sau khi lọc.....201

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI VÀ QUẢN LÝ DỰ ÁN

1.1. BỐI CẢNH VÀ LÝ DO CHỌN ĐỀ TÀI

Sự phát triển mạnh mẽ của thương mại điện tử đã thúc đẩy sự hình thành và mở rộng của Social Commerce, trong đó TikTok Shop nổi bật nhờ kết hợp hoạt động mua sắm với nội dung giải trí thông qua video ngắn, livestream và tiếp thị liên kết. Mô hình Shoppertainment này không chỉ tạo ra trải nghiệm mua sắm trực quan, tương tác mà còn làm gia tăng lượng dữ liệu phát sinh từ các hoạt động kinh doanh, bao gồm thông tin đơn hàng, sản phẩm, khách hàng, chương trình khuyến mãi và nguồn truy cập. Việc khai thác những dữ liệu này giúp doanh nghiệp đánh giá tình hình kinh doanh, xác định xu hướng tiêu dùng và nâng cao hiệu quả hoạt động bán hàng.

Tuy nhiên, dữ liệu chỉ thực sự có giá trị khi được tổ chức và khai thác hiệu quả. Trong thực tế, dữ liệu có thể tồn tại các vấn đề như thiếu thông tin, trùng lặp, sai định dạng hoặc không nhất quán, gây ảnh hưởng đến độ chính xác của quá trình phân tích. Bên cạnh đó, việc theo dõi riêng lẻ các chỉ số kinh doanh khiến doanh nghiệp gặp khó khăn trong việc đánh giá toàn diện hiệu quả bán hàng, nhận diện xu hướng tiêu dùng và phát hiện những biến động bất thường. Vì vậy, nhu cầu ứng dụng các công cụ phân tích dữ liệu và trực quan hóa ngày càng trở nên cần thiết, giúp chuyển đổi dữ liệu thô thành thông tin có ý nghĩa, hỗ trợ quá trình đánh giá và ra quyết định.

Xuất phát từ thực tiễn đó, nhóm lựa chọn đề tài phân tích dữ liệu bán hàng TikTok Shop nhằm tìm hiểu khả năng ứng dụng các công nghệ dữ liệu trong lĩnh vực thương mại điện tử. Đề tài có ý nghĩa thực tiễn trong việc vận dụng kiến thức về quản trị cơ sở dữ liệu, xử lý dữ liệu, phân tích kinh doanh và học máy vào một bối cảnh cụ thể. Đồng thời, đây cũng là cơ hội để nhóm nâng cao kỹ năng làm việc với dữ liệu, phát triển tư duy phân tích và tiếp cận quy trình xây dựng hệ thống hỗ trợ ra quyết định dựa trên dữ liệu.

1.2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu tổng quát: Xây dựng hệ thống phân tích và quản lý dữ liệu bán hàng TikTok Shop, kết hợp MongoDB Atlas, Pandas, Power BI, các kỹ thuật học máy và website nhằm hỗ trợ xử lý dữ liệu, trực quan hóa kết quả kinh doanh, phân tích hành vi khách hàng và quản lý dữ liệu tập trung.

Mục tiêu cụ thể:

- Thu thập và lưu trữ dữ liệu: Thu thập dữ liệu định dạng CSV/JSON, khởi tạo cơ sở dữ liệu và collection trên MongoDB Atlas.
- Tiền xử lý dữ liệu: Khảo sát chất lượng dữ liệu; xử lý giá trị thiếu, bản ghi trùng lặp, dữ liệu không hợp lệ; chuẩn hóa chuỗi, kiểu dữ liệu và xử lý giá trị ngoại lệ bằng Pandas.
- Xây dựng mô hình dữ liệu: Thiết kế Star Schema trên Power BI, xác định mối quan hệ giữa các bảng và xây dựng các công thức DAX phục vụ tính toán chỉ số.
- Trực quan hóa dữ liệu: Thiết kế dashboard tương tác với các chỉ số KPI, biểu đồ và bộ lọc, hỗ trợ theo dõi doanh thu, sản phẩm, khách hàng, chiết khấu và giao dịch bất thường.
- Ứng dụng học máy: Phân cụm khách hàng bằng K-Means kết hợp RFM; huấn luyện, đánh giá mô hình dự đoán và phát hiện giao dịch bất thường.
- Phát triển website: Xây dựng chức năng đăng nhập và phân quyền, tìm kiếm và lọc dữ liệu, CRUD, xuất báo cáo thống kê, sao lưu và phục hồi dữ liệu.
- Đánh giá kết quả: Đưa ra nhận định từ dữ liệu và đánh giá khả năng hỗ trợ quản lý, phân tích và ra quyết định của hệ thống.

1.3. QUY TRÌNH QUẢN LÝ MÃ NGUỒN VÀ LÀM VIỆC NHÓM TRÊN GITHUB

Đường dẫn repository GitHub của dự án:

https://github.com/24126157-ux/03_17_PhanTichDuLieuTrenSanThuongMaiDienTuTiktokShop

Nhóm sử dụng GitHub để quản lý mã nguồn và phối hợp thực hiện dự án theo chiến lược Feature Branching. Theo đó, mỗi thành viên tạo một nhánh riêng cho từng nhiệm vụ hoặc tính năng, thực hiện chỉnh sửa và lưu lại tiến độ bằng các commit trước khi đẩy mã nguồn lên repository chung. Cách làm này giúp hạn chế ảnh hưởng trực tiếp đến nhánh main, đồng thời hỗ trợ theo dõi lịch sử thay đổi và xác định mức độ đóng góp của từng thành viên.

Thư mục dự án được đặt tên theo cấu trúc

03_17_PhanTichDuLieuTrenSanThuongMaiDienTuTiktokShop, bao gồm:

- source-code: Chứa mã nguồn và chương trình xử lý dữ liệu.

- reports: Chứa báo cáo Word, PDF, slide thuyết trình, bảng phân công, bảng tự chấm và video (nếu có).
- dataset: Chứa dữ liệu sử dụng trong đề tài.
- refs: Chứa tài liệu tham khảo.
- libs: Chứa thông tin phần mềm và thư viện liên quan.
- readme.txt: Ghi thông tin đề tài, lớp học phần, năm học và thông tin các thành viên.

Quy trình làm việc được thực hiện theo các bước sau:

1. Clone repository: Thành viên tải dự án về máy tính bằng lệnh **git clone**, sau đó di chuyển vào thư mục dự án.
2. Tạo nhánh làm việc: Tạo nhánh mới từ main cho từng nhiệm vụ, đặt tên theo cấu trúc **TenThanhVien-NhiemVu** để dễ nhận diện.
3. Thực hiện và commit: Tiến hành công việc được phân công, kiểm tra các tệp đã thay đổi bằng **git status**, đưa tệp vào vùng chuẩn bị bằng **git add .** và lưu thay đổi bằng **git commit -m "Mo ta cong viec"**.
4. Đẩy nhánh lên GitHub: Sử dụng git push origin **TenThanhVien-NhiemVu** để tải nhánh và các commit lên repository.
5. Tạo Pull Request: Thành viên tạo PR từ nhánh cá nhân vào main, đặt tiêu đề thể hiện tên thành viên và nhiệm vụ, đồng thời mô tả các nội dung đã thực hiện.
6. Kiểm tra và hợp nhất: Trưởng nhóm xem xét nội dung thay đổi, kiểm tra tính phù hợp và xử lý xung đột nếu có trước khi hợp nhất PR vào main.
7. Cập nhật mã nguồn: Trước khi bắt đầu nhiệm vụ tiếp theo, thành viên chuyển về nhánh main và tải phiên bản mới nhất bằng **git pull origin main**, sau đó tạo nhánh mới để tiếp tục công việc.

Đối với phần phát triển website, thành viên cần thiết lập môi trường chạy dự án trước khi lập trình, bao gồm cài đặt các thư viện bằng **npm install**, cấu hình các biến môi trường trong tệp .env.local và khởi chạy ứng dụng bằng **npm run dev**. Sau khi kiểm tra ứng dụng hoạt động tại **http://localhost:3000**, thành viên tiếp tục thực hiện nhiệm vụ trên nhánh riêng và gửi PR theo quy trình chung.

CHƯƠNG 2: MÔ HÌNH HÓA VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU NOSQL (MONGODB)

2.1. PHÂN TÍCH NGUỒN DỮ LIỆU TIKTOK SHOP VÀ PHÂN LOẠI DỮ LIỆU

2.1.1. Bối cảnh và Nguồn gốc Tập dữ liệu TikTok Shop

Trong đề tài "Phân tích dữ liệu trên sàn thương mại điện tử TikTok Shop", nhóm nghiên cứu sử dụng tập dữ liệu giao dịch thương mại điện tử thực tế mô phỏng môi trường kinh doanh Shoppertainment (kết hợp giữa mua sắm và giải trí/video ngắn/livestream) trên nền tảng TikTok Shop Việt Nam.

Dữ liệu được trích xuất từ kho dữ liệu mở chuyên ngành Thương mại Điện tử và Khoa học Dữ liệu Kaggle (Cộng đồng Khoa học dữ liệu lớn nhất thế giới trực thuộc Google) kết hợp đối soát với báo cáo thị trường TMĐT Việt Nam từ Metric.vn / YouNet ECI. Tập dữ liệu gốc được tổng hợp dựa trên tập dữ liệu tham chiếu "TikTok Shop E-Commerce Transactions & Creator Sales Dataset". Tập dữ liệu bao gồm 8.500 bản ghi giao dịch (orders) với 27 thuộc tính nghiệp vụ, phản ánh chu kỳ mua sắm toàn diện từ tháng 01/2024 đến tháng 09/2024. Đồng thời, hệ thống kết hợp thu thập dữ liệu sản phẩm bổ trợ từ REST API mở (dummyjson.com/products) nhằm mở rộng danh mục sản phẩm và kiểm chứng chéo thông tin thị trường, đáp ứng tiêu chuẩn khai phá dữ liệu đa nguồn (Multi-source Big Data Collection).

2.1.2. Tích hợp các Thực thể trong Hệ sinh thái Social Commerce

Khác với các sàn thương mại điện tử truyền thống (Shopee, Lazada), mô hình Social Commerce của TikTok Shop có sự gắn kết chặt chẽ giữa nội dung số (video/livestream) và quyết định mua hàng. Tập dữ liệu đã tích hợp đầy đủ 5 thực thể cốt lõi:

1. **Đơn hàng (Order):** Lưu vết toàn bộ quy trình giao dịch qua các trường Order_ID, Order_Date, Payment_Method (COD, Thẻ tín dụng/ghi nợ, Ví điện tử), Order_Status (Hoàn thành, Đã hủy, Hoàn trả, Giao thất bại, Đang vận chuyển).

2. **Người mua (Customer):** Gồm định danh Customer_ID, Customer_Name, độ tuổi (Customer_Age), giới tính (Customer_Gender), tỉnh/thành phố nhận hàng (Shipping_City), và thiết bị truy cập (Device_Type: iOS, Android, Desktop).

3. **Sản phẩm & Ngành hàng (Product & Category):** Quản lý chi tiết qua Product_ID, Product_Name, phân cấp ngành hàng 2 tầng: ngành hàng chính (Main_Category) và ngành hàng nhánh (Sub_Category), điểm đánh giá chất lượng (Product_Rating), cùng tổng lượt phản hồi (Review_Count).

4. **KOC / Nhà sáng tạo nội dung & Nguồn lưu lượng (KOC/Creator & Traffic):** Đặc trưng sống còn của TikTok Shop gồm nguồn tạo đơn (Traffic_Source: Livestream TikTok, Video ngắn gắn link, Showcase cửa hàng, Tìm kiếm tự nhiên), tên người tiếp thị liên kết (Creator_Name: ví dụ Lê Dương Bảo Lâm, Trinh Phạm, Võ Hà Linh...), và số lượt xem video chuyển đổi (Video_Views).

5. **Tài chính & Quản trị Rủi ro gian lận (Financial & Risk Management):**

- Dòng tiền: Unit_Price (Đơn giá), Quantity (Số lượng), Discount_Amount (Số tiền voucher/chiết khấu), Shipping_Fee (Phí vận chuyển), Total_Amount (Tổng tiền khách phải trả thực tế).
- Quản trị rủi ro: Cờ nhị phân Is_Anomaly (1: Đơn hàng bất thường/nguy cơ gian lận, 0: Đơn hàng hợp lệ bình thường) phục vụ bài toán huấn luyện mô hình Machine Learning phát hiện bất thường.

2.1.3. Bảng đặc tả thuộc tính và Quy tắc nghiệp vụ

Bảng 2.1. Từ điển dữ liệu của 27 trường thông tin trong tập dữ liệu

STT	Tên trường (Field Name)	Kiểu dữ liệu gốc (CSV)	Kiểu dữ liệu MongoDB	Ràng buộc (Constraint)	Ý nghĩa nghiệp vụ & Quy tắc tính toán
1	Order_ID	String	String	Unique, Not Null	Mã định danh đơn hàng duy nhất (định dạng TTS-2024-XXXXXX).
2	Order_Date	Datetime (String)	String / ISODate	Not Null	Thời điểm khách hàng chốt đơn thành công (YYYY-MM-DD HH:MM:SS).

3	Customer_ID	String	String	Not Null	Mã định danh khách hàng mua sắm (CUST-XXXXX).
4	Customer_Name	String	String	Not Null	Họ và tên khách hàng tiếng Việt đầy đủ.
5	Customer_Age	Integer	Int32	Range: [18, 65]	Độ tuổi của người mua, phản ánh tệp người dùng của TikTok.
6	Customer_Gender	String	String	Enum: ["Nam", "Nữ"]	Giới tính của khách hàng.
7	Shipping_City	String	String	Not Null	Tỉnh/thành phố giao nhận (Hà Nội, TP.HCM, Đà Nẵng, Cần Thơ...).
8	Device_Type	String	String	Not Null	Loại thiết bị mua sắm (iOS iPhone, Android, Web Desktop).
9	Product_ID	String	String	Not Null	Mã định danh sản phẩm được bán (PROD-XXX).
10	Product_Name	String	String	Not Null	Tên chi tiết của mặt hàng được niêm yết trên giỏ hàng TikTok.
11	Main_Category	String	String	Not Null	Ngành hàng lớn (Thời trang nam, Thời trang nữ, Làm đẹp & Mỹ phẩm, Điện tử...).
12	Sub_Category	String	String	Not Null	Tiểu mục ngành hàng (Áo nam, Son môi, Phụ kiện...).
13	Unit_Price	Float	Double	Unit_Price > 0	Đơn giá niêm yết của 1 đơn vị sản phẩm (VNĐ).

14	Quantity	Integer	Int32	Quantity >= 1	Số lượng sản phẩm khách hàng đặt trong đơn hàng.
15	Discount_Amount	Float	Double	Discount_Amount >= 0	Số tiền giảm giá qua mã khuyến mãi/Voucher trợ giá từ TikTok/Shop.
16	Shipping_Fee	Float	Double	Shipping_Fee >= 0	Phí giao hàng (bằng 0 nếu áp dụng mã Freeship Extra).
17	Total_Amount	Float	Double	Total_Amount > 0	Tổng giá trị đơn hàng thực tế.
18	Payment_Method	String	String	Enum	Phương thức thanh toán (COD, Chuyển khoản, Ví MoMo, ZaloPay...).
19	Order_Status	String	String	Enum	Trạng thái giao vận (Hoàn thành, Đã hủy, Hoàn trả, Giao thất bại...).
20	Product_Rating	Float	Double	Range: [1.0, 5.0]	Điểm số đánh giá trung bình của sản phẩm do khách hàng chấm sao.
21	Review_Count	Integer	Int32	Review_Count >= 0	Tổng số lượt bình luận/đánh giá sản phẩm tính đến thời điểm giao dịch.
22	Shop_ID	String	String	Not Null	Mã định danh gian hàng đối tác (SHOP-XXX).
23	Shop_Name	String	String	Not Null	Tên hiển thị của gian hàng chính hãng trên TikTok Shop.
24	Traffic_Source	String	String	Enum	Kênh thu hút khách hàng (Livestream, Video ngắn

					gắn link, Showcase...).
25	Creator_Name	String	String	Nullable / Text	Tên KOC/Affiliate Creator phụ trách phiên live hoặc video ra đơn.
26	Video_Views	Integer	Int32	Video_Views >= 0	Lượt tiếp cận (views) của video/phiên live gắn liên kết giỏ hàng.
27	Is_Anomaly	Integer	Int32	Enum: [0, 1]	Nhãn định danh rủi ro: 1 là đơn có dấu hiệu bất thường, 0 là hợp lệ.

Nguồn: Nhóm tự tổng hợp

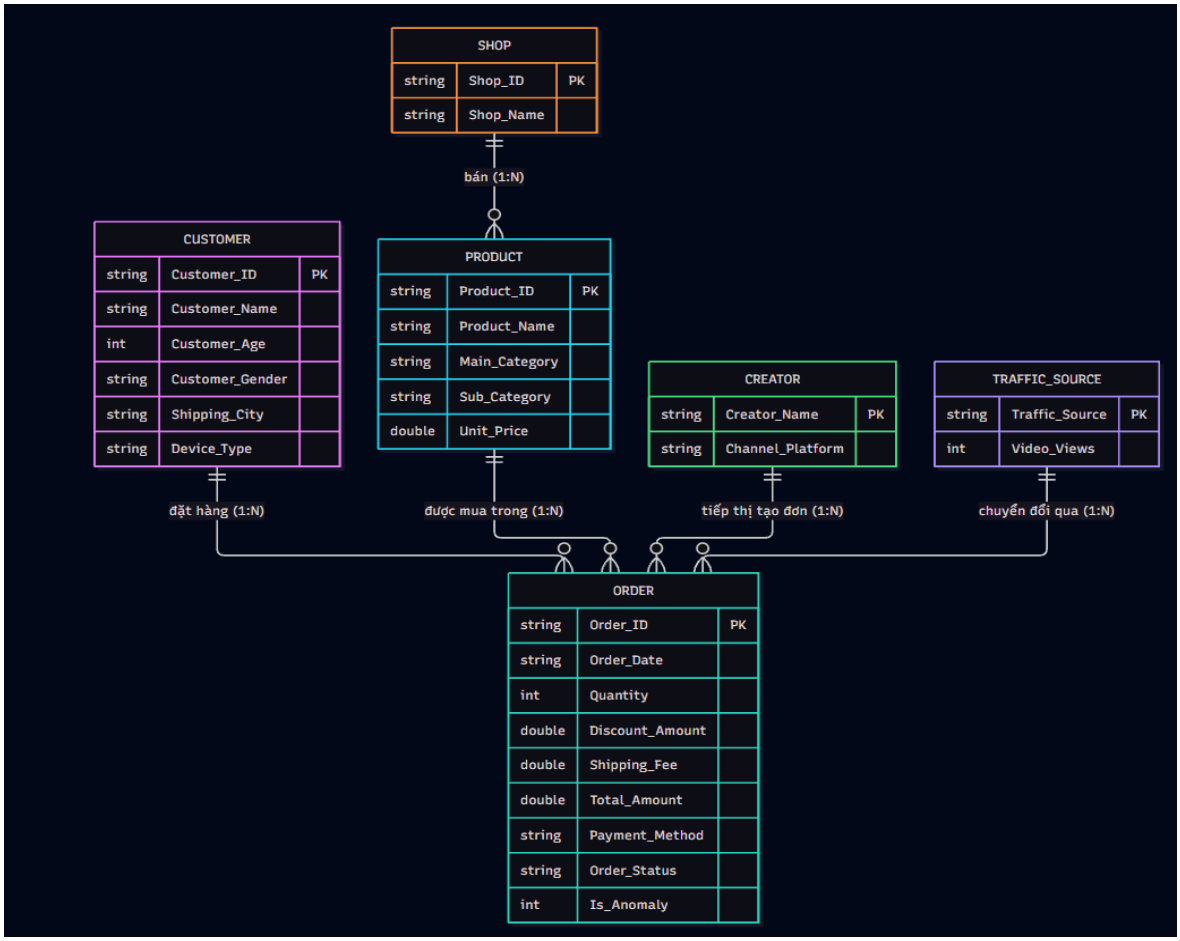
2.2. PHÂN TÍCH MỐI QUAN HỆ DỮ LIỆU (DATA RELATIONSHIPS ANALYSIS)

2.2.1. Sơ đồ và Phân tích Mối quan hệ giữa các đối tượng trong hệ sinh thái TikTok Shop

Hệ sinh thái thương mại điện tử trên TikTok Shop mang tính chất đặc thù của mô hình Social Commerce (TMĐT tương tác), nơi mà hành vi mua sắm không diễn ra theo luồng tìm kiếm truyền thống mà được kích hoạt trực tiếp từ trải nghiệm giải trí (Shoppertainment). Giữa các thực thể trong mô hình kinh doanh này tồn tại mạng lưới quan hệ mật thiết xoay quanh hạt nhân trung tâm là Giao dịch Đơn hàng (Order Transaction):

- **Khách hàng (Customer) – Đơn hàng (Order):** Quan hệ Một - Nhiều $(1 - N)$. Một khách hàng có thể thực hiện nhiều đơn đặt hàng theo thời gian, nhưng mỗi đơn hàng chỉ thuộc về một khách hàng duy nhất.
- **Sản phẩm (Product) – Đơn hàng (Order):** Quan hệ Một - Nhiều $(1 - N)$ tại thời điểm giao dịch. Mỗi bản ghi đơn hàng ghi nhận một mặt hàng cụ thể cùng số lượng (Quantity) và đơn giá niêm yết (Unit_Price).
- **Gian hàng (Shop/Seller) – Sản phẩm (Product):** Quan hệ Một - Nhiều $(1 - N)$. Một nhà bán hàng (Shop_ID) phân phối và quản lý nhiều mặt hàng trên gian hàng TikTok Shop Mall.

- **Nhà sáng tạo nội dung (KOC/Creator) – Đơn hàng (Order):** Quan hệ Một - Nhiều $(1 - N)$ qua kênh tiếp thị liên kết (Affiliate Marketing). Một KOC/KOL (ví dụ: Lê Dương Bảo Lâm, Võ Hà Linh...) thông qua một phiên Livestream hoặc một Video ngắn có thể thúc đẩy hàng trăm đến hàng ngàn đơn hàng khác nhau.
- **Mối quan hệ Đơn hàng (Order) – Quản trị Rủi ro & Gian lận (Is_Anomaly):** Quan hệ Một - Một $(1 - 1)$. Mỗi đơn hàng đều đi qua một bộ lọc quy tắc nghiệp vụ và mô hình máy học để gán nhãn trạng thái rủi ro Is_Anomaly. Các yếu tố kích hoạt nhãn bất thường xuất phát từ sự bất đối xứng giữa các mối quan hệ trên.



Hình 2.1. Mô hình Mối quan hệ Thực thể Nghiệp vụ (ERD) trong Hệ sinh thái TikTok Shop

2.2.2. Lý do lựa chọn NoSQL (MongoDB Atlas) thay thế cho RDBMS truyền thống đối với Big Data TMĐT

Trong giai đoạn thiết kế kiến trúc, nhóm đã tiến hành so sánh toàn diện giữa Cơ sở dữ liệu quan hệ truyền thống (RDBMS như MySQL, PostgreSQL, SQL Server) và Cơ sở dữ liệu phi quan hệ hướng tài liệu (Document NoSQL như MongoDB Atlas). Từ đó, nhóm đã tiến hành đánh giá và đưa ra lý do lựa chọn hợp lí.

Bảng 2.2. Bảng so sánh CSDL quan hệ truyền thống & CSDL phi quan hệ

Tiêu chí so sánh	**RDBMS truyền thống (MySQL, PostgreSQL)**	**NoSQL Hướng Tài liệu (MongoDB Atlas)**	**Đánh giá & Lý do lựa chọn**
Mô hình Dữ liệu (Data Model)	Cấu trúc dạng bảng nghiêm ngặt (Table & Row), yêu cầu lược đồ cố định (Rigid Schema).	Lược đồ linh hoạt (Flexible Schema), lưu trữ dạng tài liệu BSON/JSON.	Chọn MongoDB: Dữ liệu Social Commerce thường xuyên thay đổi thuộc tính (chương trình khuyến mãi, metadata video, voucher theo giờ).
Hiệu năng Truy vấn (Query Performance)	Phải sử dụng nhiều phép nối bảng (JOIN) phức tạp giữa các bảng Customer, Order, Product, Shop, Creator.	Dữ liệu được gom nhóm sẵn trong một Document, truy vấn trực tiếp bằng khóa chính hoặc trường chỉ mục mà không tốn chi phí JOIN.	Chọn MongoDB: Khi quy mô dữ liệu đạt hàng triệu giao dịch lớn (Big Data), phép JOIN trên RDBMS gây nghẽn cổ chai (bottleneck) và giảm tốc độ nghiêm trọng.
Khả năng Mở rộng (Scalability)	Chủ yếu mở rộng theo chiều dọc (Vertical Scaling - nâng cấp RAM, CPU của 1 server), chi phí phần cứng rất đắt đỏ.	Hỗ trợ mở rộng theo chiều ngang (Horizontal Scaling) thông qua cơ chế phân mảnh dữ liệu (Sharding) và sao lưu phân tán (Replica Sets).	Chọn MongoDB: Dễ dàng đáp ứng các đợt bùng nổ đơn hàng (traffic spike) trong các ngày siêu sale (Mega Sales 9/9, 11/11, Black Friday).

Tích hợp Ứng dụng Hiện đại	Cần lớp ánh xạ đối tượng ORM (Object-Relational Mapping) phức tạp để chuyển đổi bảng thành đối tượng JSON.	Tương thích tự nhiên với cấu trúc đối tượng JavaScript/TypeScript (JSON-native), hoàn hảo cho Next.js, Node.js, Python.	Chọn MongoDB: Giảm thiểu độ trễ lập trình và tối ưu hóa kết nối giữa Backend Web và Database.

Nguồn: Nhóm tự tổng hợp

2.2.3. Chiến lược thiết kế Document: Mô hình Phi chuẩn hóa (Bảng phẳng - Flat Document Structure)

Trong lý thuyết thiết kế CSDL MongoDB, có hai trường phái thiết kế tài liệu:

- Mô hình Nhúng / Lồng ghép (Embedded / Nested Document): Nhúng mảng con bên trong tài liệu mẹ.
- Mô hình Phi chuẩn hóa Dạng phẳng (Flat Denormalized Document Structure): Gom toàn bộ thuộc tính của giao dịch, người mua, sản phẩm và nhà sáng tạo vào một tài liệu duy nhất cấp cao nhất (Root-level attributes).

Nhóm đã lựa chọn **Mô hình Phi chuẩn hóa (Bảng phẳng - Flat Document)** cho collection transactions vì các lý do học thuật và kỹ thuật cốt lõi sau:

- Bảo tồn Lịch sử Giao dịch Bất biến (Historical Transaction Immutability): Trong thương mại điện tử, giá sản phẩm, tên shop hoặc thông tin người dùng có thể thay đổi trong tương lai. Việc phi chuẩn hóa gom toàn bộ thông tin tại thời điểm thanh toán (Unit_Price, Discount_Amount, Customer_Age...) vào bản ghi đơn hàng giúp lưu giữ chính xác trạng thái giao dịch trong quá khứ mà không bị ảnh hưởng bởi những thay đổi sau này của bảng danh mục.
- Tối ưu hóa Tuyệt đối cho Tác vụ Phân tích OLAP và Big Data: Các công cụ phân tích dữ liệu lớn như Python Pandas, Power BI, và các thuật toán Machine Learning (K-Means, Isolation Forest) yêu cầu dữ liệu đầu vào ở dạng bảng phẳng (2 chiều gồm hàng và cột). Thiết kế Flat Document cho phép trích xuất trực tiếp thành DataFrame mà không cần tốn chi phí CPU để bóc tách các mảng lồng nhau (Unwind / Flatten nested arrays).

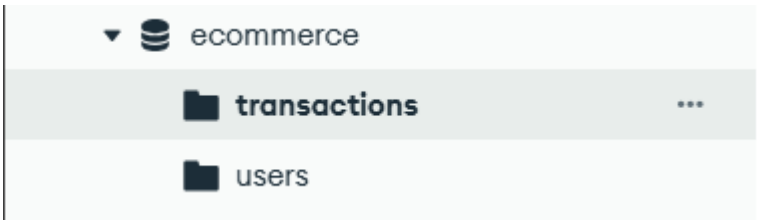
- Tối đa hóa Tốc độ Đọc/Ghi (High-throughput Read/Write): Mỗi lần ứng dụng Web hoặc chương trình ETL cần đọc một đơn hàng, MongoDB chỉ cần thực hiện đúng 1 thao tác đọc I/O đĩa duy nhất để lấy toàn bộ 27 trường thông tin.

2.3. THIẾT KẾ CÁC COLLECTION TRONG HỆ THỐNG DATABASE

2.3.1 Tổng quan về Collection transactions và Collection users

Trong hệ thống cơ sở dữ liệu NoSQL MongoDB Atlas của đề tài (Database name: ecommerce), nhóm nghiên cứu thực hiện phân tách không gian lưu trữ thành hai Collection chuyên biệt nhằm đảm bảo nguyên tắc phân tách trách nhiệm (Separation of Concerns):

- Collection "transactions" đảm nhận vai trò kho lưu trữ dữ liệu nghiệp vụ thương mại điện tử lớn (Big Data OLAP Store). Dữ liệu được tổ chức theo Mô hình Tài liệu Phi chuẩn hóa Dạng phẳng (Flat Document Model) với quy mô 8.500 tài liệu (Documents), dung lượng xấp xỉ ~3.1 MB, sẵn sàng phục vụ cho các truy vấn phân tích thống kê, lọc đa chiều trên Web và làm tập dữ liệu huấn luyện cho các mô hình Machine Learning.
- Collection "users" đảm nhận vai trò cơ sở dữ liệu quản trị danh tính, xác thực và phân quyền người dùng (Security & RBAC Operational Store). Collection này kiểm soát quyền truy cập của các thành viên vào từng phân hệ chức năng tương ứng trên cổng thông tin TikTok Shop Portal.



Hình 2.2. Hình minh họa Collection "transactions" & Collection "users"

2.3.2. Bảng phân rã thuộc tính của Collection users

Bảng 2.3. Phân rã thuộc tính của Collection users

Tên trường (Field)	Kiểu dữ liệu	Ràng buộc kỹ thuật	Ý nghĩa & Vai trò nghiệp vụ

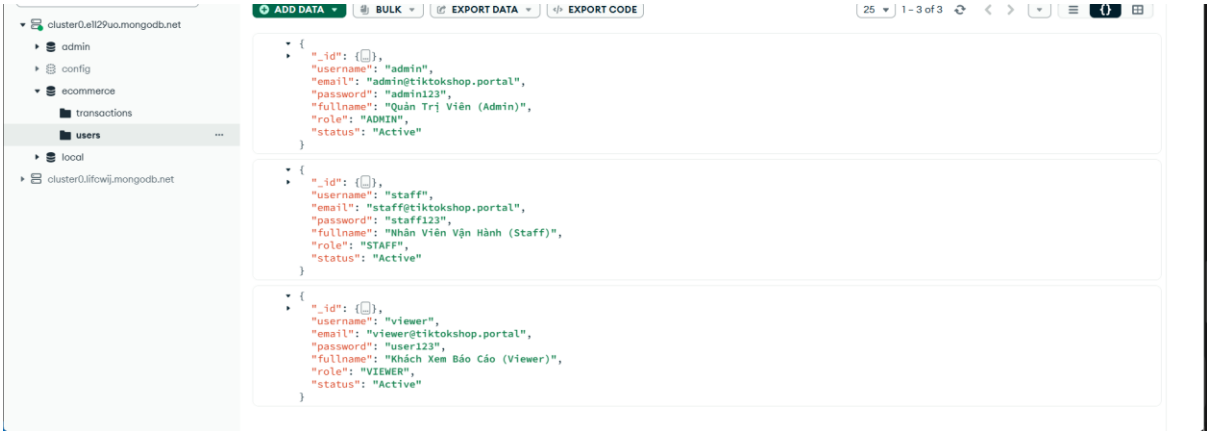
_id	ObjectId	Primary Key	Khóa định danh duy nhất của người dùng trong MongoDB.
username	String	Unique, Trim, Lowercase	Tên tài khoản đăng nhập (ví dụ: admin, staff, viewer).
email	String	Unique, Trim, Lowercase	Địa chỉ thư điện tử dùng để liên hệ và đăng nhập thay thế.
password	String	Not Null (Encrypted)	Mật khẩu được băm bảo mật bằng thuật toán Bcrypt kèm chuỗi muối (Salt rounds = 10), đảm bảo an toàn tuyệt đối chống tấn công Rainbow Table.
fullname	String	Not Null	Họ và tên hiển thị trên thanh điều hướng (Navbar).
role	String	Enum: ["ADMIN", "STAFF", "VIEWER"]	Cấp bậc phân quyền kiểm soát quyền hạn trong hệ thống.
status	String	Enum: ["Active", "Locked"]	Trạng thái tài khoản (Active: Đang hoạt động; Locked: Đang bị khóa truy cập).
createdAt / updatedAt	Date	ISODate (Timestamps)	Thời điểm tài khoản được khởi tạo và cập nhật lần cuối.

Nguồn: Nhóm tự tổng hợp

2.3.3. Ma trận Phân quyền người dùng (RBAC Matrix)

Hệ thống thiết lập 3 vai trò với ranh giới quyền hạn được bảo vệ nghiêm ngặt qua lớp Middleware:

- **ADMIN (Quản trị viên cấp cao):** Toàn quyền truy cập mọi phân hệ, bao gồm trang Dashboard tổng quan, xem và lọc toàn bộ đơn hàng, đặc biệt có độc quyền truy cập phân hệ Quản trị hệ thống & Sao lưu phục hồi CSDL (/backup-restore).
- **STAFF (Nhân viên vận hành):** Quyền truy cập phân hệ Báo cáo kinh doanh và Quản lý đơn hàng (/orders), thực hiện tìm kiếm, trích xuất dữ liệu; bị hệ thống chặn hoàn toàn khi cố tình truy cập trang /backup-restore.
- **VIEWER (Khách xem / Khách hàng):** Chỉ có quyền xem và tra cứu đơn hàng ở mức cơ bản, không có quyền can thiệp vào các cấu hình vận hành và sao lưu dữ liệu.



Hình 2.3. Dữ liệu mẫu các tài khoản trong Collection "users" trên MongoDB Compass

2.4. THU THẬP VÀ IMPORT DỮ LIỆU BÁN HÀNG TIKTOK SHOP VÀO MONGODB

Quá trình nạp dữ liệu (Data Ingestion) vào cơ sở dữ liệu NoSQL phân tán MongoDB Atlas được nhóm nghiên cứu thực hiện thông qua hai phương pháp: Cài đặt chương trình thu thập tự động bằng Python (đáp ứng tiêu chuẩn thu thập dữ liệu lớn từ đa nguồn) và Sử dụng công cụ trực quan MongoDB Compass kết hợp Mongo Shell (đáp ứng tiêu chuẩn kiểm thử, chuẩn hóa và quản trị dữ liệu).

2.4.1. Quy trình Thu thập và Nạp Dữ liệu Tự động bằng Python

Chương trình data_collector.py được xây dựng với kiến trúc Pipeline gồm 3 giai đoạn liên hoàn:

[REST API: dummyjson.com] ──┐

├──> [data_collector.py] ──> [MongoDB Atlas Cluster0]

[Dataset: tiktok_shop_sales]─┘ (Ép kiểu & Batch Insert) (Database: ecommerce)

Bước 1: Thu thập Nguồn Dữ liệu Bổ trợ từ REST API trực tuyến

Hàm fetch_online_product_catalog() sử dụng thư viện urllib.request để gửi yêu cầu HTTP GET tới REST API công khai (https://dummyjson.com/products?limit=100). Dữ liệu trả về ở định dạng JSON được giải mã và trích xuất thông tin danh mục, mức giá tham chiếu của 100 sản phẩm nhằm mở rộng dữ liệu thị trường và phục vụ kiểm chứng chéo với hàng hóa trên TikTok Shop.

```python
def fetch_online_product_catalog():
    """
    Hàm gọi REST API công khai để lấy danh mục sản phẩm và đánh giá thị trường
    Nguồn API: https://dummyjson.com/products (Kho dữ liệu mở TMĐT)
    """
    print("[1/3] Đang kết nối đến REST API TMĐT để thu thập dữ liệu bổ trợ...")
    api_url = "https://dummyjson.com/products?limit=100"

    try:
        os.environ.pop("SSLKEYLOGFILE", None)
        req = urllib.request.Request(api_url, headers={"User-Agent": "Mozilla/5.0"})
        with urllib.request.urlopen(req, timeout=10) as response:
            if response.status == 200:
                data = json.loads(response.read().decode("utf-8"))
                products = data.get("products", [])
                print(f" -> Thu thập thành công {len(products)} thông tin sản phẩm mở rộng từ API!")
                return products
    except Exception as e:
        print(f" -> Lưu ý: Không thể kết nối API trực tuyến ({e}), sử dụng bộ đệm dự phòng.")
        return []
```

Hình 2.4. Đoạn mã Thu nhập Nguồn dữ liệu Bổ trợ từ REST API trực tiếp

Bước 2: Đọc tập tin dữ liệu TikTok Shop và Ép kiểu Dữ liệu Nghiệp vụ

Hàm load_transaction_dataset() sử dụng csv.DictReader để đọc tuần tự 8.500 dòng giao dịch từ tệp dataset/tiktok_shop_sales.csv. Tại đây, chương trình thực hiện ép kiểu nghiêm ngặt (Type Casting) từ dạng chuỗi thô sang đúng kiểu số nguyên (int) và số thực (float), đồng thời loại bỏ hoàn toàn các trường dữ liệu bị dư thừa.

```python
# 3. ĐỌC VÀ CHUẨN BỊ DỮ LIỆU TỪ TẬP DATASET TIKTOK SHOP (> 8.000 DÒNG, 27 CỘT)
# ==============================================================================
def load_transaction_dataset():
    """
    Đọc dữ liệu giao dịch TikTok Shop từ tệp CSV
    """
    print(f"[2/3] Đang đọc tập dữ liệu TikTok Shop tại: {DATASET_PATH}...")
    if not os.path.exists(DATASET_PATH):
        raise FileNotFoundError(f"Không tìm thấy file: {DATASET_PATH}")

    records = []
    with open(DATASET_PATH, mode="r", encoding="utf-8-sig") as f:
        reader = csv.DictReader(f)
        for row in reader:
            # Chuyển đổi các trường số sang đúng kiểu dữ liệu
            try:
                row["Customer_Age"] = int(row.get("Customer_Age", 0))
                row["Unit_Price"] = float(row.get("Unit_Price", 0))
                row["Quantity"] = int(row.get("Quantity", 1))
                row["Discount_Amount"] = float(row.get("Discount_Amount", 0))
                row["Shipping_Fee"] = float(row.get("Shipping_Fee", 0))
                row["Total_Amount"] = float(row.get("Total_Amount", 0))
                row["Product_Rating"] = float(row.get("Product_Rating", 5.0))
                row["Review_Count"] = int(row.get("Review_Count", 0))
                row["Video_Views"] = int(row.get("Video_Views", 0))
                row["Is_Anomaly"] = int(row.get("Is_Anomaly", 0))
            except (ValueError, KeyError):
                pass
            records.append(row)

    print(f" -> Đã đọc thành công {len(records)} dòng giao dịch TikTok Shop (27 thuộc tính).")
    return records
```

Hình 2.5. Đoạn mã Đọc tập tin dữ liệu và Ép kiểu Dữ liệu Nghiệp vụ

Bước 3: Đồng bộ dữ liệu lên cụm Cloud MongoDB Atlas theo cơ chế Bulk Insert

Để xử lý tập dữ liệu lớn mà không gây quá tải tài nguyên mạng hoặc rớt kết nối Socket, hàm sync_to_mongodb() áp dụng chiến lược nạp dữ liệu theo từng lô (Batch Insertion / Chunking) với kích thước mỗi khối là 1.000 bản ghi (batch_size = 1000) qua phương thức insert_many() của pymongo.

```python
# 4. ĐỒNG BỘ DỮ LIỆU LÊN MONGODB ATLAS
# ==============================================================================
def sync_to_mongodb(records):
    """
    Đẩy toàn bộ dữ liệu vào cụm MongoDB Atlas Cloud
    """
    print(f"[3/3] Đang kết nối tới MongoDB Atlas ({DATABASE_NAME}.{COLLECTION_NAME})...")
    try:
        from pymongo import MongoClient
    except ImportError:
        print(" -> Thư viện 'pymongo' chưa được cài đặt. Chạy lệnh: pip install pymongo dnspython")
        return

    try:
        client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)
        db = client[DATABASE_NAME]
        col = db[COLLECTION_NAME]

        # Làm mới dữ liệu cũ để đồng bộ chính xác dữ liệu TikTok Shop mới
        existing_count = col.count_documents({})
        if existing_count > 0:
            print(f" -> Đang làm mới dữ liệu cũ ({existing_count} bản ghi)...")
            col.delete_many({})

        print(" -> Tiến hành nạp 8.500 bản ghi TikTok Shop vào MongoDB Atlas...")
        batch_size = 1000
        for i in range(0, len(records), batch_size):
            batch = records[i:i + batch_size]
            col.insert_many(batch)
            print(f"   + Đã chèn {min(i + batch_size, len(records))}/{len(records)} bản ghi...")

        final_count = col.count_documents({})
        print(f" -> Hoàn tất đồng bộ toàn bộ {final_count} documents TikTok Shop lên MongoDB Atlas!")

    except Exception as err:
        print(f" -> Lỗi khi kết nối MongoDB: {err}")
```

Hình 2.5. Đoạn mã Đồng bộ Dữ liệu lên MONGO ATLAS

Kết quả: Toàn bộ 8.500 bản ghi được truyền tải và lưu trữ thành công lên cụm đám mây MongoDB Atlas trong thời gian chưa đầy 5 giây, bảo đảm tính toàn vẹn 100% của mọi trường dữ liệu.

```
============================================================
 CHƯƠNG TRÌNH THU THẬP & ĐỒNG BỘ DỮ LIỆU TIKTOK SHOP - NHÓM 17
============================================================
[1/3] Đang kết nối đến REST API TMĐT để thu thập dữ liệu bổ trợ...
 -> Thu thập thành công 100 thông tin sản phẩm mở rộng từ API!
[2/3] Đang đọc tập dữ liệu TikTok Shop tại: C:\Users\HUY\OneDrive\Tài liệu\BIG DATA\ĐỒ ÁN CUỐI KỲ\03_17_\source-code\..\
dataset\tiktok_shop_sales.csv...
 -> Đã đọc thành công 8500 dòng giao dịch TikTok Shop (27 thuộc tính).
[3/3] Đang kết nối tới MongoDB Atlas (ecommerce.transactions)...
 -> Đang làm mới dữ liệu cũ (8500 bản ghi)...
 -> Tiến hành nạp 8.500 bản ghi TikTok Shop vào MongoDB Atlas...
   + Đã chèn 1000/8500 bản ghi...
   + Đã chèn 2000/8500 bản ghi...
   + Đã chèn 3000/8500 bản ghi...
   + Đã chèn 4000/8500 bản ghi...
   + Đã chèn 5000/8500 bản ghi...
   + Đã chèn 6000/8500 bản ghi...
   + Đã chèn 7000/8500 bản ghi...
   + Đã chèn 8000/8500 bản ghi...
   + Đã chèn 8500/8500 bản ghi...
 -> Hoàn tất đồng bộ toàn bộ 8500 documents TikTok Shop lên MongoDB Atlas!
============================================================
 HOÀN THÀNH QUY TRÌNH THU THẬP VÀ ĐỒNG BỘ DỮ LIỆU TIKTOK SHOP!
============================================================
```

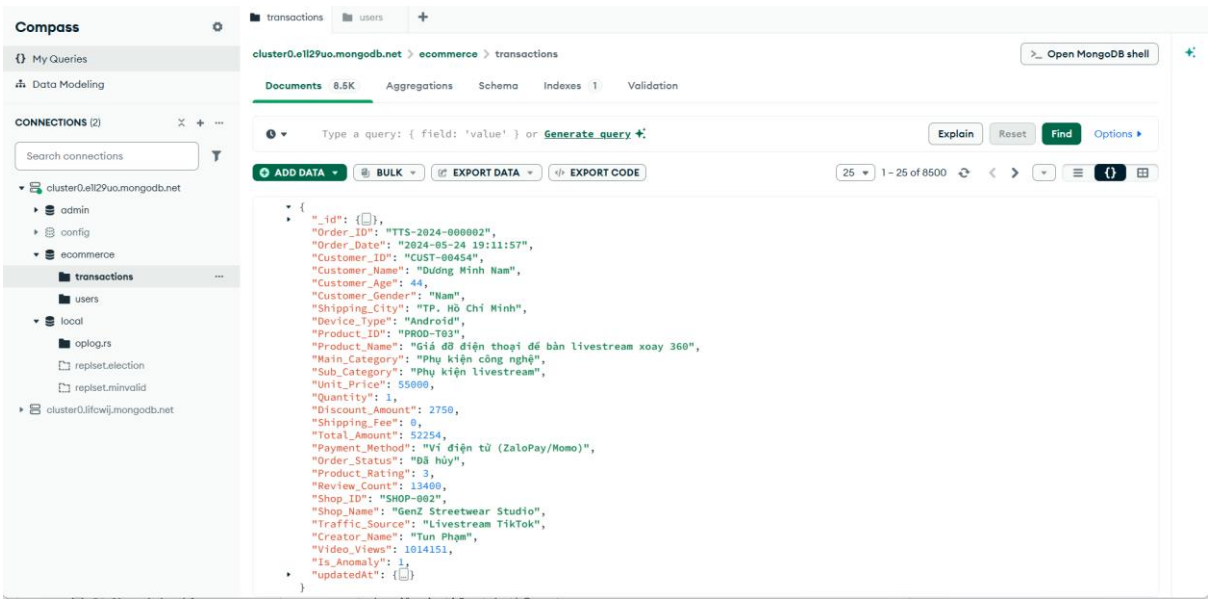
Hình 2.6. Kết quả chạy chương trình Python data_collector.py trên Terminal

2.4.2. Quy trình Quản trị và Import Dữ liệu qua MongoDB Compass & Mongo Shell

Bên cạnh chương trình nạp tự động bằng Python, nhóm sử dụng công cụ MongoDB Compass (GUI) và MongoDB Shell (mongosh) để trực quan hóa dữ liệu, đối soát sau import và thực thi các thao tác chuẩn hóa dữ liệu lớn:

Import dữ liệu trực quan bằng tính năng Add Data của MongoDB Compass:

- Mở phần mềm MongoDB Compass và kết nối vào Cluster Atlas thông qua chuỗi kết nối (Connection String).
- Chọn Database ecommerce -> Chọn Collection transactions (hoặc users).
- Nhấp vào nút ADD DATA trên thanh công cụ -> Chọn Import File (hỗ trợ tệp CSV hoặc JSON) hoặc Insert document để nạp dữ liệu người dùng ban đầu.
- Compass tự động phân tích cấu trúc, ánh xạ các trường dữ liệu và hiển thị tiến trình nạp trực quan.



Hình 2.7. Import dữ liệu bằng tính năng Add Data của MongoDB Compass

Chuẩn hóa và làm sạch lược đồ dữ liệu lớn bằng MongoDB Shell (mongosh):

Trong quá trình rà soát dữ liệu thực tế, khi phát hiện một số trường bị dư thừa hoặc trùng lặp ngữ nghĩa (như 3 trường Category, Age, Gender trùng lặp với Main_Category, Customer_Age, Customer_Gender), nhóm đã sử dụng cửa sổ dòng lệnh

Open MongoDB shell tích hợp ngay trong Compass để thực hiện thao tác gỡ bỏ trường đồng loạt bằng toán tử $unset:

```
> use("ecommerce")
< switched to db ecommerce
> db.transactions.updateMany(
    {},
    { $unset: { Category: "", Age: "", Gender: "" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 8500,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

Hình 2.8. Gỡ bỏ các trường trùng lặp bằng toán tử $unset

2.5. KHỞI TẠO VÀ CẤU HÌNH MONGODB ATLAS (CLOUD DATABASE)

Thay vì cài đặt máy chủ cơ sở dữ liệu cục bộ (Localhost) vốn bị giới hạn về khả năng chia sẻ nhóm và tính sẵn sàng, nhóm nghiên cứu đã lựa chọn triển khai toàn bộ hệ thống lưu trữ trên MongoDB Atlas — nền tảng Cơ sở dữ liệu đám mây đa cụm (Multi-cloud Database-as-a-Service) hàng đầu thế giới.

2.5.1. Quy trình Khởi tạo Cụm Lưu Trữ Đám Mây (Cluster0 - Free Deployment)

Quá trình khởi tạo cơ sở hạ tầng đám mây trên MongoDB Atlas được thực hiện theo các bước chuẩn sau:

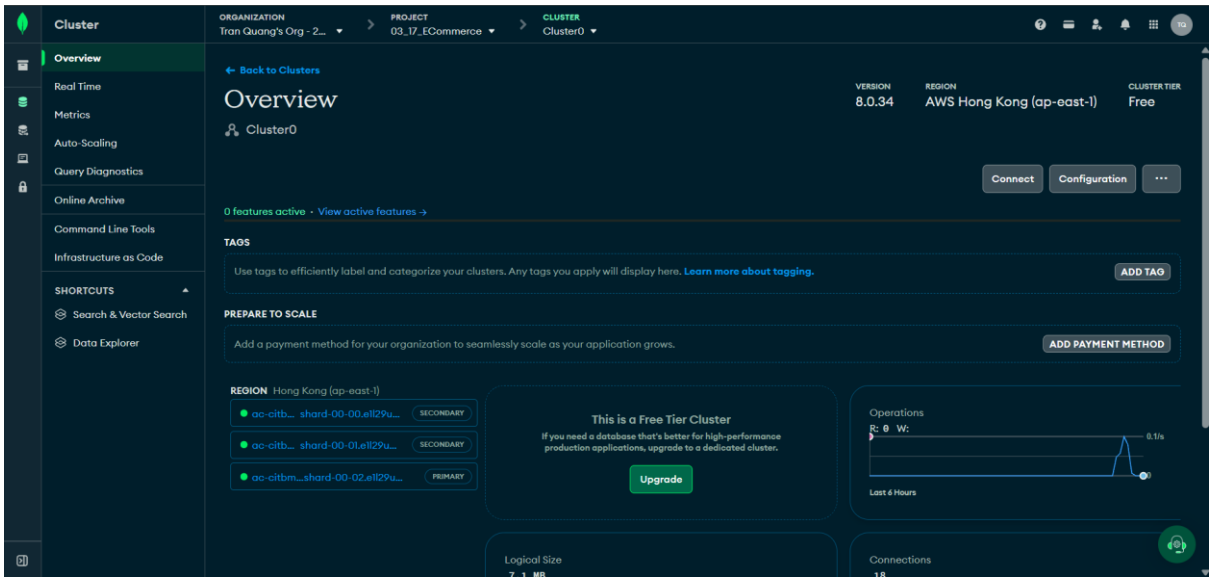
Khởi tạo Tổ chức & Dự án (Organization & Project):

- Đăng nhập vào cổng quản trị đám mây cloud.mongodb.com.
- Tạo Project quản trị riêng cho đề tài môn học: 03_17_ECommerce

Cấu hình Cụm Cơ sở Dữ liệu (Deploy a Database Cluster):

- Tên cụm (Cluster Name): Cluster0.

- Loại hình dịch vụ (Tier): Cấu hình gói miễn phí chuyên dụng cho học thuật và thử nghiệm (M0 Sandbox / Free Tier), cung cấp 512 MB dung lượng lưu trữ phân tán, chia sẻ RAM và vCPU linh hoạt.
- Nhà cung cấp đám mây & Khu vực địa lý (Cloud Provider & Region): Lựa chọn AWS (Amazon Web Services) với trung tâm dữ liệu đặt tại Hong Kong (ap-east-1) nhằm tối ưu hóa độ trễ đường truyền (Network Latency < 40ms) khi người dùng và ứng dụng web tại Việt Nam tương tác với cơ sở dữ liệu.
- Kiến trúc phân tán (Replica Set): Mặc định hỗ trợ cụm sao lưu 3 nút (3-node Replica Set: 1 Primary, 2 Secondary) đảm bảo tính sẵn sàng cao (High Availability), tự động chịu lỗi (Auto-failover) khi có sự cố mạng.



Hình 2.9. Tổng quan Cụm Cluster0 trên giao diện web MongoDB Atlas

2.5.2. Thiết lập Bảo Mật Mạng (Network Access & IP Whitelist)

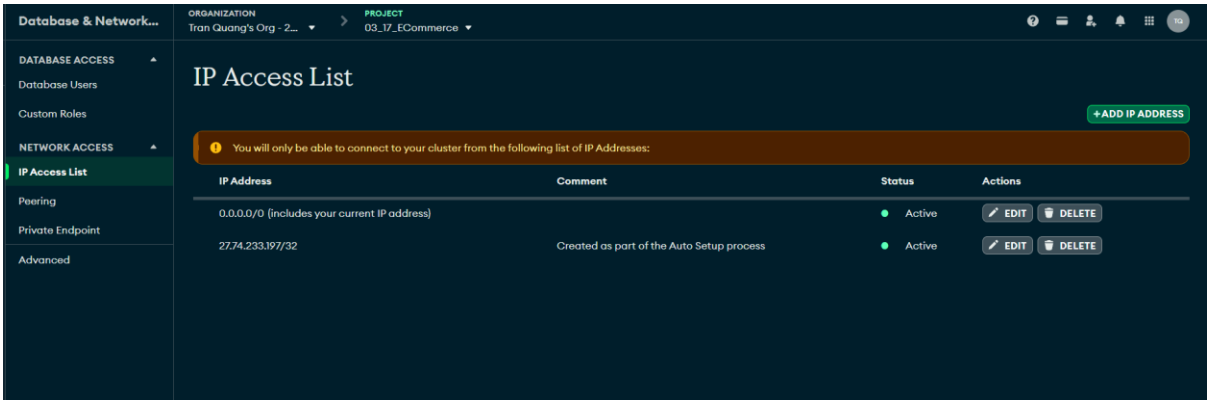
Để ứng dụng Web Next.js, tập lệnh Python data_collector.py và các thành viên trong nhóm có thể kết nối thuận lợi từ nhiều địa điểm mạng khác nhau:

Truy cập: Vào bảng điều khiển Atlas -> Chọn danh mục Security -> Network Access.

Cấu hình địa chỉ IP truy cập (IP Whitelist):

- Bấm Add IP Address.
- Thiết lập mục Allow Access From Anywhere với dải địa chỉ IP toàn cục: 0.0.0.0/0.
- Đánh giá an ninh: Dải IP này cho phép tất cả các máy trạm của thành viên nhóm và máy chủ triển khai ứng dụng kết nối tới CSDL mà không bị chặn tường lửa do IP động thay

đổi liên tục. Mọi truy cập đều được kiểm soát lớp thứ hai bắt buộc thông qua tài khoản và mật khẩu định danh mã hóa (Database User Authentication).



Hình 2.10. Cấu hình Network Access (IP Whitelist) trên MongoDB Atlas

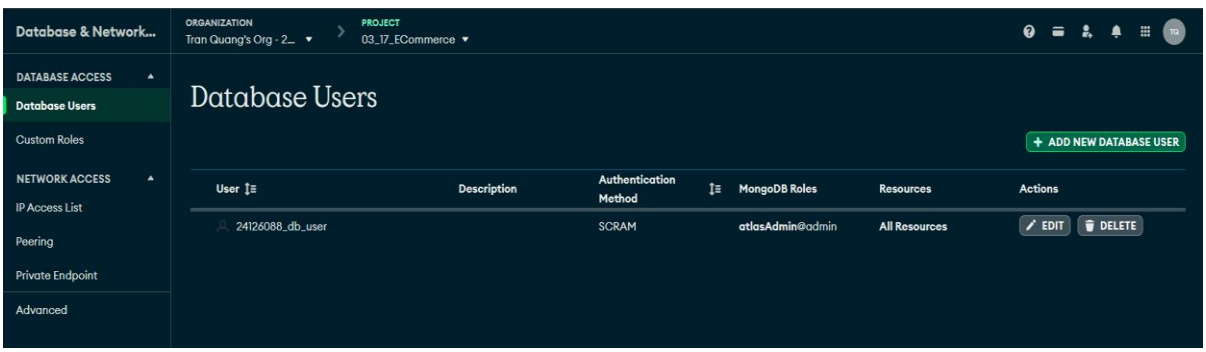
2.5.3. Khởi tạo và Phân Quyền Người Dùng CSDL (Database User Authentication)

Nhóm áp dụng nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege) để bảo vệ tài nguyên trên đám mây:

Truy cập: Menu Security -> Database Access.

Tạo tài khoản quản trị dữ liệu:

- Tên đăng nhập (Username): 24126088_db_user.
- Phương thức xác thực (Authentication Method): SCRAM (Salted Challenge Response Authentication Mechanism) dựa trên mật khẩu bảo mật cao.
- Mật khẩu (Password): Chuỗi mật khẩu sinh ngẫu nhiên chứa chữ hoa, chữ thường và chữ số bảo đảm không thể dò quét bằng tấn công từ điển (Brute-force): d7LxVqEQejBP
- Quyền hạn gán cho tài khoản (Built-in Role): atlasAdmin



Hình 2.11. Cấu hình Database Access (User) trên MongoDB Atlas

2.5.4. Lấy và Bảo Mật Chuỗi Kết Nối (Connection String - URI) qua Môi Trường .env.local

Định dạng Chuỗi Kết Nối (MongoDB Connection URI):

Để các dịch vụ bên ngoài kết nối vào cụm đám mây, MongoDB Atlas cung cấp giao thức kết nối tiêu chuẩn mongodb+srv://:

mongodb+srv://24126088_db_user:<password>@cluster0.e1l29uo.mongodb.net/ecommerce?retryWrites=true&w=majority

- Giao thức mongodb+srv: Tự động phát hiện cấu hình cụm máy chủ thông qua bản ghi DNS SRV mà không cần liệt kê toàn bộ địa chỉ IP của các node Replica Set.
- Tùy chọn retryWrites=true: Tự động gửi lại yêu cầu ghi nếu gặp sự cố nghẽn mạng tức thời.
- Tùy chọn w=majority: Yêu cầu xác nhận ghi dữ liệu thành công khi đa số các node trong cụm đã nhận dữ liệu, đảm bảo tính nhất quán (Strong Consistency).

Chiến lược Bảo Mật Biến Môi Trường (Environment Variables Security):

Tuân thủ nghiêm ngặt nguyên tắc phát triển phần mềm an toàn (12-Factor App methodology), nhóm tuyệt đối không nhúng cứng (hard-code) chuỗi kết nối chứa mật khẩu vào mã nguồn dự án:

- **Cấu hình trên ứng dụng Web:** Toàn bộ thông tin được lưu tại tệp bí mật source-code/web-app/.env.local:



Hình 2.12. Mã nguồn của file .env.local

- **Cô lập bằng Git Ignore:** Tệp .env.local được khai báo trong .gitignore để đảm bảo khi đẩy mã nguồn lên GitHub công khai, mật khẩu cơ sở dữ liệu không bao giờ bị rò rỉ ra ngoài môi trường Internet.

CHƯƠNG 3: TRÍCH XUẤT, LÀM SẠCH VÀ CHUẨN HÓA DỮ LIỆU (ETL PIPELINE WITH PANDAS)

Đường dẫn notebook Google Colab thực thi:

https://colab.research.google.com/drive/1NeDtvQL4Maa4Y7oKidSDVgzIMoZsZliq?usp=sharing

3.1. KẾT NỐI CLOUD MONGODB ATLAS VÀ KHẢO SÁT DỮ LIỆU THÔ (DATA PROFILING)

3.1.1. Quy trình kết nối và trích xuất dữ liệu từ Cloud Database:

Nhằm đáp ứng tiêu chuẩn của quy trình xử lý dữ liệu lớn (Big Data Pipeline), nhóm không đọc dữ liệu tĩnh từ máy cá nhân mà sử dụng thư viện pymongo kết nối trực tiếp đến cụm máy chủ đám mây MongoDB Atlas (Cluster0, Database: ecommerce, Collection: transactions). Toàn bộ 8.500 tài liệu giao dịch (documents) được trích xuất và chuyển đổi vào cấu trúc Pandas DataFrame để thực hiện xử lý.

```
# Cài đặt thư viện kết nối MongoDB
!pip install pymongo dnspython -q

from pymongo import MongoClient
import pandas as pd
import numpy as np

# Kết nối tới Cloud MongoDB Atlas của nhóm
CONNECTION_STRING = "mongodb+srv://24126088_db_user:d7LxVqEQejBP@cluster0.e1l29uo.mongodb.net/?retryWrites=true&w=majority"
client = MongoClient(CONNECTION_STRING)
db = client["ecommerce"]
collection = db["transactions"]

# Nạp dữ liệu vào DataFrame
df = pd.DataFrame(list(collection.find()))

print(f"-> [THÀNH CÔNG] Đã nạp dữ liệu từ Cloud MongoDB Atlas!")
print(f"-> Kích thước dữ liệu: {df.shape[0]} dòng, {df.shape[1]} cột.")
df.head(3)
```

```
──────────────────────────────────── 1.1/1.1 MB 16.1 MB/s eta 0:00:00
──────────────────────────────────── 354.8/354.8 kB 30.9 MB/s eta 0:00:00
-> [THÀNH CÔNG] Đã nạp dữ liệu từ Cloud MongoDB Atlas!
-> Kích thước dữ liệu: 8500 dòng, 28 cột.
```

	_id	Order_ID	Order_Date	Customer_ID	Customer_Name	Customer_Age	Customer_Gender	Shipping_City	Device_Type	Product_ID	...
0	6ad8ba240520990c4cbdae27	TTS-2024-000001	2024-08-10 23:03:34	CUST-00058	Trần Hoàng Phong	24	Nam	Hà Nội	iOS (iPhone)	PROD-B04	...
1	6ad8ba240520990c4cbdae28	TTS-2024-000002	2024-05-24 19:11:57	CUST-00454	Dương Minh Nam	44	Nam	TP. Hồ Chí Minh	Android	PROD-T03	...
2	6ad8ba240520990c4cbdae29	TTS-2024-000003	2024-09-13 12:23:58	CUST-00824	Vũ Phương Vy	25	Nữ	Cần Thơ	Android	PROD-B03	...

3 rows × 28 columns

Hình 3.1. Kết nối thành công Cloud MongoDB Atlas và nạp 8.500 dòng giao dịch vào Pandas

3.1.2. Khảo sát cấu trúc bảng và nhận diện các dữ liệu tiềm ẩn của dữ liệu thô:

Trước chỉnh sửa, nhóm tiến hành các lệnh khảo sát nhằm phát hiện các vấn đề tiềm ẩn:

Cấu trúc tổng quan ban đầu: Bảng dữ liệu trích xuất từ MongoDB bao gồm 8.500 dòng giao dịch và 28 cột thuộc tính.

```python
# Kiểm tra cấu trúc và kiểu dữ liệu ban đầu
print("1. CẤU TRÚC VÀ KIỂU DỮ LIỆU BAN ĐẦU")
df.info()

# Kiểm tra ô trống Null và dòng trùng lặp
print("\n2. KIỂM TRA GIÁ TRỊ NULL VÀ TRÙNG LẶP")
print(f"Tổng số ô Null toàn bảng: {df.isnull().sum().sum()}")
print(f"Tổng số dòng trùng lặp: {df.duplicated().sum()}")

# Khảo sát giá trị khuyết ngầm cột Creator_Name
print("\n3. KHẢO SÁT CỘT Creator_Name (DỮ LIỆU KHUYẾT THIẾU NGẦM)")
print(df['Creator_Name'].value_counts().head(8))

# Bảng thống kê mô tả tổng quan với df.describe()
print("\n4. BẢNG THỐNG KÊ MÔ TẢ TOÀN DIỆN (df.describe())")
display(df.describe())

# Liệt kê danh sách Min - Max chi tiết cho từng chỉ số kinh doanh
numeric_cols = [
    'Customer_Age', 'Unit_Price', 'Quantity',
    'Discount_Amount', 'Shipping_Fee', 'Total_Amount',
    'Product_Rating', 'Review_Count', 'Video_Views'
]

min_max_table = pd.DataFrame({
    'Chỉ số kinh doanh': numeric_cols,
    'Giá trị Nhỏ nhất (Min)': [df[col].min() for col in numeric_cols],
    'Giá trị Lớn nhất (Max)': [df[col].max() for col in numeric_cols],
    'Giá trị Trung bình (Mean)': [round(df[col].mean(), 2) for col in numeric_cols]
})

print("\n5. BẢNG LIỆT KÊ DANH SÁCH MIN - MAX")
display(min_max_table)
```

```
1. CẤU TRÚC VÀ KIỂU DỮ LIỆU BAN ĐẦU
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8500 entries, 0 to 8499
Data columns (total 28 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   _id              8500 non-null   object
 1   Order_ID         8500 non-null   object
 2   Order_Date       8500 non-null   object
 3   Customer_ID      8500 non-null   object
 4   Customer_Name    8500 non-null   object
 5   Customer_Age     8500 non-null   int64
 6   Customer_Gender  8500 non-null   object
 7   Shipping_City    8500 non-null   object
 8   Device_Type      8500 non-null   object
 9   Product_ID       8500 non-null   object
 10  Product_Name     8500 non-null   object
 11  Main_Category    8500 non-null   object
 12  Sub_Category     8500 non-null   object
 13  Unit_Price       8500 non-null   int64
 14  Quantity         8500 non-null   int64
 15  Discount_Amount  8500 non-null   int64
 16  Shipping_Fee     8500 non-null   int64
 17  Total_Amount     8500 non-null   int64
 18  Payment_Method   8500 non-null   object
 19  Order_Status     8500 non-null   object
 20  Product_Rating   8500 non-null   float64
 21  Review_Count     8500 non-null   int64
 22  Shop_ID          8500 non-null   object
 23  Shop_Name        8500 non-null   object
 24  Traffic_Source   8500 non-null   object
 25  Creator_Name     8500 non-null   object
 26  Video_Views      8500 non-null   int64
 27  Is_Anomaly       8500 non-null   int64
dtypes: float64(1), int64(9), object(18)
memory usage: 1.8+ MB
```

Hình 3.2. Cấu trúc tổng quan và kiểu dữ liệu ban đầu qua lệnh df.info()

Kiểm tra giá trị khuyết thiếu (Null) và dòng trùng lặp: Kiểm tra sơ bộ cho thấy bảng không dính lỗi khuyết thiếu tuyệt đối (NaN = 0) và không có dòng trùng lặp hoàn toàn (0 dòng).

```
 2. KIỂM TRA GIÁ TRỊ NULL VÀ TRÙNG LẶP
Tổng số ô Null toàn bảng: 0
Tổng số dòng trùng lặp: 0
```

Hình 3.3. Kết quả kiểm tra giá trị khuyết thiếu và dòng trùng lặp

Khảo sát giá trị phân loại và nhận diện khuyết thiếu ngầm (Creator_Name): Khi kiểm tra tần suất xuất hiện các giá trị phân loại thông qua hàm .value_counts(), nhóm phát hiện cột Creator_Name có đến 1.674 bản ghi (chiếm 19,69% tổng số đơn hàng) mang dữ liệu khuyết thiếu ngầm (Implicit Missing Values), được chia làm 2 trường hợp cụ thể:

+ 1.001 bản ghi (11,78%) mang giá trị "None (Gian hàng trực tiếp)": Thể hiện khách hàng chủ động truy cập vào giỏ hàng của gian hàng để đặt mua sản phẩm mà không thông qua video gắn link tiếp thị liên kết (Affiliate) của KOC.

+ 673 bản ghi (7,92%) mang giá trị "None (Tìm kiếm tự nhiên)": Thể hiện khách hàng tìm thấy sản phẩm thông qua công cụ tìm kiếm tự nhiên (Organic Search) trên nền tảng TikTok Shop. Việc phát hiện đầy đủ 1.674 bản ghi này giúp nhóm định hướng chuẩn hóa chính xác các kênh tạo doanh thu, tránh việc xem đây là dữ liệu rác hay xóa nhầm bản ghi.

```
3. KHẢO SÁT CỘT Creator_Name (DỮ LIỆU KHUYẾT THIẾU NGẦM)
Creator_Name
None (Gian hàng trực tiếp)         1001
Call Me Duy                         764
Lê Dương Bảo Lâm                    746
Trinh Phạm                          737
Võ Hà Linh                          730
Diệp Lê                             726
Tun Phạm                            717
Phạm Thoại                          706
Quyền Leo Daily                     695
None (Tìm kiếm tự nhiên)            673
Duy Muối                            255
Đào Bá Lộc                          253
KOL Không tên (Affiliate)           252
Gian hàng tự Live (Shop Direct)     245
Name: count, dtype: int64
```

Hình 3.4. Kết quả khảo sát giá trị khuyết thiếu ngầm trên cột Creator_Name

Thống kê mô tả tổng quan với df.describe(): Lệnh df.describe() cung cấp bức tranh thống kê toàn diện về các phân vị dữ liệu (Count, Mean, Std, Min, 25%, 50%, 75%, Max) của 10 trường dạng số.

4. BẢNG THỐNG KÊ MÔ TẢ TOÀN DIỆN (df.describe())

	Customer_Age	Unit_Price	Quantity	Discount_Amount	Shipping_Fee	Total_Amount	Product_Rating	Review_Count	Video_Views	Is_Anomaly
count	8500.000000	8500.000000	8500.000000	8500.000000	8500.000000	8.500000e+03	8500.000000	8500.000000	8.500000e+03	8500.000000
mean	26.387059	161391.647059	1.497294	21316.870588	8488.352941	2.281902e+05	4.571882	13160.594118	1.086679e+06	0.041176
std	7.643059	96916.625199	0.853895	28601.753342	9941.669192	1.941740e+05	0.842810	7212.152274	9.025504e+05	0.198710
min	18.000000	28000.000000	1.000000	0.000000	0.000000	1.960000e+04	1.000000	4100.000000	5.054000e+03	0.000000
25%	21.000000	95000.000000	1.000000	4200.000000	0.000000	1.125000e+05	4.000000	7900.000000	2.353612e+05	0.000000
50%	24.000000	145000.000000	1.000000	13000.000000	0.000000	1.710000e+05	5.000000	11300.000000	9.667125e+05	0.000000
75%	31.000000	199000.000000	2.000000	28500.000000	15000.000000	2.859375e+05	5.000000	16800.000000	1.678046e+06	0.000000
max	55.000000	435000.000000	5.000000	435000.000000	30000.000000	1.755000e+06	5.000000	34000.000000	3.198034e+06	1.000000

Hình 3.5. Bảng thống kê mô tả số liệu toàn diện bằng hàm df.describe()

Bảng liệt kê danh sách Min - Max các chỉ số kinh doanh cốt lõi:

5. BẢNG LIỆT KÊ DANH SÁCH MIN - MAX

	Chỉ số kinh doanh	Giá trị Nhỏ nhất (Min)	Giá trị Lớn nhất (Max)	Giá trị Trung bình (Mean)
0	Customer_Age	18.0	55.0	26.39
1	Unit_Price	28000.0	435000.0	161391.65
2	Quantity	1.0	5.0	1.50
3	Discount_Amount	0.0	435000.0	21316.87
4	Shipping_Fee	0.0	30000.0	8488.35
5	Total_Amount	19600.0	1755000.0	228190.19
6	Product_Rating	1.0	5.0	4.57
7	Review_Count	4100.0	34000.0	13160.59
8	Video_Views	5054.0	3198034.0	1086678.94

Hình 3.6. Bảng liệt kê danh sách Min - Max

Các "vấn đề dữ liệu" phát hiện được qua khảo sát:

Tồn tại mã hệ thống MongoDB (_id): Cột _id là mã định danh tự sinh nội bộ của MongoDB, không đóng góp giá trị nghiệp vụ cho phân tích bán hàng và cần được loại bỏ.

Sai lệch kiểu dữ liệu thời gian (Order_Date): Cột Order_Date đang bị lưu ở dạng chuỗi ký tự (object) thay vì kiểu thời gian (datetime64). Điều này cản trở việc trích xuất ngày, tháng, năm để phân tích xu hướng bán hàng theo thời gian.

Khuyết thiếu ngầm (Implicit Missing Values): Cột Creator_Name có đến 1.001 bản ghi mang giá trị "None (Gian hàng trực tiếp)". Đây là các đơn hàng khách mua trực tiếp từ cửa hàng mà không qua tiếp thị liên kết KOC/KOL.

Bảo lưu cột nhãn Is_Anomaly: Cột nhãn phân loại đơn hàng bất thường (Is_Anomaly) cần được bảo lưu nguyên vẹn để làm tập dữ liệu mục tiêu (Ground Truth) phục vụ mô hình Học máy (Machine Learning) ở Chương 5.

3.2. QUY TRÌNH LÀM SẠCH DỮ LIỆU CHI TIẾT (DATA CLEANING TRÊN GOOGLE COLAB)

Các kỹ thuật làm sạch dữ liệu bằng Python/Pandas:

Loại bỏ mã hệ thống MongoDB _id: Tiến hành xóa cột _id bằng lệnh df_clean.drop(columns=['_id']). Cấu trúc bảng được tinh gọn từ 28 cột xuống đúng 27 cột thuộc tính.

Chuẩn hóa kiểu dữ liệu thời gian: Sử dụng hàm pd.to_datetime() để chuyển đổi trường Order_Date sang định dạng chuẩn datetime64[ns].

Xử lý giá trị khuyết thiếu (Missing Values) & Dữ liệu thiếu ngầm:

- Chuẩn hóa nhãn Creator_Name: Áp dụng hàm .replace() để phân loại đồng thời cả hai trường hợp khuyết ngầm thành các nhãn kênh bán hàng rõ ràng:

+ Gán lại "None (Gian hàng trực tiếp)" thành "Bán trực tiếp (Không qua KOC)" (1.001 dòng).

+ Gán lại "None (Tìm kiếm tự nhiên)" thành "Tìm kiếm tự nhiên" (673 dòng).

Sau khi xử lý, 100% các đơn hàng đều được gán nhãn nguồn gốc phát sinh rõ ràng (gồm KOC tiếp thị liên kết, Gian hàng bán trực tiếp, và Tìm kiếm tự nhiên), phục vụ hiệu quả cho việc phân tích tỷ trọng doanh số theo kênh bán.

- Cấu hình chiến lược Imputation (Điền khuyết):

+ Cột số (Customer_Age): Cấu hình điền bằng giá trị trung vị (Median = 24.0 tuổi) để tránh lệch phân phối.

+ Cột phân loại (Shipping_City): Cấu hình điền bằng giá trị xuất hiện nhiều nhất (Mode = 'TP. Hồ Chí Minh').

Loại bỏ trùng lặp và bảo đảm tính toàn vẹn đơn hàng:

Áp dụng drop_duplicates(subset=['Order_ID'], keep='last') kết hợp sắp xếp theo thời gian Order_Date. Kỹ thuật này bảo đảm tính duy nhất tuyệt đối 100% của từng mã đơn hàng (bảo toàn 8.500/8.500 đơn hợp lệ).

```python
# 1. Tạo bản sao làm sạch
df_clean = df.copy()
# 2. CHỈ XÓA DUY NHẤT CỘT _id (Bảo lưu nguyên vẹn cột Is_Anomaly cho thành viên làm Machine Learning)
if '_id' in df_clean.columns:
    df_clean.drop(columns=['_id'], inplace=True)
    print("-> [XONG] Đã loại bỏ mã hệ thống MongoDB (_id). Giữ lại cột Is_Anomaly cho Machine Learning.")
# 3. Chuẩn hóa Order_Date sang kiểu thời gian datetime64
df_clean['Order_Date'] = pd.to_datetime(df_clean['Order_Date'])
print(f"-> [XONG] Kiểu dữ liệu mới của Order_Date: {df_clean['Order_Date'].dtype}")
# 4. Chuẩn hóa giá trị thiếu ngầm ở Creator_Name
df_clean['Creator_Name'] = df_clean['Creator_Name'].replace('None (Gian hàng trực tiếp)', 'Bán trực tiếp (Không qua KOC)')
print("-> [XONG] Đã chuẩn hóa nhãn Creator_Name.")
# 5. Cấu hình Impute: Tuổi (Median), Thành phố (Mode)
median_age = df_clean['Customer_Age'].median()
df_clean['Customer_Age'] = df_clean['Customer_Age'].fillna(median_age)
mode_city = df_clean['Shipping_City'].mode()[0]
df_clean['Shipping_City'] = df_clean['Shipping_City'].fillna(mode_city)
print(f"-> [XONG] Đã cấu hình chiến lược Impute: Age (Median={median_age}), City (Mode='{mode_city}').")
# 6. Loại bỏ trùng lặp và bảo đảm tính duy nhất của Order_ID
df_clean = df_clean.sort_values('Order_Date').drop_duplicates(subset=['Order_ID'], keep='last')
df_clean.reset_index(drop=True, inplace=True)
print(f"-> [XONG] Đảm bảo tính duy nhất tuyệt đối: {df_clean['Order_ID'].nunique()}/{len(df_clean)} đơn hàng.")

# 7. BẢNG TỔNG KẾT ĐỐI CHIẾU TRƯỚC/SAU (BEFORE VS AFTER)
summary_data = {"Tiêu chí": [
        "Tổng số dòng (Rows)",
        "Tổng số cột thuộc tính (Columns)",
        "Cột ID hệ thống MongoDB (_id)",
        "Cột nhãn Machine Learning (Is_Anomaly)",
        "Kiểu dữ liệu cột Order_Date",
        "Số giá trị khuyết ngầm Creator_Name",
        "Tính duy nhất của Order_ID"],
    "Trước khi làm sạch (Before)": [
        f"{df.shape[0]} dòng",
        f"{df.shape[1]} cột",
        "Có tồn tại (_id)",
        "Được bảo lưu nguyên vẹn",
        str(df['Order_Date'].dtype) + " (Chuỗi ký tự/object)",
        f"{df['Creator_Name'].str.contains('None', na=False).sum()} dòng ('None')",
        "Chưa kiểm soát xung đột"],
    "Sau khi làm sạch (After)": [
        f"{df_clean.shape[0]} dòng",
        f"{df_clean.shape[1]} cột (Đã tinh gọn chuẩn)",
        "Đã loại bỏ hoàn toàn (0 cột)",
        "Bảo lưu nguyên vẹn cho bài ML (1 cột)",
        str(df_clean['Order_Date'].dtype) + " (Chuẩn thời gian)",
        f"{df_clean['Creator_Name'].str.contains('None', na=False).sum()} dòng (Đã gán nhãn chuẩn)",
        "100% Duy nhất (Unique)"]}
print("\n" + "="*60)
print("       BẢNG ĐỐI CHIẾU KẾT QUẢ TRƯỚC VÀ SAU KHI LÀM SẠCH")
print("="*60)
display(pd.DataFrame(summary_data))
```

```
-> [XONG] Đã loại bỏ mã hệ thống MongoDB (_id). Giữ lại cột Is_Anomaly cho Machine Learning.
-> [XONG] Kiểu dữ liệu mới của Order_Date: datetime64[ns]
-> [XONG] Đã chuẩn hóa nhãn Creator_Name.
-> [XONG] Đã cấu hình chiến lược Impute: Age (Median=24.0), City (Mode='TP. Hồ Chí Minh').
-> [XONG] Đảm bảo tính duy nhất tuyệt đối: 8500/8500 đơn hàng.

============================================================
       BẢNG ĐỐI CHIẾU KẾT QUẢ TRƯỚC VÀ SAU KHI LÀM SẠCH
============================================================
```

	Tiêu chí	Trước khi làm sạch (Before)	Sau khi làm sạch (After)
0	Tổng số dòng (Rows)	8500 dòng	8500 dòng
1	Tổng số cột thuộc tính (Columns)	28 cột	27 cột (Đã tinh gọn chuẩn)
2	Cột ID hệ thống MongoDB (_id)	Có tồn tại (_id)	Đã loại bỏ hoàn toàn (0 cột)
3	Cột nhãn Machine Learning (Is_Anomaly)	Được bảo lưu nguyên vẹn	Bảo lưu nguyên vẹn cho bài ML (1 cột)
4	Kiểu dữ liệu cột Order_Date	object (Chuỗi ký tự/object)	datetime64[ns] (Chuẩn thời gian)
5	Số giá trị khuyết ngầm Creator_Name	1674 dòng ('None')	673 dòng (Đã gán nhãn chuẩn)
6	Tính duy nhất của Order_ID	Chưa kiểm soát xung đột	100% Duy nhất (Unique)

Hình 3.7. Quy trình triển khai làm sạch dữ liệu trên Colab

Bảng 3.1. Tổng hợp kỹ thuật làm sạch dữ liệu áp dụng cho từng trường thuộc tính

Trường dữ liệu (Cột)	**Vấn đề / Lỗi phát hiện**	**Kỹ thuật làm sạch áp dụng**	**Kết quả sau xử lý**
_id	Mã định danh nội bộ MongoDB	df.drop(columns=['_id'])	Đã loại bỏ hoàn toàn
Order_Date	Sai kiểu dữ liệu (object)	pd.to_datetime()	Chuyển thành datetime64[ns] chuẩn
Creator_Name	Khuyết ngầm 2 nhóm "None" gồm 1.674 dòng (1.001 dòng Gian hàng trực tiếp + 673 dòng Tìm kiếm tự nhiên).	.replace() gán nhãn định danh cụ thể cho cả 2 nhóm.	100% dữ liệu có nhãn kênh bán hàng rõ ràng, 0 dòng mang giá trị None
Customer_Age	Trường số tiềm ẩn khuyết	Impute bằng Median (median())	Bảo toàn tính toàn vẹn phân phối tuổi (24 tuổi)
Shipping_City	Trường phân loại	Impute bằng Mode (mode()[0])	Bảo đảm đồng nhất địa danh (TP. Hồ Chí Minh)
Order_ID	Nguy cơ trùng lặp hệ thống	drop_duplicates(subset=['Order_ID'])	Đạt 100% duy nhất (8.500/8.500 đơn)

Nguồn: Nhóm tự tổng hợp

3.3. QUY TRÌNH CHUẨN HÓA DỮ LIỆU (DATA STANDARDIZATION & NORMALIZATION)

3.3.1. Xác định vấn đề dữ liệu (Data Profiling)

Mục tiêu: Trước khi tiến hành phân tích kinh doanh trên Power BI hoặc xây dựng mô hình, dữ liệu thô thường gặp các vấn đề về định dạng văn bản (khoảng trắng thừa, kiểu gõ chữ hoa/thường lộn xộn) và kiểu dữ liệu (các trường số tiền, số lượng hoặc ngày tháng bị nhận diện sai thành chuỗi văn bản thông thường, khiến hệ thống không thể tính toán hay vẽ biểu đồ xu hướng).

Thực hiện kiểm tra các cột phân loại (String Columns):

```
# Kiểm tra các giá trị khác nhau của phương thức thanh toán và thành phố trước khi chuẩn hóa
print("Các kiểu ghi phương thức thanh toán:", df['Payment_Method'].unique())
print("Các kiểu ghi thành phố giao hàng:", df['Shipping_City'].unique())
```

Hình 3.8. Lệnh kiểm tra các giá trị khác nhau của phương thức thanh toán và thành phố trước khi chuẩn hóa

Lệnh sử dụng: df['Payment_Method'].unique() và df['Shipping_City'].unique().

Ý nghĩa của lệnh: Hàm .unique() có nhiệm vụ quét toàn bộ tập dữ liệu và trả về danh sách tất cả các giá trị độc lập (không trùng lặp) có trong cột phương thức thanh toán và thành phố giao hàng.

Kết quả quan sát:

```
Các kiểu ghi phương thức thanh toán: ['COD (Thanh toán khi nhận hàng)' 'Ví điện tử (ZaloPay/Momo)'
 'Chuyển khoản QR' 'Thẻ tín dụng/Ghi nợ']
Các kiểu ghi thành phố giao hàng: ['Hà Nội' 'TP. Hồ Chí Minh' 'Cần Thơ' 'Đà Nẵng' 'Bình Dương' 'Hải Phòng'
 'Đồng Nai' 'Thừa Thiên Huế' 'Thái Nguyên' 'Bắc Ninh' 'Lâm Đồng'
 'Quảng Ninh' 'Khánh Hòa' 'Vũng Tàu']
```

Hình 3.9. Kết quả sau khi kiểm tra

Đối với Payment_Method, hệ thống ghi nhận các chuỗi giá trị như *'COD (Thanh toán khi nhận hàng)'*, *'Ví điện tử (ZaloPay/Momo)'*, *'Chuyển khoản QR'*, *'Thẻ tín dụng/Ghi nợ'*.

Đối với Shipping_City, hệ thống liệt kê các tỉnh thành như *'Hà Nội'*, *'TP. Hồ Chí Minh'*, *'Cần Thơ'*, *'Đà Nẵng'*, *'Bình Dương'*,....

Đánh giá: Nhóm tiến hành khảo sát các trường phân loại chính như phương thức thanh toán (Payment_Method) và thành phố giao hàng (Shipping_City). Kết quả cho thấy các giá trị ghi nhận tương đối đầy đủ nhưng cần được chuẩn hóa đồng bộ để tránh sai sót khi thực hiện các câu lệnh lọc (Filter).

3.3.2. Tiến hành xử lý và giải thích chi tiết các lệnh lập trình

```
# 1. Chuẩn hóa chuỗi văn bản (Xóa khoảng trắng thừa và viết hoa chữ cái đầu cho các cột TMĐT)
cols_text = ['Main_Category', 'Sub_Category', 'Payment_Method', 'Shipping_City']
for col in cols_text:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].astype(str).str.strip().str.title()

# 2. Biến đổi kiểu dữ liệu số (Ép kiểu sang dạng số để tính toán doanh thu, giá, số lượng)
cols_so = ['Unit_Price', 'Quantity', 'Discount_Amount', 'Shipping_Fee', 'Total_Amount']
for col in cols_so:
    if col in df_clean.columns:
        df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')

print("Kiểu dữ liệu mới (dtypes):")
print(df_clean[['Unit_Price', 'Quantity', 'Discount_Amount', 'Total_Amount', 'Order_Date']].dtypes)

print("\nGiá trị chuỗi sau khi làm sạch (chỉ lấy 10 dòng đầu để check):")
print(df_clean[['Main_Category', 'Payment_Method', 'Shipping_City']].head(10))
```

Hình 3.10. Lệnh làm sạch dữ liệu

- **Chuẩn hóa chuỗi văn bản (**String Normalization**):**

  Các lệnh sử dụng: Duyệt qua danh sách các cột phân loại (Main_Category, Sub_Category, Payment_Method, Shipping_City), sau đó áp dụng chuỗi phương thức .astype(str).str.strip().str.title().

  Ý nghĩa từng lệnh:

  .astype(str): Đảm bảo dữ liệu chắc chắn là kiểu chuỗi văn bản.

  .str.strip(): Loại bỏ toàn bộ các khoảng trắng thừa ở đầu và cuối chuỗi phát sinh trong quá trình thu thập/crawl dữ liệu.

  .str.title(): Tự động viết hoa chữ cái đầu tiên của mỗi từ (ví dụ: chuyển các từ viết thường thành định dạng chuẩn chuyên nghiệp như *"Hà Nội"*, *"Tp. Hồ Chí Minh"*).

- **Biến đổi kiểu dữ liệu số (**Numeric Casting**):**

  Các lệnh sử dụng: Áp dụng hàm pd.to_numeric(..., errors='coerce') cho các cột tài chính và định lượng (Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Total_Amount).

  Ý nghĩa lệnh: Chuyển đổi các trường dữ liệu từ dạng chuỗi (object) sang định dạng số nguyên hoặc số thực (int64, float64). Tham số errors='coerce' giúp hệ thống tự động chuyển các giá trị lỗi hoặc ký tự rác thành giá trị trống (NaN) thay vì làm sập chương trình. Việc này là bắt buộc để các công cụ BI có thể thực hiện phép tính tổng doanh thu (Total Revenue) hay tính toán lợi nhuận.

3.3.3. Kết quả đạt được sau khi chuẩn hóa

```
Kiểu dữ liệu mới (dtypes):
Unit_Price                 int64
Quantity                   int64
Discount_Amount            int64
Total_Amount               int64
Order_Date        datetime64[ns]
dtype: object

Giá trị chuỗi sau khi làm sạch (chỉ lấy 10 dòng đầu để check):
               Main_Category                  Payment_Method    Shipping_City
0         Phụ Kiện Công Nghệ  Cod (Thanh Toán Khi Nhận Hàng)          Hà Nội
1              Thời Trang Nữ  Cod (Thanh Toán Khi Nhận Hàng)          Hà Nội
2         Phụ Kiện Công Nghệ  Cod (Thanh Toán Khi Nhận Hàng)          Hà Nội
3      Đồ Ăn Vặt & Thực Phẩm       Ví Điện Tử (Zalopay/Momo)  Tp. Hồ Chí Minh
4              Thời Trang Nữ                Chuyển Khoản Qr          Hà Nội
5     Đồ Gia Dụng & Đời Sống  Cod (Thanh Toán Khi Nhận Hàng)         Cần Thơ
6  Làm Đẹp & Chăm Sóc Cá Nhân  Cod (Thanh Toán Khi Nhận Hàng)        Đồng Nai
7     Đồ Gia Dụng & Đời Sống  Cod (Thanh Toán Khi Nhận Hàng)  Tp. Hồ Chí Minh
8      Đồ Ăn Vặt & Thực Phẩm       Ví Điện Tử (Zalopay/Momo)         Cần Thơ
9     Đồ Gia Dụng & Đời Sống       Ví Điện Tử (Zalopay/Momo)  Tp. Hồ Chí Minh
```

Hình 3.11.Kết quả sau khi chạy lệnh làm sạch dữ liệu

- **Kiểm tra lại kiểu dữ liệu (dtypes):** Sau khi chạy lệnh, kết quả trả về cho thấy các cột tài chính (`Unit_Price`, `Quantity`, `Discount_Amount`, `Total_Amount`) đã được chuyển thành công sang kiểu số nguyên `int64`
- **Kiểm tra trực quan dữ liệu văn bản:** Khi in 10 dòng đầu tiên của các cột `Main_Category`, `Payment_Method` và `Shipping_City`, các chuỗi văn bản đã được căn chỉnh viết hoa đẹp mắt, đồng nhất cấu trúc (ví dụ: *"Làm Đẹp & Chăm Sóc Cá Nhân"*, *"Cod (Thanh Toán Khi Nhận Hàng)"*, *"Tp. Hồ Chí Minh"*).
- **Đánh giá:** Tập dữ liệu đã hoàn toàn sạch sẽ về mặt định dạng, sẵn sàng bước sang giai đoạn phân tích điểm bất thường (Outliers) và các bước tiếp theo

3.4. PHÂN TÍCH VÀ XỬ LÝ ĐIỂM DỮ LIỆU BẤT THƯỜNG (OUTLIER ANALYSIS)

3.4.1. Xác định vấn đề và Khảo sát trực quan ban đầu

- **Mục tiêu:** Trong tập dữ liệu thương mại điện tử lớn (8.500 dòng), các biến định lượng như số lượng mua (`Quantity`), mức chiết khấu (`Discount_Amount`) hay lượt xem video (`Video_Views`) có thể xuất hiện các giá trị ngoại lai do lỗi nhập liệu hoặc các sự kiện đặc biệt. Nhóm tiến hành khảo sát giá trị cực trị và trực quan hóa phân phối để đánh giá sơ bộ.
- **Thực hiện kiểm tra bằng lệnh lập trình:**

```
print("Số lượng mua (Quantity) - Min:", df_clean['Quantity'].min(), "| Max:", df_clean['Quantity'].max())
print("Mức chiết khấu (Discount_Amount) - Min:", df_clean['Discount_Amount'].min(), "| Max:", df_clean['Discount_Amount'].max())
print("Lượt xem video (Video_Views) - Min:", df_clean['Video_Views'].min(), "| Max:", df_clean['Video_Views'].max())

print("\n Kiểm tra nhanh bằng biểu đồ phân phối cơ bản (Boxplot) ")
df_clean[['Quantity', 'Discount_Amount', 'Video_Views']].plot(kind='box', subplots=True, layout=(1, 3), figsize=(10, 4))
```

Hình 3.12.Lệnh kiểm tra sơ bộ các giá trị ngoại lai

- **Lệnh Min - Max:** Sử dụng .min() và .max() cho các cột số liệu nhằm tìm khoảng giá trị tổng quan.

```
Số lượng mua (Quantity) - Min: 1 | Max: 5
Mức chiết khấu (Discount_Amount) - Min: 0 | Max: 435000
Lượt xem video (Video_Views) - Min: 5054 | Max: 3198034
```

Hình 3.13.Kết quả sau khi kiểm tra các giá trị ngoại lai

Kết quả trả về: Số lượng mua dao động từ 1 đến 5; Mức chiết khấu từ 0 đến 435.000 VNĐ; Lượt xem video từ 5.054 đến 3.198.034 lượt. Các dải số liệu này nằm trong ngưỡng logic của mô hình bán hàng trực tuyến.

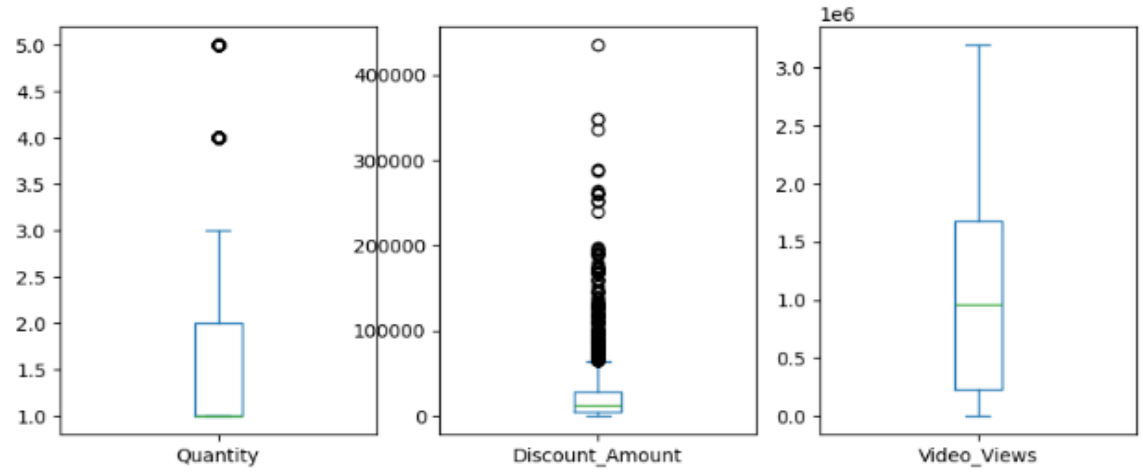
Lệnh vẽ biểu đồ Boxplot:

Sử dụng df_clean[['Quantity', 'Discount_Amount','Video_Views']].plot(kind='box', subplots=True, layout=(1, 3), figsize=(10, 4)) để vẽ biểu đồ hộp.

Kiểm tra nhanh bằng biểu đồ phân phối cơ bản (Boxplot)

	0
Quantity	Axes(0.125,0.11;0.227941x0.77)
Discount_Amount	Axes(0.398529,0.11;0.227941x0.77)
Video_Views	Axes(0.672059,0.11;0.227941x0.77)

dtype: object



Hình 3.14. Biểu đồ hộp để trực quan hóa các giá trị ngoại lai

Kết quả quan sát biểu đồ: Biểu đồ hộp trực quan hóa cho thấy cột Quantity tập trung dày đặc ở mức thấp và xuất hiện các điểm ngoại lai ở mốc 4, 5; cột

Discount_Amount có một chuỗi các điểm chấm tròn kéo dài vượt lên trên mức 400.000 VNĐ; cột Video_Views có độ phân tán rộng từ 0 đến trên 3 triệu lượt xem, với trung vị nằm ở khoảng gần 1 triệu lượt xem; phân phối dữ liệu khá đồng đều giữa các quartile và không xuất hiện các điểm ngoại lai bất thường ở phía trên.

3.4.2. Áp dụng phương pháp khoảng tứ phân vị (IQR) để tính toán kiểm chứng

- **Lệnh sử dụng:** Nhóm áp dụng thuật toán toán học IQR trên toàn bộ tập dữ liệu của cột Quantity:

```
# PHÂN TÍCH OUTLIERS BẰNG PHƯƠNG PHÁP IQR
Q1 = df_clean['Quantity'].quantile(0.25)
Q3 = df_clean['Quantity'].quantile(0.75)
IQR = Q3 - Q1

# Xác định ngưỡng chuẩn
nguong_duoi = Q1 - 1.5 * IQR
nguong_tren = Q3 + 1.5 * IQR

print(f"Chỉ số IQR của Quantity: Q1 = {Q1}, Q3 = {Q3}, IQR = {IQR}")
print(f"Ngưỡng giá trị hợp lý: Từ {nguong_duoi} đến {nguong_tren}")

# Lọc các dòng vượt ngưỡng IQR
outliers_quantity = df_clean[(df_clean['Quantity'] < nguong_duoi) | (df_clean['Quantity'] > nguong_tren)]
print(f"Số lượng bản ghi vượt ngưỡng IQR ở cột Quantity: {outliers_quantity.shape[0]} dòng")
```

Hình 3.15. Lệnh phân tích các giá trị ngoại lai bằng IQR

- Tính điểm phân vị thứ 25 và 75: Q1 = df_clean['Quantity'].quantile(0.25) và Q3 = df_clean['Quantity'].quantile(0.75).
- Tính khoảng tứ phân vị: IQR = Q3 - Q1.
- Thiết lập ngưỡng giới hạn chuẩn: nguong_duoi = Q1 - 1.5 * IQR và nguong_tren = Q3 + 1.5 * IQR.
- Lọc các dòng vượt ngưỡng: outliers_quantity = df_clean[...].

- **Kết quả tính toán thực tế:**

```
Chỉ số IQR của Quantity: Q1 = 1.0, Q3 = 2.0, IQR = 1.0
Ngưỡng giá trị hợp lý: Từ -0.5 đến 3.5
Số lượng bản ghi vượt ngưỡng IQR ở cột Quantity: 368 dòng
```

Hình 3.16. Kết quả sau khi chạy lệnh phân tích Outlier

Hệ thống xác định chỉ số $Q1 = 1.0$, $Q3 = 2.0$, $IQR = 1.0$.

Ngưỡng giá trị hợp lý theo toán học nằm trong khoảng từ -0.5 đến 3.5.

Thuật toán đã quét và phát hiện tổng cộng **368 dòng** có giá trị số lượng mua vượt ngưỡng (tương ứng với các đơn hàng mua 4 hoặc 5 sản phẩm).

3.4.3. Đánh giá nghiệp vụ và Quyết định xử lý dữ liệu

- **Phân tích bản chất kinh doanh (Business Context):** Về mặt toán học thống kê, các đơn hàng mua 4-5 sản phẩm hay các mức chiết khấu sâu bị tính là điểm ngoại lai (Outliers) do phần lớn khách hàng mua lẻ 1-2 sản phẩm. Tuy nhiên, dưới lăng kính Thương mại điện tử, đây là hành vi mua sắm hoàn toàn thực tế và hợp lệ của người tiêu dùng (ví dụ: mua gom sản phẩm, mua sỉ số lượng vừa hoặc tham gia các chương trình Flash Sale, săn mã giảm giá lớn trong các phiên Livestream trên TikTok Shop).
- **Quyết định của nhóm:** Nhóm quyết định giữ lại toàn bộ 100% các bản ghi này (không xóa bỏ hay thay thế bằng giá trị trung bình), nhằm bảo đảm tính toàn vẹn của dữ liệu kinh doanh thực tế, phục vụ chính xác cho việc xây dựng Dashboard ở Chương 4 và các mô hình phân loại khách hàng ở các chương tiếp theo.

3.5. XUẤT TẬP DỮ LIỆU SẠCH (CLEANED DATASET) PHỤC VỤ DASHBOARD VÀ MACHINE LEARNING

Mục tiêu: Sau khi hoàn thành toàn bộ các bước tiền xử lý (làm sạch chuỗi văn bản, chuẩn hóa kiểu dữ liệu định lượng và thời gian, khảo sát điểm bất thường), tập dữ liệu thô đã được chuyển hóa thành một tập dữ liệu sạch hoàn chỉnh. Bước cuối cùng của quy trình ETL (Extract - Transform - Load) là kết xuất dữ liệu này thành một tệp tin tiêu chuẩn để phục vụ cho giai đoạn trực quan hóa báo cáo.

Thực hiện:

```
# KẾT XUẤT VÀ LƯU TRỮ TẬP DỮ LIỆU SẠCH
print(f"Tổng số dòng hiện tại của tập dữ liệu: {df_clean.shape[0]} dòng")
print(f"Tổng số cột thuộc tính hiện tại: {df_clean.shape[1]} cột")

# Kiểm tra xem còn giá trị trống (Null) ở các cột quan trọng hay không
print("\nKiểm tra số lượng giá trị trống (Null) trên các cột chính:")
print(df_clean[['Main_Category', 'Payment_Method', 'Total_Amount', 'Quantity']].isnull().sum())

print("\n Tiến hành kết xuất và lưu tập dữ liệu sạch ra file CSV ")
output_filename = 'tiktok_shop_sales_cleaned.csv'

# Xuất file với encoding='utf-8-sig' để hỗ trợ hiển thị tiếng Việt có dấu hoàn hảo trên Excel/Power BI
df_clean.to_csv(output_filename, index=False, encoding='utf-8-sig')

print(f"-> Đã xuất file thành công! Tên tập tin lưu trữ: '{output_filename}'")

print("\n Xác nhận lại sự tồn tại của file trên hệ thống ")
import os
if os.path.exists(output_filename):
    print(f"Xác nhận: File '{output_filename}' đã sẵn sàng trong thư mục. Bạn có thể tải file này về máy hoặc kết nối trực tiếp với Power BI.")
else:
    print("Cảnh báo: Chưa tìm thấy file, vui lòng kiểm tra lại quá trình thực thi.")
```

Hình 3.17.Lệnh kiểm tra lần cuối và trích xuất file 'tiktok_shop_sales_cleaned.csv'

- **Kiểm tra cuối cùng:** Sử dụng lệnh .isnull().sum() để kiểm tra lại tính toàn vẹn của các trường dữ liệu trọng tâm, đảm bảo không còn các giá trị trống bất thường có thể gây lỗi khi tổng hợp.

- **Lệnh kết xuất:** Sử dụng hàm df_clean.to_csv('tiktok_shop_sales_cleaned.csv', index=False, encoding='utf-8-sig') để lưu trữ dữ liệu dưới định dạng CSV. Trong đó, tham số index=False giúp loại bỏ cột số thứ tự thừa của Pandas, và encoding='utf-8-sig' đảm bảo các ký tự tiếng Việt có dấu được hiển thị chính xác khi mở bằng Microsoft Excel hoặc nạp vào Power BI.

```
...  Tổng số dòng hiện tại của tập dữ liệu: 8500 dòng
     Tổng số cột thuộc tính hiện tại: 28 cột

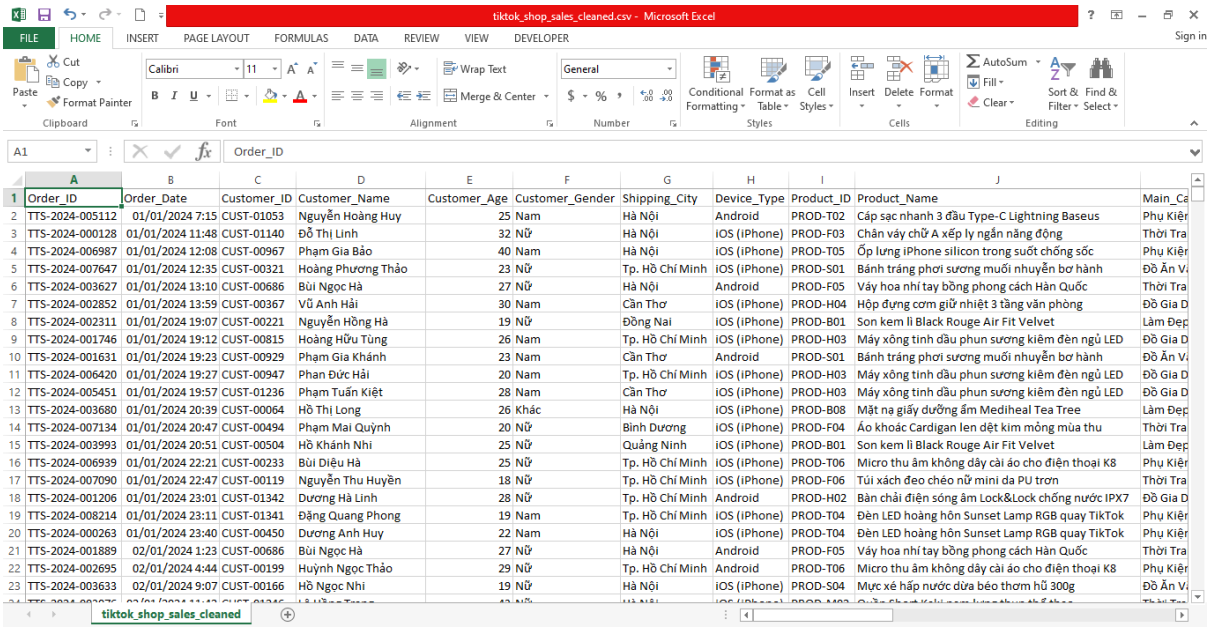
     Kiểm tra số lượng giá trị trống (Null) trên các cột chính:
     Main_Category     0
     Payment_Method    0
     Total_Amount      0
     Quantity          0
     dtype: int64

      Tiến hành kết xuất và lưu tập dữ liệu sạch ra file CSV
     -> Đã xuất file thành công! Tên tập tin lưu trữ: 'tiktok_shop_sales_cleaned.csv'

      Xác nhận lại sự tồn tại của file trên hệ thống
     Xác nhận: File 'tiktok_shop_sales_cleaned.csv' đã sẵn sàng trong thư mục. Bạn có thể tải file này về máy hoặc kết nối trực tiếp với Power BI.
```

Hình 3.18.Kết quả sau khi chạy lệnh trích xuất file tiktok_shop_sales_cleaned.csv

Kết quả đạt được: Tập dữ liệu sạch với quy mô đầy đủ các bản ghi đã được lưu thành công dưới tên tiktok_shop_sales_cleaned.csv, nhóm bắt đầu tải file về máy và sử dụng cho các phần sau.



	Order_ID	Order_Date	Customer_ID	Customer_Name	Customer_Age	Customer_Gender	Shipping_City	Device_Type	Product_ID	Product_Name	Main_Ca
2	TTS-2024-005112	01/01/2024 7:15	CUST-01053	Nguyễn Hoàng Huy	25	Nam	Hà Nội	Android	PROD-T02	Cáp sạc nhanh 3 đầu Type-C Lightning Baseus	Phụ Kiệ
3	TTS-2024-000128	01/01/2024 11:48	CUST-01140	Đỗ Thị Linh	32	Nữ	Hà Nội	iOS (iPhone)	PROD-F03	Chân váy chữ A xếp ly ngắn năng động	Thời Tra
4	TTS-2024-006987	01/01/2024 12:08	CUST-00967	Phạm Gia Bảo	40	Nam	Hà Nội	iOS (iPhone)	PROD-T05	Ốp lưng iPhone silicon trong suốt chống sốc	Phụ Kiệ
5	TTS-2024-007647	01/01/2024 12:35	CUST-00321	Hoàng Phương Thảo	23	Nữ	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-S01	Bánh tráng phơi sương muối nhuyễn bơ hành	Đồ Ăn V
6	TTS-2024-003627	01/01/2024 13:10	CUST-00686	Bùi Ngọc Hà	27	Nữ	Hà Nội	Android	PROD-F05	Váy hoa nhí tay bồng phong cách Hàn Quốc	Thời Tra
7	TTS-2024-002852	01/01/2024 13:59	CUST-00367	Vũ Anh Hải	30	Nam	Cần Thơ	iOS (iPhone)	PROD-H04	Hộp đựng cơm giữ nhiệt 3 tầng văn phòng	Đồ Gia D
8	TTS-2024-002311	01/01/2024 19:07	CUST-00221	Nguyễn Hồng Hà	19	Nữ	Đồng Nai	iOS (iPhone)	PROD-B01	Son kem lì Black Rouge Air Fit Velvet	Làm Đẹp
9	TTS-2024-001746	01/01/2024 19:12	CUST-00815	Hoàng Hữu Tùng	26	Nam	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-H03	Máy xông tinh dầu phun sương kiêm đèn ngủ LED	Đồ Gia D
10	TTS-2024-001631	01/01/2024 19:23	CUST-00929	Phạm Gia Khánh	23	Nam	Cần Thơ	Android	PROD-S01	Bánh tráng phơi sương muối nhuyễn bơ hành	Đồ Ăn V
11	TTS-2024-006420	01/01/2024 19:27	CUST-00947	Phan Đức Hải	20	Nam	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-H03	Máy xông tinh dầu phun sương kiêm đèn ngủ LED	Đồ Gia D
12	TTS-2024-005451	01/01/2024 19:57	CUST-01236	Phạm Tuấn Kiệt	28	Nam	Cần Thơ	iOS (iPhone)	PROD-H03	Máy xông tinh dầu phun sương kiêm đèn ngủ LED	Đồ Gia D
13	TTS-2024-003680	01/01/2024 20:39	CUST-00064	Hồ Thị Long	26	Khác	Hà Nội	iOS (iPhone)	PROD-B08	Mặt nạ giấy dưỡng ẩm Mediheal Tea Tree	Làm Đẹp
14	TTS-2024-007134	01/01/2024 20:47	CUST-00494	Phạm Mai Quỳnh	20	Nữ	Bình Dương	iOS (iPhone)	PROD-F04	Áo khoác Cardigan len dệt kim mỏng mùa thu	Thời Tra
15	TTS-2024-003993	01/01/2024 20:51	CUST-00504	Hồ Khánh Nhi	25	Nữ	Quảng Ninh	iOS (iPhone)	PROD-B01	Son kem lì Black Rouge Air Fit Velvet	Làm Đẹp
16	TTS-2024-006939	01/01/2024 22:21	CUST-00233	Bùi Diệu Hà	25	Nữ	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-T06	Micro thu âm không dây cài áo cho điện thoại K8	Phụ Kiệ
17	TTS-2024-007090	01/01/2024 22:47	CUST-00119	Nguyễn Thu Huyền	18	Nữ	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-F06	Túi xách đeo chéo nữ mini da PU trơn	Thời Tra
18	TTS-2024-001206	01/01/2024 23:01	CUST-01342	Dương Hà Linh	28	Nữ	Tp. Hồ Chí Minh	Android	PROD-H02	Bàn chải điện sóng âm Lock&Lock chống nước IPX7	Đồ Gia D
19	TTS-2024-008214	01/01/2024 23:11	CUST-01341	Đặng Quang Phong	19	Nam	Tp. Hồ Chí Minh	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay TikTok	Phụ Kiệ
20	TTS-2024-000263	01/01/2024 23:40	CUST-00450	Dương Anh Huy	22	Nam	Hà Nội	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay TikTok	Phụ Kiệ
21	TTS-2024-001889	02/01/2024 1:23	CUST-00686	Bùi Ngọc Hà	27	Nữ	Hà Nội	Android	PROD-F05	Váy hoa nhí tay bồng phong cách Hàn Quốc	Thời Tra
22	TTS-2024-002695	02/01/2024 4:44	CUST-00199	Huỳnh Ngọc Thảo	29	Nữ	Tp. Hồ Chí Minh	Android	PROD-T06	Micro thu âm không dây cài áo cho điện thoại K8	Phụ Kiệ
23	TTS-2024-003633	02/01/2024 9:07	CUST-00166	Hồ Ngọc Nhi	19	Nữ	Hà Nội	iOS (iPhone)	PROD-S04	Mực xé hấp nước dừa béo thơm hũ 300g	Đồ Ăn V

Hình 3.19. Kết quả file tiktok_shop_sales_cleaned.csv

CHƯƠNG 4: MÔ HÌNH HÓA VÀ TRỰC QUAN HÓA DỮ LIỆU (POWER BI DASHBOARD)

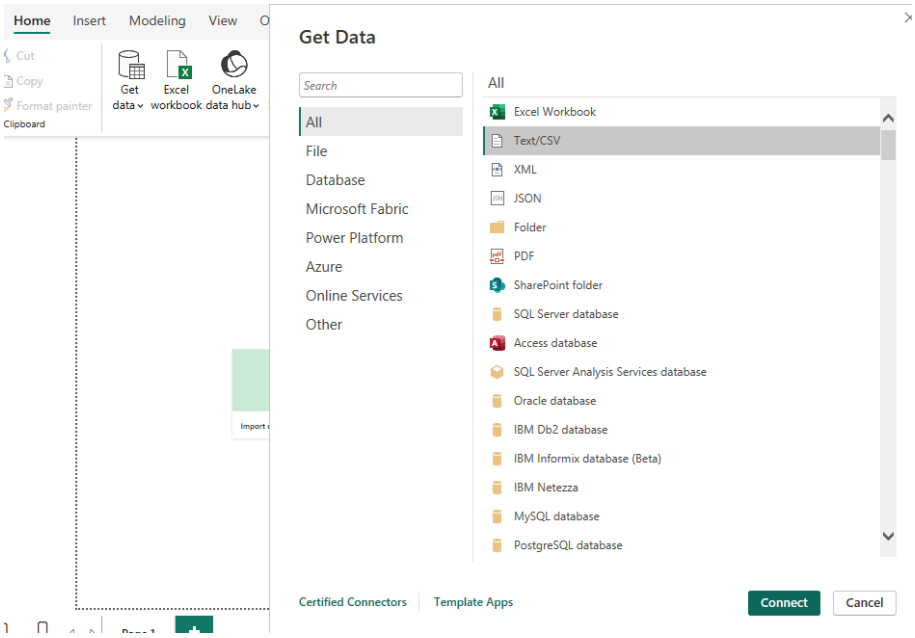
4.1. NẠP VÀ BIẾN ĐỔI DỮ LIỆU TRÊN POWER QUERY EDITOR

4.1.1. Mục đích và Quy trình trích xuất dữ liệu (Extract & Load)

Mục tiêu: Đưa tệp dữ liệu sạch đã được xử lý từ giai đoạn trước (tiktok_shop_sales_cleaned.csv) vào phần mềm Power BI Desktop, tiến hành kiểm tra chất lượng sơ bộ và chuyển hóa vào môi trường Power Query Editor để chuẩn bị cho việc xây dựng mô hình dữ liệu đa chiều.

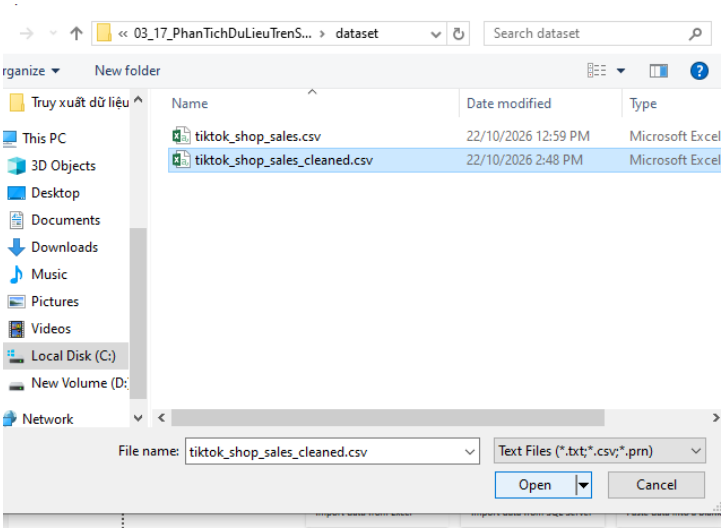
Các bước thực hiện:

1. Khởi động ứng dụng Power BI Desktop, tại thẻ giao diện chính **Home**, nhóm chọn **Get Data → Text/CSV**.



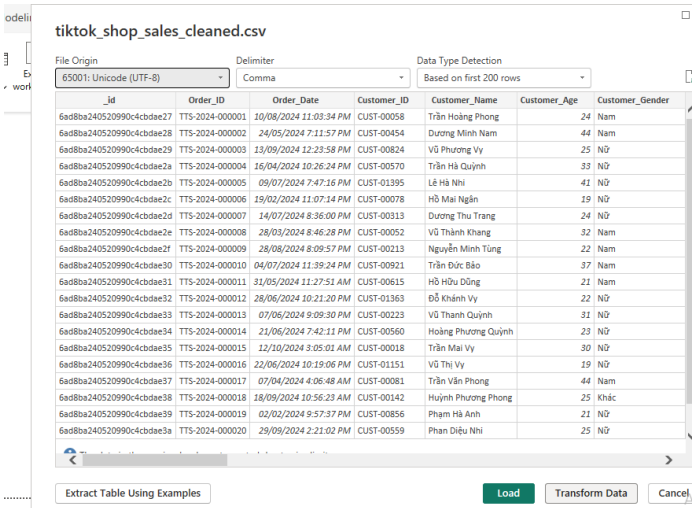
Hình 4.1.Thao tác kết nối nguồn dữ liệu CSV (Get Data) trong Power BI

2. Trỏ đường dẫn tới tệp dữ liệu sạch tiktok_shop_sales_cleaned.csv.



Hình 4.2. Thao tác chọn file 'tiktok_shop_sales_cleaned.csv' để nhập vào Power BI

3. Trong bảng cửa sổ xem trước (Preview) hiển thị dữ liệu, nhóm không bấm nút Load ngay mà chọn Transform Data để mở giao diện Power Query Editor. Thao tác này cho phép nhóm kiểm soát sâu hơn các thuộc tính dữ liệu trước khi nạp chính thức vào mô hình.

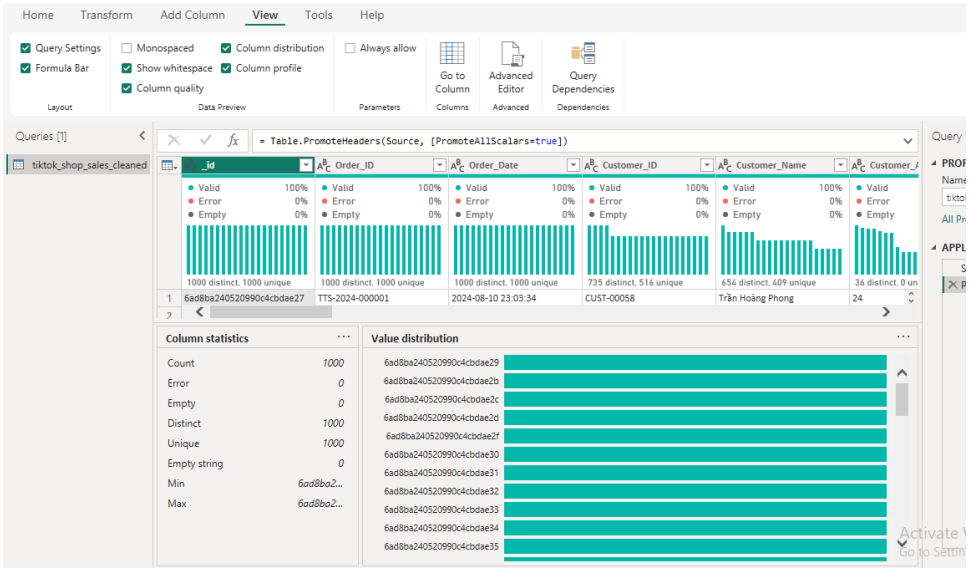


Hình 4.3. Giao diện sau khi nhấn nhập dữ liệu vào Power BI

4.1.2. Kiểm tra tính toàn vẹn của dữ liệu (Data Profiling)

Để đảm bảo tập dữ liệu không phát sinh lỗi định dạng hoặc thiếu sót sau khi nhập, nhóm đã kích hoạt các công cụ kiểm tra chất lượng tích hợp sẵn trong Power Query Editor:

- **Thao tác:** Trên thanh menu, chọn thẻ **View** → Tích chọn đồng thời 3 ô: **Column Quality**, **Column Distribution**, và **Column Profile**.
- **Kết quả rà soát:**

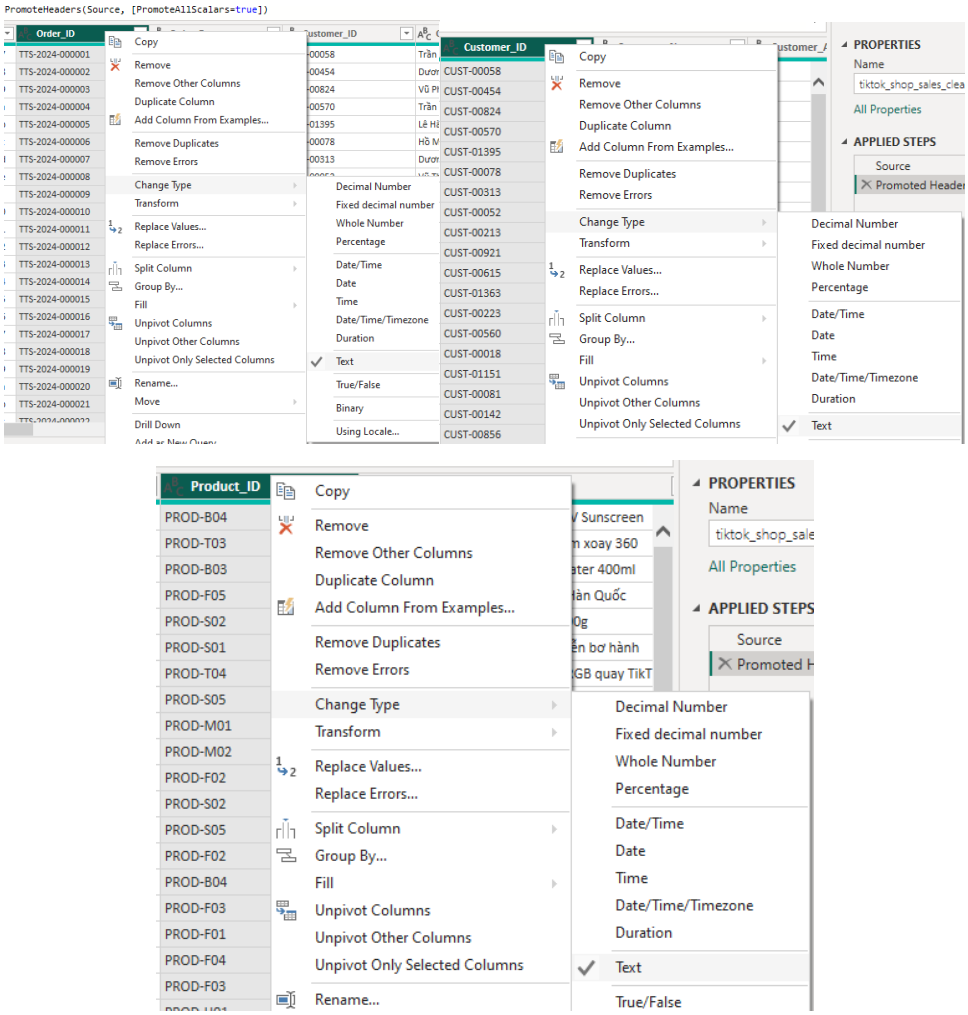


Hình 4.4.Kiểm tra tính toàn vẹn và chất lượng dữ liệu (Data Profiling) trên Power Query Editor

- Các cột dữ liệu đạt tỷ lệ 100% Valid (hợp lệ), 0% Error (không có lỗi) và 0% Empty (không còn ô trống).
- Điều này chứng minh quá trình làm sạch dữ liệu ở Chương 3 đã được thực hiện triệt để, tạo nền tảng vững chắc để chuyển sang giai đoạn mô hình hóa.

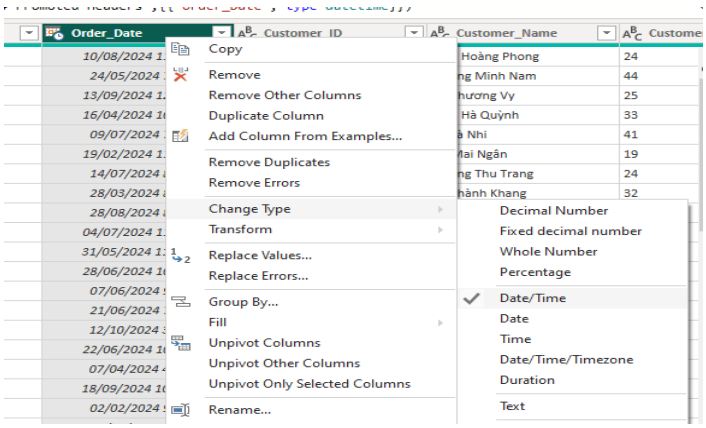
4.1.3. Chuẩn hóa định dạng kiểu dữ liệu (Data Types)

Trước khi tiến hành phân tách bảng, nhóm kiểm tra và thiết lập lại định dạng chuẩn cho từng nhóm thuộc tính:



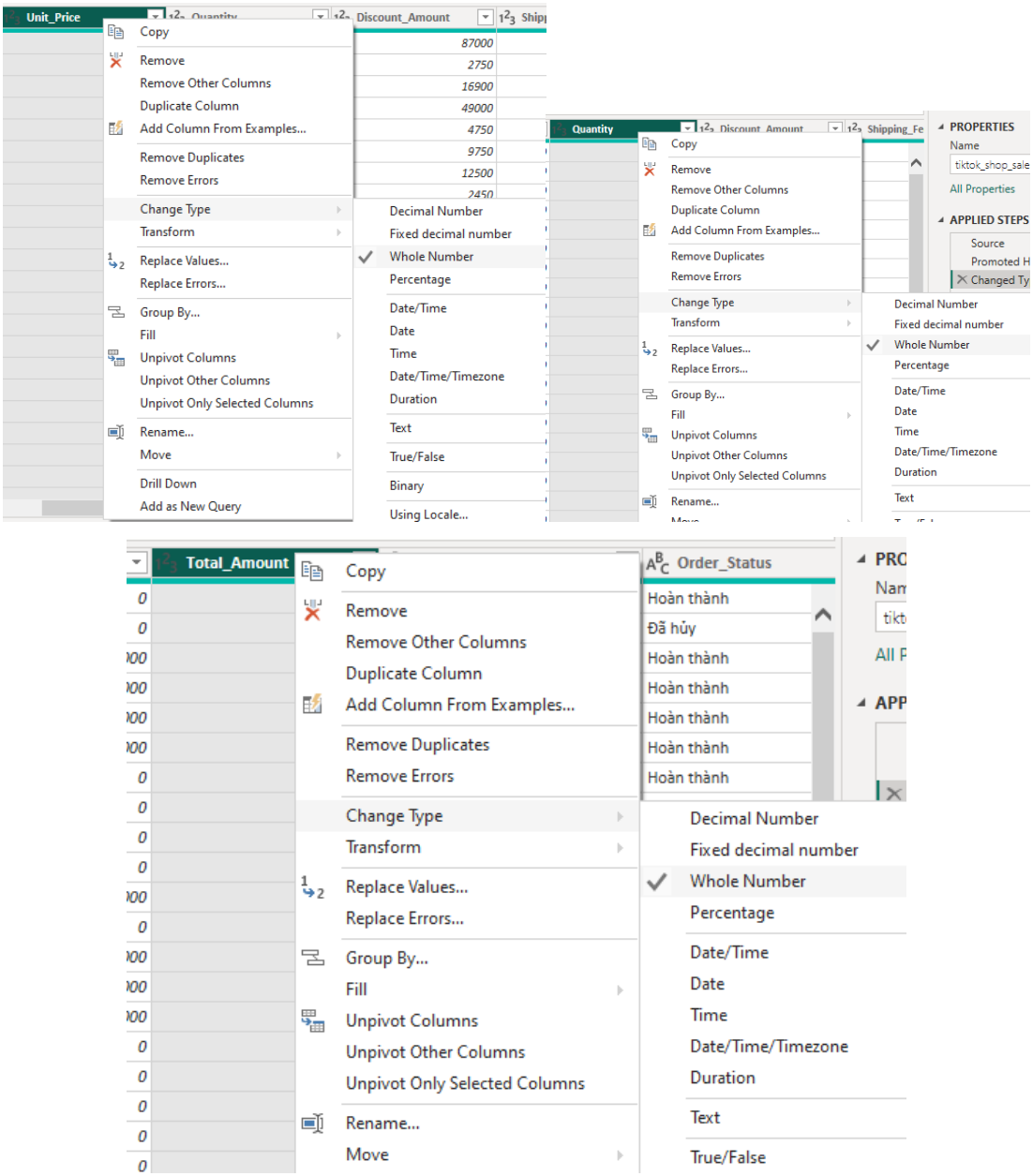
Hình 4.5.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột định danh trong Power Query

- **Cột định danh (IDs):** Các trường như Order_ID, Customer_ID, Product_ID được định dạng chuẩn ở dạng chuỗi văn bản (**Text** - biểu tượng ABC), đóng vai trò là các khóa liên kết giữa các bảng.



Hình 4.6.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột thời gian

- **Cột thời gian:** Trường Order_Date được gán định dạng ngày tháng (**Date** - biểu tượng tờ lịch), hỗ trợ cho các hàm phân tích thời gian (Time Intelligence) sau này.



Hình 4.7.Thao tác chuẩn hóa và thay đổi kiểu dữ liệu (Change Type) cho các cột về định lượng và tài chính

- **Cột định lượng & tài chính:** Các trường như Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Total_Amount được chuyển sang định dạng số nguyên hoặc số thực (Whole Number / Decimal Number), phục vụ trực tiếp cho các phép tính tổng doanh thu và thống kê kinh doanh.

4.2. XÂY DỰNG MÔ HÌNH DỮ LIỆU ĐA CHIỀU (DATA MODELING - STAR SCHEMA)

4.2.1. Phương pháp luận và Kiến trúc Lược đồ hình sao (Star Schema)

- **Mục tiêu:** Chuyển hóa bảng dữ liệu phẳng ban đầu (tiktok_shop_sales_cleaned.csv) thành một mô hình dữ liệu quan hệ đa chiều chuẩn công nghiệp. Việc này giúp tối ưu

hóa hiệu năng truy vấn của Power BI, giảm thiểu sự dư thừa dữ liệu và hỗ trợ đắc lực cho việc tổng hợp các chỉ số kinh doanh.

- **Kiến trúc mô hình:** Nhóm áp dụng mô hình **Lược đồ hình sao (Star Schema)**, bao gồm:

  - **Bảng sự kiện trung tâm (**Fact_Orders**):** Đặt ở vị trí cốt lõi của mô hình, lưu trữ các khóa ngoại liên kết cùng các chỉ số định lượng phục vụ tính toán tài chính (như đơn giá, số lượng, chiết khấu, phí vận chuyển và tổng doanh thu).
  - **Các bảng chiều vệ tinh (**Dim_Customer**,** Dim_Product**,** Dim_Date**):** Vây xung quanh bảng trung tâm, chứa các thông tin mô tả chi tiết về khách hàng, sản phẩm và mốc thời gian phát sinh giao dịch.

Bảng 4.1. Tổng hợp các trường thông tin trong mô hình dữ liệu

Bảng	**Các trường**
Fact_Orders	Order_ID, Customer_ID, Product_ID, Order_Date, Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Total_Amount, Order_Status, Traffic_Source, Payment_Method, Creator_Name, Video_Views, Is_Anomaly
Dim_Customer	Customer_ID, Customer_Name, Customer_Age, Customer_Gender, Shipping_City, Device_Type
Dim_Product	Product_ID, Product_Name, Main_Category, Sub_Category, Product_Rating, Review_Count, Shop_ID, Shop_Name
Dim_Date	Date, Year, Quarter, Month, Day, Day of Week

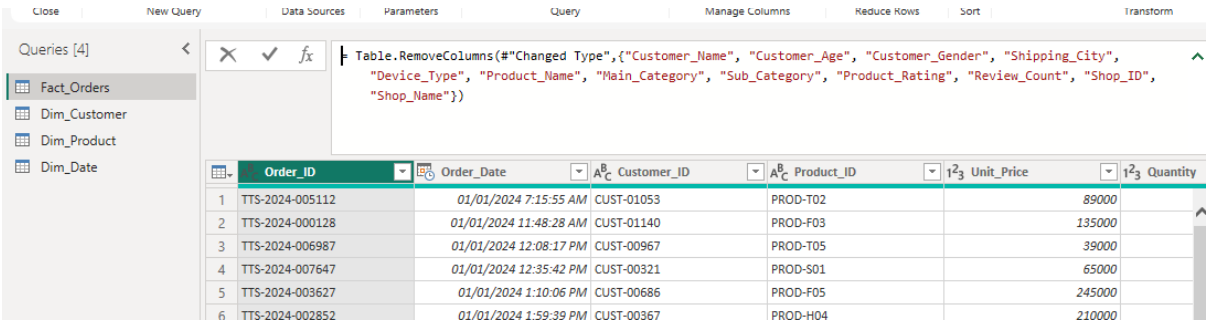
Nguồn: Nhóm tự tổng hợp

4.2.2. Chi tiết các bước phân tách bảng trên Power Query Editor

Để xây dựng cấu trúc mô hình trên, nhóm đã sử dụng tính năng Dupplicate từ bảng nguồn sạch để nhân bản và lọc trường độc lập cho từng bảng:

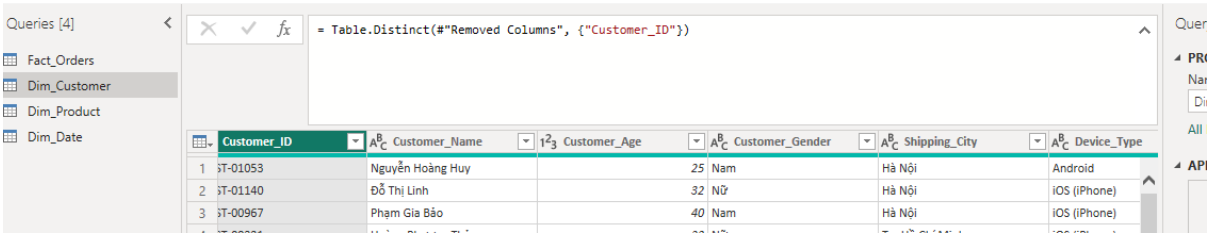
- **Xây dựng bảng sự kiện trung tâm (**Fact_Orders**):**

- Giữ lại các trường khóa liên kết cốt lõi gồm Order_ID, Customer_ID, Product_ID, Order_Date.
- Giữ lại các cột chỉ số đo lường tài chính phục vụ kinh doanh: Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Total_Amount, Order_Status, Traffic_Source, Payment_Method, Creator_Name, Video_Views, Is_Anomaly
- Loại bỏ toàn bộ các thuộc tính mô tả chi tiết rườm rà của khách hàng và sản phẩm để tối ưu dung lượng bộ nhớ.



Hình 4.8.Giao diện xây dựng bảng sự kiện trung tâm (Fact_Orders)

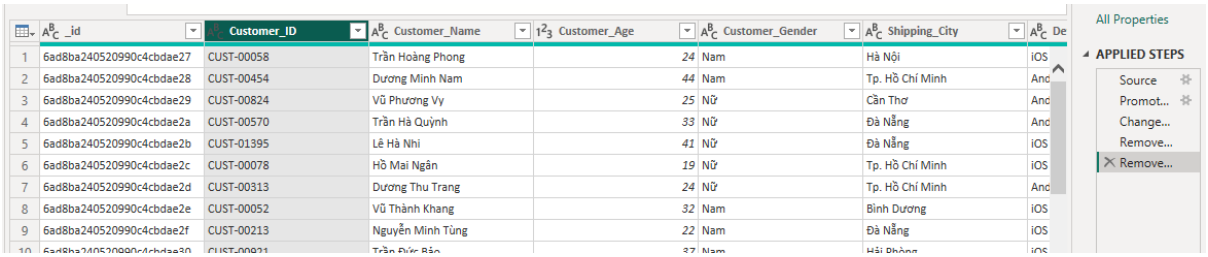
- **Xây dựng bảng chiều Khách hàng (Dim_Customer)**
  - Tham chiếu từ bảng gốc, giữ lại các thuộc tính đặc trưng của người mua: Customer_ID, Customer_Name, Customer_Age, Customer_Gender, Shipping_City, Device_Type.
  - **Xử lý khóa chính:** Thực hiện lệnh Remove Duplicates trên cột Customer_ID để đảm bảo mỗi khách hàng chỉ xuất hiện duy nhất 1 lần trong bảng chiều, tránh lỗi quan hệ dữ liệu.



Hình 4.9.Giao diện xây dựng bảng chiều khách hàng (Dim_Customer) trên Power Query Editor

- **Xây dựng bảng chiều Sản phẩm (Dim_Product):**
  - Tham chiếu từ bảng gốc, giữ lại các thuộc tính sản phẩm: Product_ID, Product_Name, Main_Category, Sub_Category, Product_Rating, Review_Count, Shop_ID, Shop_Name

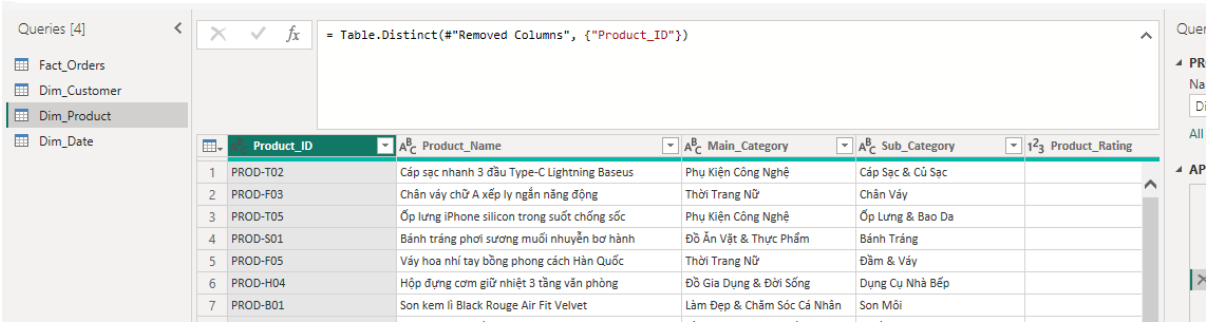
- Thực hiện lệnh Remove Duplicates trên cột Product_ID để thiết lập khóa chính sản phẩm độc lập, không lặp lại.



	_id	Customer_ID	Customer_Name	Customer_Age	Customer_Gender	Shipping_City	De
1	6ad8ba240520990c4cbdae27	CUST-00058	Trần Hoàng Phong	24	Nam	Hà Nội	iOS
2	6ad8ba240520990c4cbdae28	CUST-00454	Dương Minh Nam	44	Nam	Tp. Hồ Chí Minh	And
3	6ad8ba240520990c4cbdae29	CUST-00824	Vũ Phương Vy	25	Nữ	Cần Thơ	And
4	6ad8ba240520990c4cbdae2a	CUST-00570	Trần Hà Quỳnh	33	Nữ	Đà Nẵng	And
5	6ad8ba240520990c4cbdae2b	CUST-01395	Lê Hà Nhi	41	Nữ	Đà Nẵng	iOS
6	6ad8ba240520990c4cbdae2c	CUST-00078	Hồ Mai Ngân	19	Nữ	Tp. Hồ Chí Minh	iOS
7	6ad8ba240520990c4cbdae2d	CUST-00313	Dương Thu Trang	24	Nữ	Tp. Hồ Chí Minh	And
8	6ad8ba240520990c4cbdae2e	CUST-00052	Vũ Thành Khang	32	Nam	Bình Dương	iOS
9	6ad8ba240520990c4cbdae2f	CUST-00213	Nguyễn Minh Tùng	22	Nam	Đà Nẵng	iOS

All Properties
APPLIED STEPS
Source
Promot...
Change...
Remove...
Remove...

Hình 4.10.Thao tác remove duplicates



Queries [4]
Fact_Orders
Dim_Customer
Dim_Product
Dim_Date

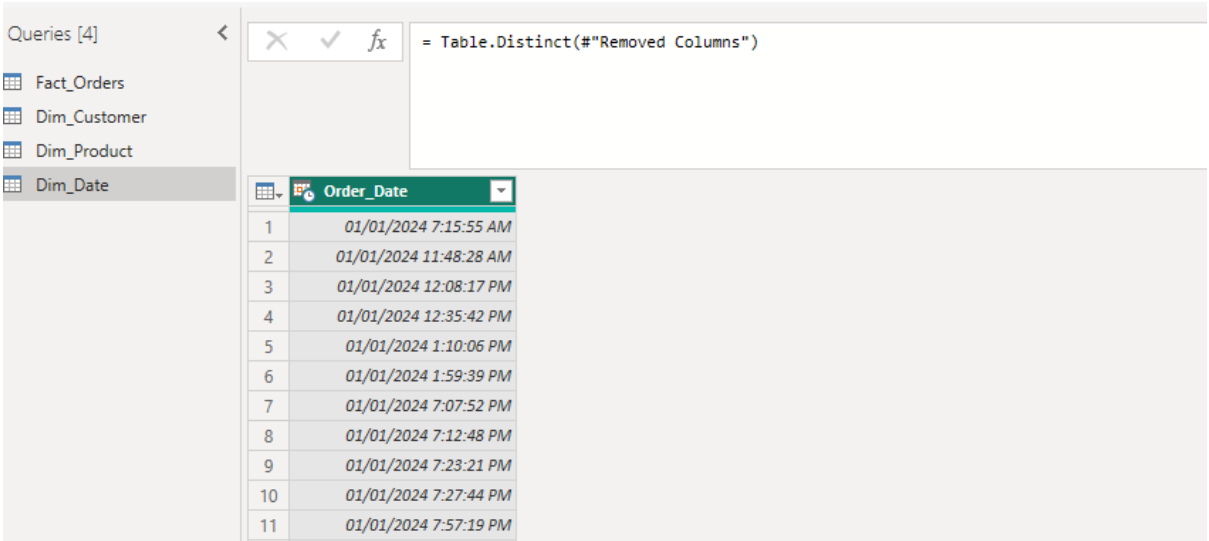
```
= Table.Distinct(#"Removed Columns", {"Product_ID"})
```

	Product_ID	Product_Name	Main_Category	Sub_Category	Product_Rating
1	PROD-T02	Cáp sạc nhanh 3 đầu Type-C Lightning Baseus	Phụ Kiện Công Nghệ	Cáp Sạc & Củ Sạc	
2	PROD-F03	Chân váy chữ A xếp ly ngắn năng động	Thời Trang Nữ	Chân Váy	
3	PROD-T05	Ốp lưng iPhone silicon trong suốt chống sốc	Phụ Kiện Công Nghệ	Ốp Lưng & Bao Da	
4	PROD-S01	Bánh tráng phơi sương muối nhuyễn bơ hành	Đồ Ăn Vặt & Thực Phẩm	Bánh Tráng	
5	PROD-F05	Váy hoa nhí tay bồng phong cách Hàn Quốc	Thời Trang Nữ	Đầm & Váy	
6	PROD-H04	Hộp đựng cơm giữ nhiệt 3 tầng văn phòng	Đồ Gia Dụng & Đời Sống	Dụng Cụ Nhà Bếp	
7	PROD-B01	Son kem lì Black Rouge Air Fit Velvet	Làm Đẹp & Chăm Sóc Cá Nhân	Son Môi	

Hình 4.11.Giao diện xây dựng bảng chiều sản phẩm (Dim_Product) trên Power Query Editor

- **Xây dựng bảng chiều Thời gian (Dim_Date):**

- Tham chiếu từ bảng gốc, trích xuất cột Order_Date, đổi tên thành Date và chạy Remove Duplicates.



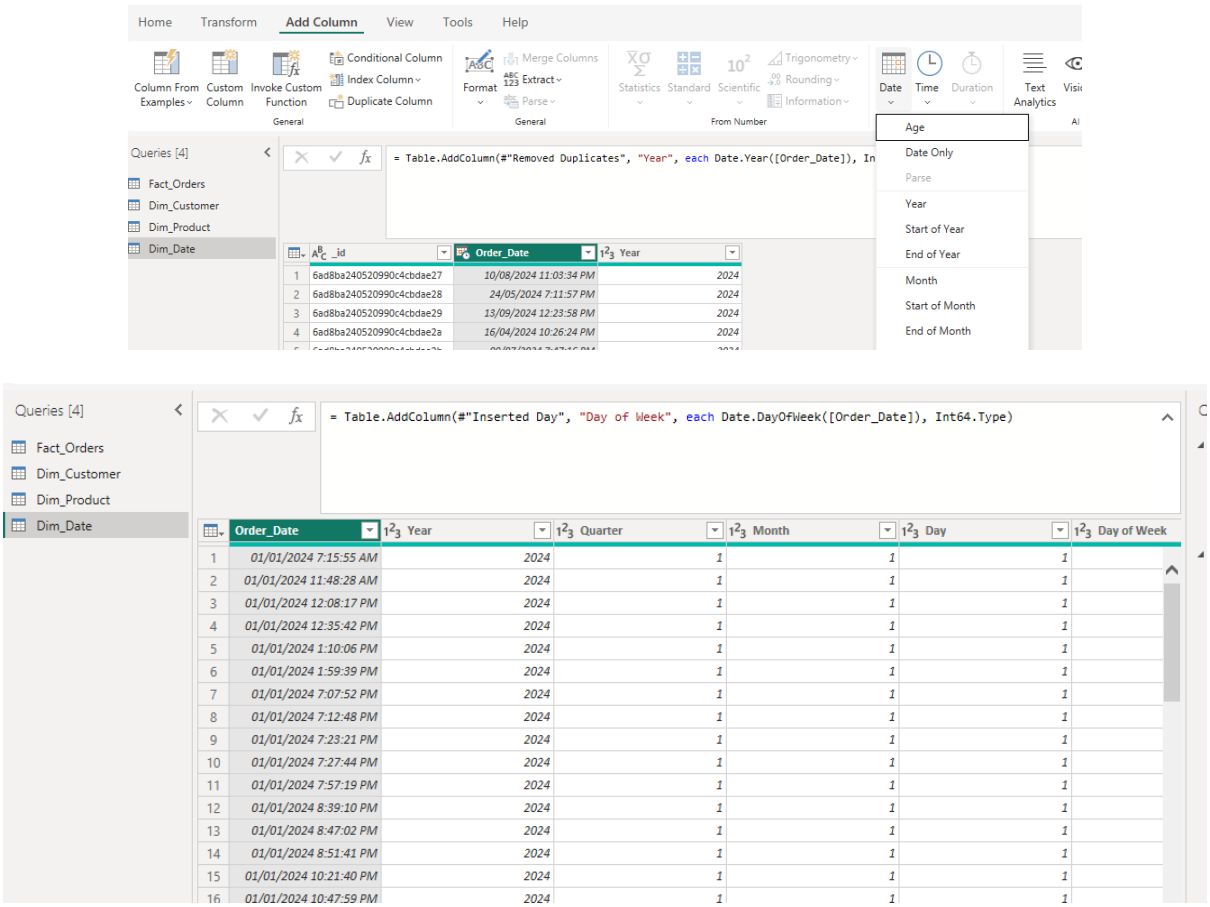
Queries [4]
Fact_Orders
Dim_Customer
Dim_Product
Dim_Date

```
= Table.Distinct(#"Removed Columns")
```

	Order_Date
1	01/01/2024 7:15:55 AM
2	01/01/2024 11:48:28 AM
3	01/01/2024 12:08:17 PM
4	01/01/2024 12:35:42 PM
5	01/01/2024 1:10:06 PM
6	01/01/2024 1:59:39 PM
7	01/01/2024 7:07:52 PM
8	01/01/2024 7:12:48 PM
9	01/01/2024 7:23:21 PM
10	01/01/2024 7:27:44 PM
11	01/01/2024 7:57:19 PM

Hình 4.12.Giao diện xây dựng bảng chiều thời gian (Dim_Date) trên Power Query Editor

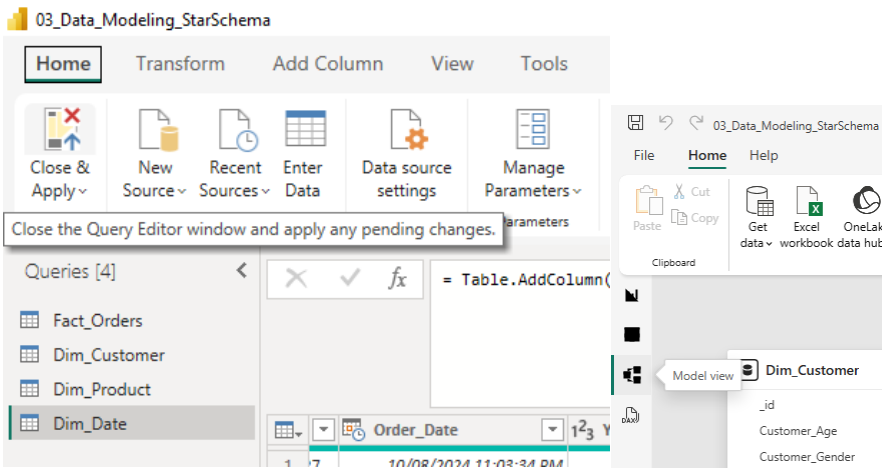
- Bổ sung các thuộc tính phân tích thời gian (Time Intelligence) ở thẻ *Add Column*, bao gồm: *Year (Năm), Quarter (Quý), Month (Tháng), Month Name (Tên tháng), Day of Week (Thứ trong tuần)* để hỗ trợ vẽ biểu đồ xu hướng.



Hình 4.13.Thao tác thêm các thuộc tính phân tích thời gian

4.2.3. Thiết lập mối quan hệ trong mô hình (Relationships - Model View)

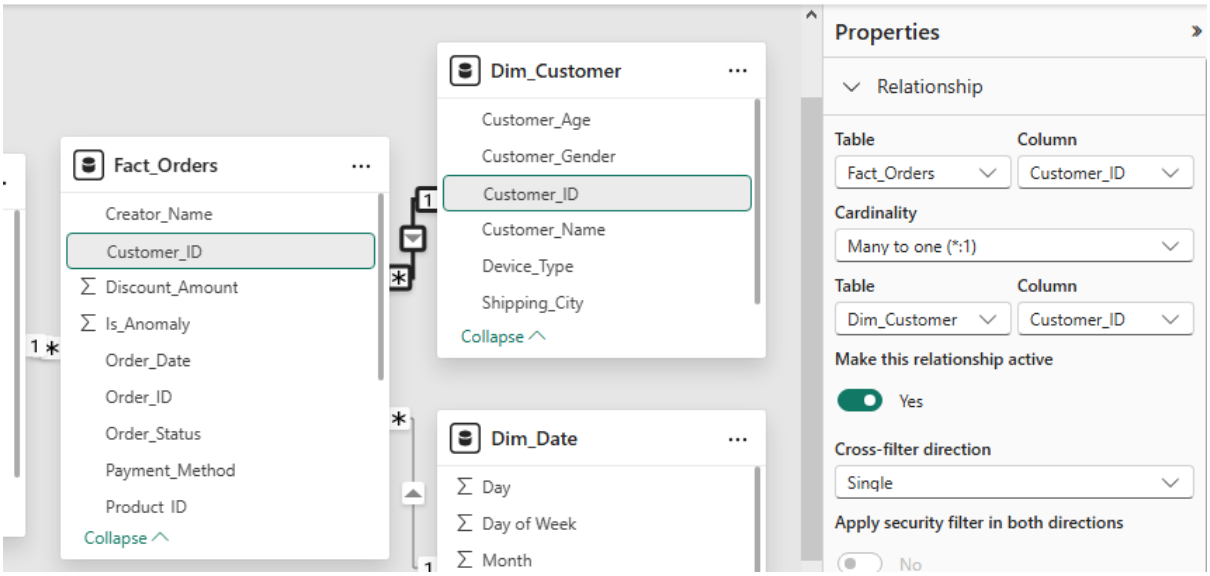
Sau khi hoàn tất quá trình nạp dữ liệu (Close & Apply), nhóm chuyển sang chế độ Model View để thiết lập liên kết quan hệ giữa các bảng:



Hình 4.14. Thao tác xây dựng mối quan hệ mô hình trong Model View

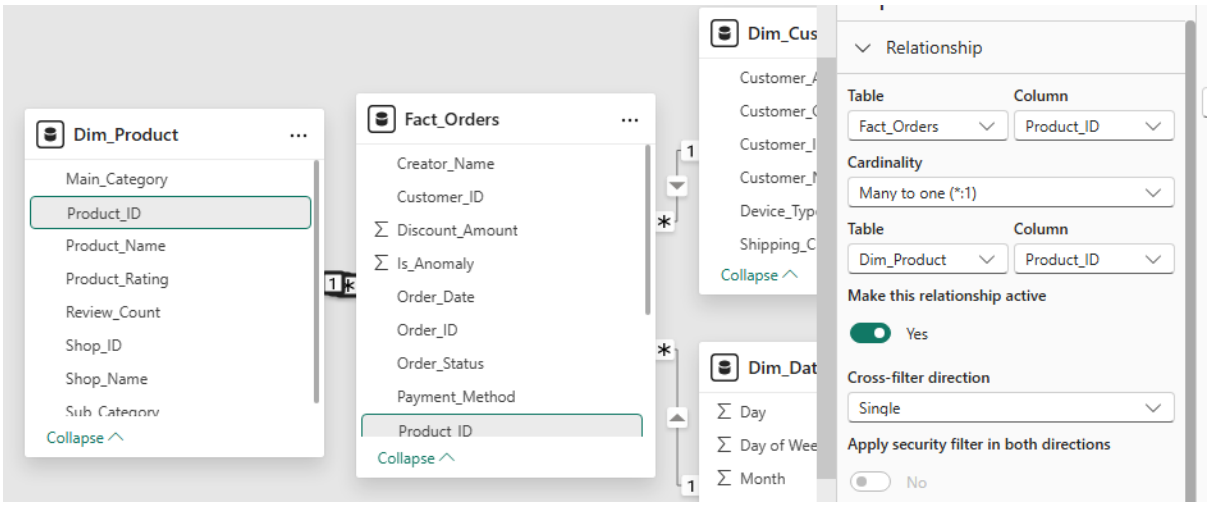
- **Thiết lập liên kết (1-to-Many):** Thực hiện kéo thả các trường khóa chung để tạo mối quan hệ một-nhiều (1:∗) theo đúng chiều từ bảng chiều sang bảng sự kiện:

  - Kéo Dim_Customer[Customer_ID] liên kết với Fact_Orders[Customer_ID].



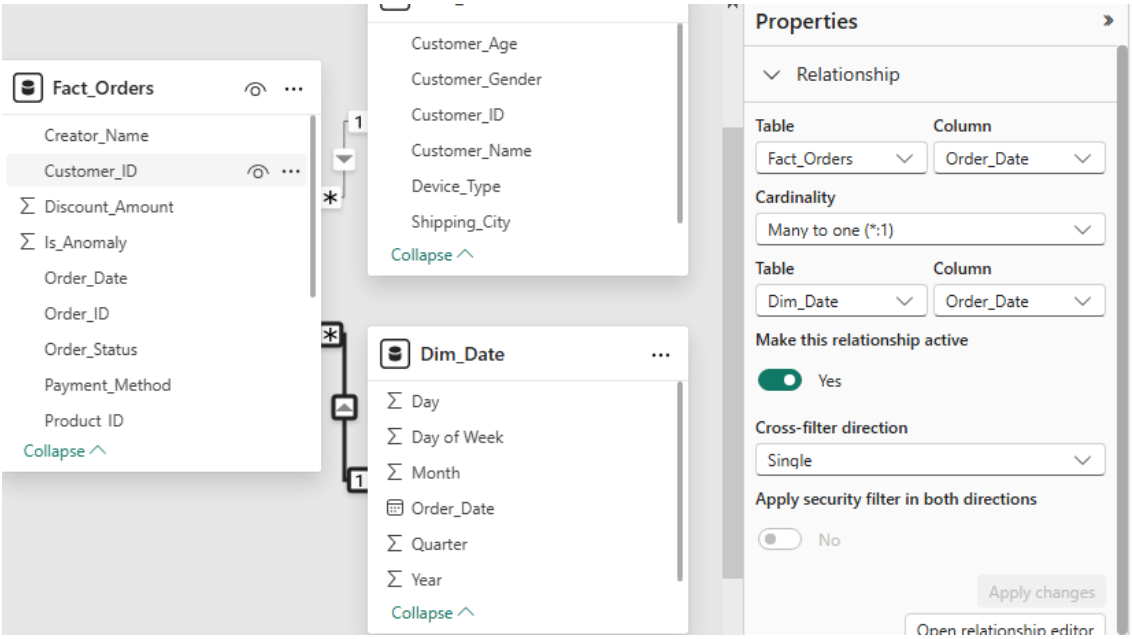
Hình 4.15. Thao tác liên kết bảng Dim_Customer với bảng Fact_Orders

- Kéo Dim_Product[Product_ID] liên kết với Fact_Orders[Product_ID].



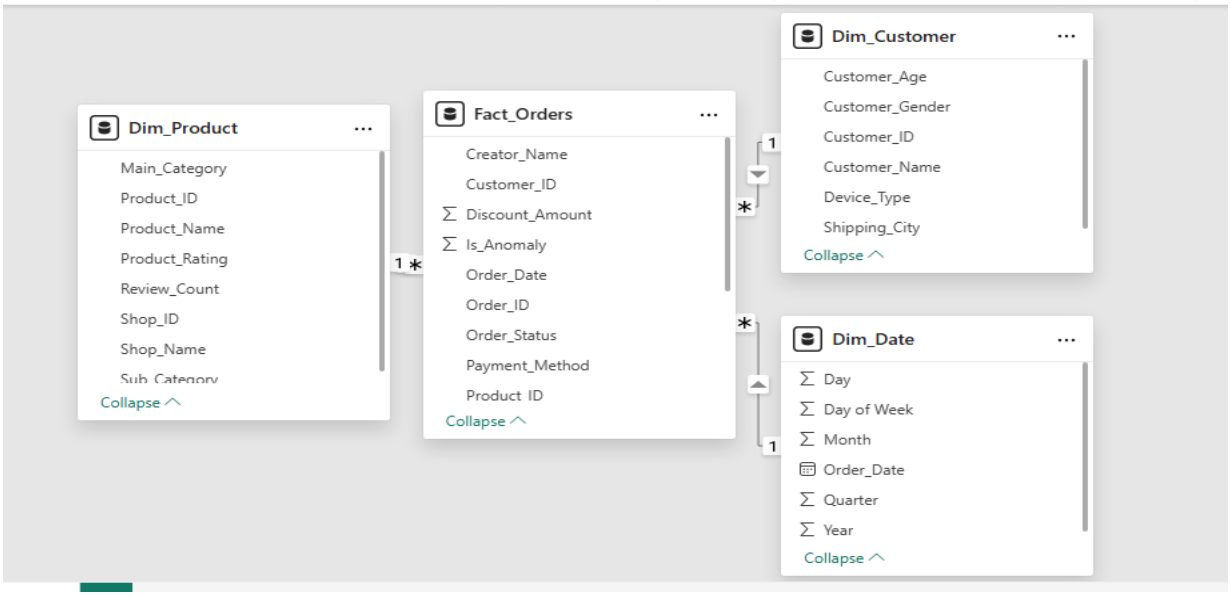
Hình 4.16.Thao tác liên kết bảng Dim_Product với bảng Fact_Orders

- Kéo Dim_Date[Date] liên kết với Fact_Orders[Order_Date].



Hình 4.17.Thao tác liên kết bảng Dim_Date với bảng Fact_Orders

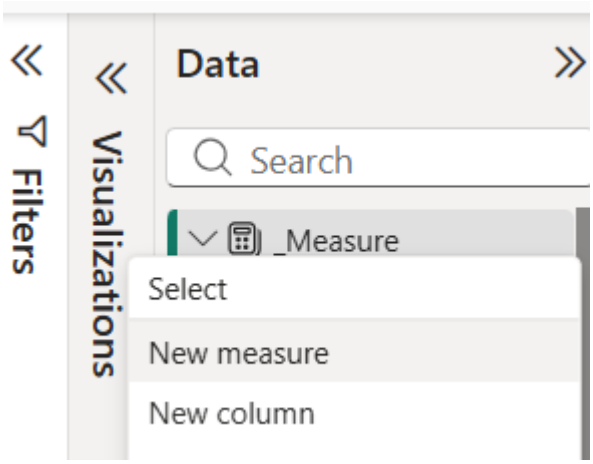
- **Cấu hình hướng lọc:** Đảm bảo thuộc tính *Cross-filter direction* ở chế độ một chiều (Single), giúp luồng dữ liệu lọc từ các bảng chiều tác động chính xác xuống bảng sự kiện, tối ưu hóa tốc độ xử lý của Dashboard trong Chương tiếp theo.
- **Sắp xếp bố cục:** Đặt bảng trung tâm Fact_Orders ở chính giữa, bao quanh là 3 bảng chiều Dim_Customer, Dim_Product, Dim_Date tạo thành hình thái Lược đồ hình sao trực quan và chuyên nghiệp.



Hình 4.18. Kết quả sau khi xây dựng mô hình Star Schema

4.3. XÂY DỰNG BỘ CÔNG THỨC DAX MEASURES CỐT LÕI

Tại giao diện chính của Power BI Desktop, trước tiên chọn Enter Data (Nhập dữ liệu) để tạo một bảng trống và đặt tên là _Measures. Sau đó, chuyển sang khu vực Data ở cạnh phải màn hình → Nhấp chuột phải vào bảng _Measures vừa tạo → Chọn New measure. Lúc này, thanh nhập công thức DAX sẽ xuất hiện ở phía trên để bạn bắt đầu viết mã lệnh.



Hình 4.19. Khởi tạo New measure

– Total Revenue (Tổng doanh thu):

- **Mục đích:** Thống kê tổng số tiền doanh thu bán hàng thu về trên toàn hệ thống.
- **Mã lệnh DAX:** Total Revenue = SUM(Fact_Orders[Total_Amount])
- **Giải thích:** Áp dụng hàm SUM để tính tổng toàn bộ giá trị ở cột Total_Amount. Cấu hình định dạng hiển thị kiểu Tiền tệ (Currency - VND) và bật dấu phân cách hàng nghìn.

```
Total Revenue = SUM(Fact_Orders[Total_Amount])
```

Hình 4.20. Nhập mã công thức DAX Measures của Total Revenue

– Total Orders (Tổng số đơn hàng):

- **Mục đích:** Đo lường chính xác lượng đơn hàng không trùng lặp được khởi tạo trong kỳ.
- **Mã lệnh DAX:** Total Orders = DISTINCTCOUNT(Fact_Orders[Order_ID]).
- **Giải thích:** Sử dụng hàm DISTINCTCOUNT trên trường Order_ID nhằm triệt tiêu các mã đơn bị lặp. Cấu hình định dạng dạng Số nguyên (Whole Number).

```
Total Orders = DISTINCTCOUNT(Fact_Orders[Order_ID])
```

Hình 4.21. Nhập mã công thức DAX Measures của Total Orders

– AOV (Giá trị đơn hàng trung bình):

- **Mục đích:** Đánh giá mức doanh thu bình quân tạo ra trên mỗi đơn hàng phát sinh.
- **Mã lệnh DAX:** AOV = DIVIDE([Total Revenue], [Total Orders], 0).
- **Giải thích:** Thực hiện phép chia Total Revenue cho Total Orders bằng hàm DIVIDE, gán giá trị mặc định là 0 để tránh lỗi phép chia cho 0. Cấu hình định dạng kiểu Tiền tệ (Currency - VND).

```
AOV = DIVIDE([Total Revenue], [Total Orders], 0)
```

Hình 4.22. Nhập mã công thức DAX Measures của AOV

– **Total Quantity (Tổng sản lượng bán):**

- **Mục đích:** Tổng hợp tổng số lượng sản phẩm được giao dịch thành công.
- **Mã lệnh DAX:** Total Quantity = SUM(Fact_Orders[Quantity]).
- **Giải thích:** Dùng hàm SUM tính tổng cho toàn bộ dữ liệu tại cột Quantity. Cấu hình định dạng dạng Số nguyên (Whole Number).

```
Total Quantity = SUM(Fact_Orders[Quantity])
```

Hình 4.23. Nhập mã công thức DAX Measures của Total Quantity

– **Total Review Count (Tổng số lượt đánh giá):**

- **Mục đích:** Tổng hợp tổng số lượt đánh giá (phản hồi) của người mua hàng đối với các sản phẩm trên sàn TikTok Shop.
- **Mã lệnh DAX:** Total Review Count = SUM(Dim_Product[Review_Count])
- **Giải thích:** Sử dụng hàm SUM tính tổng cho toàn bộ dữ liệu tại cột Review_Count. Cấu hình định dạng dạng **Số nguyên (Whole Number)** kèm dấu phân cách hàng nghìn.

```
Total Review Count = SUM(Dim_Product[Review_Count])
```

Hình 4.24. Nhập mã công thức DAX Measures của Total Review Count

– **Average Product Rating (Đánh giá trung bình):**

- **Mục đích:** Đo lường điểm đánh giá (số sao) trung bình của toàn bộ các sản phẩm.
- **Mã lệnh DAX:** Average Product Rating = AVERAGE(Dim_Product[Product_Rating])
- **Giải thích:** Hàm AVERAGE tính trung bình cộng điểm số ở cột Product_Rating. Cấu hình định dạng thập phân (1 chữ số).

```
Average Product Rating = AVERAGE(Dim_Product
[Product_Rating])
```

Hình 4.25. Nhập mã công thức DAX Measures của Average Product Rating

– **Total Video Views (Tổng lượt xem video/livestream):**

- **Mục đích:** Tổng hợp số lượng lượt xem từ các nội dung mang lại đơn hàng.
- **Mã lệnh DAX:** Total Video Views = SUM(Fact_Orders[Video_Views])
- **Giải thích:** Áp dụng SUM trên cột Video_Views. Cấu hình định dạng số nguyên có phân cách hàng nghìn.

```
Total Video Views = SUM(Fact_Orders[Video_Views])
```

Hình 4.26. Nhập mã công thức DAX Measures của Total Video Views

– **Total Customers (Tổng số khách hàng):**

- **Mục đích:** Đếm số lượng khách hàng thực tế đã thực hiện giao dịch (loại bỏ trường hợp một khách mua nhiều lần).
- **Mã lệnh DAX:**

  Total Customers = DISTINCTCOUNT(Dim_Customer[Customer_ID]).
- **Giải thích:** Áp dụng DISTINCTCOUNT lên cột Customer_ID của bảng Dim_Customer. Định dạng Whole Number.

```
Total Customers = DISTINCTCOUNT(Dim_Customer
[Customer_ID])
```

Hình 4.27. Nhập mã công thức DAX Measures của Total Customers

– **Total Discount Amount (Tổng chi phí chiết khấu):**

- **Mục đích:** Xác định tổng giá trị tiền chiết khấu (Voucher/Coupon) mà hệ thống đã áp dụng cho các giao dịch.
- **Mã lệnh DAX:** Total Discount Amount = SUM(Fact_Orders[Discount_Amount]).
- **Giải thích:** Hàm SUM cộng toàn bộ giá trị tại trường Discount_Amount. Định dạng Currency (VND).

```
Total Discount Amount = SUM(Fact_Orders
[Discount_Amount])
```

Hình 4.28. Nhập mã công thức DAX Measures của Total Discount Amount

− Total Shipping Fee (Tổng chi phí vận chuyển):

- **Mục đích:** Tổng hợp tổng số tiền chi phí vận chuyển/giao hàng (Shipping Fee) phát sinh từ tất cả các đơn hàng trên hệ thống, giúp nhà quản trị kiểm soát ngân sách logistics và trợ giá vận chuyển.
- **Mã lệnh DAX:**

  Total Shipping Fee = SUM(Fact_Orders[Shipping_Fee])
- **Giải thích:** Sử dụng hàm SUM tính tổng toàn bộ các giá trị tại cột Shipping_Fee trong bảng dữ liệu Fact_Orders. Cấu hình định dạng kiểu **Tiền tệ (Currency - VND)** và bật dấu phân cách hàng nghìn.

```
Total Shipping Fee = SUM(Fact_Orders[Shipping_Fee])
```

Hình 4.29. Nhập mã công thức DAX Measures của Total Shipping Fee

− Cancel Rate (Tỷ lệ hủy đơn):

- **Mục đích:** Tính toán tỷ lệ phần trăm các đơn hàng bị hủy/hoàn trả so với tổng đơn hàng.
- **Mã lệnh DAX:**

```
Cancel Rate =
VAR CancelledOrders =
    CALCULATE(
        [Total Orders],
        Fact_Orders[Order_Status] = "Đã hủy"
    )
RETURN
    DIVIDE(CancelledOrders, [Total Orders], 0)
```

- **Giải thích:** Dùng CALCULATE kết hợp lọc Order_Status để đếm riêng các đơn hủy, sau đó dùng DIVIDE chia cho tổng số đơn hàng (Total Orders). Định dạng hiển thị Percentage (%).

```
Cancel Rate =
VAR CancelledOrders =
    CALCULATE(
        [Total Orders],
        Fact_Orders[Order_Status] == "Đã hủy"
    )
RETURN
    DIVIDE(CancelledOrders, [Total Orders], 0)
```

Hình 4.30. Nhập mã công thức DAX Measures của Cancel Rate

− Return Rate (Tỷ lệ hoàn hàng):

- **Mục đích:** Đánh giá tỷ lệ phần trăm các đơn hàng phát sinh bị trả hàng/hoàn tiền so với tổng số đơn hàng nhằm kiểm soát chất lượng dịch vụ và sản phẩm.
- **Mã lệnh DAX:**

```
Return Rate =
VAR ReturnedOrders =
  CALCULATE(
    [Total Orders],
    Fact_Orders[Order_Status] = "Hoàn trả / Khiếu nại"
  )
RETURN
  DIVIDE(ReturnedOrders, [Total Orders], 0)
```

- **Giải thích:** Sử dụng hàm CALCULATE kết hợp bộ lọc điều kiện Order_Status để đếm riêng các đơn có trạng thái "Trả hàng/Hoàn tiền", sau đó dùng hàm DIVIDE chia cho tổng số đơn hàng (Total Orders). Cấu hình định dạng hiển thị kiểu **Percentage (%)**.

```
Return Rate =
VAR ReturnedOrders =
    CALCULATE(
        [Total Orders],
        Fact_Orders[Order_Status] == "Hoàn trả / Khiếu nại"
    )
RETURN
    DIVIDE(ReturnedOrders, [Total Orders], 0)
```

Hình 4.31. Nhập mã công thức DAX Measures của Return Rate

– **Anomaly Rate (Tỷ lệ giao dịch bất thường):**

- **Mục đích:** Đánh giá phần trăm các đơn hàng bị gán cờ bất thường/gian lận (Is_Anomaly = 1).
- **Mã lệnh DAX:**

  Anomaly Rate =

  VAR AnomalyOrders = SUM(Fact_Orders[Is_Anomaly])

  RETURN

  DIVIDE(AnomalyOrders, [Total Orders], 0)

- **Giải thích:** Vì cột Is_Anomaly chứa giá trị 1 (bất thường) và 0 (bình thường), dùng hàm SUM sẽ đếm được số ca bất thường, rồi dùng DIVIDE chia tỷ lệ phần trăm so với tổng số đơn. Cấu hình Percentage (%).

```
Anomaly Rate =
VAR AnomalyOrders = SUM(Fact_Orders[Is_Anomaly])
RETURN
    DIVIDE(AnomalyOrders, [Total Orders], 0)
```

Hình 4.32. Nhập mã công thức DAX Measures của Anomaly Rate

4.4. TỔNG HỢP THỐNG KÊ SỐ LIỆU

Bảng 4.2. Thống kê Min–Max–Mean của các biến số liệu

Biến số liệu	Min	Max	Mean	Đơn vị
Unit Price – Đơn giá	28.000	435.000	161.391,65	VNĐ

Total Amount – Giá trị đơn hàng	19.600	1.755.000	228.190,19	VNĐ
Quantity – Số lượng sản phẩm/đơn	1	5	1,50	Sản phẩm
Discount Amount – Tiền chiết khấu	0	435.000	21.316,87	VNĐ
Shipping Fee – Phí vận chuyển	0	30.000	8.488,35	VNĐ
Video Views – Lượt xem ghi nhận/đơn	5.054	3.198.034	1.086.679,00	Lượt
Customer Age – Độ tuổi khách hàng	18	55	26,39	Tuổi
Product Rating – Điểm đánh giá	2	5	4,52	Điểm
Review Count – Lượt đánh giá/sản phẩm	4.100	34.000	13.271,21	Lượt
Is Anomaly – Cờ giao dịch bất thường	0	1	4,12%	Tỷ lệ bản ghi

Nguồn: Nhóm tự tổng hợp

Kết quả cho thấy giá trị đơn hàng dao động từ 19.600 đến 1.755.000 đồng, với mức trung bình khoảng 228.190 đồng. Số lượng sản phẩm mỗi đơn từ 1 đến 5 sản phẩm, trung bình khoảng 1,50 sản phẩm. Chiết khấu trung bình đạt 21.316,87 đồng/đơn, trong khi phí vận chuyển trung bình là 8.488,35 đồng/đơn.

Đối với dữ liệu sản phẩm, điểm đánh giá trung bình đạt 4,52/5, với mức thấp nhất là 2 và cao nhất là 5 điểm. Lượt đánh giá trung bình đạt khoảng 13.271 lượt/sản phẩm. Bên cạnh đó, độ tuổi khách hàng dao động từ 18 đến 55 tuổi, trung bình khoảng 26,39 tuổi tính theo các bản ghi đơn hàng. Các kết quả này giúp xác định phạm vi dữ liệu, phát hiện những giá trị có độ chênh lệch lớn và làm cơ sở lựa chọn cách thể hiện phù hợp trên Dashboard.

Bảng 4.3. Tổng hợp giá trị các chỉ số kinh doanh

Chỉ số	**Giá trị tổng thể**	**Đơn vị**
Total Revenue	1.939.616.600	VNĐ
Total Orders	8.500	Đơn hàng
AOV	228.190,19	VNĐ/đơn
Total Quantity	12.727	Sản phẩm
Total Review Count	437.950	Lượt
Average Product Rating	4,52	Điểm/5
Total Video Views	9.236.771.022	Lượt
Total Customers	1.398	Khách hàng
Total Discount Amount	181.193.400	VNĐ
Total Shipping Fee	72.151.000	VNĐ
Cancel Rate	6,14%	%
Return Rate	3,25%	%
Anomaly Rate	4,12%	%

Nguồn: Nhóm tự tổng hợp

Nhìn chung, hệ thống ghi nhận gần 1,94 tỷ đồng doanh thu, 8.500 đơn hàng và 12.727 sản phẩm được giao dịch. Chi phí chiết khấu đạt khoảng 181,19 triệu đồng, còn phí vận chuyển là 72,15 triệu đồng. Tỷ lệ hủy đơn đạt 6,14%, tỷ lệ hoàn hàng đạt 3,25%

và tỷ lệ giao dịch bất thường đạt 4,12%. Đây là những chỉ số quan trọng để đánh giá quy mô kinh doanh, hiệu quả sử dụng chi phí và rủi ro vận hành.

4.5. THIẾT KẾ BÁO CÁO QUẢN TRỊ KINH DOANH TIKTOK SHOP TƯƠNG TÁC (DASHBOARD DESIGN)

4.5.1 Dashboard Tổng quan Tổng quan Doanh thu & Quy mô Kinh doanh

Chủ đề: Giúp Ban Giám đốc & Trưởng phòng Kinh doanh theo dõi bức tranh tổng thể về Doanh thu, Đơn hàng, Kênh bán và Địa lý.

Bộ Thẻ KPI Banner (Top Banner)

- **Mục tiêu:** Đo lường các chỉ số kinh doanh cốt lõi của toàn sàn TikTok Shop tại khu vực nhận diện ưu tiên theo sơ đồ Gutenberg.
- **Vị trí đặt:** Top-Left đến Top-Right.
- **Loại biểu đồ (Visual Type):** Card Visual.
- **Các bước kéo thả dữ liệu:**
  - Chọn biểu đồ **Card** từ thẻ *Visualizations*.
  - Lần lượt tạo 4 thẻ riêng biệt và kéo các DAX Measures tương ứng từ bảng _Measures vào khung **Fields**:
    - **Card 1 (Tổng Doanh Thu):** Fields = _Measures[Total Revenue]
    - **Card 2 (Giá Trị Đơn Trung Bình - AOV):** Fields = _Measures[AOV]
    - **Card 3 (Tổng Đơn Hàng):** Fields = _Measures[Total Orders]
    - **Card 4 (Tổng Sản Lượng Bán):** Fields = _Measures[Total Quantity]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Nền Thẻ #FFFFFF, Viền Thẻ #E0E0E0 (Bo góc 20px, Width = 5px).
  - **Nhãn dữ liệu (Callout Value):** Màu chữ Callout Value chỉnh thành #009990 (Segoe UI, Bold, 26pt). Bật phân cách hàng nghìn cho số nguyên.
  - **Tiêu đề (Category Label / Title):** Đổi tên nhãn các thẻ lần lượt thành *"Tổng doanh thu (VNĐ)"*, *"Tổng đơn hàng"*, *"Giá trị đơn trung bình (VNĐ)"*, *"Tổng sản lượng bán"*. Màu chữ chỉnh thành #E66C37 (Tahoma, Bold, 20pt)

- **Hiệu ứng đặc biệt:** Đặt khoảng cách giữa các thẻ cách đều nhau (Distribute Horizontally), bật hiệu ứng đổ bóng nhẹ (Drop Shadow).



Hình 4.33. Bộ thẻ KPI Cốt lõi

Bộ Lọc Tương Tác (Slicers Header)

- **Mục tiêu:** Cho phép cấp quản lý khoanh vùng và lọc dữ liệu động theo Thời gian, Khu vực trên toàn bộ trang báo cáo.
- **Vị trí đặt:** Top-Right.
- **Loại biểu đồ (Visual Type):** Slicer.
- **Các bước kéo thả dữ liệu:**
  - Chọn biểu đồ **Slicer** từ thẻ *Visualizations*.
  - Kéo trường thông tin vào khung **Field**:
    - **Slicer 1 (Khu vực):** Field = Dim_Customer[Shipping_City]
    - **Slicer 2 (Khoảng thời gian):** Field = Dim_Date[Order_Date]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Nền bộ lọc #FFFFFF, Nút/Thanh trượt chọn active màu #009990.
  - **Kiểu hiển thị (Slicer Settings):** Slicer 1 chọn dạng Dropdown (Danh sách thả xuống); Slicer 2 chọn Between (Date Range Slider).
  - **Tiêu đề (Slicer Header / Title):** Đổi tiêu đề tương ứng thành *"Khoảng thời gian"*, *"Khu vực"*.
  - **Hiệu ứng đặc biệt:** Kiểm tra *Edit Interactions* đảm bảo Slicer tự động tác động lọc tới tất cả biểu đồ còn lại trên trang.



Hình 4.34. Bộ lọc Dữ liệu Slicer

Top 10 Tỉnh/Thành Phố Có Doanh Thu Cao Nhất

- **Mục tiêu:** Xác định các thị trường địa lý trọng điểm tiêu thụ hàng hóa tốt nhất để tập trung ngân sách tiếp thị.
- **Vị trí đặt biểu đồ:** Center-Left.
- **Loại biểu đồ (Visual Type):** Clustered Bar Chart (Biểu đồ thanh ngang).
- **Các bước kéo thả dữ liệu:**
  - **Trục Y (Y-Axis):** Dim_Customer[Shipping_City]
  - **Trục X (X-Axis):** _Measures[Total Revenue]
  - **Filters (Bộ lọc nâng cao):** Kéo _Measures[Total Revenue] vào thẻ **Top N Filter** → Chọn Top 10 theo Total Revenue.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Bars Color tô đồng nhất màu #009990 (Xanh Mòng Két).
  - **Nhãn dữ liệu (Data Labels):** Bật On, căn lề *Inside End*, định dạng font chữ màu xám #605E5C (Bold, 9pt).
  - **Tiêu đề (Title):** *"Top 10 tỉnh/thành Phố có doanh thu cao nhất"*.
  - **Hiệu ứng đặc biệt:** Tắt đường lưới phụ (Gridlines) ở trục X để tránh gây rối mắt (Noise Suppression).



Hình 4.35. Biểu đồ Top 10 Thành phố Dẫn Đầu Doanh Thu

Top 5 Sản Phẩm Có Doanh Thu Cao Nhất

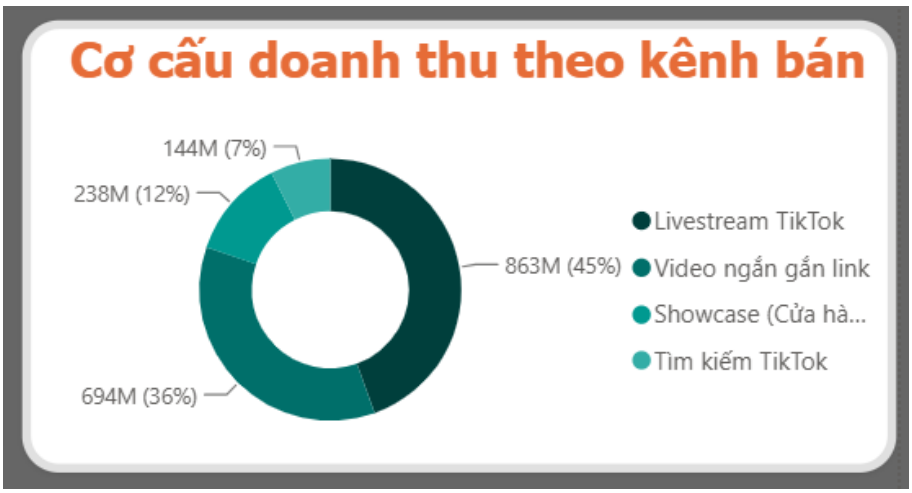
- **Mục tiêu:** Nhận diện nhanh 5 mã sản phẩm bán chạy nhất đóng góp nguồn thu cốt lõi cho sàn.
- **Vị trí đặt biểu đồ:** Center.
- **Loại biểu đồ (Visual Type):** Clustered Bar Chart (Biểu đồ thanh ngang).
- **Các bước kéo thả dữ liệu:**
  - **Trục Y (Y-Axis):** Dim_Product[Product_Name]
  - **Trục X (X-Axis):** _Measures[Total Revenue]
  - **Filters (Bộ lọc riêng):** Kéo _Measures[Total Revenue] vào thẻ **Top N Filter** → Chọn lấy Top 5 theo Total Revenue.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Bars Color sử dụng tông màu #009990.
  - **Nhãn dữ liệu (Data Labels):** Bật On, căn lề *Inside End*, màu chữ xám.
  - **Tiêu đề (Title):** Đổi tên thành *"Top 5 sản phẩm có doanh thu cao nhất"*.



Hình 4.36. Biểu đồ Top 5 Sản Phẩm Đạt Doanh Thu Cao Nhất

Cơ Cấu Doanh Thu Theo Kênh Bán

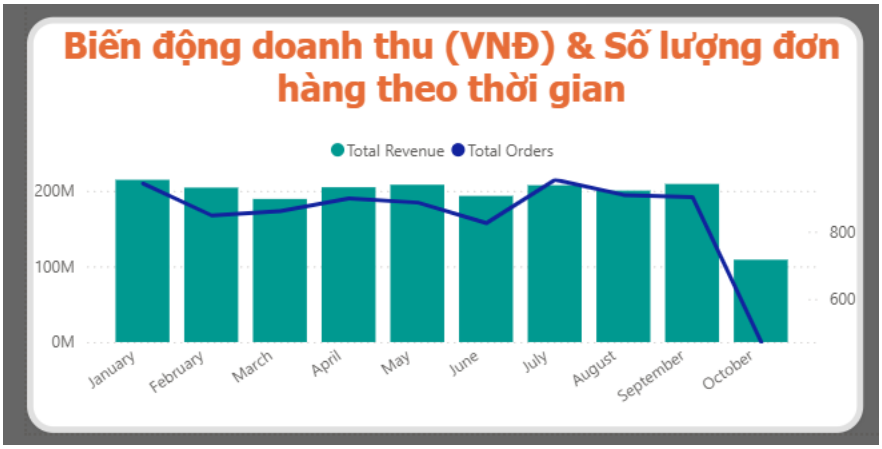
- **Mục tiêu:** Đánh giá đóng góp tỷ trọng doanh thu giữa các nguồn lưu lượng gồm Livestream, Showcase và Tìm kiếm TikTok.
- **Vị trí đặt biểu đồ:** Center-Right.
- **Loại biểu đồ (Visual Type):** Donut Chart (Biểu đồ bánh hình khuyên).
- **Các bước kéo thả dữ liệu:**
  - **Legend (Chú giải):** Fact_Orders[Traffic_Source]
  - **Values (Giá trị):** _Measures[Total Revenue]
  - **Filters:** Không cài đặt.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo (Slices Color):** Lát cắt Livestream TikTok màu #003F3C, Video ngắn gắn link màu #006F6A, Showcase màu #009990, Tìm kiếm TikTok màu #33ADA6.
  - **Nhãn dữ liệu (Detail Labels):** Bật On, vị trí *Outside*, hiển thị Data Value & Percent of Total (Giá trị Millions VNĐ và Tỷ lệ %).
  - **Tiêu đề (Title):** *"Cơ cấu doanh thu theo kênh bán"*.



Hình 4.37. Biểu đồ Cơ Cấu Doanh Thu Theo Kênh Bán

Biến Động Doanh Thu & Số Lượng Đơn Hàng Theo Thời Gian

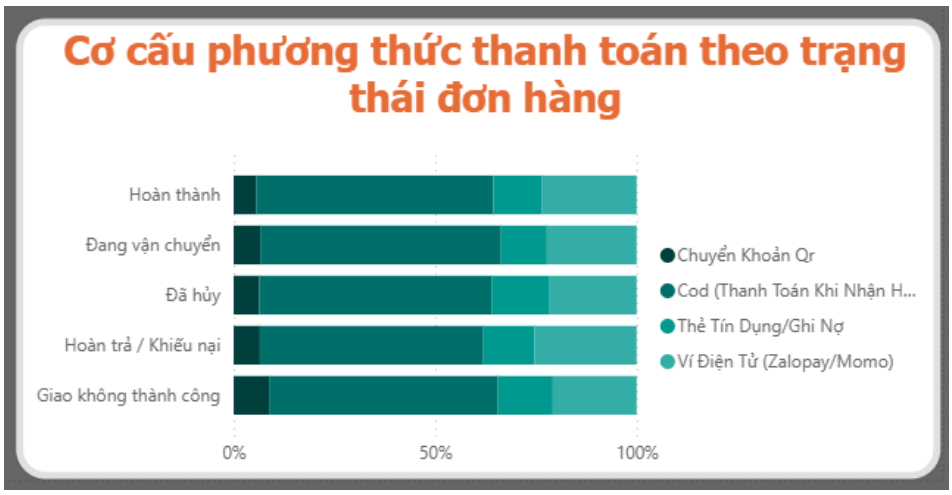
- **Mục tiêu:** Theo dõi biến động doanh thu và sản lượng đơn hàng qua chuỗi thời gian để nhận diện tính mùa vụ và các mốc tăng trưởng bứt phá.
- **Vị trí đặt biểu đồ:** Bottom-Left.
- **Loại biểu đồ (Visual Type):** Line and Stacked Column Chart (Biểu đồ kết hợp Cột & Đường).
- **Các bước kéo thả dữ liệu:**
  - **Trục X (Shared Axis):** Dim_Date[Month]
  - **Trục Y Cột (Column Values):** _Measures[Total Revenue]
  - **Trục Y Đường (Line Values):** _Measures[Total Orders]
  - **Filters:** Không cài đặt bộ lọc riêng (nhận tác động từ Slicer chung).
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Cột Doanh thu chỉnh màu #009990, Đường Đơn hàng màu #12239E (Độ dày nét Stroke Width: 3px).
  - **Nhãn dữ liệu (Data Labels):** Bật Off cho cả Cột và Đường.
  - **Tiêu đề (Title):** *"Biến động doanh thu (VNĐ) & Số lượng đơn hàng theo thời gian".*



Hình 4.38. Biểu đồ Biến Động Doanh Thu & Số Lượng Đơn Hàng Theo Thời Gian

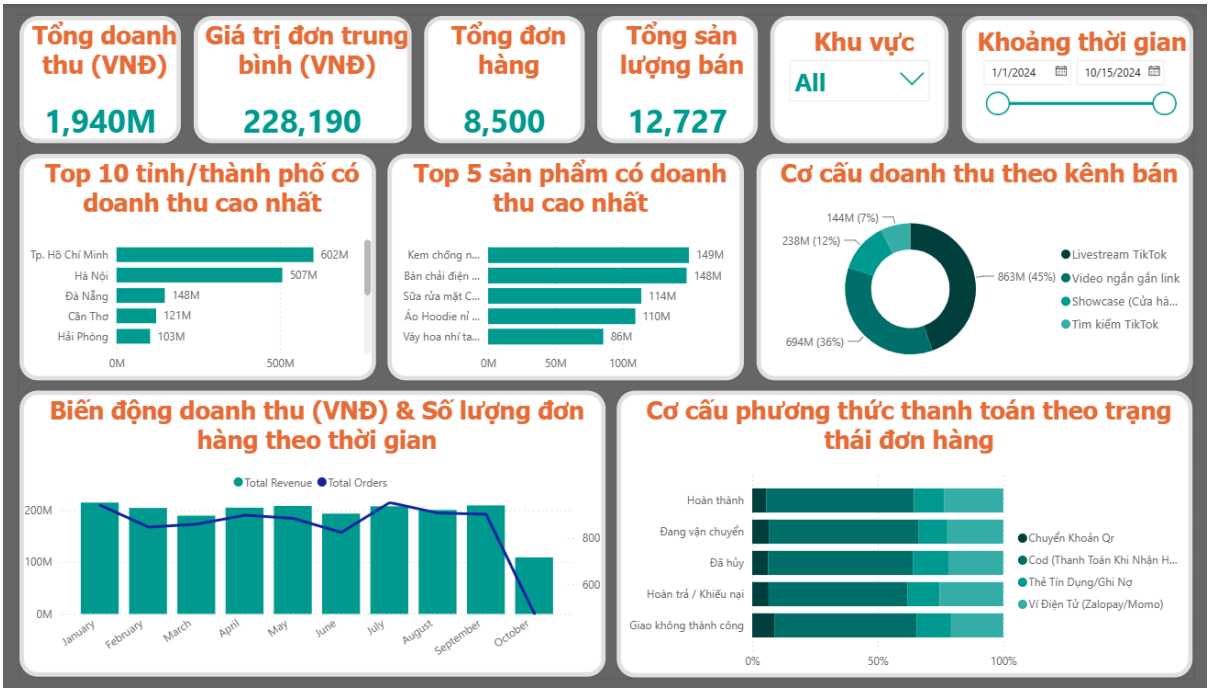
Phân Bố Đơn Hàng Theo Trạng Thái & Phương Thức Thanh Toán

- **Mục tiêu:** Đánh giá và so sánh cơ cấu tỷ trọng phần trăm (%) các phương thức thanh toán bên trong từng trạng thái đơn hàng, giúp nhận diện xu hướng chọn phương thức giao dịch đối với từng nhóm kết quả đơn hàng.
- **Vị trí đặt biểu đồ:** Bottom-Right.
- **Loại biểu đồ (Visual Type):** 100% Stacked Bar Chart (Biểu đồ thanh xếp chồng 100%).
- **Các bước kéo thả dữ liệu:**
  - **Trục Y (Y-Axis):** Fact_Orders[Order_Status].
  - **Trục X (X-Axis):** _Measures[Total Orders].
  - **Chú giải (Legend):** Fact_Orders[Payment_Method].
  - **Filters:** Không cài đặt bộ lọc riêng (nhận tác động tương tác chung từ Slicer trên Dashboard).
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo (Bars Color):** Phương thức Chuyển khoản QR màu #003F3C, Cod màu #006F6A, Thẻ tín dụng/Ghi nợ màu #009990, Ví điện tử màu #33ADA6.
  - **Tiêu đề (Title):** *"Cơ cấu phương thức thanh toán theo trạng thái đơn hàng"* .



Hình 4.39. Biểu đồ Cơ Cấu Phương Thức Thanh Toán Theo Trạng Thái Đơn Hàng

Màn hình Giao diện Dashboard hoàn chỉnh



Hình 4.40. Màn hình giao diện Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh hoàn chỉnh

Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh cung cấp cái nhìn tổng thể về tình hình kinh doanh trên TikTok Shop thông qua các chỉ số doanh thu, đơn hàng, sản lượng bán, khu vực, sản phẩm, kênh bán và phương thức thanh toán.

Bộ thẻ KPI cốt lõi cho thấy tổng doanh thu đạt khoảng 1,940 tỷ đồng, giá trị đơn hàng trung bình đạt 228.190 đồng, tổng số đơn hàng là 8.500 đơn và tổng sản lượng bán đạt 12.727 sản phẩm.

Biểu đồ Top 10 tỉnh/thành phố có doanh thu cao nhất cho thấy TP. Hồ Chí Minh dẫn đầu với khoảng 602,15 triệu đồng, tiếp theo là Hà Nội với 507,03 triệu đồng. Đà Nẵng đạt khoảng 147,58 triệu đồng, Cần Thơ đạt 121,13 triệu đồng và Hải Phòng đạt 102,68 triệu đồng. Doanh thu của TP. Hồ Chí Minh và Hà Nội cao hơn đáng kể so với các địa phương còn lại, cho thấy hai thị trường này đóng góp phần lớn doanh thu trong nhóm địa bàn dẫn đầu. Đây là cơ sở để doanh nghiệp cân nhắc ưu tiên nguồn lực bán hàng và tiếp thị tại các thị trường có doanh thu cao.

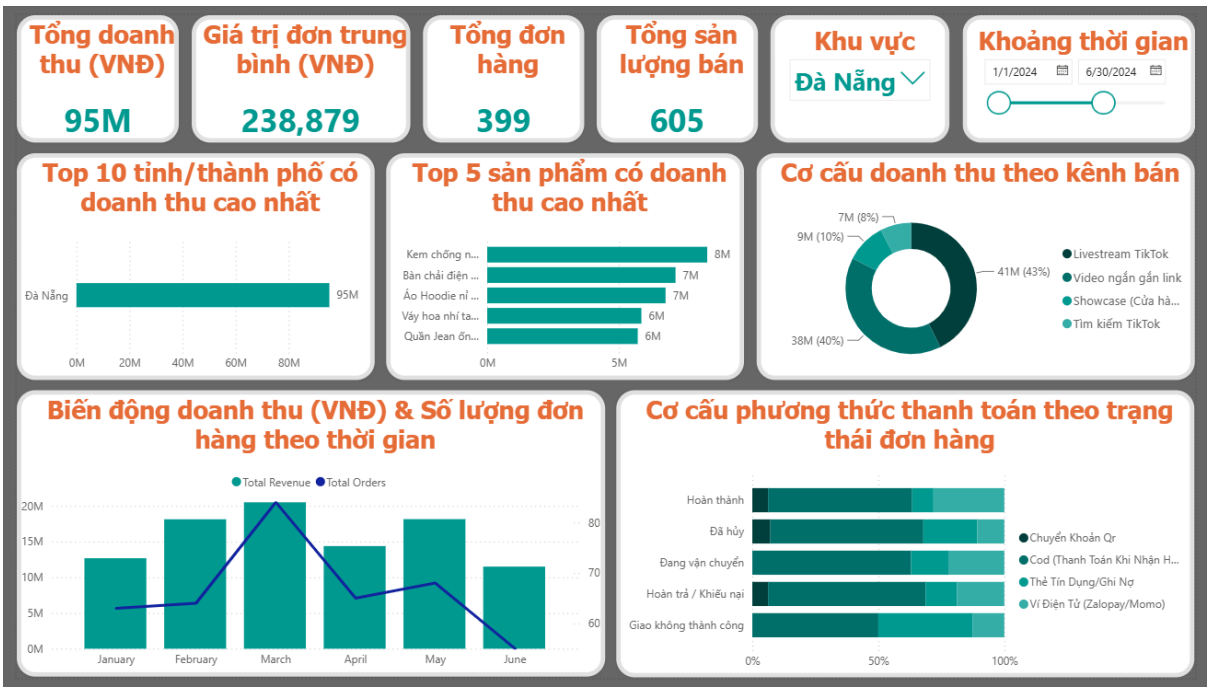
Biểu đồ Top 5 sản phẩm có doanh thu cao nhất cho thấy Kem chống nắng Anessa Perfect UV Sunscreen đứng đầu với khoảng 149,47 triệu đồng, theo sau là Bàn chải điện sóng âm Lock&Lock chống nước IPX7 với 147,72 triệu đồng. Sữa rửa mặt CeraVe Foaming Cleanser 236ml đạt khoảng 113,99 triệu đồng, Áo Hoodie nỉ bông dày dặn form rộng đạt 109,58 triệu đồng và Váy hoa nhí tay bồng phong cách Hàn Quốc đạt 85,70 triệu đồng. Kết quả cho thấy doanh thu tập trung vào các sản phẩm thuộc nhiều nhóm hàng khác nhau, đặc biệt là chăm sóc cá nhân, thiết bị gia dụng và thời trang.

Biểu đồ Cơ cấu doanh thu theo kênh bán cho thấy Livestream TikTok là kênh đóng góp doanh thu lớn nhất, đạt khoảng 863,21 triệu đồng, tương đương 45% tổng doanh thu. Video ngắn gắn link đứng thứ hai với 694,37 triệu đồng (36%), tiếp theo là Showcase (Cửa hàng) với 237,98 triệu đồng (12%) và Tìm kiếm TikTok với 144,06 triệu đồng (7%). Hai kênh Livestream TikTok và Video ngắn gắn link đóng góp tổng cộng khoảng 81% doanh thu, cho thấy nội dung video và hoạt động bán hàng trực tiếp giữ vai trò chủ đạo trong hoạt động kinh doanh.

Biểu đồ Biến động doanh thu và số lượng đơn hàng theo thời gian thể hiện sự thay đổi của hai chỉ số từ tháng 1 đến tháng 10/2024. Doanh thu đạt mức cao nhất vào tháng 1 với khoảng 214,55 triệu đồng, giảm xuống 189,26 triệu đồng vào tháng 3, sau đó dao động và phục hồi trong các tháng tiếp theo. Tháng 9 ghi nhận doanh thu khoảng 209,04 triệu đồng, trong khi tháng 10 chỉ đạt 108,90 triệu đồng. Số lượng đơn hàng cũng giảm mạnh trong tháng 10, từ 903 đơn vào tháng 9 xuống còn 472 đơn. Tuy nhiên, do dữ liệu tháng 10 chỉ được ghi nhận đến ngày 15/10/2024, mức giảm này chưa phản ánh đầy đủ xu hướng doanh thu và đơn hàng của cả tháng. Nhìn chung, doanh thu và số lượng đơn hàng có biến động qua các tháng, vì vậy cần tiếp tục theo dõi trên cùng khoảng thời gian đầy đủ để đánh giá chính xác xu hướng kinh doanh.

Biểu đồ Cơ cấu phương thức thanh toán theo trạng thái đơn hàng thể hiện tỷ trọng các phương thức thanh toán trong từng nhóm trạng thái, gồm Hoàn thành, COD là phương thức được sử dụng phổ biến nhất, trong khi Chuyển khoản QR có tỷ trọng thấp nhất. Biểu đồ xếp chồng 100% giúp so sánh cơ cấu thanh toán giữa các trạng thái đơn hàng, hỗ trợ doanh nghiệp đánh giá thói quen thanh toán và xem xét mối liên hệ giữa phương thức giao dịch với kết quả xử lý đơn hàng. Tuy nhiên, tỷ trọng phương thức thanh toán không tự nó chứng minh phương thức nào có tỷ lệ hủy hoặc hoàn trả cao hơn; cần phân tích thêm số lượng và tỷ lệ theo từng nhóm trạng thái để đưa ra kết luận chính xác.

Nhìn chung, Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh tổng hợp được các chỉ số quan trọng, đồng thời làm nổi bật những thị trường, sản phẩm và kênh bán đóng góp doanh thu chủ yếu. Các kết quả có thể hỗ trợ nhà quản lý xác định khu vực cần ưu tiên, điều chỉnh danh mục sản phẩm, tối ưu hoạt động tiếp thị và theo dõi hiệu quả vận hành.



Hình 4.41. Màn hình giao diện Dashboard Tổng quan Doanh thu & Quy mô Kinh doanh lọc theo khu vực Đà Nẵng và thời gian từ tháng 1 đến tháng 6/2024

Khi lựa chọn khu vực Đà Nẵng và khoảng thời gian từ tháng 1 đến hết tháng 6/2024, doanh thu đạt khoảng 95 triệu đồng, với 399 đơn hàng, 605 sản phẩm được bán ra và giá trị đơn hàng trung bình đạt 238.879 đồng.

Doanh thu tập trung chủ yếu vào Livestream TikTok (43%) và Video ngắn gắn link (40%), chiếm tổng cộng khoảng 83% doanh thu. Trong khi đó, Showcase đóng góp khoảng 10% và Tìm kiếm TikTok khoảng 8%. Điều này cho thấy hai kênh nội dung giữ vai trò chủ lực, là cơ sở để ưu tiên nguồn lực tiếp thị và cải thiện hiệu quả các kênh còn lại.

Xét theo sản phẩm, kem chống nắng dẫn đầu với doanh thu khoảng 8 triệu đồng, tiếp theo là bàn chải điện và áo hoodie với khoảng 7 triệu đồng mỗi sản phẩm. Các sản phẩm còn lại trong Top 5 đạt khoảng 6 triệu đồng. Kết quả cho thấy doanh thu đến từ nhiều nhóm hàng khác nhau, đồng thời giúp xác định những sản phẩm nổi bật để ưu tiên quảng bá tại Đà Nẵng.

Theo thời gian, tháng 3 ghi nhận doanh thu cao nhất, khoảng hơn 20 triệu đồng, đồng thời có số lượng đơn hàng cao nhất trong sáu tháng. Doanh thu giảm vào tháng 4, phục hồi trong tháng 5 rồi tiếp tục giảm vào tháng 6, còn khoảng hơn 11 triệu đồng. Diễn biến này cho thấy kết quả kinh doanh chưa ổn định, cần xem xét hiệu quả các chiến dịch và hoạt động kích cầu theo từng tháng.

Về phương thức thanh toán, COD chiếm tỷ trọng đáng kể trong nhiều trạng thái đơn hàng, đặc biệt ở nhóm đơn đã hủy và đang vận chuyển. Cơ cấu thanh toán khác nhau giữa các trạng thái là cơ sở để kiểm tra thêm mối liên hệ giữa phương thức thanh toán và kết quả xử lý đơn hàng, từ đó đề xuất giải pháp giảm tỷ lệ hủy hoặc giao hàng không thành công.

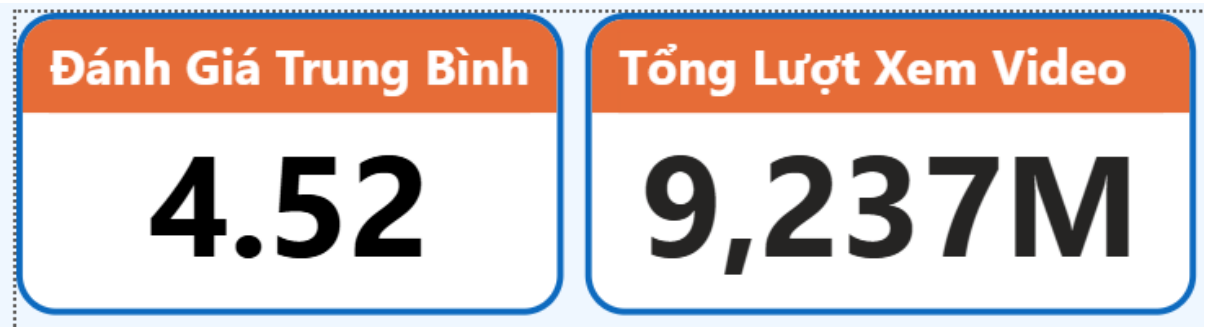
Nhìn chung, trong sáu tháng đầu năm 2024, Đà Nẵng ghi nhận doanh thu nổi bật với hai kênh Livestream và Video ngắn gắn link, cùng nhóm sản phẩm dẫn đầu thuộc ngành hàng chăm sóc cá nhân và thiết bị gia dụng. Tháng 3 là giai đoạn kinh doanh nổi bật nhất, trong khi mức giảm doanh thu vào tháng 6 là tín hiệu cần theo dõi để điều chỉnh chiến lược bán hàng và phân bổ ngân sách tiếp thị phù hợp.

4.5.2 Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi

Chủ đề: Dành cho Brand Manager, Booking KOL/KOC và Marketing nhằm tối ưu sản phẩm bán chạy, chiến dịch Livestream và giá bán.

Bộ Thẻ KPI Banner Sản Phẩm (Top Banner)

- **Mục tiêu:** Đo lường các chỉ số chất lượng đánh giá và quy mô tiếp thị truyền thông của gian hàng.
- **Vị trí đặt:** Top-Left.
- **Loại biểu đồ (Visual Type):** Card Visual.
- **Các bước kéo thả dữ liệu:**
  - **Card 1 (Đánh Giá Trung Bình):** Fields = _Measures[Average Product Rating]
  - **Card 2 (Tổng Lượt Xem Video):** Fields = _Measures[Total Video Views]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Viền thẻ bo góc màu Xanh Dương (#0D6ABF), dải thanh tiêu đề phủ tràn màu Cam Đất (#E66C37) với chữ in đậm màu trắng (#FFFFFF), nền thẻ màu trắng (#FFFFFF).
  - **Nhãn dữ liệu (Callout Value):** Màu chữ đen (#000000), phông chữ Segoe UI Bold, cỡ chữ 40pt. Định dạng hiển thị lượt xem dạng rút gọn (Display Units = Millions) và điểm rating dạng thập phân 2 chữ số.
  - **Tiêu đề (Category Label / Title):** Đổi tên nhãn tiêu đề tương ứng thành *"Đánh Giá Trung Bình"* và *"Tổng Lượt Xem Video"*.
  - **Hiệu ứng đặc biệt:** Đặt khoảng cách giữa hai thẻ cách đều nhau (Distribute Horizontally).

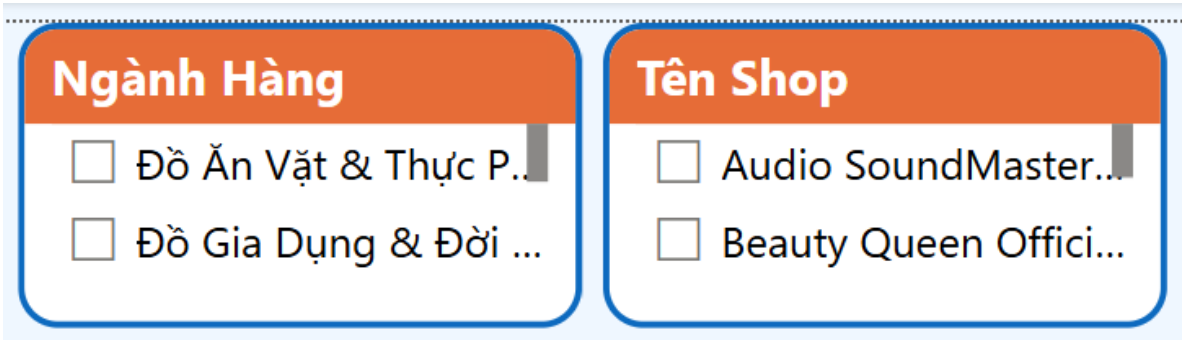


Hình 4.42. Biểu đồ Bộ thẻ KPI Banner Sản phẩm

Bộ Lọc Slicer Ngành Hàng & Shop

- **Mục tiêu:** Lọc nhanh dữ liệu tương tác trên toàn trang theo Ngành hàng chính và Tên Shop bán hàng.
- **Vị trí đặt:** Top-Center / Top-Right.

- **Loại biểu đồ (Visual Type):** Slicer.
- **Các bước kéo thả dữ liệu:**
  - **Slicer 1 (Ngành Hàng):** Field = Dim_Product[Main_Category]
  - **Slicer 2 (Tên Shop):** Field = Dim_Product[Shop_Name]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Thanh tiêu đề phủ màu Cam Đất (#E66C37) chữ in đậm màu trắng (#FFFFFF), viền khung bo góc màu Xanh Dương (#0D6ABF), ô chọn Checkbox active màu #E66C37.
  - **Kiểu hiển thị (Slicer Settings):** Chọn kiểu hiển thị danh sách dạng Vertical List với các ô tích Checkbox.
  - **Tiêu đề (Slicer Header / Title):** Đổi tiêu đề tương ứng thành *"Ngành Hàng"* và *"Tên Shop"*.
  - **Hiệu ứng đặc biệt:** Kiểm tra *Edit Interactions* đảm bảo Slicer tự động tác động lọc tới toàn bộ các visual còn lại trên trang.



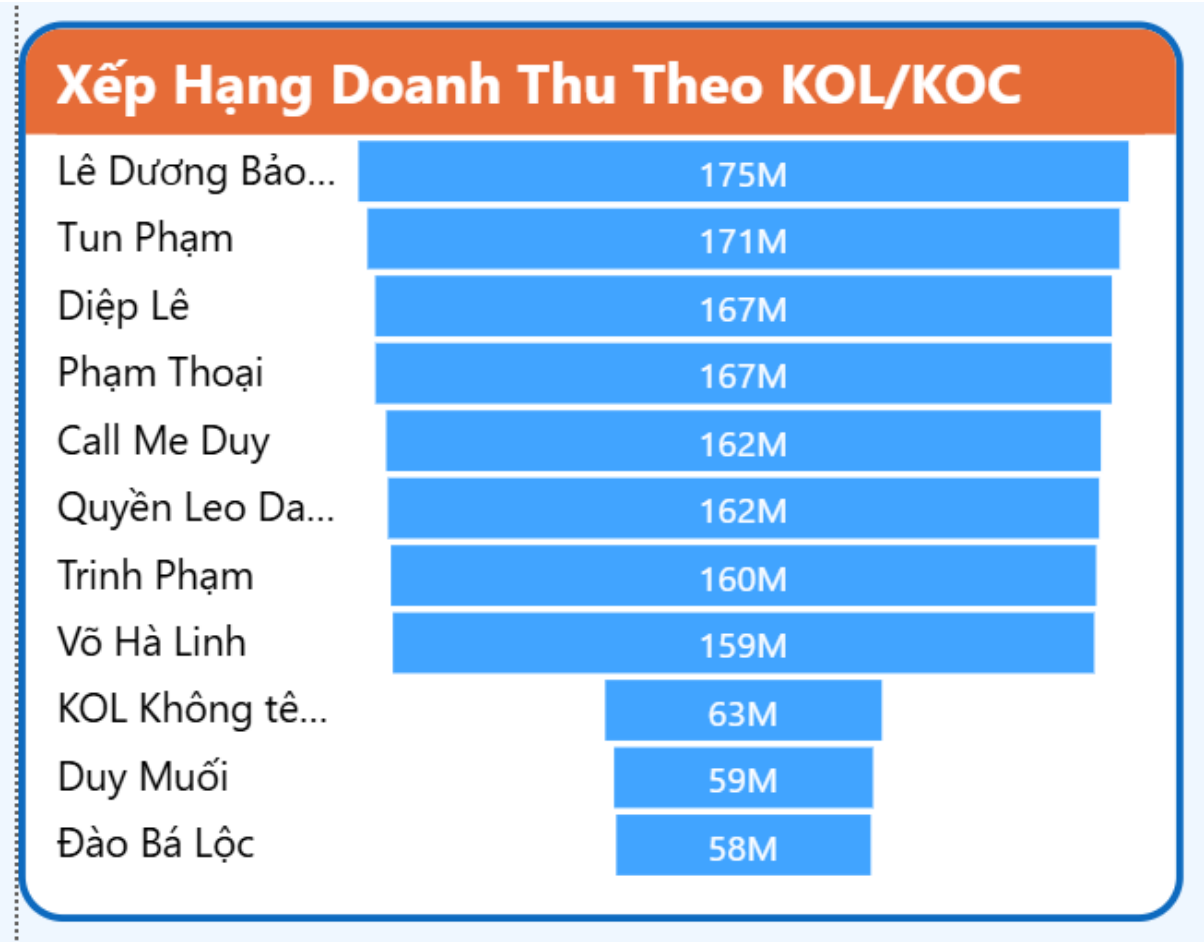
Hình 4.43. Biểu đồ Bộ lọc Dữ liệu Slicer Ngành Hàng & Shop

Xếp Hạng Doanh Thu Theo KOL/KOC

- **Mục tiêu:** Xếp hạng thứ bậc doanh thu đóng góp của các Nhà sáng tạo nội dung (KOL/KOC) theo dạng phễu thu nhỏ giảm dần, giúp bộ phận Booking đánh giá chính xác hiệu quả từng Creator.
- **Vị trí đặt biểu đồ:** Center-Left.
- **Loại biểu đồ (Visual Type):** Funnel Chart (Biểu đồ phễu).
- **Các bước kéo thả dữ liệu:**

- **Category:** Fact_Orders[Creator_Name]
- **Values:** _Measures[Total Revenue]
- **Filters:** Bật bộ lọc loại bỏ nhóm không phải là KOL/KOC (Gian hàng bán trực tiếp,...).

- **Thiết lập định dạng & Màu sắc (Format):**
- **Màu sắc chủ đạo:** Nền dải phễu tô màu Xanh Dương sáng (#41A4FF), viền khung bo góc màu Xanh Dương (#0D6ABF), thanh tiêu đề màu Cam Đất (#E66C37).
- **Nhãn dữ liệu (Data Labels):** Bật On, vị trí căn *Inside center* (nằm gọn bên trong thanh), màu chữ trắng (#FFFFFF), hiển thị số tiền dạng rút gọn (Millions VNĐ).
- **Tiêu đề (Title):** *"Xếp Hạng Doanh Thu Theo KOL/KOC"*.

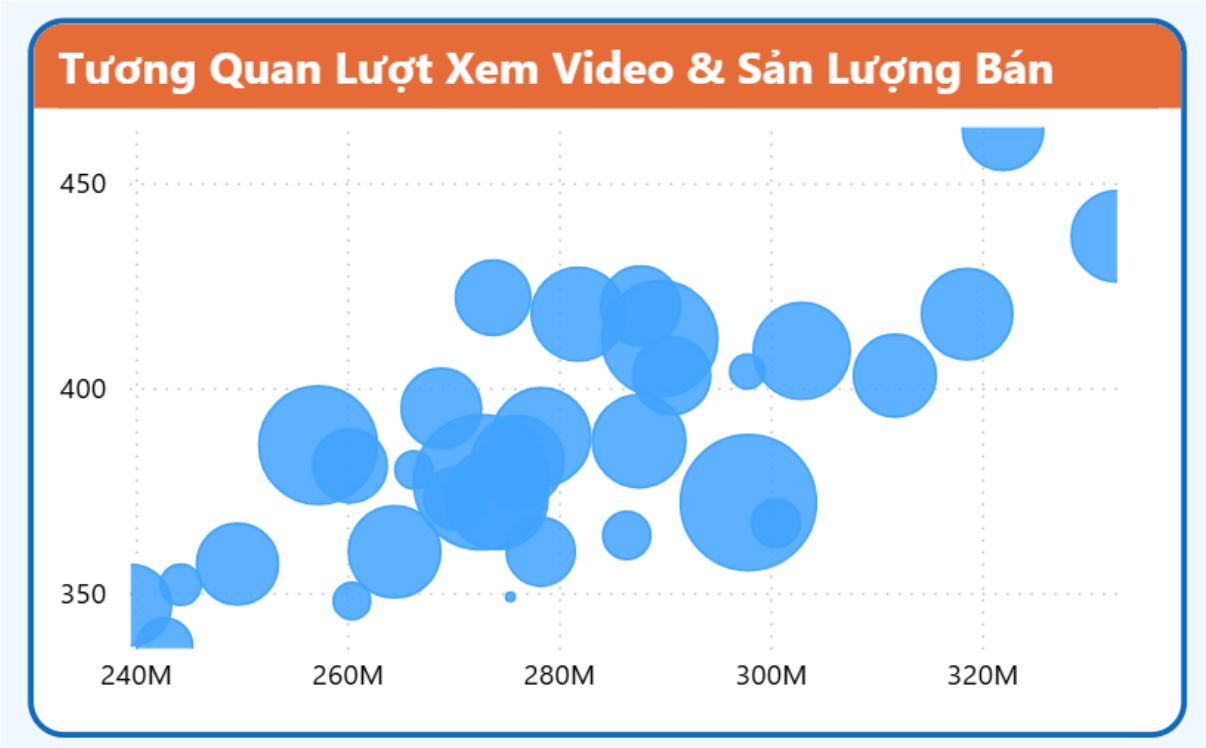


Hình 4.44. Biểu đồ Xếp Hạng Doanh Thu Theo KOL/KOC

Tương Quan Lượt Xem Video & Sản Lượng Bán

- **Mục tiêu:** Khảo sát mối tương quan giữa số lượt xem video/livestream và sản lượng chốt đơn để phân loại hiệu quả chuyển đổi sản phẩm, giúp nhận diện mã hàng có tỷ lệ chuyển đổi cao (Conversion Rate).

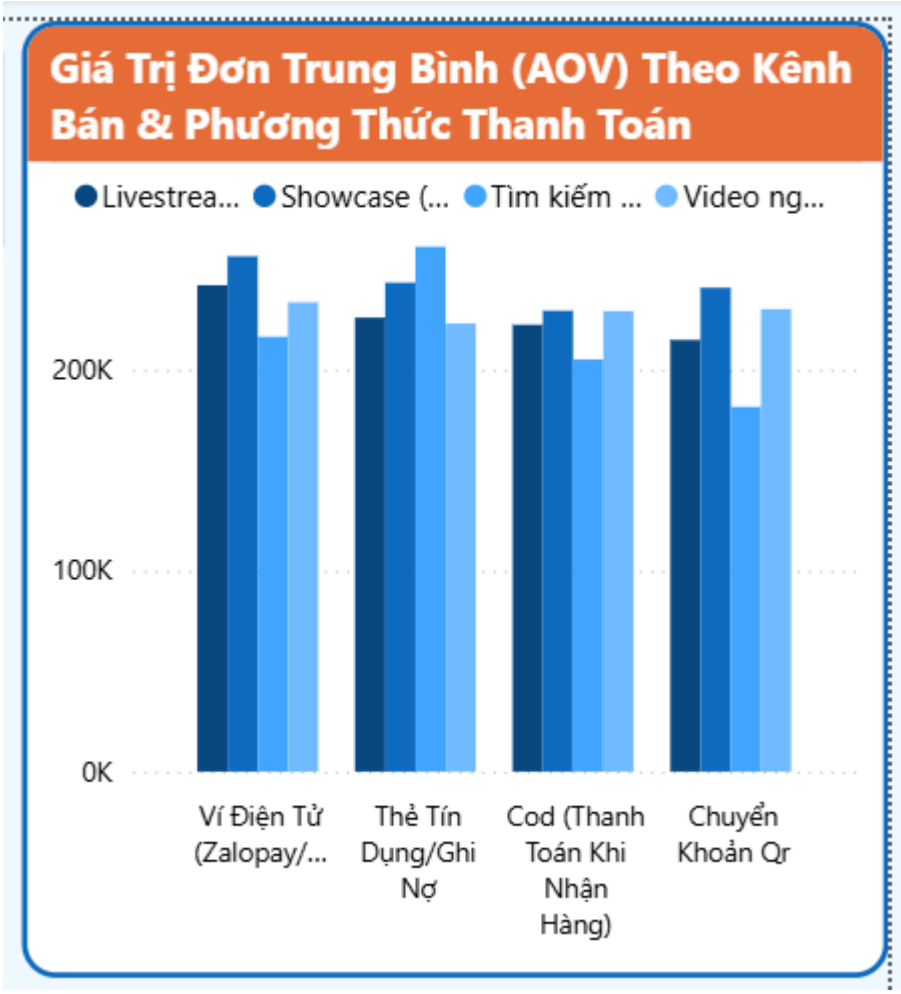
- **Vị trí đặt biểu đồ:** Center-Middle.
- **Loại biểu đồ (Visual Type):** Scatter Chart (Biểu đồ phân tán).
- **Các bước kéo thả dữ liệu:**
  - **Values:** Dim_Product[Product_Name]
  - **Trục X (X-Axis):** _Measures[Total Video Views]
  - **Trục Y (Y-Axis):** _Measures[Total Quantity]
  - **Size (Kích thước bóng):** _Measures[Total Revenue]
  - **Filters:** Không cài đặt bộ lọc riêng.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc chủ đạo:** Các bóng tròn biểu thị sản phẩm sử dụng tông màu Xanh Dương ngọc (#41A4FF).
  - **Khung & Tiêu đề:** Thanh tiêu đề phủ tràn màu Cam Đất (#E66C37) với chữ in đậm màu trắng (#FFFFFF), viền khung bo góc màu Xanh Dương (#0D6ABF).
  - **Nhãn dữ liệu (Data Labels):** Tắt nhãn bóng mặc định để tránh đè chữ, hiển thị thông tin chi tiết qua Tooltip khi di chuột.
  - **Tiêu đề (Title):** *"Tương Quan Lượt Xem Video & Sản Lượng Bán"*.



Hình 4.45. Biểu đồ Tương Quan Lượt Xem Video & Sản Lượng Bán

Giá Trị Đơn Trung Bình (AOV) Theo Kênh Bán & Phương Thức Thanh Toán

- **Mục tiêu:** So sánh và đánh giá mức chi tiêu bình quân trên mỗi đơn hàng (AOV) giữa các phương thức thanh toán, phân rã chi tiết theo dải màu của từng kênh bán nhằm phát hiện sự kết hợp tạo ra giá trị giao dịch cao nhất.
- **Vị trí đặt biểu đồ:** Center-Right.
- **Loại biểu đồ (Visual Type):** Clustered Column Chart (Biểu đồ cột cụm nhóm).
- **Các bước kéo thả dữ liệu:**
  - **Trục X (X-Axis):** Fact_Orders[Payment_Method]
  - **Trục Y (Y-Axis):** _Measures[AOV]
  - **Chú giải (Legend):** Fact_Orders[Traffic_Source]
  - **Filters:** Không cài đặt bộ lọc riêng.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc các cột (Columns Color):** Phối cụm dải màu Xanh Tím Than (#094780), Xanh Tím (#0D6ABF), Xanh Dương (#41A4FF), Xanh Nhạt (#70BBFF).
  - **Khung & Tiêu đề:** Thanh tiêu đề phủ tràn màu Cam Đất (#E66C37) với chữ in đậm màu trắng (#FFFFFF), viền khung bo góc màu Xanh Dương (#0D6ABF).
  - **Tiêu đề (Title):** *"Giá Trị Đơn Trung Bình (AOV) Theo Kênh Bán & Phương Thức Thanh Toán"*.
  - **Hiệu ứng đặc biệt:** Tắt đường lưới ngang (Gridlines) ở trục Y để tăng độ thoáng cho visual.

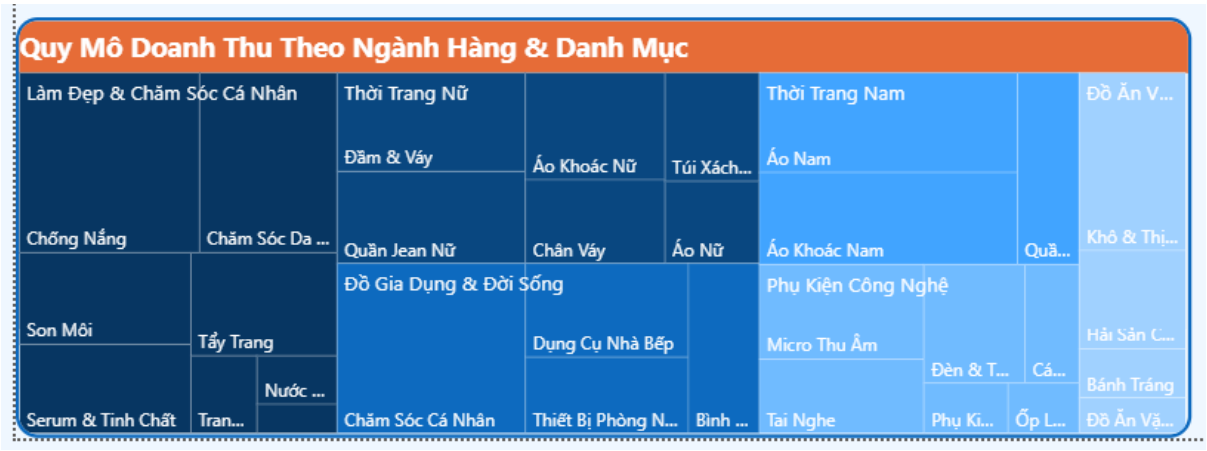


Hình 4.46. Biểu đồ Giá Trị Đơn Trung Bình (AOV) Theo Kênh Bán & Phương Thức Thanh Toán

Quy Mô Doanh Thu Theo Ngành Hàng & Danh Mục

- **Mục tiêu:** Trực quan hóa tỷ trọng quy mô dòng tiền đóng góp giữa các Ngành hàng chính (Main Category) và Danh mục con (Sub Category).
- **Vị trí đặt biểu đồ:** Bottom-Left.
- **Loại biểu đồ (Visual Type):** Treemap Chart (Biểu đồ ô khối Treemap).
- **Các bước kéo thả dữ liệu:**
  - **Category:** Dim_Product[Main_Category]
  - **Details:** Dim_Product[Sub_Category]
  - **Values:** _Measures[Total Revenue]
  - **Filters:** Không cài đặt bộ lọc riêng.
- **Thiết lập định dạng & Màu sắc (Format):**

- **Màu sắc khối:** Phối bộ màu chuyển sắc từ Tím Than (#0D6ABF), Xanh Dương (#41A4FF) đến Xanh Nhạt theo thứ tự quy mô giảm dần.
- **Khung & Tiêu đề:** Thanh tiêu đề màu Cam Đất (#E66C37), chữ in đậm màu trắng (#FFFFFF), viền khung Xanh Dương bo góc (#0D6ABF).
- **Tiêu đề (Title):** *"Quy Mô Doanh Thu Theo Ngành Hàng & Danh Mục"*.



Hình 4.47. Biểu đồ Quy Mô Doanh Thu Theo Ngành Hàng & Danh Mục

Bảng Tra Cứu Chi Tiết Sản Phẩm & Đánh Giá

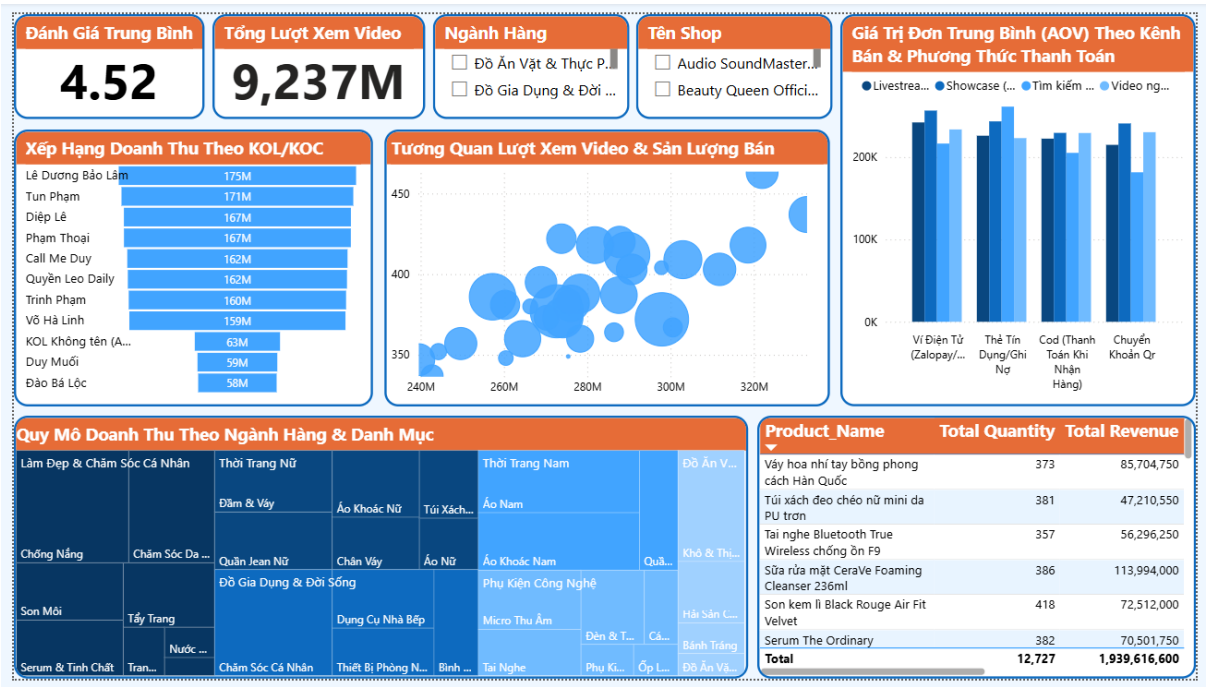
- **Mục tiêu:** Bảng tra chi tiết thông số bán hàng của từng mã sản phẩm, hỗ trợ tra cứu chính xác sản lượng, doanh thu và điểm đánh giá rating.
- **Vị trí đặt biểu đồ:** Bottom-Right.
- **Loại biểu đồ (Visual Type):** Table.
- **Các bước kéo thả dữ liệu:**
- **Rows / Columns:** Dim_Product[Product_Name], _Measures[Total Quantity], _Measures[Total Revenue], _Measures[Average Product Rating], _Measures[Total Review Count]
- **Thiết lập định dạng & Màu sắc (Format):**
- **Column Headers:** Dải tiêu đề cột tràn viền tô phủ tràn màu Cam Đất (#E66C37), chữ in đậm màu trắng (#FFFFFF).
- **Values & Style:** Dòng dữ liệu hiển thị dạng dòng trắng xen kẽ xanh nhạt (Alternating Rows), dòng **Total** in đậm kèm viền Xanh Dương dưới cùng.

- **Tiêu đề (Title):** Tự động theo tên các cột (Product_Name, Total Quantity, Total Revenue,...).
- **Hiệu ứng đặc biệt:** Bật chức năng cuộn dọc (Scrollbar) giúp tra cứu toàn bộ danh mục sản phẩm.

Product_Name	Total Quantity	Total Revenue
Váy hoa nhí tay bồng phong cách Hàn Quốc	373	85,704,750
Túi xách đeo chéo nữ mini da PU trơn	381	47,210,550
Tai nghe Bluetooth True Wireless chống ồn F9	357	56,296,250
Sữa rửa mặt CeraVe Foaming Cleanser 236ml	386	113,994,000
Son kem lì Black Rouge Air Fit Velvet	418	72,512,000
Serum The Ordinary	382	70,501,750
Total	**12,727**	**1,939,616,600**

Hình 4.48. Bảng Tra Cứu Chi Tiết Sản Phẩm & Đánh Giá

Màn hình Giao diện Dashboard hoàn chỉnh



Hình 4.49. Màn hình giao diện Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi hoàn chỉnh

Dashboard **Phân tích Sản phẩm & Kích cầu Chuyển đổi** cung cấp cái nhìn tổng quan về hiệu quả kinh doanh theo sản phẩm, ngành hàng, KOL/KOC và kênh bán trên TikTok Shop. Thông qua các chỉ số đánh giá sản phẩm, lượt xem video, doanh thu, sản lượng bán và giá trị đơn hàng trung bình, dashboard hỗ trợ bộ phận Marketing và Booking đánh giá hiệu quả quảng bá, xác định sản phẩm tiềm năng và tối ưu hoạt động kích cầu chuyển đổi.

Bộ thẻ KPI Đánh giá trung bình và Tổng lượt xem video cho thấy điểm đánh giá sản phẩm trung bình đạt 4,52/5 điểm, phản ánh mức độ đánh giá tương đối tích cực từ khách hàng. Tổng lượt xem video đạt khoảng 9,237 triệu lượt, cho thấy nội dung video có khả năng tiếp cận lượng người xem đáng kể.

Biểu đồ Xếp hạng doanh thu theo KOL/KOC cho thấy Lê Dương Bảo Lâm dẫn đầu với khoảng 175 triệu đồng, tiếp theo là Tun Phạm với 171 triệu đồng và Diệp Lê với 167 triệu đồng. Nhóm nhà sáng tạo có doanh thu cao tạo ra mức đóng góp nổi bật, trong khi các vị trí cuối bảng chỉ đạt khoảng 58–63 triệu đồng. Chênh lệch này giúp doanh nghiệp đánh giá hiệu quả hợp tác và cân nhắc phân bổ ngân sách Booking phù hợp.

Biểu đồ Tương quan lượt xem video và sản lượng bán thể hiện xu hướng sản lượng bán có sự thay đổi theo lượt xem, nhưng các điểm dữ liệu phân tán cho thấy lượt xem cao chưa đồng nghĩa với sản lượng bán cao nhất. Một số sản phẩm đạt sản lượng khoảng 400–420 đơn vị ở các mức lượt xem khác nhau. Điều này cho thấy doanh nghiệp cần kết hợp lượt xem với doanh thu và sản lượng thực tế để đánh giá hiệu quả chuyển đổi, thay vì chỉ dựa vào mức độ tiếp cận.

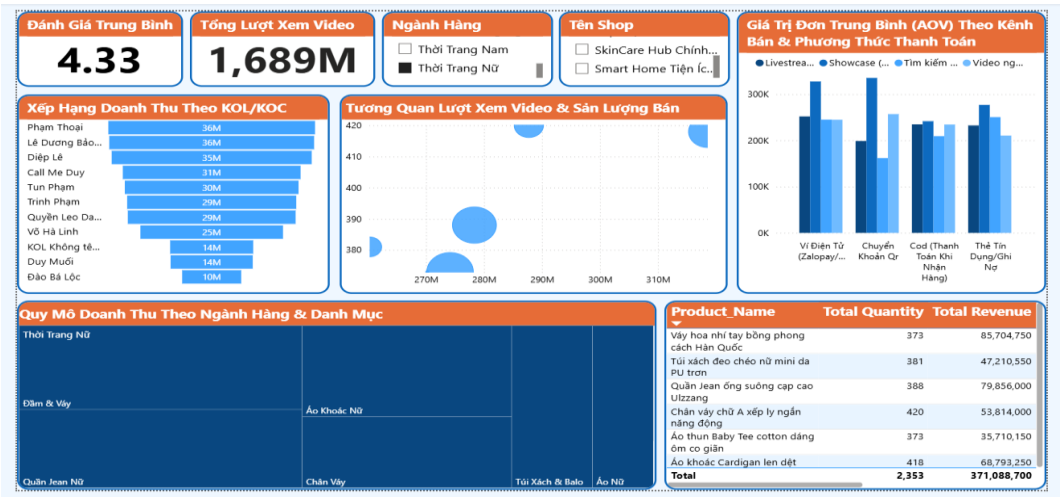
Biểu đồ Giá trị đơn trung bình (AOV) theo kênh bán và phương thức thanh toán cho thấy giá trị đơn hàng bình quân có sự khác biệt giữa các phương thức thanh toán và nguồn truy cập. Các mức AOV trên biểu đồ chủ yếu nằm trong khoảng 180.000–260.000 đồng/đơn hàng. Một số tổ hợp giữa kênh bán và phương thức thanh toán đạt giá trị cao hơn các nhóm còn lại, trong khi thanh toán QR qua nguồn tìm kiếm có mức AOV tương đối thấp. Kết quả này giúp doanh nghiệp xác định những tổ hợp kênh bán

và phương thức thanh toán có giá trị giao dịch tốt để tối ưu hoạt động tiếp thị và chuyển đổi.

Biểu đồ Quy mô doanh thu theo ngành hàng và danh mục cho thấy doanh thu được phân bổ giữa nhiều nhóm sản phẩm, trong đó Làm đẹp và Chăm sóc cá nhân chiếm diện tích nổi bật trên biểu đồ Treemap ở trạng thái dashboard tổng thể. Các danh mục như Chống nắng, Son môi, Serum và Tinh chất đóng góp đáng kể trong nhóm này. Thời trang nữ, Đồ gia dụng và Đời sống cùng Phụ kiện công nghệ cũng có nhiều danh mục con được thể hiện. Sự phân bổ này cho thấy danh mục kinh doanh tương đối đa dạng, đồng thời giúp doanh nghiệp nhận diện các nhóm sản phẩm cần ưu tiên phân tích và phân bổ nguồn lực tiếp thị.

Bảng Tra cứu chi tiết sản phẩm cung cấp thông tin về sản lượng và doanh thu của từng sản phẩm. Tổng cộng, toàn bộ dữ liệu ghi nhận 12.727 sản phẩm được bán ra, đạt doanh thu 1.939.616.600 đồng. Trong bảng, Sữa rửa mặt CeraVe Foaming Cleanser 236ml đạt doanh thu khoảng 113,99 triệu đồng, nổi bật hơn các sản phẩm được hiển thị khác.

Nhìn chung, Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi hỗ trợ đánh giá đồng thời hiệu quả KOL/KOC, mức độ quan tâm qua lượt xem video, giá trị giao dịch, cơ cấu ngành hàng và kết quả bán hàng theo từng sản phẩm. Các thông tin này giúp bộ phận Marketing và Booking lựa chọn nhà sáng tạo nội dung phù hợp, tối ưu kênh bán và tập trung nguồn lực vào những danh mục có tiềm năng doanh thu.



Hình 4.50. Màn hình giao diện Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi lọc theo ngành hàng Thời trang nữ

Khi lựa chọn ngành hàng Thời Trang Nữ, Dashboard Phân tích Sản phẩm & Kích cầu Chuyển đổi tập trung vào hiệu quả kinh doanh của nhóm sản phẩm thời trang dành cho nữ, với tổng doanh thu đạt khoảng 371,09 triệu đồng, tổng sản lượng 2.353 sản phẩm và điểm đánh giá trung bình 4,33/5. Tổng lượt xem video đạt khoảng 1,689 triệu lượt, cung cấp cơ sở đánh giá mức độ tiếp cận nội dung và kết quả bán hàng trong phân khúc này.

Về hiệu quả KOL/KOC, Phạm Thoại và Lê Dương Bảo Lâm cùng dẫn đầu với doanh thu khoảng 36 triệu đồng, tiếp theo là Diệp Lê với 35 triệu đồng. Các KOL/KOC còn lại trong nhóm dẫn đầu tạo ra doanh thu khoảng 25–31 triệu đồng, trong khi một số nhóm có doanh thu thấp hơn, khoảng 10–14 triệu đồng. Kết quả cho thấy doanh thu giữa các nhà sáng tạo có sự chênh lệch, giúp doanh nghiệp xác định những đối tác tiềm năng để ưu tiên hợp tác và phân bổ ngân sách booking.

Xét mối quan hệ giữa lượt xem video và sản lượng bán, các điểm dữ liệu phân bố khá rộng, với sản lượng chủ yếu trong khoảng 370–420 sản phẩm và lượt xem khoảng 270–330 triệu theo thang đo hiển thị. Một số sản phẩm có lượt xem cao nhưng sản lượng không tăng tương ứng, cho thấy lượt xem đơn thuần chưa đủ để phản ánh hiệu quả chuyển đổi. Doanh nghiệp cần kết hợp thêm dữ liệu doanh thu và tỷ lệ chuyển đổi để xác định sản phẩm được quảng bá hiệu quả.

Về giá trị đơn hàng trung bình theo kênh bán và phương thức thanh toán, biểu đồ cho thấy mức AOV khác nhau giữa các tổ hợp. Một số nhóm thanh toán qua chuyển khoản QR và ví điện tử đạt mức khoảng 300.000–330.000 đồng/đơn hàng, trong khi một số tổ hợp khác nằm quanh 160.000–250.000 đồng. Sự chênh lệch này có thể hỗ trợ doanh nghiệp đánh giá nhóm giao dịch có giá trị cao, tuy nhiên cần đối chiếu số lượng đơn hàng của từng nhóm trước khi đưa ra quyết định.

Cơ cấu doanh thu theo ngành hàng và danh mục cho thấy doanh thu hiện tập trung trong các danh mục như Đầm & Váy, Quần Jean Nữ, Áo khoác nữ và Chân váy. Đây là những nhóm sản phẩm đáng chú ý để doanh nghiệp cân nhắc ưu tiên trưng bày, xây dựng nội dung quảng bá và triển khai chương trình kích cầu.

Bảng chi tiết sản phẩm cho thấy Váy hoa nhí tay bồng phong cách Hàn Quốc đạt doanh thu 85.704.750 đồng với 373 sản phẩm; Quần Jean ống suông cạp cao Ulzzang đạt 79.856.000 đồng với 388 sản phẩm; và Áo khoác Cardigan len dệt đạt 68.793.250

đồng với 418 sản phẩm. Trong khi đó, Chân váy chữ A xếp ly ngắn năng động bán được 420 sản phẩm, cao nhất trong các dòng đang hiển thị, với doanh thu 53.814.000 đồng. Điều này cho thấy sản phẩm bán nhiều nhất không nhất thiết là sản phẩm tạo doanh thu cao nhất, vì kết quả còn phụ thuộc vào giá bán và giá trị giao dịch.

Nhìn chung, phân khúc Thời Trang Nữ ghi nhận doanh thu nổi bật ở một số KOL/KOC và danh mục váy, quần, áo khoác; trong khi hiệu quả giữa lượt xem và sản lượng chưa đồng đều. Doanh nghiệp có thể dựa vào kết quả này để ưu tiên sản phẩm chủ lực, lựa chọn đối tác quảng bá phù hợp và tối ưu chuyển đổi thay vì chỉ tập trung gia tăng lượt xem.

4.5.3 Dashboard Phân tích Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường

Chủ đề: Tối ưu chi phí Khuyến mãi/Discount, quản lý rủi ro đơn bất thường (Is_Anomaly), phí vận chuyển và hành vi khách hàng theo vị trí địa lý.

Bộ Thẻ KPI Banner Rủi Ro & Khách Hàng

- **Mục tiêu:** Theo dõi tổng chi phí khuyến mãi, quy mô tệp khách hàng và kiểm soát rủi ro tỷ lệ hủy đơn, hoàn hàng tại khu vực nhận diện ưu tiên theo sơ đồ Gutenberg.
- **Vị trí đặt:** Top-Left
- **Loại biểu đồ (Visual Type):** Card Visual
- **Các bước kéo thả dữ liệu:**
  - **Card 1 (Tổng Tiền Chiết Khấu):** Fields = _Measures[Total Discount Amount]
  - **Card 2 (Tổng Khách Hàng):** Fields = _Measures[Total Customers]
  - **Card 3 (Tỷ Lệ Hủy Đơn):** Fields = _Measures[Cancel Rate]
  - **Card 4 (Tỷ Lệ Hoàn Hàng):** Fields = _Measures[Return Rate]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Khung & Nền:** Background tô màu #00201F (Xanh Đen Tối), Border bật **On** màu trắng #FFFFFF với độ dày **Width = 1px**, bo góc **Rounded Corners = 20px**.
  - **Tiêu đề (Title):** Font **Tahoma**, cỡ chữ **14pt**, in đậm (Bold), màu chữ trắng #FFFFFF, căn giữa.

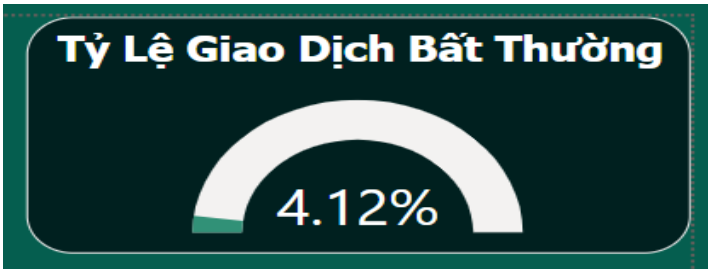
- **Giá trị chỉ số (Callout Value):** Font **Segoe UI**, cỡ chữ **24pt**, màu chữ #37FF9F. Đổi tên hiển thị các thẻ lần lượt thành *"Tổng Tiền Chiết Khấu (VNĐ)"*, *"Tổng Khách Hàng"*, *"Tỷ Lệ Hủy Đơn"* và *"Tỷ Lệ Hoàn Hàng"*.



Hình 4.51. Biểu đồ Bộ thẻ KPI Banner Rủi Ro & Khách Hàng

Tỷ Lệ Giao Dịch Bất Thường

- **Mục tiêu:** Kiểm soát mức độ an toàn hệ thống và phát hiện kịp thời tỷ lệ các đơn hàng gian lận/outlier (Is_Anomaly = 1) để đưa ra cảnh báo quản trị.
- **Vị trí đặt:** Top-Middle
- **Loại biểu đồ (Visual Type):** Gauge Chart (Biểu đồ đồng hồ đo).
- **Các bước kéo thả dữ liệu:**
  - **Value (Giá trị hiện tại):** _Measures[Anomaly Rate]
  - **Maximum value (Giá trị tối đa):** Nhập giá trị cố định 0.1.
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc vạch đo (Fill Color):** Tô màu xanh mòng két sáng #319177 cho phần dải đo đạt được.
  - **Khung & Nền:** Background màu #00201F, Visual Border màu trắng #FFFFFF (**Width = 1px**), bo góc **20px**.
  - **Tiêu đề (Title):** *"Tỷ Lệ Giao Dịch Bất Thường"*
  - **Nhãn & Số liệu (Data Labels / Callout):** Font **Segoe UI**, màu chữ trắng #FFFFFF.



Hình 4.52. Biểu đồ Đồng hồ đo Tỷ Lệ Giao Dịch Bất Thường

Bộ Lọc Slicer Độ Tuổi & Giới Tính & Thiết Bị

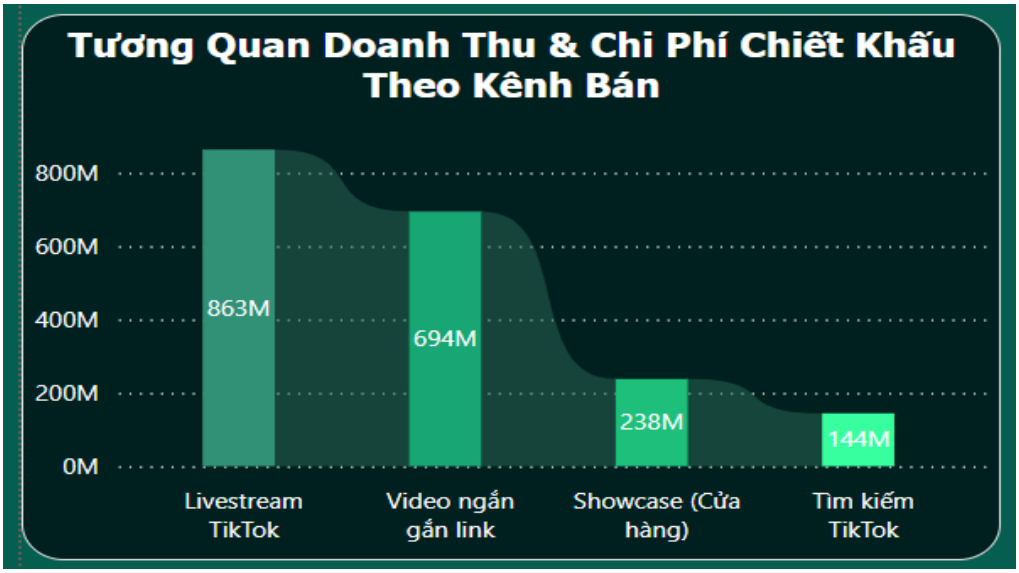
- **Mục tiêu:** Cho phép cấp quản lý khoanh vùng và lọc dữ liệu động theo các thuộc tính nhân khẩu học và thiết bị của khách hàng trên toàn bộ Dashboard.
- **Vị trí đặt:** Top-Right
- **Loại biểu đồ (Visual Type):** Slicer.
- **Các bước kéo thả dữ liệu:**
  - **Slicer 1 (Độ tuổi):** Field = Dim_Customer[Customer_Age]
  - **Slicer 2 (Giới tính):** Field = Dim_Customer[Customer_Gender]
  - **Slicer 3 (Thiết bị):** Field = Dim_Customer[Device_Type]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc & Nút chọn:** Nền Slicer màu #00201F, viền khung trắng #FFFFFF (**1px**, bo góc **20px**). Các tùy chọn item active tô màu xanh ngọc #37FF9F.
  - **Kiểu hiển thị (Slicer Settings):** Chọn kiểu danh sách thả xuống (Dropdown).
  - **Tiêu đề (Slicer Header / Title):** Đổi tiêu đề tương ứng thành *"Độ Tuổi"*, *"Giới Tính"* và *"Thiết Bị"*.
  - **Giá trị danh sách (Values):** Font **Segoe UI**, màu chữ #2C6958.
  - **Hiệu ứng đặc biệt:** Kiểm tra *Edit Interactions* đảm bảo các Slicer tự động lọc dữ liệu cho toàn bộ các biểu đồ trên trang.



Hình 4.53. Biểu đồ Bộ lọc Dữ liệu Slicer Nhân khẩu học & Thiết bị

Tương Quan Doanh Thu & Chi Phí Chiết Khấu Theo Kênh Bán

- **Mục tiêu:** Đánh giá và so sánh thứ bậc quy mô Doanh thu (Total Revenue) và Chi phí Chiết khấu (Total Discount Amount) giữa các Kênh bán hàng (*Livestream, Showcase, Tìm kiếm, Video ngắn*), giúp nhận diện kênh bán nào tạo dòng tiền cao nhất và kênh nào ngốn nhiều tiền khuyến mãi nhất.
- **Vị trí đặt biểu đồ:** Center-Left.
- **Loại biểu đồ (Visual Type):** Ribbon Chart (Biểu đồ dải băng).
- **Các bước kéo thả dữ liệu:**
  - **Trục X (X-Axis):** Fact_Orders[Traffic_Source]
  - **Trục Y (Y-Axis):** _Measures[Total Revenue]
  - **Tooltips (Chú giải công cụ):** _Measures[Total Discount Amount]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc nhóm cột (Columns Colors):** Phối phân dải màu cột theo đúng thứ tự lần lượt: #319177, #18A674, #1DBF7A, #37FF9F.
  - **Tiêu đề (Title):** *"Tương Quan Doanh Thu & Chi Phí Chiết Khấu Theo Kênh Bán"*
  - **Nhãn dữ liệu & Trục (Data Labels & Axis):** Bật **On**, Font **Segoe UI**, toàn bộ nhãn số liệu và chữ chú giải hiển thị full màu trắng #FFFFFF.



Hình 4.54. Biểu đồ Ribbon Chart Tương Quan Doanh Thu & Chi Phí Chiết Khấu Theo Kênh Bán

Tương Quan Doanh Thu & Phí Ship Theo Tỉnh/Thành Phố

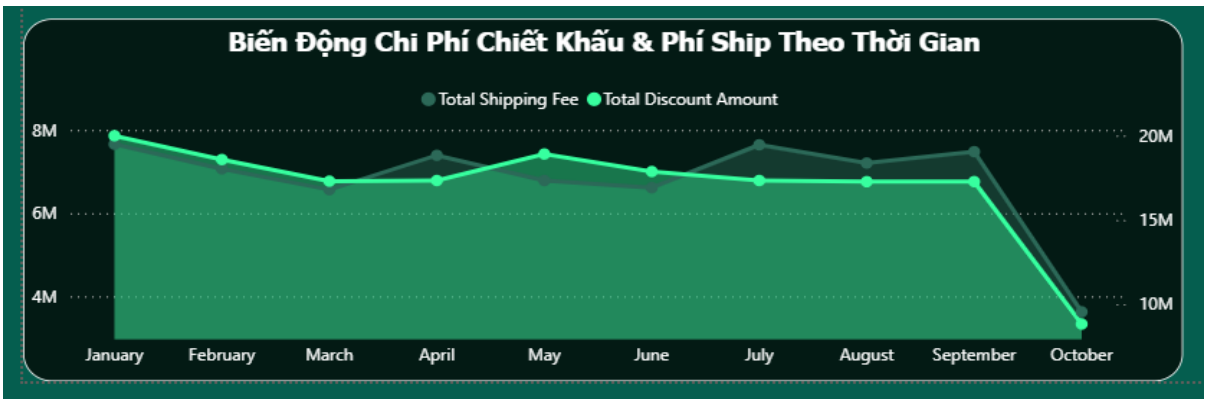
- **Mục tiêu:** Đo lường tổng chi phí vận chuyển phát sinh theo từng Tỉnh/Thành phố nhằm phát hiện các khu vực địa lý đang tốn nhiều chi phí logistics so với doanh thu tạo ra.
- **Vị trí đặt biểu đồ:** Bottom-Left.
- **Loại biểu đồ (Visual Type):** Line and Clustered Column Chart (Biểu đồ kết hợp Cột & Đường).
- **Các bước kéo thả dữ liệu:**
  - **Trục X (Shared Axis):** Dim_Customer[Shipping_City]
  - **Trục Y Cột (Column Values):** _Measures[Total Revenue]
  - **Trục Y Đường (Line Values):** _Measures[Total Shipping Fee]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc nhóm cột (Columns Colors):** #319177.
  - **Màu đường (Line Color):** Đường Phí Ship tô màu #2C6958 (độ đậm nét Stroke Width: **3px**).
  - **Khung & Nền:** Background màu #00201F, Visual Border màu trắng #FFFFFF (**1px**), bo góc **20px**.
  - **Tiêu đề (Title):** *"Tương Quan Doanh Thu & Chi Phí Vận Chuyển Theo Tỉnh/Thành Phố"*.



Hình 4.55. Biểu đồ Tương Quan Doanh Thu & Chi Phí Vận Chuyển Theo Tỉnh/Thành Phố

Biến Động Chi Phí Chiết Khấu & Phí Ship Theo Thời Gian

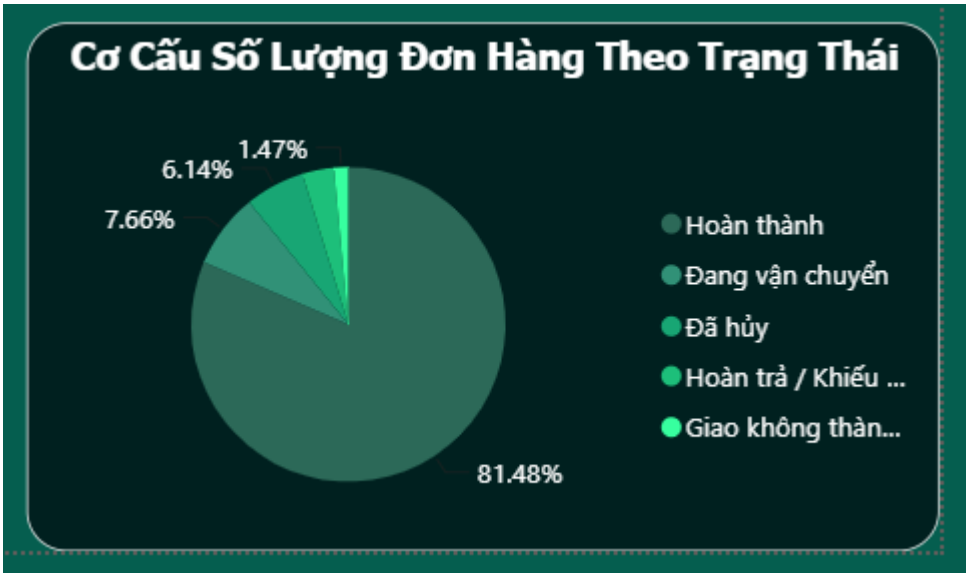
- **Mục tiêu:** Theo dõi biến động liên tục và diện tích tích lũy của ngân sách Khuyến mãi (Total Discount Amount) và Phí vận chuyển (Total Shipping Fee) qua từng tháng.
- **Vị trí đặt biểu đồ:** Bottom-Left
- **Loại biểu đồ (Visual Type):** Area Chart (Biểu đồ vùng).
- **Các bước kéo thả dữ liệu:**
  - **Trục X (X-Axis):** Dim_Date[Month]
  - **Trục Y chính (Y-Axis):** _Measures[Total Shipping Fee]
  - **Trục Y phụ (Secondary Y-Axis):** _Measures[Total Discount Amount]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc các vùng (Area Colors):**
    - Vùng Phí Vận Chuyển (Total Shipping Fee): Tô màu xanh ngọc đậm #2C6958 (độ trong suốt Transparency **60%**).
    - Vùng Chiết khấu (Total Discount Amount): Tô màu xanh dạ quang #37FF9F (độ trong suốt **60%**).
  - **Khung & Nền:** Background màu #00201F, Visual Border màu trắng #FFFFFF (**1px**), bo góc **20px**.
  - **Tiêu đề (Title):** *"Biến Động Chi Phí Chiết Khấu & Phí Ship Theo Thời Gian"*.
  - **Nhãn dữ liệu & Trục (Data Labels & Axis):** Font **Segoe UI**, toàn bộ nhãn số liệu và thông tin trục X/Y màu trắng #FFFFFF.



Hình 4.56. Biểu đồ Area Chart Biến Động Chi Phí Chiết Khấu & Phí Ship Theo Thời Gian

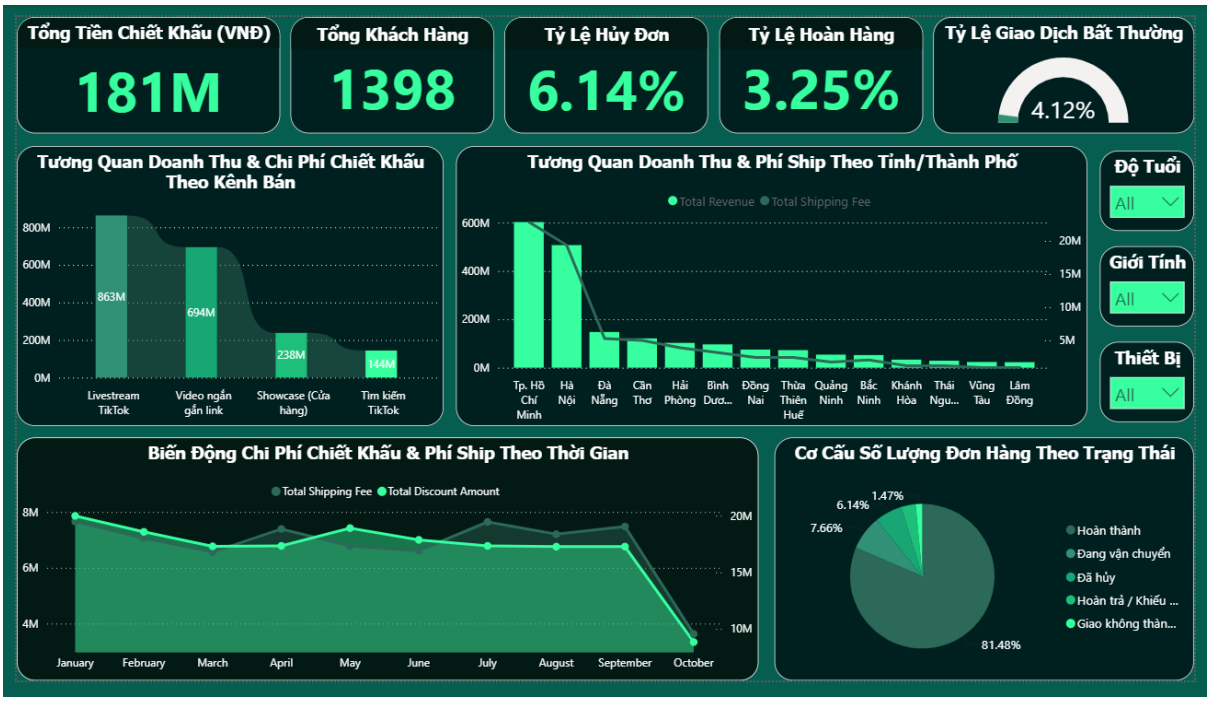
Biểu Đồ Cơ Cấu Số Lượng Đơn Hàng Theo Trạng Thái

- **Mục tiêu:** Trực quan hóa tỷ trọng phần trăm (%) số lượng đơn hàng theo từng Trạng thái (*Hoàn thành, Đang vận chuyển, Đã hủy, Trả hàng/Khiếu nại, Giao không thành công*) để kiểm soát chất lượng vận hành.
- **Vị trí đặt biểu đồ:** Bottom-Right
- **Loại biểu đồ (Visual Type):** Pie Chart.
- **Các bước kéo thả dữ liệu:**
  - **Legend (Chú giải):** Fact_Orders[Order_Status]
  - **Values (Giá trị):** _Measures[Total Orders]
- **Thiết lập định dạng & Màu sắc (Format):**
  - **Màu sắc các lát cắt (Slices Colors):** Phối dải màu theo thứ tự tương ứng các trạng thái:
    - *Hoàn thành / Đã giao:* Tô màu #2C6958
    - *Đang vận chuyển / Đang xử lý:* Tô màu #319177
    - *Đã hủy:* Tô màu #18A674
    - *Trả hàng / Hoàn tiền / Khiếu nại:* Tô màu #1DBF7A
    - *Giao không thành công:* Tô màu #37FF9F
  - **Khung & Nền:** Background màu #00201F, Visual Border màu trắng #FFFFFF (**1px**), bo góc **20px**.
  - **Tiêu đề (Title):** *"Cơ Cấu Số Lượng Đơn Hàng Theo Trạng Thái"*.
  - **Nhãn dữ liệu (Detail Labels):** Bật **On**, vị trí chọn **Outside**, hiển thị Percent of Total. Font **Segoe UI**, cỡ chữ9pt màu trắng #FFFFFF.
  - **Chú giải (Legend):** Font **Segoe UI**, toàn bộ chữ màu trắng #FFFFFF.



Hình 4.57. Biểu đồ Cơ Cấu Số Lượng Đơn Hàng Theo Trạng Thái

Màn hình Giao diện Dashboard hoàn chỉnh



Hình 4.58. Màn hình giao diện Dashboard Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường hoàn chỉnh

Dashboard **Phân tích Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường** cung cấp cái nhìn tổng quan về chi phí khuyến mãi, phí vận chuyển, đặc điểm khách hàng và tình trạng đơn hàng trên TikTok Shop. Thông qua các chỉ số KPI, biểu đồ doanh thu theo kênh bán, phân tích chi phí theo địa lý và thời gian, dashboard hỗ trợ nhà quản lý kiểm soát chi phí vận hành, đánh giá chất lượng đơn hàng và nhận diện những giao dịch có dấu hiệu bất thường.

Bộ thẻ KPI Rủi ro và Khách hàng cho thấy tổng tiền chiết khấu đạt khoảng 181 triệu đồng, tổng số khách hàng là 1.398, tỷ lệ hủy đơn đạt 6,14% và tỷ lệ hoàn hàng là 3,25%. Các chỉ số phản ánh quy mô khách hàng, ngân sách khuyến mãi và tình hình xử lý đơn hàng.

Biểu đồ Tỷ lệ giao dịch bất thường ghi nhận mức 4,12%. Đây là chỉ số hỗ trợ theo dõi tỷ lệ đơn hàng được đánh dấu bất thường trong dữ liệu. Tuy nhiên, để xác định mức độ rủi ro thực tế, doanh nghiệp cần kiểm tra tiêu chí gắn cờ bất thường và đối chiếu với các trường hợp phát sinh cụ thể, thay vì xem toàn bộ giao dịch được đánh dấu là gian lận.

Biểu đồ Tương quan doanh thu và chi phí chiết khấu theo kênh bán cho thấy Livestream TikTok dẫn đầu với doanh thu khoảng 863 triệu đồng, tiếp theo là Video ngắn gắn link với 694 triệu đồng, Showcase đạt 238 triệu đồng và Tìm kiếm TikTok đạt 144 triệu đồng. Như vậy, Livestream và Video ngắn gắn link đóng góp phần lớn doanh thu trong bốn kênh được thể hiện.

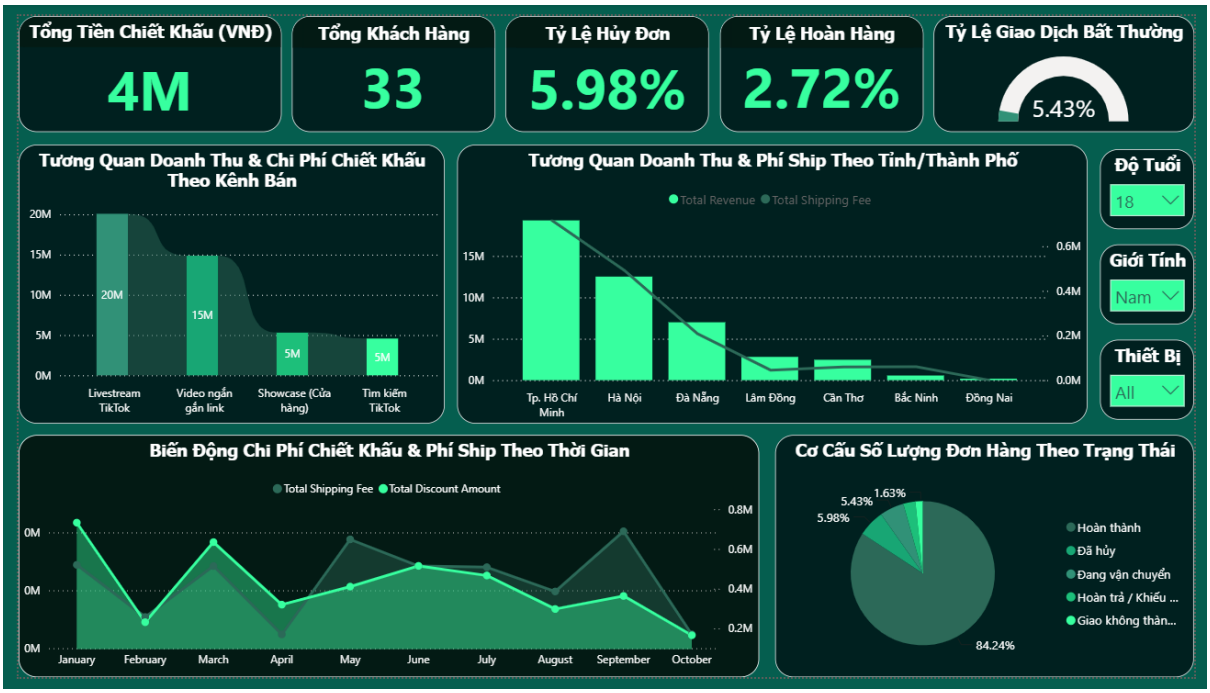
Biểu đồ Tương quan doanh thu và phí vận chuyển theo tỉnh/thành phố cho thấy TP. Hồ Chí Minh có doanh thu cao nhất, khoảng 600 triệu đồng, tiếp theo là Hà Nội với hơn 500 triệu đồng. Các địa phương còn lại có doanh thu thấp hơn đáng kể. Đường biểu diễn phí vận chuyển cũng thay đổi theo địa bàn, phản ánh sự khác biệt về chi phí logistics giữa các thị trường. Doanh nghiệp có thể sử dụng kết quả này để xác định những khu vực cần ưu tiên kiểm soát chi phí vận chuyển.

Biểu đồ Biến động chi phí chiết khấu và phí vận chuyển theo thời gian cho thấy hai khoản chi phí có sự thay đổi qua các tháng từ tháng 1 đến tháng 10. Phí vận chuyển dao động tương đối ổn định trong phần lớn giai đoạn, trong khi chi phí chiết khấu cũng biến động nhưng có xu hướng giảm ở một số tháng. Cả hai chỉ số đều giảm mạnh vào tháng 10. Do dữ liệu tháng 10 chỉ được ghi nhận đến ngày 15/10/2024, mức giảm này chưa thể hiện đầy đủ tình hình của cả tháng. Vì vậy, cần đối chiếu trên cùng khoảng thời gian hoặc chuẩn hóa theo số ngày ghi nhận để đánh giá chính xác xu hướng chi phí.

Biểu đồ Cơ cấu số lượng đơn hàng theo trạng thái cho thấy đơn hàng hoàn thành chiếm tỷ trọng lớn nhất, đạt 81,48%. Tiếp theo là đơn hàng đang vận chuyển với 7,66%, đơn hàng đã hủy chiếm 6,14%, đơn hàng hoàn trả/khiếu nại chiếm khoảng 3,25%

và giao không thành công chiếm 1,47%. Như vậy, phần lớn đơn hàng đã hoàn thành, nhưng tổng tỷ trọng các đơn chưa hoàn thành hoặc gặp vấn đề vẫn cần được theo dõi để hạn chế tổn thất và cải thiện trải nghiệm khách hàng.

Nhìn chung, Dashboard Phân tích Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường hỗ trợ nhà quản lý theo dõi đồng thời ngân sách khuyến mãi, chi phí vận chuyển, hành vi khách hàng và chất lượng đơn hàng. Việc kết hợp các chỉ số này giúp xác định kênh bán chủ lực, theo dõi biến động chi phí và phát hiện những vấn đề cần kiểm tra sâu hơn. Từ đó, doanh nghiệp có cơ sở điều chỉnh chính sách khuyến mãi, tối ưu logistics và nâng cao hiệu quả vận hành trên TikTok Shop.



Hình 4.59. Màn hình giao diện Dashboard Tối ưu Chiết khấu, Hành vi Khách hàng & Cảnh báo Giao dịch Bất thường lọc theo khách hàng nam 18 tuổi

Khi lọc khách hàng ở độ tuổi 18, giới tính nam và không giới hạn thiết bị, Dashboard ghi nhận tổng tiền chiết khấu khoảng 4 triệu đồng từ 33 khách hàng. Tỷ lệ hủy đơn đạt 5,98%, tỷ lệ hoàn hàng là 2,72%, trong khi tỷ lệ giao dịch bất thường lên đến 5,43%. Đáng chú ý, tỷ lệ giao dịch bất thường cao hơn tỷ lệ hoàn hàng và chỉ thấp hơn tỷ lệ hủy đơn 0,55 điểm phần trăm, cho thấy phân khúc này cần được chú ý hơn trong công tác giám sát rủi ro giao dịch.

Xét theo kênh bán, Livestream TikTok đóng góp doanh thu cao nhất, khoảng 20 triệu đồng, tiếp theo là Video ngắn gắn link với 15 triệu đồng. Hai kênh này chiếm phần

lớn doanh thu, cho thấy nội dung video và livestream vẫn là nguồn tạo doanh thu chủ lực của nhóm khách hàng được lọc.

Về địa lý, TP. Hồ Chí Minh dẫn đầu với doanh thu khoảng 19 triệu đồng, tiếp theo là Hà Nội với hơn 12 triệu đồng và Đà Nẵng với khoảng 7 triệu đồng. Phí vận chuyển cũng tập trung nhiều hơn tại các địa phương có doanh thu cao, đặc biệt là TP. Hồ Chí Minh, phản ánh quy mô giao dịch có liên quan đến chi phí logistics phát sinh.

Theo thời gian, chi phí chiết khấu và phí vận chuyển biến động đáng kể giữa các tháng, trong đó tháng 3 ghi nhận chiết khấu tăng trở lại, còn phí vận chuyển nổi bật vào tháng 5 và tháng 9. Đến tháng 10, cả hai khoản chi phí đều giảm mạnh. Điều này cho thấy cần theo dõi hiệu quả sử dụng ngân sách và chi phí giao hàng theo từng giai đoạn.

Về trạng thái đơn hàng, đơn hoàn thành chiếm tỷ trọng lớn nhất với 84,24%; đơn đã hủy chiếm 5,98%, đơn đang vận chuyển chiếm khoảng 5,43%, trong khi giao dịch hoàn trả/khiếu nại và giao không thành công chiếm tỷ lệ thấp hơn.

Nhìn chung, nhóm khách hàng nam 18 tuổi tạo ra doanh thu chủ yếu qua Livestream TikTok và tập trung tại các thành phố lớn, nhưng tỷ lệ giao dịch bất thường tương đối cao so với các chỉ số rủi ro khác, đòi hỏi doanh nghiệp tăng cường xác minh đơn hàng và theo dõi dấu hiệu bất thường trước khi mở rộng hoạt động kích cầu.

CHƯƠNG 5: TRIỂN KHAI CÁC MÔ HÌNH HỌC MÁY (MACHINE LEARNING)

Đường dẫn notebook Google Colab thực thi:

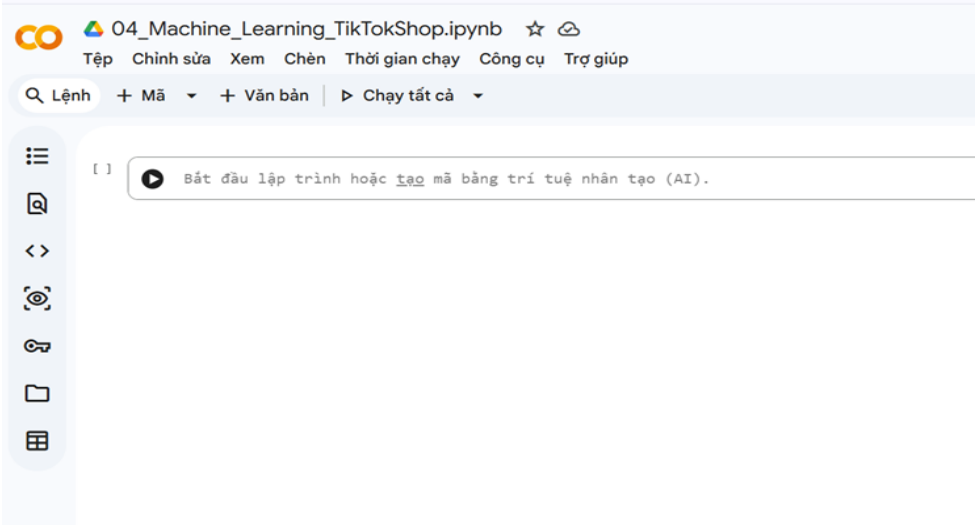
https://colab.research.google.com/drive/1LIgDjy0Bw2N4DavE3zUzhhYYN7dzMh2a?usp=sharing

5.1 MÔ HÌNH HỌC MÁY KHÔNG GIÁM SÁT

5.1.1 Thiết lập môi trường thực nghiệm

Để triển khai các mô hình học máy phục vụ phân tích dữ liệu thương mại điện tử TikTok Shop, nhóm sử dụng Google Colab làm môi trường thực thi chương trình Python. Google Colab cho phép viết và chạy mã nguồn trực tiếp trên trình duyệt, đồng thời hỗ trợ tổ chức chương trình thành các ô lệnh riêng biệt. Cách tổ chức này giúp quá trình xử lý dữ liệu, huấn luyện mô hình và quan sát kết quả được thực hiện tuần tự, thuận tiện cho việc kiểm tra và đánh giá.

Trong phạm vi thực hiện, nhóm tạo notebook với tên 04_Machine_Learning_TikTokShop.ipynb. Notebook được sử dụng để triển khai ba thuật toán chính: K-Means kết hợp bộ chỉ số RFM nhằm phân cụm khách hàng, Random Forest Classifier nhằm dự đoán trạng thái đơn hàng và Isolation Forest nhằm phát hiện các điểm dữ liệu bất thường.



Hình 5.1. Giao diện notebook Machine Learning trên Google Colab

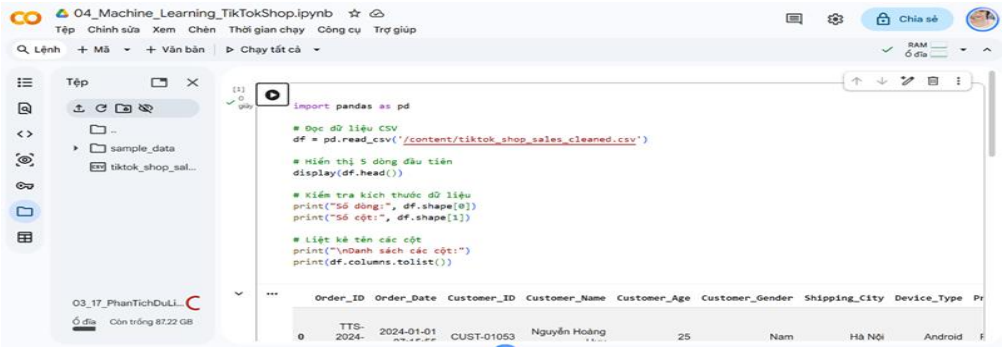
Hình 5A.1 thể hiện môi trường Google Colab được sử dụng để xây dựng và thực thi chương trình học máy. Notebook được đặt tên theo nội dung phân tích, tạo cơ sở để tổ chức các bước xử lý dữ liệu và triển khai mô hình ở những phần tiếp theo.

5.1.2. Nạp dữ liệu đầu vào

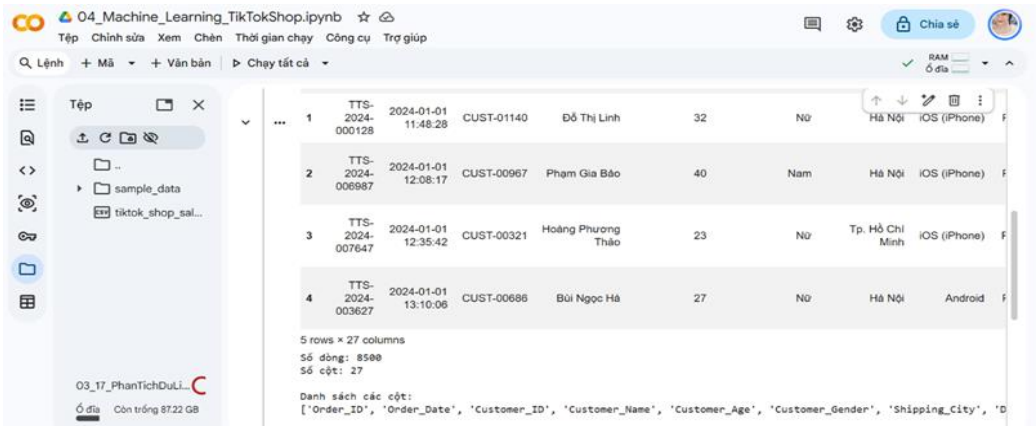
Dữ liệu đầu vào được sử dụng là tập dữ liệu đã qua bước làm sạch, có tên tiktok_shop_sales_cleaned.csv. Tệp dữ liệu được tải lên môi trường Google Colab và đọc bằng thư viện Pandas thông qua hàm read_csv(). Sau khi đọc thành công, dữ liệu được lưu trong biến df dưới dạng DataFrame, cho phép thực hiện các thao tác kiểm tra, thống kê và xử lý bằng Python.

Để xác nhận dữ liệu đã được nạp đúng, chương trình sử dụng hàm head() để hiển thị năm dòng đầu tiên, thuộc tính shape để xác định số dòng và số cột, đồng thời sử dụng columns.tolist() để liệt kê tên các trường dữ liệu.

Kết quả thực nghiệm cho thấy tập dữ liệu được nạp thành công vào Google Colab, gồm **8.500 dòng và 27 cột**. Năm dòng đầu tiên được hiển thị nhằm quan sát cấu trúc bản ghi và một số thông tin ban đầu, chẳng hạn như mã đơn hàng, ngày đặt hàng, mã khách hàng, tên khách hàng, độ tuổi, giới tính, thành phố giao hàng và loại thiết bị.



Hình 5.2. Giao diện Pull Request cập nhật chức năng Backup/Restore và hệ thống thông báo trên GitHub.



Hình 5.3. Kết quả nạp và kiểm tra cấu trúc dữ liệu đầu vào

Hình 5.3 minh họa kết quả thực thi chương trình đọc dữ liệu CSV, hiển thị năm dòng đầu tiên và kiểm tra kích thước tập dữ liệu. Kết quả này xác nhận tệp dữ liệu đã

được đọc thành công và có thể tiếp tục được khảo sát trước khi đưa vào các thuật toán học máy.

5.1.3. Vai trò của bước chuẩn bị dữ liệu

Việc thiết lập môi trường và nạp dữ liệu là bước khởi đầu cần thiết trong quy trình triển khai học máy. Thông qua việc xác nhận kích thước tập dữ liệu và quan sát các bản ghi ban đầu, nhóm có cơ sở kiểm tra tính phù hợp của dữ liệu với mục tiêu phân tích.

Cụ thể, thông tin khách hàng, mã đơn hàng và ngày đặt hàng là cơ sở để xây dựng các chỉ số Recency, Frequency và Monetary trong mô hình K-Means RFM. Các trường liên quan đến trạng thái đơn hàng được sử dụng để xem xét bài toán phân loại bằng Random Forest Classifier. Trong khi đó, các trường thông tin phù hợp cùng nhãn bất thường, nếu có, sẽ được kiểm tra để phục vụ việc triển khai và đánh giá Isolation Forest.

Sau bước chuẩn bị này, dữ liệu cần tiếp tục được khảo sát về kiểu dữ liệu, giá trị thiếu, bản ghi trùng lặp và phân bố nhãn. Kết quả khảo sát sẽ là căn cứ để lựa chọn cách xử lý phù hợp trước khi huấn luyện mô hình, tránh đưa ra kết luận khi chưa kiểm tra đầy đủ chất lượng dữ liệu.

5.1.4. Khảo sát và đánh giá chất lượng dữ liệu đầu vào

- **Kiểm tra kiểu dữ liệu và giá trị thiếu**

Sau khi nạp tập dữ liệu tiktok_shop_sales_cleaned.csv vào Google Colab, chương trình sử dụng thư viện Pandas để kiểm tra kiểu dữ liệu của các trường thông tin thông qua thuộc tính dtypes. Bên cạnh đó, hàm isnull().sum() được sử dụng để thống kê số lượng giá trị thiếu trên từng cột.

Kết quả thực nghiệm cho thấy tổng số ô thiếu được phát hiện là 0. Điều này cho thấy tập dữ liệu không có giá trị khuyết thiếu theo kết quả kiểm tra bằng phương thức isnull(). Do đó, tại bước khảo sát này, chưa cần thực hiện thao tác bổ sung hoặc thay thế giá trị thiếu.

```python
# Kiểm tra kiểu dữ liệu
display(df.dtypes.to_frame(name="Data Type"))

# Kiểm tra giá trị thiếu
missing = df.isnull().sum()
print("Tổng số ô thiếu:", missing.sum())
print(missing[missing > 0])

# Kiểm tra dữ liệu trùng lặp
print("Số dòng trùng lặp:", df.duplicated().sum())

# Thống kê trạng thái đơn hàng
if "Order_Status" in df.columns:
    print(df["Order_Status"].value_counts(dropna=False))

# Thống kê nhãn bất thường
if "Is_Anomaly" in df.columns:
    print(df["Is_Anomaly"].value_counts(dropna=False))

# Kiểm tra các cột phục vụ RFM
for col in ["Customer_ID", "Order_ID", "Order_Date"]:
    print(col, ":", col in df.columns)
```

Hình 5.4. Code khảo sát chất lượng dữ liệu

- **Kiểm tra dữ liệu trùng lặp**

Để đánh giá khả năng xuất hiện các bản ghi trùng lặp, chương trình sử dụng hàm duplicated().sum() nhằm đếm số dòng có nội dung trùng hoàn toàn với những dòng xuất hiện trước đó.

Kết quả kiểm tra ghi nhận 0 dòng trùng lặp hoàn toàn. Điều này cho thấy chương trình không phát hiện bản ghi nào bị lặp lại trên toàn bộ các cột dữ liệu theo tiêu chí kiểm tra đã sử dụng. Tuy nhiên, kết quả này không đồng nghĩa với việc mọi mã đơn hàng đều duy nhất, vì việc kiểm tra trùng toàn bộ dòng khác với việc kiểm tra trùng theo một trường định danh cụ thể.

- **Thống kê trạng thái đơn hàng**

Chương trình sử dụng phương thức value_counts() để thống kê số lượng bản ghi theo từng trạng thái đơn hàng. Kết quả thu được như sau:

Bảng 5.1.Kết quả khảo sát dữ liệu trên Google Colab của nhóm.

Trạng thái đơn hàng	**Số lượng**
Hoàn thành	6.926
Đang vận chuyển	651

Đã hủy	522
Hoàn trả / Khiếu nại	276
Giao không thành công	125
Tổng cộng	**8.500**

N*guồn: Nhóm tự tổng hợp*

Kết quả cho thấy trạng thái Hoàn thành chiếm số lượng lớn nhất với 6.926 bản ghi, trong khi trạng thái Giao không thành công có 125 bản ghi. Tổng số lượng của năm trạng thái bằng 8.500, phù hợp với tổng số dòng của tập dữ liệu ban đầu.

Việc thống kê trạng thái đơn hàng giúp nhận diện các nhóm kết quả giao dịch hiện có, đồng thời cung cấp cơ sở để xem xét bài toán phân loại bằng Random Forest Classifier. Trước khi huấn luyện mô hình, cần xác định rõ những trạng thái nào được sử dụng làm nhãn mục tiêu và cách mã hóa các nhãn này.

- **Kiểm tra nhãn dữ liệu bất thường**

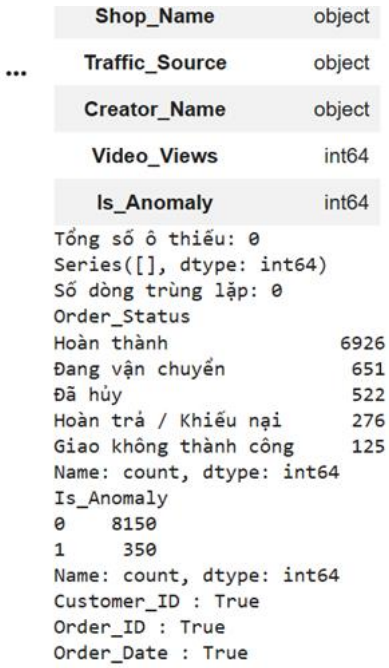
Để xác định phân bố nhãn bất thường có sẵn trong tập dữ liệu, chương trình thống kê số lượng bản ghi theo trường Is_Anomaly. Kết quả cho thấy có 8.150 bản ghi mang nhãn 0 và 350 bản ghi mang nhãn 1.

Tỷ lệ bản ghi được gắn nhãn bất thường được tính theo công thức:

Tỷ lệ bất thường = (Số bản ghi có Is_Anomaly = 1 / Tổng số bản ghi) × 100%.

Thay số liệu thực tế:

Tỷ lệ bất thường = (350 / 8.500) × 100% ≈ 4,12%.



Shop_Name	object
Traffic_Source	object
Creator_Name	object
Video_Views	int64
Is_Anomaly	int64

```
Tổng số ô thiếu: 0
Series([], dtype: int64)
Số dòng trùng lặp: 0
Order_Status
Hoàn thành               6926
Đang vận chuyển           651
Đã hủy                    522
Hoàn trả / Khiếu nại      276
Giao không thành công     125
Name: count, dtype: int64
Is_Anomaly
0    8150
1     350
Name: count, dtype: int64
Customer_ID : True
Order_ID : True
Order_Date : True
```

Hình 5.5. Kết quả khảo sát chất lượng dữ liệu

Như vậy, khoảng 4,12% bản ghi trong tập dữ liệu được gắn nhãn bất thường, trong khi 95,88% còn lại mang nhãn 0. Thông tin này có ý nghĩa đối với quá trình triển khai Isolation Forest và phân tích các giao dịch có dấu hiệu bất thường.

Is_Anomaly là nhãn có sẵn trong dữ liệu. Nhãn này cần được phân biệt với kết quả dự đoán do Isolation Forest tạo ra, bởi thuật toán có thể xác định các điểm bất thường khác với những bản ghi đã được gắn nhãn trước đó.

- **Đánh giá kết quả khảo sát dữ liệu**

Từ các kết quả thực nghiệm, tập dữ liệu có 8.500 dòng và 27 cột, không phát hiện giá trị thiếu hoặc dòng trùng lặp hoàn toàn theo các phép kiểm tra đã thực hiện. Đồng thời, dữ liệu có đầy đủ các nhóm trạng thái đơn hàng được thống kê và trường nhãn Is_Anomaly với 350 bản ghi mang giá trị 1.

Các kết quả trên cho thấy dữ liệu có thể được sử dụng làm cơ sở để tiếp tục xây dựng các mô hình học máy. Tuy nhiên, trước khi huấn luyện, cần kiểm tra thêm tính hợp lệ của các trường dữ liệu, cách xác định nhãn mục tiêu và sự phù hợp của các thuộc tính đầu vào đối với từng thuật toán.

Đối với K-Means RFM, cần sử dụng các trường mã khách hàng, mã đơn hàng, ngày đặt hàng và giá trị chi tiêu phù hợp để tính toán các chỉ số Recency, Frequency và Monetary. Đối với Random Forest Classifier, cần xác định rõ nhóm trạng thái đơn hàng

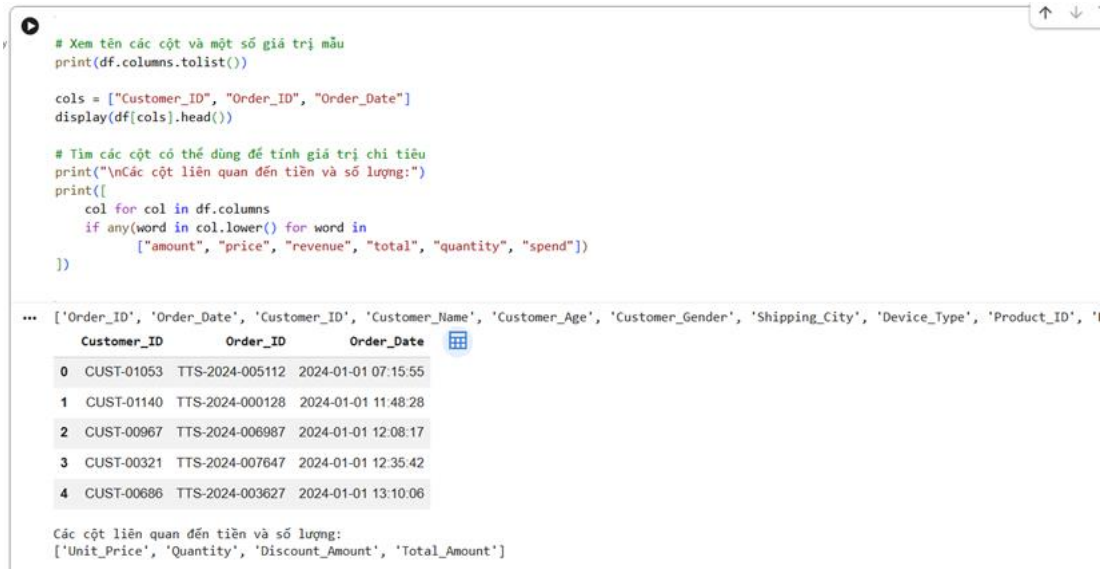
được dự đoán. Đối với Isolation Forest, cần lựa chọn các thuộc tính đầu vào phù hợp và đánh giá kết quả phát hiện bất thường của mô hình độc lập với nhãn có sẵn trong dữ liệu.

5.1.5. Xây dựng bộ chỉ số RFM

- **Xác định các trường dữ liệu phục vụ xây dựng RFM**

Để thực hiện phân cụm khách hàng bằng thuật toán K-Means, nhóm tiến hành xác định các trường dữ liệu cần thiết để xây dựng bộ chỉ số RFM từ tập dữ liệu tiktok_shop_sales_cleaned.csv. Qua quá trình kiểm tra trên Google Colab, nhóm xác định được các trường Customer_ID, Order_ID, Order_Date, Order_Status và Total_Amount.

Trong đó, Customer_ID được sử dụng để xác định khách hàng; Order_ID phục vụ việc đếm số đơn hàng duy nhất; Order_Date được dùng để xác định thời điểm mua hàng; Order_Status hỗ trợ phân loại trạng thái giao dịch; và Total_Amount được sử dụng để tính tổng giá trị đơn hàng của khách hàng.



```
# Xem tên các cột và một số giá trị mẫu
print(df.columns.tolist())

cols = ["Customer_ID", "Order_ID", "Order_Date"]
display(df[cols].head())

# Tìm các cột có thể dùng để tính giá trị chi tiêu
print("\nCác cột liên quan đến tiền và số lượng:")
print([
    col for col in df.columns
    if any(word in col.lower() for word in
           ["amount", "price", "revenue", "total", "quantity", "spend"])
])
```

```
['Order_ID', 'Order_Date', 'Customer_ID', 'Customer_Name', 'Customer_Age', 'Customer_Gender', 'Shipping_City', 'Device_Type', 'Product_ID', '
```

	Customer_ID	Order_ID	Order_Date
0	CUST-01053	TTS-2024-005112	2024-01-01 07:15:55
1	CUST-01140	TTS-2024-000128	2024-01-01 11:48:28
2	CUST-00967	TTS-2024-006987	2024-01-01 12:08:17
3	CUST-00321	TTS-2024-007647	2024-01-01 12:35:42
4	CUST-00686	TTS-2024-003627	2024-01-01 13:10:06

```
Các cột liên quan đến tiền và số lượng:
['Unit_Price', 'Quantity', 'Discount_Amount', 'Total_Amount']
```

Hình 5.6. Xác định các trường dữ liệu phục vụ xây dựng RFM

Ngoài các trường phục vụ RFM, chương trình còn xác định những trường liên quan đến giá trị và số lượng như Unit_Price, Quantity, Discount_Amount và Total_Amount. Trong phạm vi xây dựng RFM, nhóm sử dụng Total_Amount làm cơ sở tính chỉ số Monetary.

- **Kiểm tra giá trị đơn hàng theo trạng thái**

Trước khi tổng hợp dữ liệu theo khách hàng, nhóm kiểm tra sự phân bố giá trị đơn hàng theo từng trạng thái bằng thư viện Pandas. Chương trình sử dụng groupby() để nhóm dữ liệu theo Order_Status, sau đó thống kê số lượng bản ghi, giá trị nhỏ nhất, giá trị trung bình, giá trị lớn nhất và tổng giá trị Total_Amount của từng nhóm.

Kết quả kiểm tra cho thấy dữ liệu có năm trạng thái đơn hàng, với số lượng và tổng giá trị như sau:

```
# Kiểm tra giá trị đơn hàng theo trạng thái
display(
    df.groupby("Order_Status")["Total_Amount"]
    .agg(["count", "min", "mean", "max", "sum"])
    .round(2)
)

# Kiểm tra kiểu dữ liệu và giá trị mẫu
print("Kiểu dữ liệu Total_Amount:", df["Total_Amount"].dtype)
display(df[["Order_ID", "Order_Date", "Order_Status", "Total_Amount"]].head(10))
```

Hình 5.7. Đoạn code thống kê giá trị đơn hàng và kiểm tra dữ liệu bằng Pandas

Order_Status	count	min	mean	max	sum
Giao không thành công	125	35100	254236.00	1596000	31779500
Hoàn thành	6926	19600	227486.66	1755000	1575572600
Hoàn trả / Khiếu nại	276	25200	254475.91	1527000	70235350
Đang vận chuyển	651	22400	219829.88	1512000	143109250
Đã hủy	522	22400	227815.90	1566000	118919900

Kiểu dữ liệu Total_Amount: int64

	Order_ID	Order_Date	Order_Status	Total_Amount
0	TTS-2024-005112	2024-01-01 07:15:55	Hoàn thành	84550
1	TTS-2024-000128	2024-01-01 11:48:28	Hoàn thành	150000
2	TTS-2024-006987	2024-01-01 12:08:17	Hoàn thành	139000
3	TTS-2024-007647	2024-01-01 12:35:42	Hoàn thành	65000
4	TTS-2024-003627	2024-01-01 13:10:06	Hoàn thành	267000
5	TTS-2024-002852	2024-01-01 13:59:39	Hoàn thành	183000
6	TTS-2024-002311	2024-01-01 19:07:52	Hoàn thành	527250
7	TTS-2024-001746	2024-01-01 19:12:48	Hoàn trả / Khiếu nại	432300
8	TTS-2024-001631	2024-01-01 19:23:21	Hoàn thành	117000
9	TTS-2024-006420	2024-01-01 19:27:44	Hoàn thành	163550

Hình 5.8. Thống kê giá trị đơn hàng theo trạng thái

Bên cạnh đó, trường Total_Amount có kiểu dữ liệu int64, cho thấy giá trị đơn hàng đang được lưu dưới dạng số nguyên. Chương trình cũng hiển thị các bản ghi mẫu để đối chiếu giữa mã đơn hàng, ngày đặt hàng, trạng thái và giá trị đơn hàng.

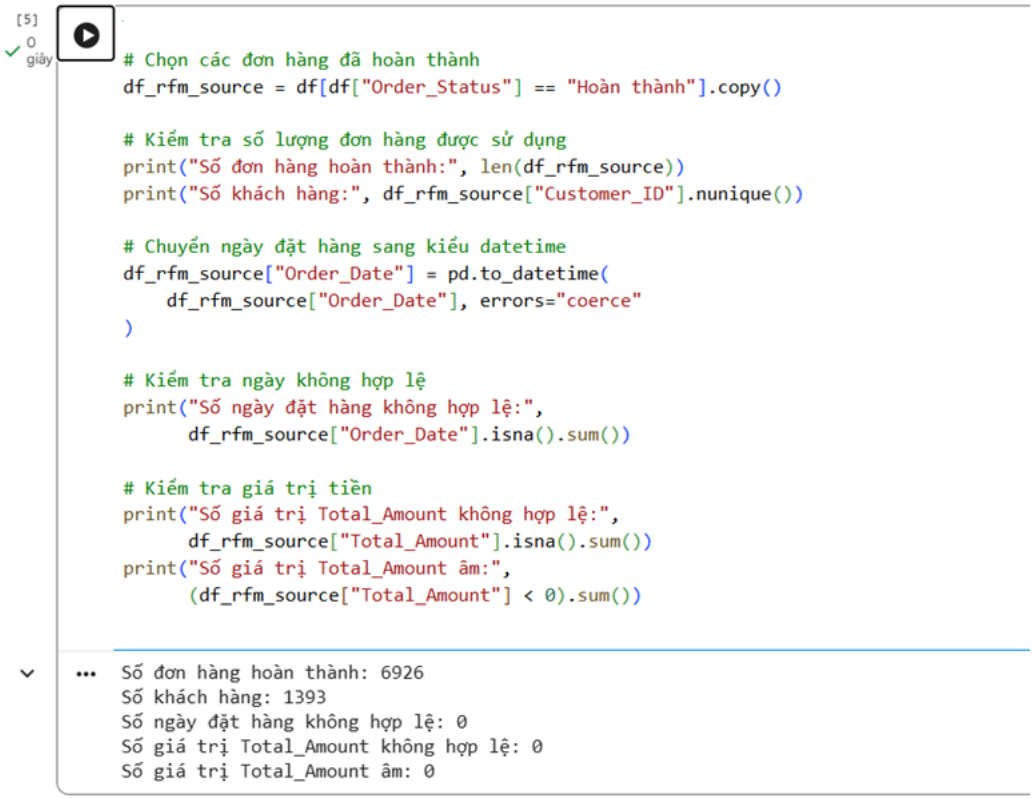
Kết quả thống kê cho thấy trạng thái Hoàn thành có số lượng lớn nhất với 6.926 đơn hàng và tổng giá trị Total_Amount là 1.575.572.600. Các trạng thái còn lại vẫn có

giá trị tiền được ghi nhận trong dữ liệu. Vì vậy, nhóm cần xác định quy tắc lựa chọn trạng thái giao dịch trước khi tính toán các chỉ số RFM, nhằm bảo đảm cách diễn giải kết quả nhất quán với mục tiêu phân tích.

- **Lựa chọn và kiểm tra dữ liệu đơn hàng hoàn thành**

Để tập trung phân tích hành vi mua hàng thông qua các giao dịch đã hoàn thành, nhóm lọc dữ liệu theo điều kiện Order_Status = "Hoàn thành". Sau bước lọc, chương trình kiểm tra số lượng đơn hàng, số khách hàng duy nhất, tính hợp lệ của ngày đặt hàng và các giá trị trong trường Total_Amount.

Kết quả thực nghiệm thu được 6.926 đơn hàng hoàn thành thuộc 1.393 khách hàng duy nhất. Số ngày đặt hàng không hợp lệ là 0; số giá trị Total_Amount bị thiếu là 0; và số giá trị Total_Amount âm là 0.



Hình 5.9. Kết quả lọc và kiểm tra dữ liệu đơn hàng hoàn thành

Các kết quả này cho thấy tập dữ liệu được lựa chọn không có giá trị thiếu trong trường Total_Amount, không phát hiện giá trị tiền âm và không có ngày đặt hàng bị chuyển thành giá trị thiếu sau khi chuẩn hóa kiểu ngày tháng.

Trong phạm vi mô hình RFM, nhóm sử dụng tập dữ liệu đơn hàng hoàn thành làm đầu vào để tổng hợp hành vi khách hàng. Các đơn hàng đang vận chuyển, đã hủy,

hoàn trả/khiếu nại và giao không thành công không được đưa vào phép tính RFM ở bước này. Đây là quy tắc xử lý được lựa chọn nhằm tập trung vào các giao dịch đã hoàn thành, đồng thời cần được duy trì nhất quán khi giải thích kết quả phân cụm.

5.1.6. Tính toán bộ chỉ số RFM theo khách hàng

Sau khi hoàn thành bước kiểm tra và lựa chọn dữ liệu, nhóm tiến hành xây dựng bộ chỉ số RFM nhằm mô tả hành vi mua sắm của khách hàng trên TikTok Shop. Dữ liệu đầu vào gồm 6.926 đơn hàng có trạng thái Hoàn thành, thuộc 1.393 khách hàng duy nhất. Quá trình tổng hợp được thực hiện bằng thư viện Pandas trên Google Colab, sử dụng các trường dữ liệu đã xác định ở bước chuẩn bị.

Bộ chỉ số RFM gồm ba thành phần:

- Recency (R): Số ngày kể từ lần đặt hàng hoàn thành gần nhất của khách hàng đến ngày chốt dữ liệu. Giá trị Recency càng nhỏ cho thấy khách hàng có giao dịch gần thời điểm chốt dữ liệu hơn.
- Frequency (F): Số lượng mã đơn hàng hoàn thành duy nhất của mỗi khách hàng, phản ánh tần suất mua hàng trong tập dữ liệu được lựa chọn.
- Monetary (M): Tổng giá trị Total_Amount của các đơn hàng hoàn thành thuộc mỗi khách hàng, thể hiện tổng giá trị giao dịch đã ghi nhận.

Trong chương trình, dữ liệu được nhóm theo Customer_ID. Ngày đặt hàng gần nhất được sử dụng để tính Recency, số mã Order_ID duy nhất được dùng để tính Frequency và tổng Total_Amount được dùng để tính Monetary. Ngày chốt dữ liệu được xác định bằng ngày đặt hàng muộn nhất trong tập dữ liệu đã lọc cộng thêm một ngày.

Kết quả thu được bảng RFM gồm 1.393 dòng, tương ứng với 1.393 khách hàng. Chương trình không phát hiện giá trị thiếu hoặc mã khách hàng trùng lặp trong bảng tổng hợp. Đây là cơ sở để nhóm tiếp tục chuẩn hóa các thuộc tính và thực hiện phân cụm khách hàng bằng thuật toán K-Means.

```python
# Xác định ngày chốt dữ liệu
snapshot_date = df_rfm_source["Order_Date"].max() + pd.Timedelta(days=1)

# Tính các chỉ số RFM theo khách hàng
rfm = df_rfm_source.groupby("Customer_ID").agg(
    Recency=("Order_Date", lambda x: (snapshot_date - x.max()).days),
    Frequency=("Order_ID", "nunique"),
    Monetary=("Total_Amount", "sum")
).reset_index()

# Hiển thị kết quả
display(rfm.head(10))

print("Số khách hàng:", len(rfm))
print("Số giá trị thiếu:", rfm.isnull().sum().sum())
print("Số dòng trùng:", rfm.duplicated(subset=["Customer_ID"]).sum())
```

Hình 5.10. Đoạn code tính toán các chỉ số RFM và kiểm tra chất lượng dữ liệu khách hàng.

	Customer_ID	Recency	Frequency	Monetary
0	CUST-00001	27	6	1380700
1	CUST-00002	199	2	525250
2	CUST-00003	46	4	1126600
3	CUST-00004	36	7	1234850
4	CUST-00005	98	4	1129550
5	CUST-00006	74	3	787050
6	CUST-00007	26	5	829500
7	CUST-00008	95	2	404500
8	CUST-00009	30	4	891600
9	CUST-00010	26	5	505000

```
Số khách hàng: 1393
Số giá trị thiếu: 0
Số dòng trùng: 0
```

Hình 5.11. Kết quả tổng hợp bộ chỉ số RFM theo khách hàng.

Hình 5.11 minh họa cấu trúc bảng RFM sau khi tổng hợp. Chẳng hạn, khách hàng CUST-00001 có Recency bằng 27 ngày, Frequency bằng 6 đơn và Monetary bằng 1.380.700. Ba giá trị này cung cấp thông tin về thời điểm mua hàng gần nhất, số đơn hàng hoàn thành và tổng giá trị giao dịch của khách hàng trong tập dữ liệu phân tích.

5.1.7. Chuẩn hóa dữ liệu và phân cụm bằng thuật toán K-Means

- **Chuẩn hóa dữ liệu và lựa chọn số cụm**

Trước khi thực hiện phân cụm, nhóm lựa chọn ba thuộc tính Recency, Frequency và Monetary làm đầu vào cho mô hình. Dữ liệu được chuẩn hóa bằng StandardScaler

thuộc thư viện Scikit-learn nhằm đưa các thuộc tính về thang đo tương đồng. Bước này cần thiết vì Monetary có độ lớn khác biệt đáng kể so với Recency và Frequency, có thể ảnh hưởng đến phép tính khoảng cách của thuật toán nếu sử dụng dữ liệu chưa chuẩn hóa.

Tiếp theo, nhóm áp dụng phương pháp Elbow để khảo sát số cụm phù hợp. Chương trình lần lượt chạy K-Means với số cụm từ 1 đến 10 và ghi nhận chỉ số WCSS. Chỉ số này thể hiện tổng bình phương khoảng cách từ các điểm dữ liệu đến tâm cụm gần nhất; WCSS thường giảm khi số cụm tăng.

```python
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Chọn các chỉ số RFM
rfm_features = rfm[["Recency", "Frequency", "Monetary"]]

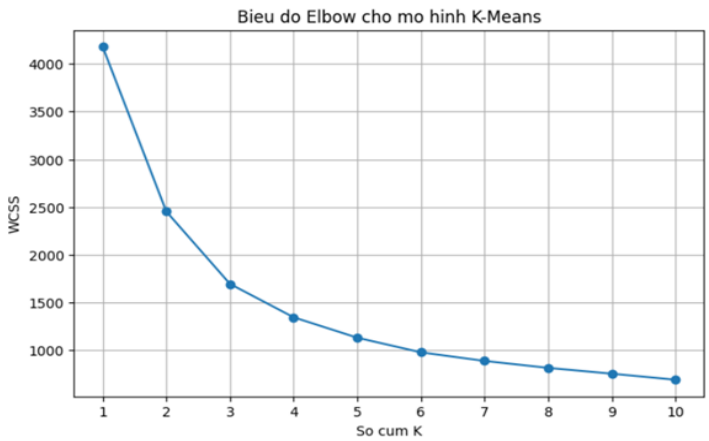
# Chuẩn hóa dữ liệu
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm_features)

# Tính WCSS với các giá trị K khác nhau
wcss = []
k_values = range(1, 11)

for k in k_values:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(rfm_scaled)
    wcss.append(model.inertia_)

# Vẽ biểu đồ Elbow
plt.figure(figsize=(8, 5))
plt.plot(list(k_values), wcss, marker="o")
plt.xlabel("So cum K")
plt.ylabel("WCSS")
plt.title("Bieu do Elbow cho mo hinh K-Means")
plt.xticks(list(k_values))
plt.grid(True)
plt.show()
```

Hình 5.12. Chuẩn hóa dữ liệu RFM bằng StandardScaler.



Hình 5.13. Biểu đồ Elbow khảo sát số cụm K.

Quan sát biểu đồ Elbow cho thấy WCSS giảm mạnh từ K = 1 đến K = 3, sau đó tốc độ giảm chậm dần. Dựa trên hình dạng đường cong, nhóm lựa chọn K = 3 để thực

hiện phân cụm ban đầu. Đây là lựa chọn dựa trên kết quả quan sát biểu đồ và sẽ được xem xét cùng đặc điểm của các cụm sau khi mô hình chạy.

- **Đánh giá số cụm bằng Silhouette Score**

Bên cạnh phương pháp Elbow, nghiên cứu sử dụng Silhouette Score để đánh giá chất lượng phân cụm và hỗ trợ lựa chọn số cụm phù hợp cho thuật toán K-Means. Chỉ số này xem xét mức độ gắn kết của các đối tượng trong cùng một cụm và mức độ phân tách giữa các cụm khác nhau. Giá trị Silhouette Score càng cao thường cho thấy kết quả phân cụm càng tốt.

Trong quá trình thực hiện, dữ liệu RFM của 1.393 khách hàng được chuẩn hóa bằng StandardScaler. Sau đó, thuật toán K-Means được chạy lần lượt với số cụm K từ 2 đến 10, sử dụng random_state = 42 và n_init = 10 để bảo đảm tính nhất quán trong quá trình thử nghiệm. Silhouette Score được tính tương ứng cho từng giá trị K và kết quả được thể hiện qua biểu đồ.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

silhouette_scores = []
k_values = range(2, 11)

for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

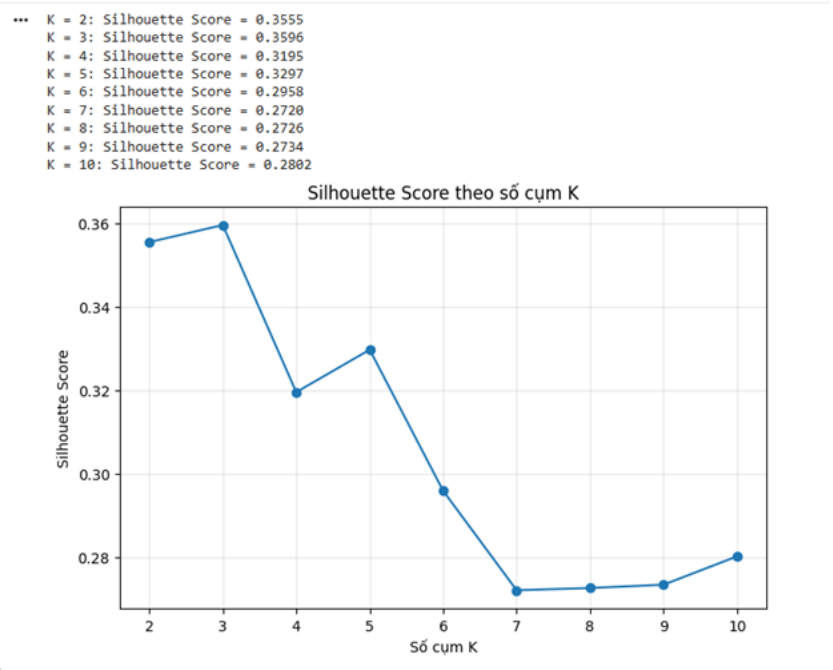
    labels = model.fit_predict(rfm_scaled)
    score = silhouette_score(rfm_scaled, labels)
    silhouette_scores.append(score)

    print(f"K = {k}: Silhouette Score = {score:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(
    list(k_values),
    silhouette_scores,
    marker="o"
)

plt.xlabel("Số cụm K")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score theo số cụm K")
plt.xticks(list(k_values))
plt.grid(True, alpha=0.3)
plt.show()
```

Hình 5.14. Đoạn code sử dụng K-Means và Silhouette Score để xác định số cụm tối ưu



Hình 5.15. Kết quả đánh giá số cụm bằng Silhouette Score

Kết quả cho thấy K = 3 đạt Silhouette Score cao nhất, ở mức 0,3596, tiếp theo là K = 2 với 0,3555. Khi số cụm tăng lên từ 4 đến 10, điểm số nhìn chung giảm và chỉ dao động nhẹ ở một số giá trị. Điều này cho thấy việc tăng số cụm không giúp cải thiện rõ rệt chất lượng phân cụm trên bộ dữ liệu đang xét.

Mặc dù điểm số tại K = 2 khá gần với K = 3, kết quả kết hợp với biểu đồ Elbow và yêu cầu phân tích các nhóm khách hàng cho thấy K = 3 là lựa chọn phù hợp. Phương án này cho phép chia khách hàng thành ba nhóm để tiếp tục so sánh về mức độ gần đây của lần mua hàng, tần suất mua và tổng giá trị giao dịch.

- **Kết quả phân cụm khách hàng**

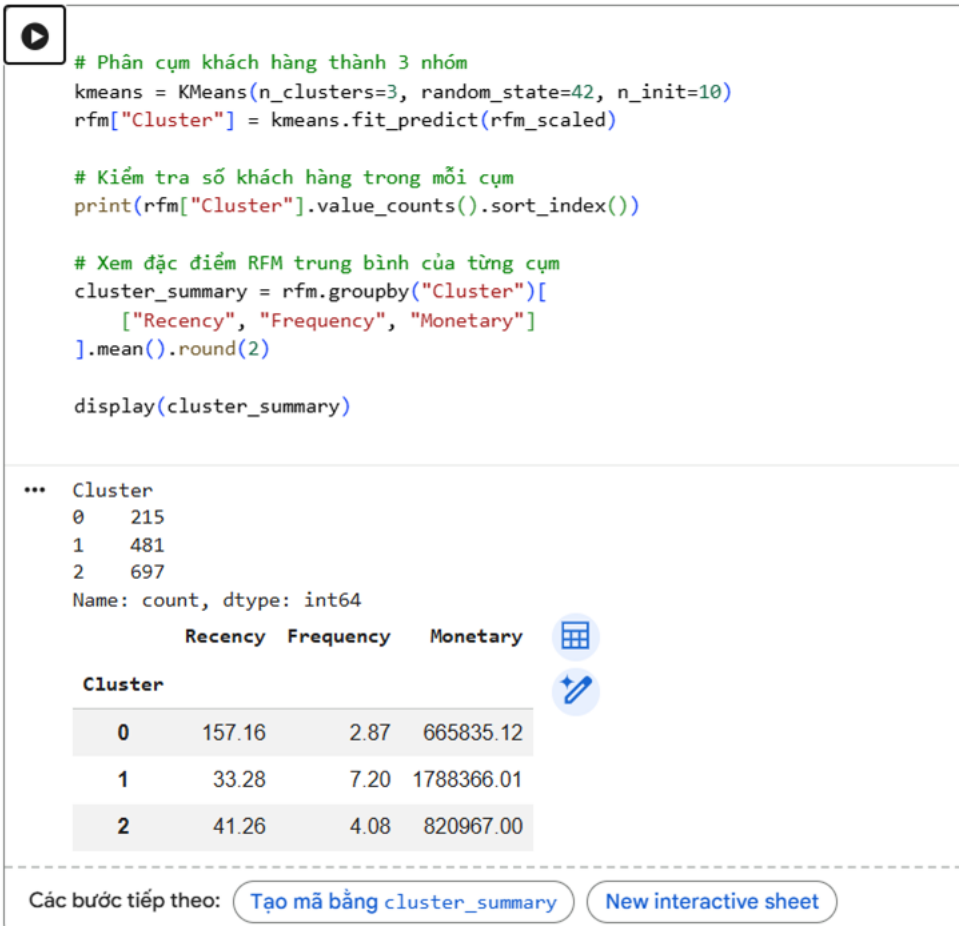
Sau khi xác định số cụm, nhóm sử dụng thuật toán K-Means với n_clusters=3, random_state=42 và n_init=10. Trong đó, n_clusters=3 quy định mô hình tạo ba cụm; random_state=42 hỗ trợ tái lập kết quả khi chạy lại cùng cấu hình; còn n_init=10 cho phép thuật toán thử nhiều lần khởi tạo tâm cụm.

Kết quả cho thấy 1.393 khách hàng được phân thành ba cụm. Quy mô và các chỉ số RFM trung bình của từng cụm được trình bày trong Bảng 5A.1.

Bảng 5.2. Thống kê các chỉ số RFM trung bình theo cụm khách hàng

Cụm	Số khách hàng	Recency trung bình (ngày)	Frequency trung bình (đơn)	Monetary trung bình
Cluster 0	215	157,16	2,87	665.835,12
Cluster 1	481	33,28	7,20	1.788.366,01
Cluster 2	697	41,26	4,08	820.967,00
Tổng	**1.393**	—	—	—

Nguồn: Nhóm tự tổng hợp



Hình 5.16. Kết quả phân cụm K-Means và thống kê RFM trung bình.

Dựa vào Bảng 5.2, Cluster 0 gồm 215 khách hàng, có Recency trung bình cao nhất là 157,16 ngày, Frequency trung bình 2,87 đơn và Monetary trung bình 665.835,12. Cluster 1 gồm 481 khách hàng, có Recency trung bình thấp nhất là 33,28 ngày, đồng

thời đạt Frequency trung bình 7,20 đơn và Monetary trung bình 1.788.366,01 - cao nhất trong ba cụm. Cluster 2 gồm 697 khách hàng, có Recency trung bình 41,26 ngày, Frequency trung bình 4,08 đơn và Monetary trung bình 820.967,00.

Các số liệu cho thấy ba cụm khác nhau về thời điểm mua hàng gần nhất, tần suất mua và tổng giá trị giao dịch. Sự khác biệt này là cơ sở để diễn giải đặc điểm từng nhóm và phục vụ bước trực quan hóa kết quả.

5.1.8. Trực quan hóa và đánh giá kết quả phân cụm

- **Trực quan hóa bằng biểu đồ Scatter Plot**

Để quan sát sự phân bố khách hàng sau khi phân cụm, nhóm xây dựng biểu đồ Scatter Plot với Frequency trên trục hoành và Monetary trên trục tung. Mỗi điểm biểu diễn một khách hàng, màu sắc khác nhau thể hiện ba cụm được tạo bởi K-Means.

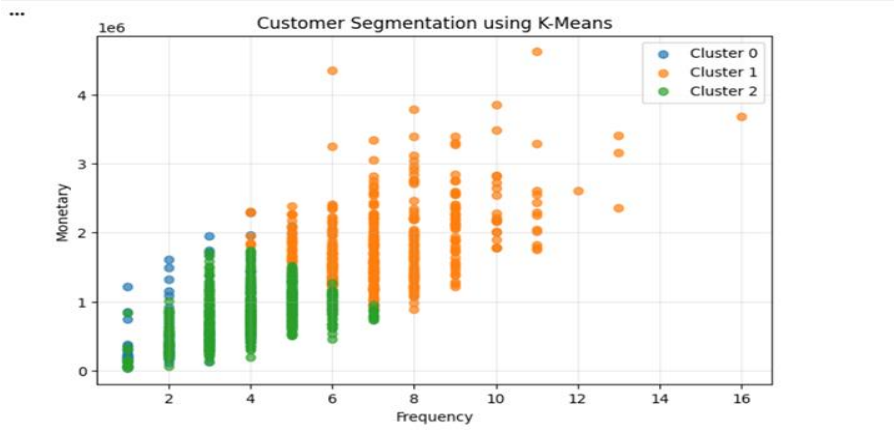
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

for cluster in sorted(rfm["Cluster"].unique()):
    data = rfm[rfm["Cluster"] == cluster]
    plt.scatter(
        data["Frequency"],
        data["Monetary"],
        label=f"Cluster {cluster}",
        alpha=0.7
    )

plt.xlabel("Frequency")
plt.ylabel("Monetary")
plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Hình 5.17. Đoạn mã trực quan hóa kết quả phân cụm khách hàng bằng thuật toán K-Means.



Hình 5.18. Biểu đồ phân cụm khách hàng bằng thuật toán K-Means.

Quan sát biểu đồ cho thấy Cluster 1 có nhiều khách hàng phân bố ở vùng Frequency và Monetary cao hơn. Cluster 2 tập trung chủ yếu ở vùng giá trị thấp đến

trung bình, trong khi Cluster 0 có số lượng khách hàng ít hơn và đặc điểm RFM cho thấy khoảng thời gian kể từ lần mua gần nhất dài hơn.

Một số điểm dữ liệu của các cụm vẫn chồng lấn trên biểu đồ. Điều này cho thấy hai thuộc tính Frequency và Monetary chưa thể hiện toàn bộ sự khác biệt giữa các nhóm. Vì vậy, biểu đồ cần được xem xét kết hợp với Recency và bảng thống kê trung bình của từng cụm.

- **Đánh giá ý nghĩa của các cụm khách hàng**

Dựa trên kết quả RFM, nhóm có thể diễn giải ba cụm theo đặc điểm mua sắm như sau:

- Cluster 0 – Nhóm ít hoạt động gần đây: Có Recency trung bình cao nhất và Frequency, Monetary trung bình thấp nhất. Doanh nghiệp có thể cân nhắc các chương trình nhắc nhớ thương hiệu hoặc khuyến khích mua lại.
- Cluster 1 – Nhóm mua thường xuyên, giá trị cao: Có Frequency và Monetary trung bình cao nhất, đồng thời Recency thấp nhất. Đây là nhóm có thể được ưu tiên trong các hoạt động chăm sóc khách hàng thân thiết và duy trì quan hệ mua sắm.
- Cluster 2 – Nhóm có mức độ mua hàng trung gian: Có tần suất và giá trị chi tiêu trung bình thấp hơn Cluster 1, nhưng hoạt động mua hàng gần đây hơn đáng kể so với Cluster 0. Doanh nghiệp có thể xem xét các chương trình khuyến khích mua hàng tiếp theo hoặc gợi ý sản phẩm phù hợp.

Những tên gọi trên là cách diễn giải từ các chỉ số RFM trung bình, không phải nhãn phân loại có sẵn trong dữ liệu.

Nhìn chung, mô hình K-Means kết hợp RFM đã chia 1.393 khách hàng thành ba cụm với đặc điểm mua sắm khác nhau. Kết quả cung cấp cơ sở tham khảo cho việc phân khúc khách hàng và đề xuất hoạt động tiếp thị phù hợp. Tuy nhiên, K = 3 là lựa chọn thực nghiệm dựa trên biểu đồ Elbow, mức độ phù hợp của mô hình cần được đánh giá trong phạm vi dữ liệu và các thuộc tính đã lựa chọn.

5.2. MÔ HÌNH HỌC MÁY CÓ GIÁM SÁT (SUPERVISED LEARNING)

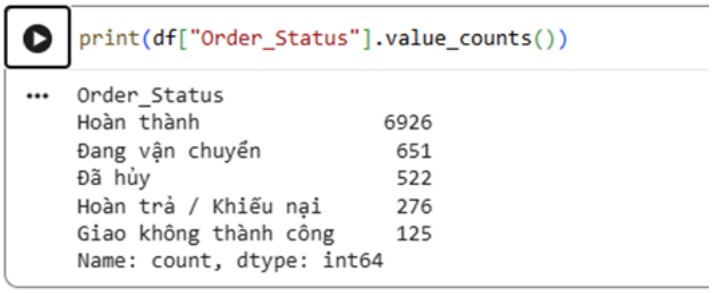
Học máy có giám sát là phương pháp sử dụng dữ liệu đã có nhãn để huấn luyện mô hình dự đoán kết quả cho các dữ liệu mới. Trong đề tài phân tích dữ liệu thương mại điện tử TikTok Shop, nghiên cứu xây dựng bài toán phân loại nhằm dự đoán nguy cơ

hủy đơn hoặc hoàn trả dựa trên các đặc trưng của giao dịch. Quy trình thực hiện gồm xác định biến mục tiêu, lựa chọn biến đầu vào và chia dữ liệu thành tập huấn luyện và tập kiểm tra.

5.2.1. Xác định bài toán phân loại

- **Khảo sát trạng thái đơn hàng ban đầu**

Bộ dữ liệu sử dụng trong nghiên cứu gồm 8.500 đơn hàng với năm trạng thái: Hoàn thành, Đang vận chuyển, Đã hủy, Hoàn trả / Khiếu nại và Giao không thành công. Trước khi xây dựng mô hình phân loại, nghiên cứu kiểm tra số lượng đơn hàng ở từng trạng thái thông qua biến Order_Status.



Hình 5.19. Phân bố trạng thái đơn hàng ban đầu

Kết quả cho thấy có 6.926 đơn hàng Hoàn thành, 651 đơn hàng Đang vận chuyển, 522 đơn hàng Đã hủy, 276 đơn hàng Hoàn trả / Khiếu nại và 125 đơn hàng Giao không thành công. Việc khảo sát phân bố trạng thái ban đầu giúp xác định các nhóm đơn hàng cần được quy đổi thành nhãn cho bài toán phân loại.

- **Mã hóa biến mục tiêu**

Để chuyển bài toán thành phân loại nhị phân, nghiên cứu xây dựng biến mục tiêu (y) từ cột Order_Status theo hai nhóm:

- **Nhãn 0:** Đơn hàng Hoàn thành hoặc Đang vận chuyển.
- **Nhãn 1:** Đơn hàng Đã hủy, Hoàn trả / Khiếu nại hoặc Giao không thành công.

Quá trình mã hóa được thực hiện bằng Python thông qua việc kiểm tra trạng thái của mỗi đơn hàng và chuyển kết quả thành giá trị số 0 hoặc 1.

```
# Mã hóa trạng thái đơn hàng thành biến mục tiêu y
cancel_return_statuses = [
    "Đã hủy",
    "Hoàn trả / Khiếu nại",
    "Giao không thành công"
]

y = df["Order_Status"].isin(cancel_return_statuses).astype(int)

print("Phân bố nhãn mục tiêu y:")
print(y.value_counts().sort_index())

print("\nTỷ lệ phần trăm:")
print((y.value_counts(normalize=True).sort_index() * 100).round(2))
```

```
Phân bố nhãn mục tiêu y:
Order_Status
0    7577
1     923
Name: count, dtype: int64

Tỷ lệ phần trăm:
Order_Status
0    89.14
1    10.86
Name: proportion, dtype: float64
```

Hình 5.20. Mã hóa biến mục tiêu cho bài toán phân loại

Sau khi mã hóa, nhãn 0 có 7.577 đơn hàng, chiếm 89,14% tổng dữ liệu; nhãn 1 có 923 đơn hàng, chiếm 10,86%. Như vậy, số lượng đơn hàng thuộc nhãn 0 lớn hơn đáng kể so với nhãn 1. Đặc điểm này cho thấy dữ liệu có sự mất cân bằng giữa hai nhóm, cần được lưu ý khi đánh giá khả năng nhận diện đơn hàng hủy hoặc gặp sự cố của mô hình.

5.2.2. Xác định biến đầu vào

Nghiên cứu lựa chọn bảy biến đầu vào (X) theo yêu cầu của bài tập, gồm Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Payment_Method, Shipping_City và Traffic_Source.

Trong đó, Unit_Price, Quantity, Discount_Amount và Shipping_Fee là các biến định lượng, phản ánh giá bán, số lượng sản phẩm, mức giảm giá và phí vận chuyển. Ba biến Payment_Method, Shipping_City và Traffic_Source là biến phân loại, lần lượt thể hiện phương thức thanh toán, thành phố giao hàng và nguồn truy cập.

```python
# Xác định các biến đầu vào theo hướng dẫn bài tập
feature_columns = [
    "Unit_Price",
    "Quantity",
    "Discount_Amount",
    "Shipping_Fee",
    "Payment_Method",
    "Shipping_City",
    "Traffic_Source"
]

X = df[feature_columns].copy()

print("Danh sách biến đầu vào:")
print(X.columns.tolist())

print("\nKích thước tập đặc trưng X:", X.shape)
print("\nKiểu dữ liệu của các biến:")
print(X.dtypes)

print("\nSố giá trị thiếu ở từng biến:")
print(X.isnull().sum())
```

```
Danh sách biến đầu vào:
['Unit_Price', 'Quantity', 'Discount_Amount', 'Shipping_Fee', 'Payment_Method', 'Shipping_City', 'Traffic_Source']

Kích thước tập đặc trưng X: (8500, 7)

Kiểu dữ liệu của các biến:
Unit_Price          int64
Quantity            int64
Discount_Amount     int64
Shipping_Fee        int64
Payment_Method     object
Shipping_City      object
Traffic_Source     object
dtype: object

Số giá trị thiếu ở từng biến:
Unit_Price         0
Quantity           0
Discount_Amount    0
Shipping_Fee       0
Payment_Method     0
Shipping_City      0
Traffic_Source     0
dtype: int64
```

Hình 5.21. Xác định và kiểm tra các biến đầu vào

Kết quả kiểm tra cho thấy tập đặc trưng (X) có 8.500 dòng và 7 cột, tương ứng với 8.500 đơn hàng và bảy biến được lựa chọn. Các biến này không có giá trị thiếu trong bộ dữ liệu hiện tại. Tuy nhiên, ba biến phân loại có kiểu dữ liệu chuỗi nên cần được mã hóa phù hợp trước khi đưa vào mô hình Random Forest.

5.2.3. Chia tập dữ liệu huấn luyện và kiểm tra

Để đánh giá khả năng dự đoán của mô hình trên dữ liệu chưa được sử dụng trong quá trình huấn luyện, nghiên cứu chia bộ dữ liệu thành hai tập theo tỷ lệ 80/20. Tập huấn luyện chiếm 80% dữ liệu, gồm 6.800 đơn hàng; tập kiểm tra chiếm 20%, gồm 1.700 đơn hàng.

Quá trình chia tập được thực hiện bằng hàm train_test_split của thư viện Scikit-learn với các tham số test_size=0.2, random_state=42 và stratify=y. Trong đó, test_size=0.2 quy định tỷ lệ dữ liệu kiểm tra; random_state=42 hỗ trợ tái lập kết quả chia dữ liệu; còn stratify=y giúp duy trì tỷ lệ các nhãn mục tiêu tương đối đồng đều giữa hai tập.

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Kích thước tập huấn luyện X_train:", X_train.shape)
print("Kích thước tập kiểm tra X_test:", X_test.shape)
print("Kích thước nhãn y_train:", y_train.shape)
print("Kích thước nhãn y_test:", y_test.shape)

print("\nTỷ lệ nhãn trong tập huấn luyện (%):")
print((y_train.value_counts(normalize=True).sort_index() * 100).round(2))

print("\nTỷ lệ nhãn trong tập kiểm tra (%):")
print((y_test.value_counts(normalize=True).sort_index() * 100).round(2))
```

```
Kích thước tập huấn luyện X_train: (6800, 7)
Kích thước tập kiểm tra X_test: (1700, 7)
Kích thước nhãn y_train: (6800,)
Kích thước nhãn y_test: (1700,)

Tỷ lệ nhãn trong tập huấn luyện (%):
Order_Status
0    89.15
1    10.85
Name: proportion, dtype: float64

Tỷ lệ nhãn trong tập kiểm tra (%):
Order_Status
0    89.12
1    10.88
Name: proportion, dtype: float64
```

Hình 5.22. Kết quả chia dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20

Kết quả thực nghiệm cho thấy nhãn 0 chiếm 89,15% tập huấn luyện và 89,12% tập kiểm tra; nhãn 1 lần lượt chiếm 10,85% và 10,88%. Các tỷ lệ này gần với phân bố nhãn ban đầu, cho thấy việc chia tập đã duy trì tương đối tốt tỷ lệ giữa hai nhóm.

5.2.4. Tiền xử lý dữ liệu trước khi huấn luyện mô hình

Trước khi huấn luyện Random Forest Classifier, nghiên cứu tiến hành mã hóa các biến phân loại để chuyển dữ liệu về dạng phù hợp với thuật toán học máy. Trong bảy biến đầu vào đã lựa chọn, ba biến Payment_Method, Shipping_City và Traffic_Source có kiểu dữ liệu chuỗi, do đó không thể đưa trực tiếp vào mô hình Random Forest của thư viện Scikit-learn.

Nghiên cứu sử dụng phương pháp One-Hot Encoding thông qua lớp OneHotEncoder kết hợp với ColumnTransformer. Phương pháp này chuyển các giá trị phân loại thành những cột biểu diễn bằng số, trong khi bốn biến định lượng Unit_Price, Quantity, Discount_Amount và Shipping_Fee được giữ nguyên.

Quá trình mã hóa được thực hiện bằng cách khớp bộ tiền xử lý trên tập huấn luyện X_train, sau đó áp dụng bộ tiền xử lý đã khớp để chuyển đổi tập kiểm tra X_test. Cách thực hiện này giúp tránh sử dụng thông tin từ tập kiểm tra trong quá trình học các giá trị mã hóa.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

categorical_features = [
    "Payment_Method",
    "Shipping_City",
    "Traffic_Source"
]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        )
    ],
    remainder="passthrough"
)

X_train_encoded = preprocessor.fit_transform(X_train)
X_test_encoded = preprocessor.transform(X_test)

print("Kích thước X_train sau mã hóa:", X_train_encoded.shape)
print("Kích thước X_test sau mã hóa:", X_test_encoded.shape)
print("Số nhãn y_train:", len(y_train))
print("Số nhãn y_test:", len(y_test))
```

```
Kích thước X_train sau mã hóa: (6800, 26)
Kích thước X_test sau mã hóa: (1700, 26)
Số nhãn y_train: 6800
Số nhãn y_test: 1700
```

Hình 5.23. Mã hóa các biến phân loại bằng One-Hot Encoding

Kết quả thực nghiệm cho thấy tập huấn luyện sau mã hóa có kích thước 6.800 dòng và 26 cột, trong khi tập kiểm tra có kích thước 1.700 dòng và 26 cột. Như vậy, dữ liệu đầu vào đã được chuyển đổi thành công, đồng thời số lượng nhãn mục tiêu vẫn tương ứng với số dòng của từng tập dữ liệu.

5.2.5. Huấn luyện mô hình Random Forest Classifier

Sau khi hoàn thành bước tiền xử lý, nghiên cứu sử dụng thuật toán Random Forest Classifier để dự đoán nhóm trạng thái đơn hàng dựa trên các đặc trưng giao dịch. Đây là mô hình học máy có giám sát, được huấn luyện bằng tập dữ liệu đã có nhãn mục tiêu y_train.

Mô hình được khởi tạo với tham số n_estimators=100 và random_state=42. Trong đó, n_estimators=100 quy định mô hình sử dụng 100 cây quyết định; random_state=42 giúp kết quả có thể được tái lập khi thực hiện lại quá trình huấn luyện với cùng dữ liệu và cấu hình.

Quá trình huấn luyện sử dụng tập đầu vào X_train_encoded và nhãn tương ứng y_train. Sau khi huấn luyện hoàn tất, mô hình có thể dự đoán nhãn cho các giao dịch trong tập kiểm tra.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train_encoded, y_train)

print("Huấn luyện Random Forest hoàn tất.")
print("Số cây trong mô hình:", len(rf_model.estimators_))
print("Số đặc trưng đầu vào:", rf_model.n_features_in_)
```

```
Huấn luyện Random Forest hoàn tất.
Số cây trong mô hình: 100
Số đặc trưng đầu vào: 26
```

Hình 5.24. Huấn luyện mô hình Random Forest Classifier

Kết quả cho thấy mô hình đã được huấn luyện thành công với 100 cây quyết định và 26 đặc trưng đầu vào sau mã hóa. Mô hình này được sử dụng ở bước tiếp theo để dự đoán trạng thái của 1.700 đơn hàng trong tập kiểm tra và đánh giá hiệu quả phân loại.

5.2.6. Đánh giá mô hình Random Forest bằng Confusion Matrix

- **Kết quả dự đoán và ma trận nhầm lẫn**

Sau khi huấn luyện, mô hình Random Forest được sử dụng để dự đoán nhãn trên tập kiểm tra thông qua phương thức predict(). Kết quả dự đoán được so sánh với nhãn thực tế bằng Confusion Matrix nhằm xác định số trường hợp phân loại đúng và sai ở từng nhóm.

```python
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report
)
import matplotlib.pyplot as plt

y_pred = rf_model.predict(X_test_encoded)

cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

print("Confusion Matrix:")
print(cm)

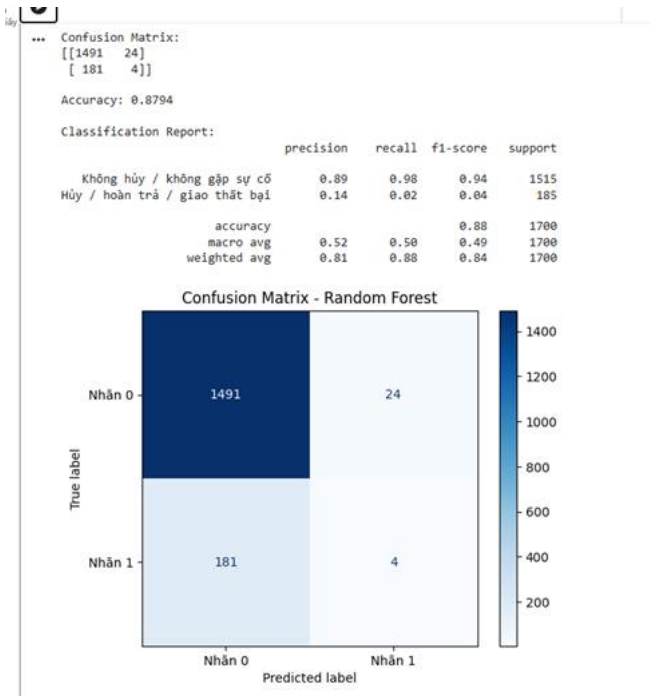
print("\nAccuracy:", round(accuracy_score(y_test, y_pred), 4))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1],
    target_names=["Không hủy / không gặp sự cố", "Hủy / hoàn trả / giao thất bại"],
    zero_division=0
))

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Nhãn 0", "Nhãn 1"]
)

display.plot(cmap="Blues", values_format="d")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

Hình 5.25. Đoạn mã đánh giá mô hình Random Forest bằng Confusion Matrix và các chỉ số phân loại.



Hình 5.26. Ma trận nhầm lẫn và các chỉ số đánh giá mô hình Random Forest

Ma trận nhầm lẫn thu được như sau:

Bảng 5.3. Ma trận nhầm lẫn cho Random Forest

Nhãn thực tế	Dự đoán nhãn 0	Dự đoán nhãn 1
Nhãn 0 – Không hủy/không gặp sự cố	1.491	24
Nhãn 1 – Hủy/hoàn trả/giao thất bại	181	4

Kết quả cho thấy mô hình dự đoán đúng 1.491 đơn hàng thuộc nhãn 0 và 4 đơn hàng thuộc nhãn 1. Bên cạnh đó, có 24 đơn hàng thuộc nhãn 0 bị dự đoán nhầm thành nhãn 1 và 181 đơn hàng thuộc nhãn 1 bị dự đoán nhầm thành nhãn 0.

Độ chính xác tổng thể (Accuracy) của mô hình đạt 87,94%. Tuy nhiên, nhãn 0 chiếm tỷ trọng lớn trong bộ dữ liệu, trong khi nhãn 1 chỉ chiếm khoảng 10,86%. Vì vậy, Accuracy chưa phản ánh đầy đủ khả năng nhận diện các đơn hàng hủy, hoàn trả hoặc giao không thành công.

- **Phân tích các chỉ số đánh giá**

Ngoài Accuracy, nghiên cứu xem xét Precision, Recall và F1-score để đánh giá khả năng phân loại của mô hình ở từng nhóm.

Đối với nhãn 0, mô hình đạt Precision khoảng 0,89, Recall khoảng 0,98 và F1-score khoảng 0,94. Kết quả này cho thấy mô hình nhận diện tương đối tốt nhóm đơn hàng không thuộc diện hủy hoặc gặp sự cố.

Ngược lại, đối với nhãn 1, Precision đạt khoảng 0,14, Recall khoảng 0,02 và F1-score khoảng 0,04. Recall thấp cho thấy mô hình chỉ phát hiện đúng 4 trong tổng số 185 đơn hàng thực tế thuộc nhóm nhãn 1 trên tập kiểm tra. Phần lớn các trường hợp hủy, hoàn trả hoặc giao không thành công vẫn bị mô hình phân loại thành nhãn 0.

Như vậy, mặc dù Accuracy đạt 87,94%, mô hình vẫn còn hạn chế đáng kể trong việc nhận diện nhóm đơn hàng có vấn đề. Đối với mục tiêu dự đoán nguy cơ hủy đơn hoặc hoàn trả, khả năng phát hiện đúng nhãn 1 là yếu tố quan trọng và cần được xem xét cùng với độ chính xác tổng thể.

- **Thử nghiệm điều chỉnh trọng số lớp**

Để kiểm tra khả năng cải thiện việc nhận diện nhóm nhãn thiểu số, nghiên cứu thực hiện thêm một thử nghiệm với tham số class_weight="balanced" trong Random Forest. Cấu hình này điều chỉnh trọng số các lớp trong quá trình huấn luyện nhằm giảm ảnh hưởng của sự chênh lệch số lượng giữa các nhãn.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

rf_balanced = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

rf_balanced.fit(X_train_encoded, y_train)

y_pred_balanced = rf_balanced.predict(X_test_encoded)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_balanced, labels=[0, 1]))

print("\nAccuracy:", round(
    accuracy_score(y_test, y_pred_balanced), 4
))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred_balanced,
    labels=[0, 1],
    target_names=["Nhãn 0", "Nhãn 1"],
    zero_division=0
))
```

Hình 5.27. Đoạn mã thử nghiệm mô hình Random Forest với tham số class_weight="balanced" nhằm cải thiện khả năng phân loại dữ liệu mất cân bằng.

```
Confusion Matrix:
[[1469   46]
 [ 181    4]]

Accuracy: 0.8665

Classification Report:
              precision    recall  f1-score   support

      Nhãn 0       0.89      0.97      0.93      1515
      Nhãn 1       0.08      0.02      0.03       185

    accuracy                           0.87      1700
   macro avg       0.49      0.50      0.48      1700
weighted avg       0.80      0.87      0.83      1700
```

Hình 5.28. Kết quả thử nghiệm Random Forest với trọng số lớp cân bằng

Kết quả thử nghiệm cho thấy Accuracy giảm từ 87,94% xuống 86,65%. Số đơn hàng nhãn 1 được dự đoán đúng vẫn là 4 trên tổng số 185 đơn; Recall của nhãn 1 tiếp tục ở mức khoảng 0,02, trong khi Precision giảm từ khoảng 0,14 xuống 0,08.

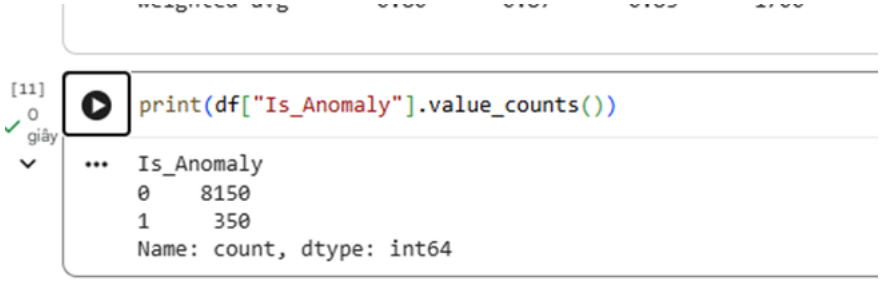
Như vậy, việc sử dụng class_weight="balanced" trong cấu hình thử nghiệm này chưa cải thiện khả năng nhận diện nhóm đơn hàng hủy, hoàn trả hoặc giao không thành công. Kết quả cho thấy cần tiếp tục xem xét các phương pháp lựa chọn đặc trưng, điều chỉnh tham số hoặc xử lý mất cân bằng dữ liệu nếu muốn nâng cao hiệu quả dự đoán nhóm nhãn 1.

5.2.7. Phát hiện giao dịch bất thường bằng Isolation Forest

- **Khảo sát nhãn bất thường ban đầu**

Bên cạnh bài toán phân loại trạng thái đơn hàng, nghiên cứu sử dụng Isolation Forest để phát hiện những giao dịch có đặc điểm khác biệt so với phần lớn dữ liệu. Trước khi huấn luyện mô hình, cột Is_Anomaly có sẵn trong bộ dữ liệu được kiểm tra để xác định phân bố nhãn bất thường ban đầu.

Kết quả cho thấy có 8.150 giao dịch mang nhãn 0 và 350 giao dịch mang nhãn 1, tương ứng khoảng 4,12% tổng số 8.500 giao dịch.



Hình 5.29. Phân bố nhãn bất thường trong dữ liệu ban đầu

- **Huấn luyện Isolation Forest**

Nghiên cứu lựa chọn bảy đặc trưng giao dịch gồm Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Payment_Method, Shipping_City và Traffic_Source. Ba biến phân loại được mã hóa bằng One-Hot Encoding trước khi đưa vào mô hình.

Isolation Forest được khởi tạo với contamination=350/8500 và random_state=42. Tham số contamination được thiết lập theo tỷ lệ nhãn bất thường có sẵn trong dữ liệu, khoảng 4,12%, để quy định tỷ lệ bất thường dự kiến trong quá trình phân loại.

Sau khi mô hình dự đoán, kết quả được chuyển thành cột IF_Anomaly, trong đó giá trị 1 biểu thị giao dịch do mô hình đánh dấu là bất thường, còn giá trị 0 biểu thị giao dịch được phân loại là bình thường.

Kết quả cho thấy mô hình đánh dấu 350 giao dịch là bất thường và 8.150 giao dịch là bình thường. Tuy nhiên, số lượng trùng với phân bố nhãn gốc không đồng nghĩa với việc các giao dịch được phát hiện trùng hoàn toàn với những giao dịch có Is_Anomaly = 1.

- **Trực quan hóa giao dịch bất thường**

Để quan sát kết quả phát hiện, nghiên cứu sử dụng biểu đồ phân tán với Quantity trên trục hoành và Total_Amount trên trục tung. Các giao dịch được mô hình phân loại là bình thường được hiển thị bằng màu xanh, trong khi các giao dịch được đánh dấu bất thường được hiển thị bằng màu đỏ.

```
import matplotlib.pyplot as plt

# Chọn hai đặc trưng để trực quan hóa giao dịch
plt.figure(figsize=(9, 6))

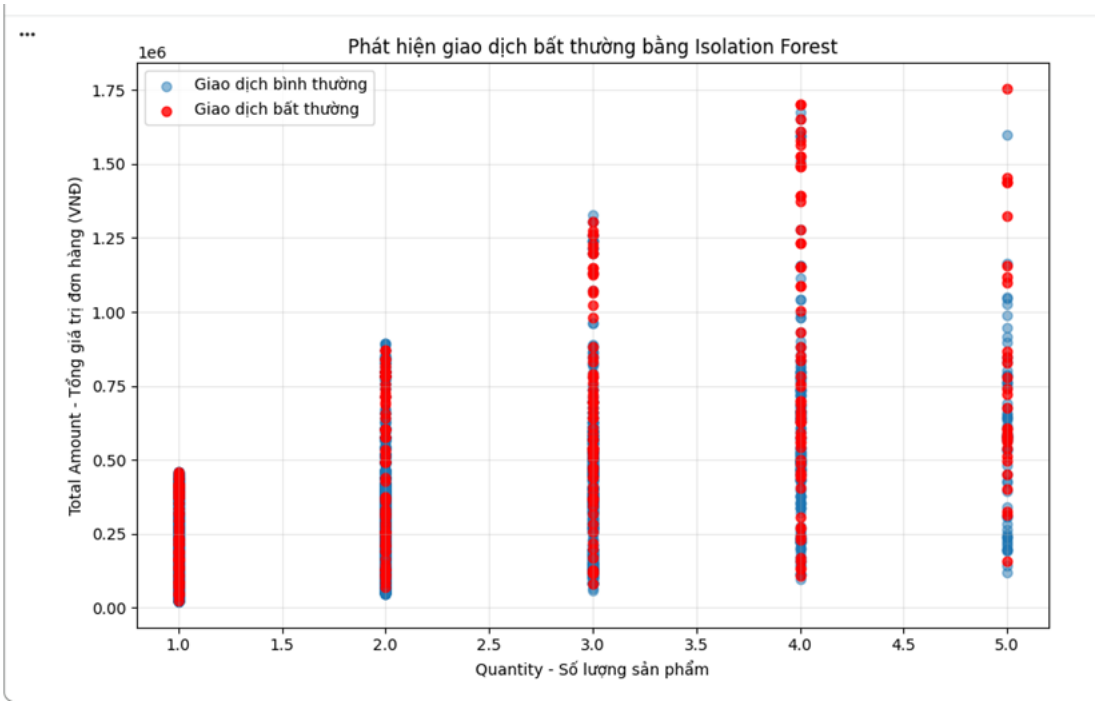
normal_data = df[df["IF_Anomaly"] == 0]
anomaly_data = df[df["IF_Anomaly"] == 1]

plt.scatter(
    normal_data["Quantity"],
    normal_data["Total_Amount"],
    label="Giao dịch bình thường",
    alpha=0.5
)

plt.scatter(
    anomaly_data["Quantity"],
    anomaly_data["Total_Amount"],
    color="red",
    label="Giao dịch bất thường",
    alpha=0.8
)

plt.xlabel("Quantity - Số lượng sản phẩm")
plt.ylabel("Total Amount - Tổng giá trị đơn hàng (VNĐ)")
plt.title("Phát hiện giao dịch bất thường bằng Isolation Forest")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Hình 5.30. Đoạn mã trực quan hóa các giao dịch bất thường bằng thuật toán Isolation Forest.

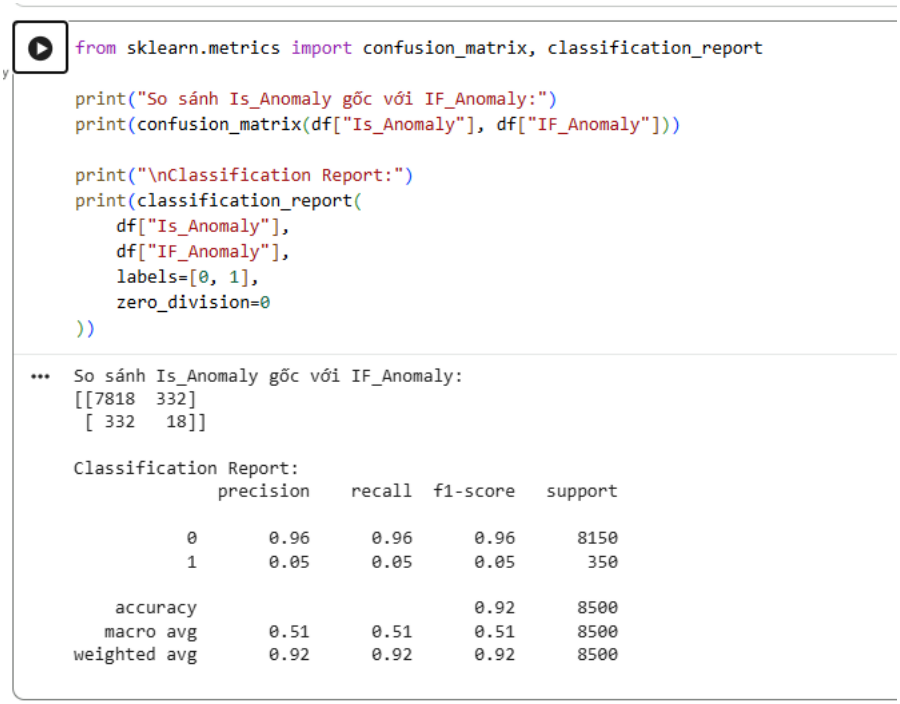


Hình 5.31. Trực quan hóa giao dịch bất thường bằng Isolation Forest

Biểu đồ cho thấy các điểm bất thường xuất hiện ở nhiều mức số lượng sản phẩm và giá trị đơn hàng khác nhau. Điều này cho thấy các giao dịch được mô hình đánh dấu không chỉ tập trung ở những đơn hàng có tổng giá trị lớn nhất. Tuy nhiên, biểu đồ chỉ thể hiện hai đặc trưng dùng để trực quan hóa, trong khi mô hình được huấn luyện trên nhiều đặc trưng hơn; do đó, những điểm nằm gần nhau trên biểu đồ vẫn có thể được phân loại khác nhau.

- **Đối chiếu kết quả Isolation Forest với nhãn gốc**

Để đánh giá mức độ tương đồng giữa kết quả dự đoán và nhãn bất thường có sẵn, nghiên cứu so sánh IF_Anomaly với Is_Anomaly bằng Confusion Matrix và các chỉ số Precision, Recall, F1-score.



```
from sklearn.metrics import confusion_matrix, classification_report

print("So sánh Is_Anomaly gốc với IF_Anomaly:")
print(confusion_matrix(df["Is_Anomaly"], df["IF_Anomaly"]))

print("\nClassification Report:")
print(classification_report(
    df["Is_Anomaly"],
    df["IF_Anomaly"],
    labels=[0, 1],
    zero_division=0
))
```

```
So sánh Is_Anomaly gốc với IF_Anomaly:
[[7818  332]
 [ 332   18]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.96      0.96      8150
           1       0.05      0.05      0.05       350

    accuracy                           0.92      8500
   macro avg       0.51      0.51      0.51      8500
weighted avg       0.92      0.92      0.92      8500
```

Hình 5.32. Đối chiếu kết quả Isolation Forest với nhãn bất thường gốc

Ma trận nhầm lẫn thu được như sau:

Bảng 5.4. Ma trận nhầm lẫn cho Isolation Forest

Nhãn gốc	**Dự đoán bình thường (0)**	**Dự đoán bất thường (1)**
Nhãn 0 – Bình thường	7.818	332
Nhãn 1 – Bất thường	332	18

Nguồn: Nhóm tự tổng hợp

Kết quả cho thấy mô hình nhận diện đúng 7.818 giao dịch thuộc nhãn bình thường và 18 giao dịch thuộc nhãn bất thường theo dữ liệu gốc. Trong khi đó, 332 giao dịch mang nhãn bất thường gốc bị mô hình phân loại thành bình thường; đồng thời, 332 giao dịch mang nhãn bình thường gốc bị mô hình đánh dấu là bất thường.

Accuracy của mô hình đạt khoảng 92%, nhưng Precision, Recall và F1-score của nhãn 1 đều chỉ khoảng 0,05. Điều này cho thấy khả năng nhận diện các giao dịch bất thường theo nhãn gốc còn hạn chế. Vì vậy, Accuracy tổng thể không đủ để khẳng định mô hình phát hiện bất thường hiệu quả.

Is_Anomaly là nhãn có sẵn trong bộ dữ liệu, còn IF_Anomaly là kết quả do Isolation Forest dự đoán. Việc đối chiếu hai cột giúp đánh giá mức độ trùng khớp giữa

mô hình và nhãn gốc, nhưng không tự chứng minh nhãn gốc phản ánh chính xác mọi trường hợp bất thường trong thực tế.

Nhận xét chung

Qua quá trình thực nghiệm, Random Forest đã được huấn luyện và đánh giá bằng Confusion Matrix, trong khi Isolation Forest được sử dụng để phát hiện và trực quan hóa các giao dịch bất thường. Các kết quả cho thấy hai mô hình đều chạy thành công về mặt kỹ thuật, nhưng khả năng nhận diện nhóm nhãn 1 còn thấp. Do đó, nghiên cứu không chỉ xem xét Accuracy mà còn phân tích Precision, Recall và F1-score để đánh giá toàn diện hơn những hạn chế của các mô hình trên bộ dữ liệu hiện tại.

5.2.8. Bài toán hồi quy (Regression): Dự báo giá trị đơn hàng (Total_Amount)

- **Xác định biến đầu vào và biến mục tiêu**

Bài toán hồi quy được xây dựng nhằm dự đoán giá trị đơn hàng (Total_Amount) dựa trên các đặc trưng liên quan đến sản phẩm và giao dịch trong bộ dữ liệu TikTok Shop. Khác với bài toán phân loại ở mục trước, mô hình hồi quy dự đoán một giá trị số liên tục thay vì xác định đơn hàng thuộc một nhóm nhãn.

Nghiên cứu lựa chọn bảy biến đầu vào gồm Unit_Price, Quantity, Discount_Amount, Shipping_Fee, Video_Views, Product_Rating và Review_Count. Các biến này phản ánh giá bán sản phẩm, số lượng mua, mức giảm giá, phí vận chuyển, lượt xem video, đánh giá sản phẩm và số lượng đánh giá.

Biến mục tiêu được xác định là Total_Amount, đại diện cho tổng giá trị đơn hàng cần dự đoán. Tập biến đầu vào được ký hiệu là (X), còn biến mục tiêu được ký hiệu là (y).

Trước khi xây dựng mô hình, nghiên cứu tiến hành kiểm tra sự tồn tại của các cột dữ liệu, kiểu dữ liệu và giá trị thiếu. Kết quả cho thấy cả bảy biến đầu vào và biến mục tiêu đều tồn tại trong bộ dữ liệu. Các biến được lựa chọn không có giá trị thiếu, bảy biến đầu vào có kiểu dữ liệu số nguyên hoặc số thực, trong khi Total_Amount có kiểu dữ liệu số nguyên. Bộ dữ liệu tại thời điểm kiểm tra gồm 8.500 dòng và 28 cột.

```python
regression_features = [
    "Unit_Price",
    "Quantity",
    "Discount_Amount",
    "Shipping_Fee",
    "Video_Views",
    "Product_Rating",
    "Review_Count"
]

regression_target = "Total_Amount"

required_columns = regression_features + [regression_target]

print("Các cột còn thiếu:")
print([
    col for col in required_columns
    if col not in df.columns
])

print("\nKích thước dữ liệu:", df.shape)

print("\nKiểu dữ liệu:")
print(df[required_columns].dtypes)

print("\nSố giá trị thiếu:")
print(df[required_columns].isnull().sum())
```

Hình 5.33. Đoạn mã kiểm tra các biến đầu vào, biến mục tiêu và chất lượng dữ liệu cho bài toán hồi quy.

```
Các cột còn thiếu:
[]

Kích thước dữ liệu: (8500, 28)

Kiểu dữ liệu:
Unit_Price           int64
Quantity             int64
Discount_Amount      int64
Shipping_Fee         int64
Video_Views          int64
Product_Rating     float64
Review_Count         int64
Total_Amount         int64
dtype: object

Số giá trị thiếu:
Unit_Price         0
Quantity           0
Discount_Amount    0
Shipping_Fee       0
Video_Views        0
Product_Rating     0
Review_Count       0
Total_Amount       0
dtype: int64
```

Hình 5.34. Kiểm tra các biến đầu vào và biến mục tiêu cho bài toán hồi quy

Kết quả kiểm tra xác nhận dữ liệu của các biến được lựa chọn đáp ứng yêu cầu ban đầu để tiếp tục thực hiện bước chia tập và huấn luyện mô hình hồi quy.

- **Chia tập dữ liệu huấn luyện và kiểm tra**

Sau khi xác định các biến, nghiên cứu tạo tập đặc trưng (X) từ bảy biến đầu vào và tập mục tiêu (y) từ cột Total_Amount. Dữ liệu được chia thành tập huấn luyện và tập kiểm tra theo tỷ lệ 80/20 bằng hàm train_test_split của thư viện Scikit-learn.

Trong quá trình chia dữ liệu, tham số test_size=0.2 được sử dụng để dành 20% dữ liệu cho kiểm tra, trong khi 80% còn lại được dùng để huấn luyện. Tham số random_state=42 giúp có thể tái lập kết quả chia dữ liệu khi thực hiện lại với cùng bộ dữ liệu và cấu hình.

```python
from sklearn.model_selection import train_test_split

# Tạo tập biến đầu vào X và biến mục tiêu y
X_reg = df[regression_features].copy()
y_reg = df[regression_target].copy()

# Chia dữ liệu theo tỷ lệ 80/20
X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)

print("Kích thước X_reg:", X_reg.shape)
print("Kích thước y_reg:", y_reg.shape)

print("\nKích thước tập huấn luyện:")
print("X_reg_train:", X_reg_train.shape)
print("y_reg_train:", y_reg_train.shape)

print("\nKích thước tập kiểm tra:")
print("X_reg_test:", X_reg_test.shape)
print("y_reg_test:", y_reg_test.shape)
```

Hình 5.35. Đoạn mã chia tập dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20 cho mô hình hồi quy.

```
Kích thước X_reg: (8500, 7)
Kích thước y_reg: (8500,)

Kích thước tập huấn luyện:
X_reg_train: (6800, 7)
y_reg_train: (6800,)

Kích thước tập kiểm tra:
X_reg_test: (1700, 7)
y_reg_test: (1700,)
```

Hình 5.36. Kết quả chia tập dữ liệu huấn luyện và kiểm tra theo tỷ lệ 80/20

Kết quả thực nghiệm cho thấy tập dữ liệu ban đầu có 8.500 mẫu và bảy biến đầu vào. Sau khi chia, tập huấn luyện gồm 6.800 mẫu, còn tập kiểm tra gồm 1.700 mẫu. Các kích thước này tương ứng với tỷ lệ chia dữ liệu 80/20 theo yêu cầu.

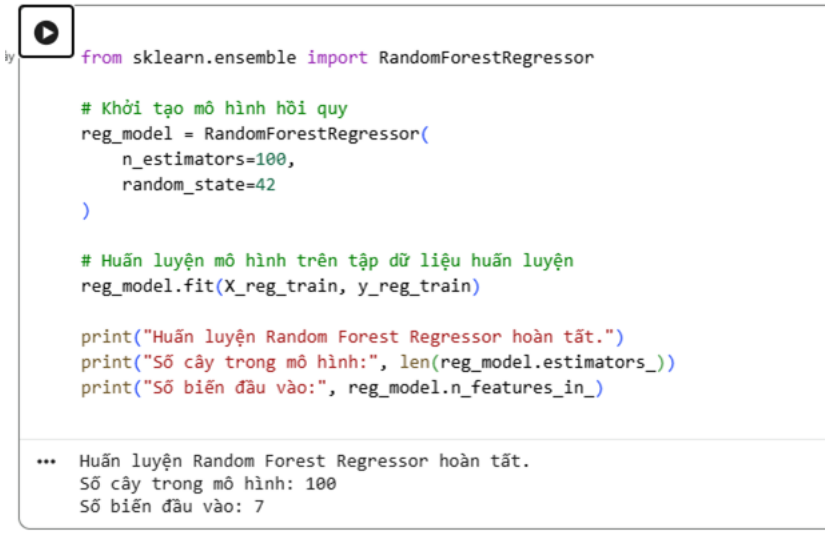
Việc chia dữ liệu tạo cơ sở để mô hình học mối quan hệ giữa các đặc trưng đầu vào và giá trị đơn hàng trên tập huấn luyện, sau đó đánh giá khả năng dự đoán trên tập kiểm tra chưa được sử dụng trong quá trình huấn luyện.

- **Huấn luyện mô hình**

Sau khi hoàn thành bước xác định biến đầu vào, biến mục tiêu và chia tập dữ liệu, nghiên cứu tiến hành xây dựng mô hình hồi quy nhằm dự đoán giá trị đơn hàng (Total_Amount) trên sàn thương mại điện tử TikTok Shop. Thuật toán được lựa chọn là Random Forest Regressor thuộc thư viện Scikit-learn.

Mô hình được khởi tạo với hai tham số chính: n_estimators=100 và random_state=42. Trong đó, n_estimators=100 quy định mô hình sử dụng 100 cây quyết định để thực hiện dự đoán, random_state=42 hỗ trợ tái lập kết quả khi huấn luyện lại mô hình với cùng dữ liệu và cấu hình.

Quá trình huấn luyện được thực hiện thông qua phương thức fit() với tập đặc trưng huấn luyện X_reg_train và biến mục tiêu y_reg_train. Dữ liệu huấn luyện gồm 6.800 mẫu với 7 biến đầu vào đã được xác định ở phần A. Mô hình học mối quan hệ giữa các đặc trưng giao dịch và tổng giá trị đơn hàng để phục vụ việc dự đoán trên tập kiểm tra.



```
from sklearn.ensemble import RandomForestRegressor

# Khởi tạo mô hình hồi quy
reg_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

# Huấn luyện mô hình trên tập dữ liệu huấn luyện
reg_model.fit(X_reg_train, y_reg_train)

print("Huấn luyện Random Forest Regressor hoàn tất.")
print("Số cây trong mô hình:", len(reg_model.estimators_))
print("Số biến đầu vào:", reg_model.n_features_in_)
```

```
Huấn luyện Random Forest Regressor hoàn tất.
Số cây trong mô hình: 100
Số biến đầu vào: 7
```

Hình 5.37. Đoạn mã huấn luyện mô hình Random Forest Regressor

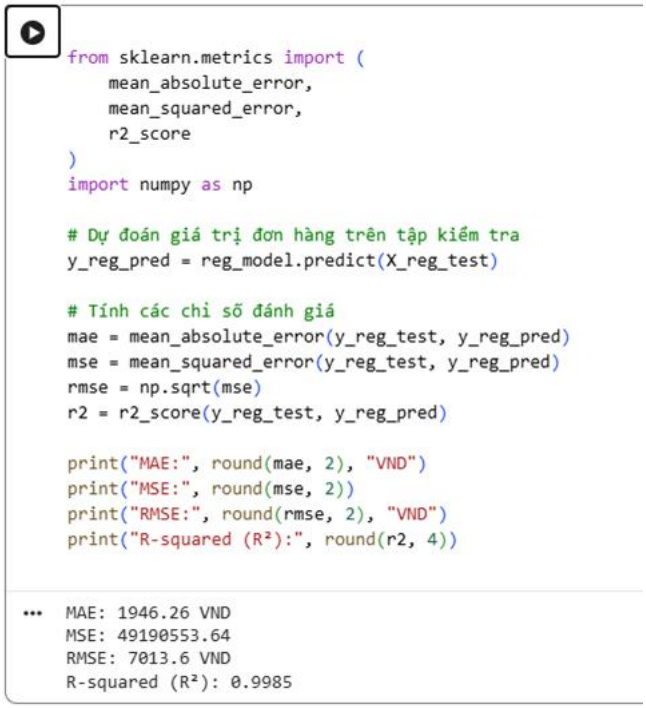
Kết quả thực nghiệm trên Google Colab xác nhận quá trình huấn luyện đã hoàn tất. Mô hình sử dụng 100 cây quyết định và tiếp nhận 7 biến đầu vào, phù hợp với cấu hình đã thiết lập.

Sau khi huấn luyện, mô hình reg_model được sử dụng để dự đoán giá trị Total_Amount trên tập kiểm tra gồm 1.700 mẫu. Kết quả dự đoán sẽ được đối chiếu với

giá trị thực tế thông qua các chỉ số MAE, MSE, RMSE và R^2 nhằm đánh giá sai số dự đoán cũng như mức độ phù hợp của mô hình.

- **Đánh giá hiệu năng mô hình hồi quy**

Sau khi huấn luyện mô hình Random Forest Regressor, nghiên cứu tiến hành đánh giá khả năng dự đoán giá trị đơn hàng trên tập kiểm tra gồm 1.700 mẫu. Giá trị dự đoán được so sánh với giá trị thực tế của biến mục tiêu Total_Amount thông qua bốn chỉ số: MAE, MSE, RMSE và R^2.



```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

# Dự đoán giá trị đơn hàng trên tập kiểm tra
y_reg_pred = reg_model.predict(X_reg_test)

# Tính các chỉ số đánh giá
mae = mean_absolute_error(y_reg_test, y_reg_pred)
mse = mean_squared_error(y_reg_test, y_reg_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_reg_test, y_reg_pred)

print("MAE:", round(mae, 2), "VND")
print("MSE:", round(mse, 2))
print("RMSE:", round(rmse, 2), "VND")
print("R-squared (R²):", round(r2, 4))
```

```
MAE: 1946.26 VND
MSE: 49190553.64
RMSE: 7013.6 VND
R-squared (R²): 0.9985
```

Hình 5.38. Đoạn mã tính toán các chỉ số đánh giá mô hình hồi quy

Kết quả thực nghiệm trên Google Colab được tổng hợp như sau:

Thứ nhất, chỉ số MAE đạt 1.946,26 VNĐ, cho thấy độ lệch tuyệt đối trung bình giữa giá trị đơn hàng dự đoán và giá trị thực tế ở mức khoảng 1.946 VNĐ. Chỉ số này giúp biểu diễn trực tiếp mức sai số trung bình theo đơn vị tiền tệ.

Thứ hai, MSE đạt 49.190.553,64. Do tính bình phương sai số trước khi lấy trung bình, chỉ số này nhạy hơn với những trường hợp dự đoán lệch lớn. RMSE đạt 7.013,60 VNĐ, phản ánh mức sai số theo cùng đơn vị với Total_Amount và thuận tiện hơn khi diễn giải trong bối cảnh giá trị đơn hàng.

Thứ ba, hệ số xác định R^2 đạt 0,9985, cho thấy mô hình có mức độ phù hợp rất cao trên tập kiểm tra. Theo cách diễn giải thông thường, khoảng 99,85% mức biến thiên

của Total_Amount trên tập kiểm tra được giải thích bởi mô hình. Tuy nhiên, R^2 cao không tự động chứng minh mô hình có khả năng dự đoán tốt trên dữ liệu mới trong thực tế.

Nhìn chung, các chỉ số thu được cho thấy mô hình Random Forest Regressor có sai số dự đoán tương đối thấp so với thang giá trị của dữ liệu và đạt hệ số R^2 cao. Dù vậy, cần kiểm tra thêm mối quan hệ giữa các biến đầu vào và cách hình thành Total_Amount để loại trừ khả năng biến mục tiêu được suy ra gần như trực tiếp từ các đặc trưng đầu vào. Vì vậy, kết quả hiện tại nên được xem là đánh giá trên tập kiểm tra của bộ dữ liệu đang sử dụng, chưa đủ để khẳng định hiệu quả tương tự trên dữ liệu thực tế khác.

- **Kiểm tra mối quan hệ giữa biến đầu vào và biến mục tiêu**

Để kiểm tra mức độ phù hợp của kết quả hồi quy, nghiên cứu tiến hành đối chiếu giá trị Total_Amount với giá trị ước tính theo công thức Unit_Price × Quantity - Discount_Amount + Shipping_Fee.

```
# Kiểm tra các biến liên quan đến tổng giá trị đơn hàng
check_columns = [
    "Unit_Price",
    "Quantity",
    "Discount_Amount",
    "Shipping_Fee",
    "Total_Amount"
]

print("10 dòng dữ liệu ban đầu:")
print(df[check_columns].head(10))

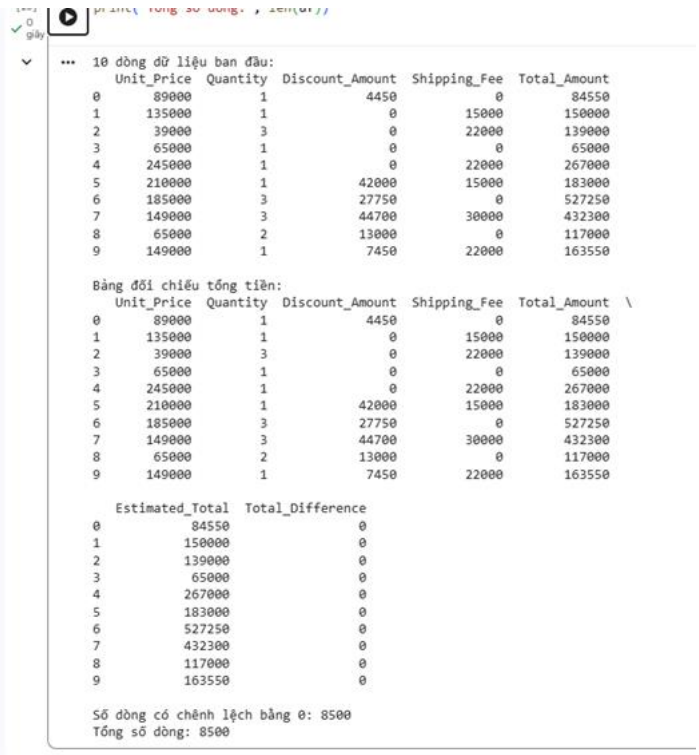
# Kiểm tra công thức tổng tiền giả định
df["Estimated_Total"] = (
    df["Unit_Price"] * df["Quantity"]
    - df["Discount_Amount"]
    + df["Shipping_Fee"]
)

df["Total_Difference"] = (
    df["Total_Amount"] - df["Estimated_Total"]
)

print("\nBảng đối chiếu tổng tiền:")
print(
    df[
        [
            "Unit_Price",
            "Quantity",
            "Discount_Amount",
            "Shipping_Fee",
            "Total_Amount",
            "Estimated_Total",
            "Total_Difference"
        ]
    ].head(10)
)

print(
    "\nSố dòng có chênh lệch bằng 0:",
    (df["Total_Difference"] == 0).sum()
)
print("Tổng số dòng:", len(df))
```

Hình 5.39. Đoạn mã kiểm tra công thức tính Total_Amount và đối chiếu giá trị thực tế với giá trị ước tính.



```
10 dòng dữ liệu ban đầu:
   Unit_Price  Quantity  Discount_Amount  Shipping_Fee  Total_Amount
0       89000         1             4450             0         84550
1      135000         1                0         15000        150000
2       39000         3                0         22000        139000
3       65000         1                0             0         65000
4      245000         1                0         22000        267000
5      210000         1            42000         15000        183000
6      185000         3            27750             0        527250
7      149000         3            44700         30000        432300
8       65000         2            13000             0        117000
9      149000         1             7450         22000        163550

Bảng đối chiếu tổng tiền:
   Unit_Price  Quantity  Discount_Amount  Shipping_Fee  Total_Amount  \
0       89000         1             4450             0         84550
1      135000         1                0         15000        150000
2       39000         3                0         22000        139000
3       65000         1                0             0         65000
4      245000         1                0         22000        267000
5      210000         1            42000         15000        183000
6      185000         3            27750             0        527250
7      149000         3            44700         30000        432300
8       65000         2            13000             0        117000
9      149000         1             7450         22000        163550

   Estimated_Total  Total_Difference
0            84550                 0
1           150000                 0
2           139000                 0
3            65000                 0
4           267000                 0
5           183000                 0
6           527250                 0
7           432300                 0
8           117000                 0
9           163550                 0

Số dòng có chênh lệch bằng 0: 8500
Tổng số dòng: 8500
```

Hình 5.40. Kết quả kiểm tra công thức tính Total_Amount và đối chiếu giá trị thực tế với giá trị ước tính.

Kết quả thực nghiệm cho thấy cả 8.500 dòng dữ liệu đều có giá trị chênh lệch bằng 0, chứng tỏ Total_Amount trong bộ dữ liệu khớp hoàn toàn với công thức tính giả định này.

Kết quả trên cho thấy biến mục tiêu có mối quan hệ xác định trực tiếp với một số biến đầu vào được sử dụng trong mô hình. Đây là yếu tố có thể góp phần giải thích hệ số R^2 rất cao của Random Forest Regressor. Vì vậy, kết quả đánh giá cần được diễn giải trong bối cảnh cấu trúc của bộ dữ liệu và chưa nên được xem là bằng chứng độc lập về khả năng dự đoán giá trị đơn hàng trên dữ liệu thực tế mới.

5.3. MÔ HÌNH KHẢO SÁT VÀ PHÁT HIỆN ĐIỂM BẤT THƯỜNG (ANOMALY DETECTION)

Để nhận diện những giao dịch có đặc điểm khác biệt trong dữ liệu TikTok Shop, nghiên cứu sử dụng thuật toán Isolation Forest thuộc thư viện Scikit-learn. Mô hình được thiết lập với contamination=0.04 và random_state=42, đồng thời sử dụng năm đặc trưng gồm Total_Amount, Discount_Amount, Shipping_Fee, Quantity và Video_Views theo yêu cầu bài tập.

5.3.1. Xây dựng mô hình Isolation Forest

Isolation Forest phát hiện điểm bất thường dựa trên khả năng cô lập các quan sát trong tập dữ liệu. Những giao dịch có đặc điểm khác biệt so với phần lớn quan sát có thể được cô lập dễ dàng hơn và được mô hình đánh dấu là bất thường.

Sau khi lựa chọn các đặc trưng, mô hình được huấn luyện bằng phương thức fit_predict(). Thuật toán trả về nhãn -1 cho giao dịch bất thường và 1 cho giao dịch bình thường. Để thuận tiện cho việc thống kê và trực quan hóa, kết quả được chuyển thành cột IF_Anomaly_Final, trong đó giá trị 1 biểu thị giao dịch bất thường và giá trị 0 biểu thị giao dịch bình thường.

```
from sklearn.ensemble import IsolationForest

# Khởi tạo mô hình theo yêu cầu bài tập
iso_model = IsolationForest(
    contamination=0.04,
    random_state=42
)

# Huấn luyện và dự đoán điểm bất thường
anomaly_labels = iso_model.fit_predict(X_anomaly)

# Chuyển kết quả: -1 là bất thường, 1 là bình thường
df["IF_Anomaly_Final"] = (anomaly_labels == -1).astype(int)

print("Phân bố kết quả Isolation Forest:")
print(df["IF_Anomaly_Final"].value_counts().sort_index())

print("\nTỷ lệ phần trăm:")
print(
    (df["IF_Anomaly_Final"].value_counts(normalize=True)
     .sort_index() * 100).round(2)
)
```

```
Phân bố kết quả Isolation Forest:
IF_Anomaly_Final
0    8160
1     340
Name: count, dtype: int64

Tỷ lệ phần trăm:
IF_Anomaly_Final
0    96.0
1     4.0
Name: proportion, dtype: float64
```

Hình 5.41. Kết quả phân loại giao dịch bằng thuật toán Isolation Forest

Kết quả cho thấy mô hình xác định 340 giao dịch bất thường, chiếm 4% tổng số 8.500 giao dịch; 8.160 giao dịch còn lại được phân loại là bình thường. Kết quả này phù hợp với tỷ lệ contamination=0.04 được thiết lập khi khởi tạo mô hình.

5.3.2. Trực quan hóa các giao dịch bất thường

Để quan sát sự phân bố của các giao dịch được phát hiện, nghiên cứu sử dụng biểu đồ phân tán giữa Discount_Amount và Total_Amount. Trục hoành thể hiện giá trị giảm giá, còn trục tung biểu diễn tổng giá trị đơn hàng. Các giao dịch bình thường được thể hiện bằng màu xanh và các giao dịch bất thường được tô màu đỏ.

```python
import matplotlib.pyplot as plt

# Tách giao dịch theo kết quả Isolation Forest
normal_data = df[df["IF_Anomaly_Final"] == 0]
anomaly_data = df[df["IF_Anomaly_Final"] == 1]

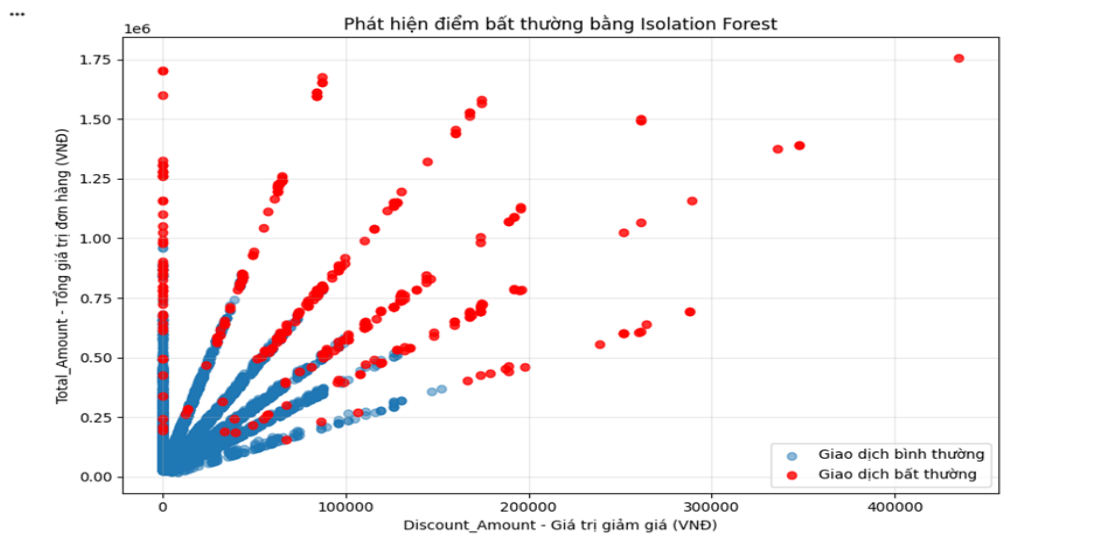
plt.figure(figsize=(9, 6))

# Giao dịch bình thường
plt.scatter(
    normal_data["Discount_Amount"],
    normal_data["Total_Amount"],
    label="Giao dịch bình thường",
    alpha=0.5
)

# Giao dịch bất thường hiển thị màu đỏ
plt.scatter(
    anomaly_data["Discount_Amount"],
    anomaly_data["Total_Amount"],
    color="red",
    label="Giao dịch bất thường",
    alpha=0.8
)

plt.xlabel("Discount_Amount - Giá trị giảm giá (VNĐ)")
plt.ylabel("Total_Amount - Tổng giá trị đơn hàng (VNĐ)")
plt.title("Phát hiện điểm bất thường bằng Isolation Forest")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Hình 5.42. Đoạn mã trực quan hóa các giao dịch bất thường bằng thuật toán Isolation Forest



Hình 5.43. Biểu đồ phân tán giữa giá trị giảm giá và tổng giá trị đơn hàng, thể hiện kết quả phát hiện bất thường bằng Isolation Forest

Quan sát biểu đồ cho thấy phần lớn giao dịch tập trung ở vùng có giá trị giảm giá và tổng giá trị đơn hàng tương đối thấp. Một số điểm màu đỏ xuất hiện tại các mức giá trị cao hoặc nằm tách biệt so với vùng tập trung chính. Những giao dịch này có đặc điểm khác biệt theo mô hình và có thể được ưu tiên kiểm tra thêm.

Tuy nhiên, biểu đồ chỉ thể hiện hai đặc trưng trên mặt phẳng hai chiều, trong khi Isolation Forest được huấn luyện bằng năm đặc trưng. Vì vậy, vị trí của một điểm trên biểu đồ không thể hiện đầy đủ cơ sở để mô hình đánh dấu giao dịch đó là bất thường.

5.3.3. Đối chiếu kết quả phát hiện với nhãn bất thường ban đầu

Để đánh giá mức độ tương đồng giữa kết quả của Isolation Forest và nhãn bất thường có sẵn trong bộ dữ liệu, nghiên cứu tiến hành đối chiếu cột IF_Anomaly_Final với cột Is_Anomaly thông qua ma trận nhầm lẫn và báo cáo phân loại.

Kết quả cho thấy mô hình phát hiện đúng 15 trong tổng số 350 giao dịch được gắn nhãn bất thường ban đầu, tương ứng với Recall khoảng 4%. Precision của nhóm bất thường cũng đạt khoảng 4%, cho thấy phần lớn các giao dịch do mô hình đánh dấu bất thường không trùng với nhãn gốc. Mặc dù Accuracy tổng thể đạt khoảng 92%, chỉ số này chịu ảnh hưởng lớn từ tỷ lệ giao dịch bình thường chiếm ưu thế trong dữ liệu.

```
from sklearn.metrics import confusion_matrix, classification_report

print("Ma trận đối chiếu nhãn gốc và nhãn Isolation Forest:")
print(
    confusion_matrix(
        df["Is_Anomaly"],
        df["IF_Anomaly_Final"],
        labels=[0, 1]
    )
)

print("\nBáo cáo phân loại:")
print(
    classification_report(
        df["Is_Anomaly"],
        df["IF_Anomaly_Final"],
        labels=[0, 1],
        target_names=["Bình thường", "Bất thường"],
        zero_division=0
    )
)
```

```
Ma trận đối chiếu nhãn gốc và nhãn Isolation Forest:
[[7825  325]
 [ 335   15]]

Báo cáo phân loại:
              precision    recall  f1-score   support

 Bình thường       0.96      0.96      0.96      8150
  Bất thường       0.04      0.04      0.04       350

    accuracy                           0.92      8500
   macro avg       0.50      0.50      0.50      8500
weighted avg       0.92      0.92      0.92      8500
```

Hình 5.44. Ma trận nhầm lẫn và báo cáo phân loại khi đối chiếu kết quả Isolation Forest với nhãn bất thường ban đầu.

Kết quả cho thấy Isolation Forest chưa tái hiện tốt các nhãn bất thường có sẵn trong bộ dữ liệu với cấu hình hiện tại. Tuy nhiên, sự khác biệt này không tự động chứng

minh mô hình phát hiện sai, vì nhãn gốc và kết quả phát hiện không nhất thiết được xây dựng theo cùng một tiêu chí. Do đó, cần kiểm tra thêm đặc điểm của các giao dịch được đánh dấu và cách tạo nhãn ban đầu trước khi đưa ra kết luận về hiệu quả phát hiện bất thường.

5.4. HOÀN THIỆN DỮ LIỆU VÀ XUẤT KẾT QUẢ PHÂN CỤM KHÁCH HÀNG RFM

Sau khi hoàn thành các mô hình học máy, nhóm tiếp tục thực hiện bước tích hợp kết quả phân cụm khách hàng RFM vào dữ liệu giao dịch ban đầu. Nội dung này nhằm hoàn thiện bộ dữ liệu đầu ra, xử lý các giao dịch chưa có nhãn phân cụm và xuất tệp dữ liệu phục vụ cho quá trình khai thác tiếp theo.

5.4.1. Ánh xạ nhãn phân cụm vào dữ liệu giao dịch

Sau khi phân cụm 1.393 khách hàng bằng thuật toán K-Means, nhóm sử dụng hai trường Customer_ID và Cluster từ bảng RFM để tạo bảng ánh xạ khách hàng sang nhãn phân cụm. Tiếp theo, nhóm sử dụng hàm merge() với khóa liên kết Customer_ID và phương thức how="left" để ghép nhãn cụm vào dữ liệu giao dịch ban đầu. Tham số validate="many_to_one" được sử dụng để kiểm tra mối quan hệ: một khách hàng có thể xuất hiện trong nhiều giao dịch nhưng chỉ tương ứng với một nhãn phân cụm trong bảng ánh xạ.

```python
# Tạo bảng ánh xạ khách hàng sang nhãn cụm
cluster_mapping = rfm[["Customer_ID", "Cluster"]].rename(
    columns={"Cluster": "Cluster_Label"}
)

# Ghép nhãn cụm vào dữ liệu giao dịch gốc
df = df.drop(columns=["Cluster_Label"], errors="ignore")

df = df.merge(
    cluster_mapping,
    on="Customer_ID",
    how="left",
    validate="many_to_one"
)

print("Kích thước dữ liệu sau khi gắn nhãn:", df.shape)

print("\nPhân bố nhãn cụm trên dữ liệu giao dịch:")
print(df["Cluster_Label"].value_counts(dropna=False).sort_index())

print("\nSố giao dịch chưa có nhãn cụm:")
print(df["Cluster_Label"].isnull().sum())

print("\nKiểm tra các cột kết quả:")
print([
    col for col in [
        "Customer_ID",
        "Cluster_Label",
        "Is_Anomaly",
        "IF_Anomaly_Final"
    ]
    if col in df.columns
])
```

Hình 5.45. Mã nguồn ánh xạ nhãn phân cụm RFM vào dữ liệu giao dịch.

Kết quả sau khi ghép cho thấy bộ dữ liệu có 8.500 dòng và 32 cột. Phân bố nhãn trên dữ liệu giao dịch gồm 897 giao dịch thuộc Cluster 0, 3.961 giao dịch thuộc Cluster 1, 3.634 giao dịch thuộc Cluster 2 và 8 giao dịch chưa có nhãn. Các giao dịch chưa có nhãn được kiểm tra riêng ở bước tiếp theo.

```
Kích thước dữ liệu sau khi gắn nhãn: (8500, 32)

Phân bố nhãn cụm trên dữ liệu giao dịch:
Cluster_Label
0.0     897
1.0    3961
2.0    3634
NaN       8
Name: count, dtype: int64

Số giao dịch chưa có nhãn cụm:
8

Kiểm tra các cột kết quả:
['Customer_ID', 'Cluster_Label', 'Is_Anomaly', 'IF_Anomaly_Final']
```

Hình 5.46. Kết quả kiểm tra dữ liệu sau khi gắn nhãn phân cụm.

5.4.2. Kiểm tra và xử lý giao dịch chưa có nhãn phân cụm

Để xác định nguyên nhân còn 8 giao dịch chưa được gán nhãn, nhóm kiểm tra trạng thái đơn hàng của các bản ghi có giá trị thiếu tại cột Cluster_Label. Kết quả cho thấy có 4 giao dịch đang vận chuyển, 2 giao dịch đã hủy và 2 giao dịch hoàn trả/khiếu nại. Không có giao dịch nào trong số này thuộc trạng thái Hoàn thành.

```python
# Kiểm tra các giao dịch chưa được phân cụm
unassigned = df[df["Cluster_Label"].isna()]

print("Số giao dịch chưa có nhãn:", len(unassigned))
print("\nTrạng thái của các giao dịch:")
print(unassigned["Order_Status"].value_counts())

print(
    "\nSố giao dịch chưa có nhãn nhưng thuộc đơn hoàn thành:",
    (unassigned["Order_Status"] == "Hoàn thành").sum()
)
```

```
Số giao dịch chưa có nhãn: 8

Trạng thái của các giao dịch:
Order_Status
Đang vận chuyển         4
Đã hủy                  2
Hoàn trả / Khiếu nại    2
Name: count, dtype: int64

Số giao dịch chưa có nhãn nhưng thuộc đơn hoàn thành: 0
```

Hình 5.47. Kết quả thống kê trạng thái của các giao dịch chưa có nhãn.

Nguyên nhân xuất phát từ quy trình xây dựng bảng RFM ở mục 5.1, khi nhóm chỉ sử dụng các đơn hàng có trạng thái "Hoàn thành" để tính toán Recency, Frequency và Monetary. Do đó, những khách hàng không có đơn hàng hoàn thành sẽ không xuất hiện trong bảng RFM và không có nhãn cụm để ánh xạ vào dữ liệu giao dịch. Nhóm tiếp tục

trích xuất thông tin mã đơn hàng, mã khách hàng, trạng thái và tổng giá trị của các giao dịch này để kiểm tra cụ thể.

```
# Kiểm tra các giao dịch chưa được gán nhãn cụm
unassigned = df[df["Cluster_Label"].isna()].copy()

print("Số giao dịch chưa có nhãn:", len(unassigned))

print("\nTrạng thái đơn hàng:")
print(unassigned["Order_Status"].value_counts(dropna=False))

print("\nThông tin các giao dịch:")
print(
    unassigned[
        ["Order_ID", "Customer_ID", "Order_Status", "Total_Amount"]
    ]
)
```

```
Số giao dịch chưa có nhãn: 8

Trạng thái đơn hàng:
Order_Status
Đang vận chuyển         4
Đã hủy                  2
Hoàn trả / Khiếu nại    2
Name: count, dtype: int64

Thông tin các giao dịch:
            Order_ID Customer_ID          Order_Status  Total_Amount
705   TTS-2024-006582  CUST-01244       Đang vận chuyển        143050
1949  TTS-2024-003282  CUST-01360               Đã hủy         95750
2640  TTS-2024-005583  CUST-00345               Đã hủy        165000
4436  TTS-2024-004053  CUST-00696       Đang vận chuyển        207250
4736  TTS-2024-001775  CUST-01137  Hoàn trả / Khiếu nại        198400
6244  TTS-2024-000359  CUST-01360       Đang vận chuyển        399000
6372  TTS-2024-002676  CUST-00345       Đang vận chuyển        124600
7889  TTS-2024-003135  CUST-00345  Hoàn trả / Khiếu nại        107100
```

Hình 5.48. Thông tin chi tiết các giao dịch chưa được gán nhãn phân cụm.

Sau khi xác định nguyên nhân, nhóm gán giá trị "Chưa có đơn hoàn thành" cho các bản ghi chưa có nhãn tại cột Cluster_Label. Đây là nhãn bổ sung nhằm phân biệt những giao dịch không đủ điều kiện tham gia quá trình xây dựng RFM với ba nhóm khách hàng được xác định bằng K-Means; không phải là cụm khách hàng thứ tư của thuật toán.

```
# Xử lý các giao dịch chưa có nhãn RFM
df["Cluster_Label"] = df["Cluster_Label"].astype("object")

df.loc[
    df["Cluster_Label"].isna(),
    "Cluster_Label"
] = "Chưa có đơn hoàn thành"

# Kiểm tra lại kết quả
print("Phân bố nhãn RFM sau khi xử lý:")
print(df["Cluster_Label"].value_counts(dropna=False))

print(
    "\nSố nhãn còn thiếu:",
    df["Cluster_Label"].isna().sum()
)
```

```
Phân bố nhãn RFM sau khi xử lý:
Cluster_Label
1.0                     3961
2.0                     3634
0.0                      897
Chưa có đơn hoàn thành     8
Name: count, dtype: int64

Số nhãn còn thiếu: 0
```

Hình 5.49. Kết quả xử lý các giao dịch chưa được gán nhãn RFM.

Sau khi xử lý, phân bố nhãn trên toàn bộ dữ liệu giao dịch được ghi nhận như sau:

Bảng 5.5. Phân bố nhãn trên toàn bộ dữ liệu

Nhãn phân cụm	**Số giao dịch**
Cluster 0	897
Cluster 1	3.961
Cluster 2	3.634
Chưa có đơn hoàn thành	8
Tổng cộng	**8.500**

Nguồn: Nhóm tự tổng hợp

Kết quả kiểm tra cho thấy số lượng giá trị thiếu tại cột Cluster_Label bằng 0. Như vậy, toàn bộ 8.500 giao dịch đã có thông tin phân loại. Trong đó, "Chưa có đơn hoàn thành" là nhãn bổ sung phục vụ xử lý dữ liệu, không phải một cụm khách hàng mới của K-Means.

5.4.3. Kiểm tra và xuất dữ liệu kết quả

Sau khi hoàn tất việc gắn nhãn, nhóm tiến hành kiểm tra kích thước dữ liệu, số lượng nhãn còn thiếu và sự tồn tại của cột Cluster_Label. Tiếp theo, dữ liệu được xuất thành tệp CSV với tên tiktok_shop_sales_ml_results.csv. Tham số index=False được sử dụng để không ghi thêm cột chỉ mục, trong khi encoding="utf-8-sig" hỗ trợ hiển thị tiếng Việt khi mở tệp bằng các phần mềm bảng tính.

```
# Kiểm tra dữ liệu trước khi xuất
print("Kích thước dữ liệu:", df.shape)
print("Số nhãn cụm bị thiếu:", df["Cluster_Label"].isna().sum())

# Xuất dữ liệu kết quả
output_file = "tiktok_shop_sales_ml_results.csv"

df.to_csv(output_file, index=False, encoding="utf-8-sig")

print("\nĐã xuất file:", output_file)
print("Số dòng trong file:", len(df))
print("Có cột Cluster_Label:", "Cluster_Label" in df.columns)
```

```
Kích thước dữ liệu: (8500, 32)
Số nhãn cụm bị thiếu: 0

Đã xuất file: tiktok_shop_sales_ml_results.csv
Số dòng trong file: 8500
Có cột Cluster_Label: True
```

Hình 5.50. Kết quả kiểm tra và xuất dữ liệu sau khi tích hợp nhãn phân cụm.

Kết quả thực nghiệm xác nhận tệp dữ liệu được xuất thành công với 8.500 dòng, 32 cột và không còn giá trị thiếu tại trường Cluster_Label. Sau đó, nhóm sử dụng chức năng tải tệp của Google Colab để lưu dữ liệu về máy tính, phục vụ cho quá trình phân tích và xử lý tiếp theo.

```
from google.colab import files

files.download("tiktok_shop_sales_ml_results.csv")
```

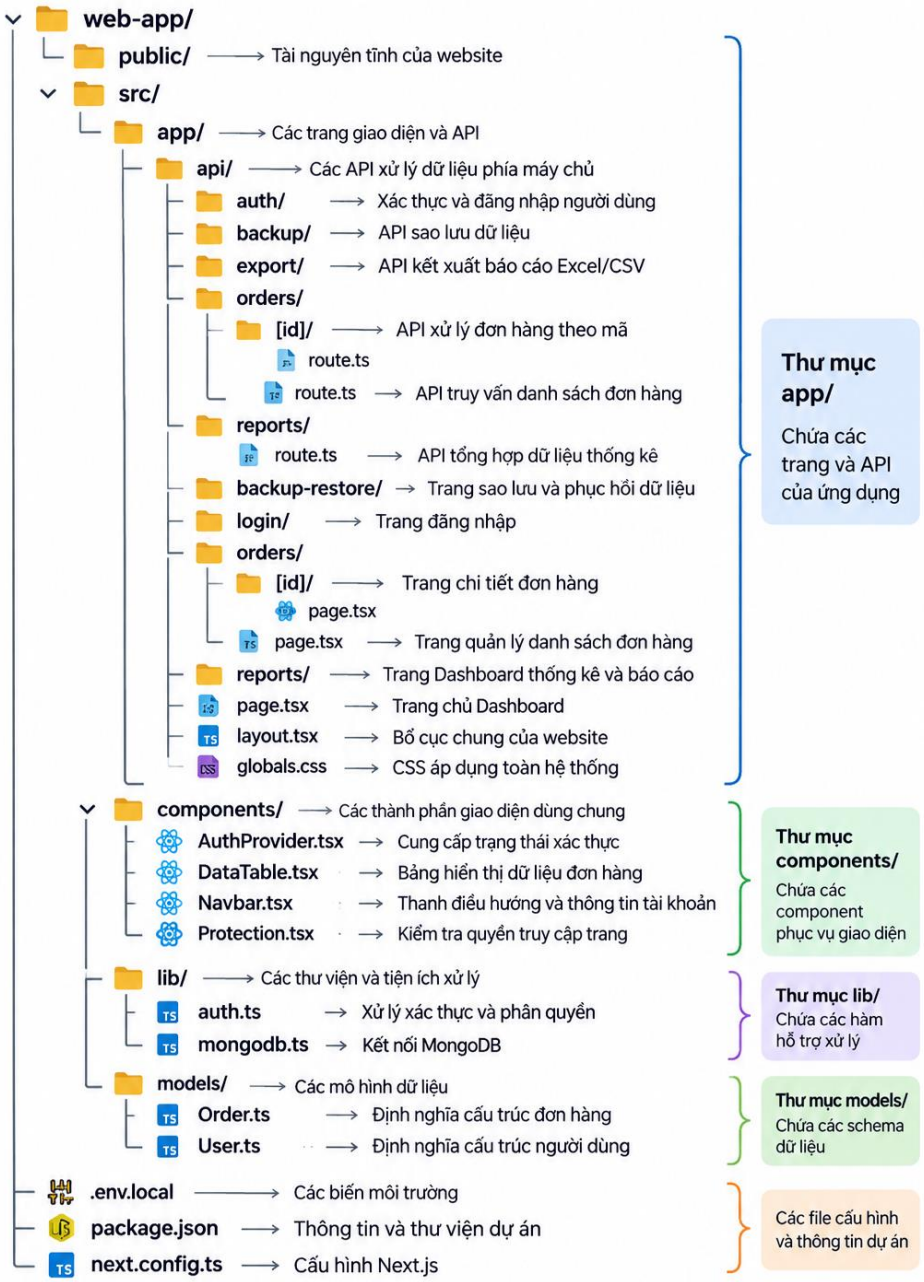
Hình 5.51. Mã lệnh tải tệp kết quả CSV từ Google Colab.

Sau khi hoàn tất quá trình xử lý dữ liệu và phân cụm khách hàng bằng thuật toán K-Means, nhóm tiến hành xuất bộ dữ liệu kết quả dưới định dạng CSV để lưu trữ và phục vụ cho các bước phân tích tiếp theo. Mã lệnh files.download() của thư viện google.colab được sử dụng để tải tệp tiktok_shop_sales_ml_results.csv trực tiếp từ môi trường Google Colab về máy tính. Tệp xuất ra bao gồm 8.500 dòng dữ liệu và 32 cột, trong đó có trường Cluster_Label thể hiện nhãn phân cụm của từng giao dịch, đồng thời đã xử lý các trường hợp chưa có nhãn phân cụm.

CHƯƠNG 6: XÂY DỰNG WEBSITE TÍCH HỢP HỆ THỐNG VÀ TÍNH NĂNG MỞ RỘNG (NEXT.JS)

6.1. KIẾN TRÚC HỆ THỐNG WEBSITE VÀ CẤU TRÚC THƯ MỤC DỰ ÁN (SOURCE-CODE/WEB-APP)

Website được xây dựng trên nền tảng Next.js, sử dụng cấu trúc thư mục src/app để tổ chức các trang giao diện và API. Các thành phần dùng chung, thư viện xử lý và mô hình dữ liệu được tách riêng nhằm thuận tiện cho việc phát triển, quản lý và bảo trì hệ thống.



Hình 6.1 Cấu trúc thư mục dự án

Bảng 6.1. Chức năng các trang giao diện chính

Trang	Chức năng
/login	Đăng nhập và xác thực tài khoản người dùng.
/orders	Quản lý danh sách đơn hàng, tìm kiếm, lọc và thực hiện các thao tác theo quyền được cấp.
/orders/[id]	Xem thông tin chi tiết và thao tác với một đơn hàng cụ thể.
/reports	Hiển thị các chỉ số KPI, biểu đồ thống kê và hỗ trợ kết xuất báo cáo Excel/CSV.
/backup-restore	Cung cấp chức năng sao lưu và phục hồi dữ liệu dành cho ADMIN.

Các nhóm thành phần hỗ trợ:

- api/: Tiếp nhận yêu cầu từ giao diện, truy vấn và xử lý dữ liệu, kết nối với các chức năng tương ứng.
- components/: Chứa các thành phần giao diện dùng chung như thanh điều hướng, bảng dữ liệu và cơ chế bảo vệ trang.
- lib/: Tập trung các chức năng xử lý xác thực, phân quyền và kết nối cơ sở dữ liệu.
- models/: Định nghĩa cấu trúc dữ liệu người dùng (User) và đơn hàng (Order) phục vụ quá trình thao tác với MongoDB.

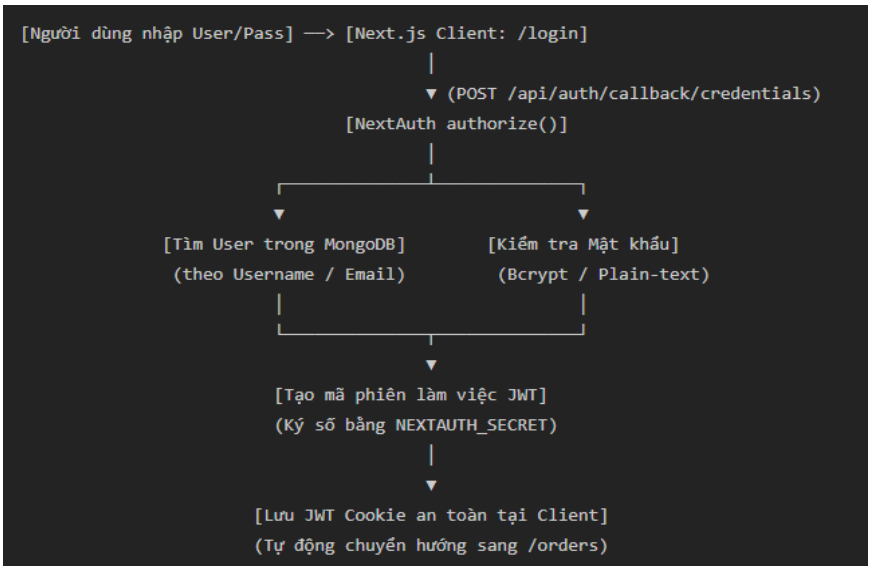
Nhìn chung, hệ thống được tổ chức thành ba nhóm chính gồm giao diện người dùng, API xử lý nghiệp vụ và các thành phần quản lý dữ liệu. Cách phân chia này giúp các chức năng đăng nhập, quản lý đơn hàng, thống kê và sao lưu dữ liệu hoạt động theo từng nhóm riêng biệt nhưng vẫn liên kết trong cùng một ứng dụng.

Hệ thống Web App Next.js đã được triển khai trực tuyến trên hạ tầng Cloud Vercel. Link đường dẫn (Live Demo Website): https://03-17-phan-tich-du-lieu-tiktok-shop.vercel.app

6.2. THIẾT LẬP ĐĂNG NHẬP VÀ PHÂN QUYỀN NGƯỜI DÙNG (ROLE-BASED AUTHORIZATION)

6.2.1. Kiến trúc Xác thực Người dùng (Authentication Flow với NextAuth.js)

Hệ thống triển khai cơ chế xác thực thông tin đăng nhập tùy chỉnh (Credentials Provider) kết nối trực tiếp với collection users trong cơ sở dữ liệu MongoDB Atlas.



Hình 6.2. Sơ đồ Kiến trúc Xác thực NextAuth

Quy trình xử lý chi tiết tại máy chủ (src/lib/auth.ts):

1. Truy vấn người dùng đa năng: Cho phép người dùng đăng nhập linh hoạt bằng username hoặc địa chỉ email thông qua toán tử $or:

```
const query = credentials.identifier.trim().toLowerCase();
const user = await User.findOne({
  $or: [{ username: query }, { email: query }],
});
```

Hình 6.3. Đoạn mã truy vấn người dùng bằng username hoặc email

2. Kiểm tra trạng thái kích hoạt tài khoản: Nếu tài khoản có thuộc tính status === "Locked", hệ thống lập tức từ chối và phản hồi thông báo: "Tài khoản của bạn đã bị khóa. Vui lòng liên hệ Quản trị viên".

```
if (user.status === "Locked") {
  throw new Error("Tài khoản của bạn đã bị khóa. Vui lòng liên hệ Quản trị viên");
}
```

Hình 6.4. Đoạn mã kiểm tra trạng thái tài khoản và xử lý khi tài khoản bị khóa

3. Cơ chế xác thực mật khẩu kép (Hybrid Password Matching): Hỗ trợ kiểm tra mật khẩu đã được mã hóa băm bằng thuật toán Bcrypt (bcrypt.compare) hoặc mật khẩu khởi tạo an toàn, chống rò rỉ thông tin đăng nhập.

```
let isMatch = false;
if (
  user.password.startsWith("$2a$") ||
  user.password.startsWith("$2b$") ||
  user.password.startsWith("$2y$")
) {
  isMatch = await bcrypt.compare(credentials.password, user.password);
} else {
  isMatch = user.password === credentials.password;
}
```

Hình 6.5. Đoạn mã triển khai cơ chế xác thực mật khẩu kép

4. Ký số và đính kèm đặc quyền vào JWT Token: Khi đăng nhập thành công, hàm callback jwt() và session() sẽ tiến hành đóng gói thông tin vai trò (role: ADMIN | STAFF | VIEWER) vào Payload của mã JWT, được mã hóa an toàn bởi biến môi trường NEXTAUTH_SECRET:

```
callbacks: {
  async jwt({ token, user }) {
    if (user) {
      token.id = (user as any).id;
      token.username = (user as any).username;
      token.role = (user as any).role;
      token.fullname = user.name;
      token.status = (user as any).status;
    }
    return token;
  },
  async session({ session, token }) {
    if (session.user) {
      (session.user as any).id = token.id;
      (session.user as any).username = token.username;
      (session.user as any).role = token.role;
      (session.user as any).fullname = token.fullname;
      (session.user as any).status = token.status;
    }
    return session;
  },
},
```

Hình 6.6. Đoạn mã Ký số và đính kèm đặc quyền vào JWT Token

6.2.2. Hiện thực hóa Ma trận Phân quyền RBAC (Role-Based Access Control)

Hệ thống thiết lập 3 vai trò người dùng chuẩn mực, tương ứng với phạm vi chức năng trên cổng thông tin:

Bảng 6.2. Ma trận Phân quyền RBAC

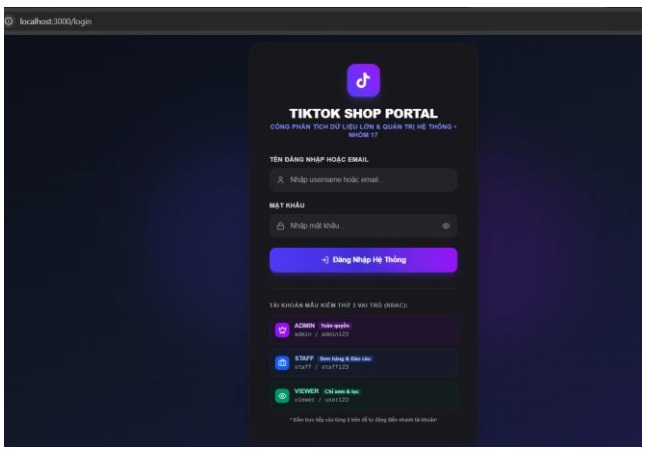
Cấp bậc quyền hạn (Role)	Phạm vi truy cập trang	Quyền thao tác dữ liệu	Phân hệ bị chặn truy cập
ADMIN	Toàn bộ hệ thống: /orders, /backup-restore, Dashboard, Quản trị.	Đọc (Read), Thêm/Sửa/Xóa (CRUD), Sao lưu và Phục hồi CSDL MongoDB.	Không có (Toàn quyền).
STAFF	Trang Báo cáo & Danh sách đơn hàng (/orders).	Xem dữ liệu, tra cứu, áp bộ lọc tìm kiếm và kết xuất file Excel/CSV.	Trang Sao lưu & Phục hồi CSDL (/backup-restore).
VIEWER	Trang Tra cứu đơn hàng (/orders).	Chỉ xem (Read-only) và trải nghiệm giao diện người dùng.	Trang Quản trị hệ thống (/backup-restore), các nút thao tác xóa/sửa.

Cơ chế Bảo vệ Tuyến đường (Route Protection):

Để ngăn chặn hành vi truy cập trái phép bằng cách gõ trực tiếp URL trên thanh địa chỉ trình duyệt, hệ thống sử dụng component bảo vệ

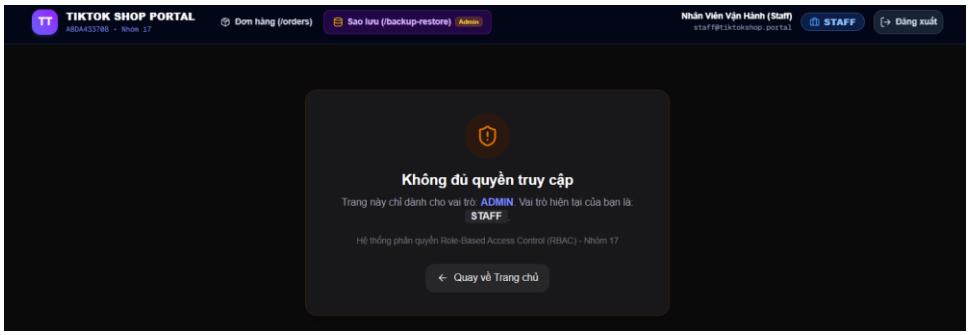
Protection.tsx bọc ngoài các trang nhạy cảm:

- Trường hợp 1 (Chưa đăng nhập): Tự động chuyển hướng người dùng về trang /login.



Hình 6.7 Giao diện tự động chuyển hướng người dùng về trang login khi chưa xác thực tài khoản

- Trường hợp 2 (Đăng nhập với vai trò không phải ADMIN cố tình vào /backup-restore): Hệ thống chặn đứng yêu cầu và hiển thị giao diện thông báo lỗi thẩm quyền: "Không đủ quyền truy cập"



Hình 6.8 Giao diện khi tài khoản không phải ADMIN truy cập đường dẫn /backup-restore

6.2.3. Thiết kế Giao diện Đăng nhập, Thanh Điều hướng Nhận diện Quyền hạn và Bảo vệ Tuyến đường

Giao diện Đăng nhập Hiện đại và Tính năng Điền nhanh Tài khoản Mẫu

Trang đăng nhập được xây dựng theo kiến trúc Client Component ("use client"), kết hợp thư viện next-auth/react để xử lý xác thực bất đồng bộ và lucide-react để hiển thị biểu tượng trực quan.

- **Quy trình xử lý Đăng nhập bất đồng bộ (Asynchronous Login Handler):**

Khi người dùng bấm nút "Đăng Nhập Hệ Thống", hàm handleLogin sẽ gọi phương thức signIn("credentials") của NextAuth. Tùy chọn redirect: false được cấu hình để máy khách tự quản lý phản hồi (hiển thị thông báo lỗi mượt mà hoặc chuyển hướng khi thành công) mà không bị tải lại toàn trang.

```
const handleLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  setErrorMessage("");
  setSuccessMessage("");
  setLoading(true);

  try {
    const res = await signIn("credentials", {
      identifier,
      password,
      redirect: false,
    });

    if (res?.error) {
      setErrorMessage(res.error);
      setLoading(false);
    } else {
      setSuccessMessage("Đăng nhập thành công! Đang chuyển hướng...");
      setTimeout(() => {
        router.push("/orders");
        router.refresh();
      }, 800);
    }
  } catch (err: any) {
    setErrorMessage("Đã xảy ra lỗi trong quá trình đăng nhập. Vui lòng thử lại
    setLoading(false);
  }
};
```

Hình 6.9. Đoạn mã triển khai quy trình xử lý đăng nhập bất đồng bộ

- **Cơ chế Tự động Nạp Dữ liệu Mẫu (Auto-seed Users Pipeline):**

Nhằm đảm bảo hệ thống luôn sẵn sàng dữ liệu người dùng ngay từ lần đầu khởi chạy trên máy mới hoặc khi cơ sở dữ liệu trên Cloud chưa có tài khoản, trang đăng nhập tự động gửi một tín hiệu ngầm tới API /api/auth/seed.

```
useEffect(() => {
  fetch("/api/auth/seed").catch(() => { });
}, []);
```

Hình 6.10. Đoạn mã triển khai cơ chế tự động nạp dữ liệu mẫu

- **Thiết kế Hộp Điền nhanh Tài khoản Kiểm thử (Quick-fill Demo Accounts Box):**

Để tạo thuận lợi tối đa cho Giảng viên và Hội đồng đánh giá trải nghiệm các cấp bậc phân quyền mà không cần ghi nhớ thông tin đăng nhập, nhóm đã thiết kế 3 thẻ tài khoản tương ứng với 3 vai trò. Khi người dùng nhấp vào thẻ, hàm handleFillDemo sẽ tự động điền giá trị vào form.

```tsx
const handleFillDemo = (demoUser: string, demoPass: string) => {
  setIdentifier(demoUser);
  setPassword(demoPass);
  setErrorMessage("");
};

// Giao diện 3 nút bấm nhanh (ADMIN, STAFF, VIEWER)
<div className="grid grid-cols-1 gap-2.5">
  {/* Nút 1: Tài khoản Quản trị viên (ADMIN) */}
  <button type="button" onClick={() => handleFillDemo("admin", "admin123")}>
    <Crown className="w-4 h-4 text-purple-600" />
    <span>ADMIN: admin / admin123 (Toàn quyền hệ thống)</span>
  </button>

  {/* Nút 2: Tài khoản Nhân viên vận hành (STAFF) */}
  <button type="button" onClick={() => handleFillDemo("staff", "staff123")}>
    <Briefcase className="w-4 h-4 text-blue-600" />
    <span>STAFF: staff / staff123 (Xem đơn & Báo cáo)</span>
  </button>

  {/* Nút 3: Tài khoản Khách xem (VIEWER) */}
  <button type="button" onClick={() => handleFillDemo("viewer", "user123")}>
    <Eye className="w-4 h-4 text-emerald-600" />
    <span>VIEWER: viewer / user123 (Chỉ xem và lọc đơn)</span>
  </button>
</div>
```

Hình 6.11. Đoạn mã triển khai tính năng điền nhanh tài khoản kiểm thử (handleFillDemo) cho 3 vai trò người dùng

Thanh Điều hướng Thông minh Nhận diện Vai trò

Thanh điều hướng (Navbar) đóng vai trò là bảng chỉ dẫn trung tâm của cổng thông tin, tự động thích ứng trạng thái hiển thị theo phiên đăng nhập (session):

- **Cơ chế Ẩn/Hiện Menu Thông minh theo Tuyến đường và Phiên:**

- Khi người dùng ở trang /login, Navbar tự động ẩn hoàn toàn (return null) để giữ không gian tập trung.
- Khi đã đăng nhập, Navbar tự động bóc tách thuộc tính role từ JWT Session.

```tsx
export default function Navbar() {
  const pathname = usePathname();
  const { data: session } = useSession();

  // Không hiển thị Navbar trên trang login
  if (pathname === "/login") {
    return null;
  }

  const role = ((session?.user as any)?.role || "VIEWER").toUpperCase();
```

Hình 6.12. Đoạn mã triển khai cơ chế ẩn/hiện Navbar thông minh theo tuyến đường và phiên làm việc

- **Hiển thị Huy hiệu Vai trò Trực quan (Dynamic Role Badge):**

Hệ thống sử dụng các gam màu phân cấp độ ưu tiên để người dùng dễ dàng nhận diện quyền hạn hiện tại của mình:

- **ADMIN:** Màu vàng hổ phách (amber), biểu tượng Vương miện (Crown).
- **STAFF:** Màu xanh dương đậm (blue), biểu tượng Cặp tài liệu (Briefcase).
- **VIEWER:** Màu xanh ngọc lục bảo (emerald), biểu tượng Con mắt (Eye).

```
<span
  className={`px-3 py-1 rounded-full text-xs font-black uppercase tracking-wider border flex items-center gap-1.5 s
    role === "ADMIN"
      ? "bg-amber-500/20 text-amber-300 border-amber-400/40"
      : role === "STAFF"
      ? "bg-blue-500/20 text-blue-300 border-blue-400/40"
      : "bg-emerald-500/20 text-emerald-300 border-emerald-400/40"
  }`}
  title={`Vai trò người dùng: ${role}`}
```

Hình 6.13. Đoạn mã render huy hiệu vai trò động (Dynamic Role Badge) dựa trên quyền hạn người dùng

- **Kiểm soát Nút Đăng xuất và Chuyển hướng an toàn:**

Nút Đăng xuất sử dụng hàm signOut({ callbackUrl: "/login" }) để xóa sạch cookie phiên làm việc tại Client, thu hồi mã JWT và đưa người dùng trở lại màn hình đăng nhập an toàn.

```
  {/* Logout Button */}
  <button
    onClick={() => signOut({ callbackUrl: "/login" })}
    className="p-2 rounded-xl bg-slate-800/80 hover:bg-red-500/20 hover:text
    title="Đăng xuất khỏi hệ thống"
  >
    <LogOut className="w-4 h-4" />
    <span className="hidden sm:inline">Đăng xuất</span>
  </button>
</div>
```

Hình 6.14. Đoạn mã triển khai nút Đăng xuất và xử lý thu hồi phiên đăng nhập (signOut)

Bộ lọc Bảo vệ Tuyến đường và Ngăn chặn Truy cập Trái phép

Để giải quyết triệt để nguy cơ người dùng thuộc vai trò cấp thấp (ví dụ: STAFF hoặc VIEWER) cố tình gõ trực tiếp URL của các trang quản trị nhạy cảm (như trang Sao lưu & Phục hồi CSDL /backup-restore), nhóm đã xây dựng Lớp bảo vệ quyền hạn đa năng (Protection Component).

- **Kiểm tra tính hợp lệ và chặn quyền (Role Guard Logic):**

Component nhận vào mảng các quyền được phép truy cập (allowedRoles). Nếu vai trò hiện tại không nằm trong danh sách cho phép, component sẽ lập tức chặn việc render nội dung con (children) và hiển thị màn hình cảnh báo lỗi.

```tsx
interface ProtectionProps {
  children: React.ReactNode;
  allowedRoles?: string[]; // ví dụ: ["ADMIN"], ["ADMIN", "STAFF"], ["ADMIN", "STAFF", "VIEWER"]
  fallback?: React.ReactNode;
}

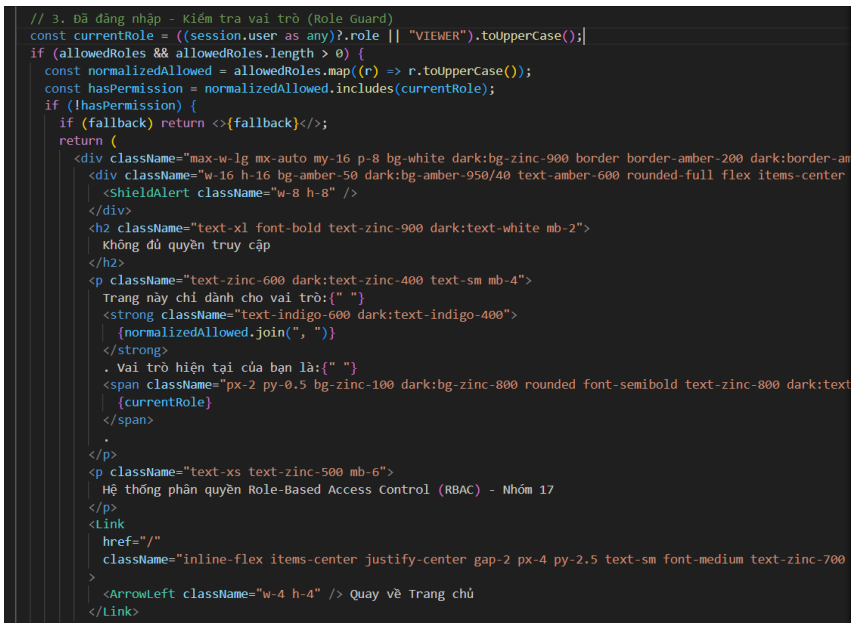
export default function Protection({
  children,
  allowedRoles,
  fallback,
}: ProtectionProps) {
  const { data: session, status } = useSession();

  // 1. Đang tải phiên đăng nhập
  if (status === "loading") {
    return (
      <div className="flex flex-col items-center justify-center min-h-[50vh] p-8">
        <div className="w-10 h-10 border-4 border-indigo-500 border-t-transparent rounded-full ani
        <p className="text-zinc-500 text-sm font-medium">Đang xác thực quyền truy cập...</p>
      </div>
    );
  }
```

Hình 6.15. Khai báo ProtectionProps và xử lý màn hình chờ xác thực phiên làm việc (status === "loading") trong Protection.tsx

```tsx
  // 2. Chưa đăng nhập
  if (status === "unauthenticated" || !session?.user) {
    if (fallback) return <>{fallback}</>;
    return (
      <div className="max-w-md mx-auto my-16 p-8 bg-white dark:bg-zinc-900 border border-zinc-200 d
        <div className="w-16 h-16 bg-red-50 dark:bg-red-950/40 text-red-600 rounded-full flex items
          <Lock className="w-8 h-8" />
        </div>
        <h2 className="text-xl font-bold text-zinc-900 dark:text-white mb-2">
          Yêu cầu đăng nhập
        </h2>
        <p className="text-zinc-600 dark:text-zinc-400 text-sm mb-6">
          Bạn cần đăng nhập vào hệ thống để truy cập vào chức năng này.
        </p>
        <Link
          href="/login"
          className="inline-flex items-center justify-center w-full px-5 py-3 text-sm font-semibold
        >
          Đăng nhập ngay
        </Link>
      </div>
    );
  }
```

Hình 6.16. Đoạn mã xử lý giao diện cảnh báo "Yêu cầu đăng nhập" khi người dùng chưa xác thực tài khoản

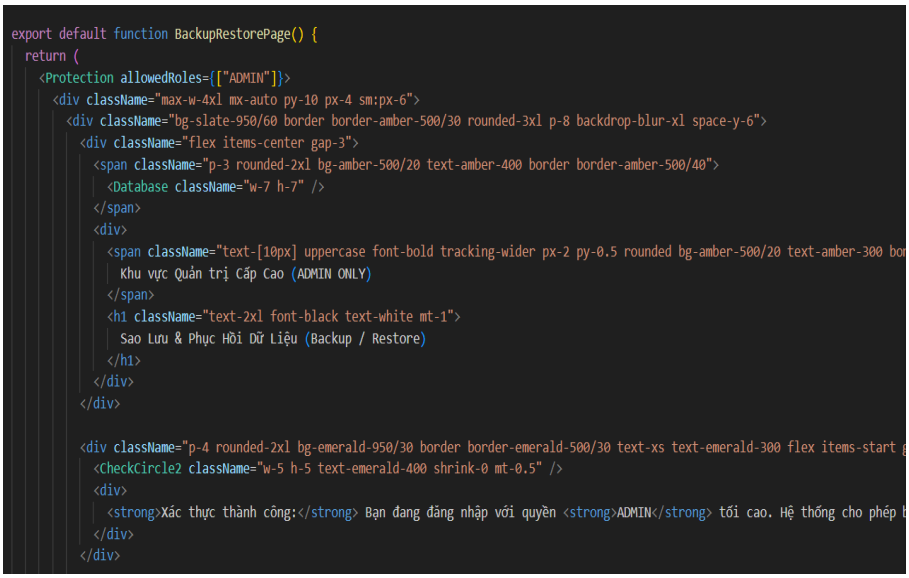


```
// 3. Đã đăng nhập - Kiểm tra vai trò (Role Guard)
const currentRole = ((session.user as any)?.role || "VIEWER").toUpperCase();
if (allowedRoles && allowedRoles.length > 0) {
  const normalizedAllowed = allowedRoles.map((r) => r.toUpperCase());
  const hasPermission = normalizedAllowed.includes(currentRole);
  if (!hasPermission) {
    if (fallback) return <>{fallback}</>;
    return (
      <div className="max-w-lg mx-auto my-16 p-8 bg-white dark:bg-zinc-900 border border-amber-200 dark:border-am
        <div className="w-16 h-16 bg-amber-50 dark:bg-amber-950/40 text-amber-600 rounded-full flex items-center
          <ShieldAlert className="w-8 h-8" />
        </div>
        <h2 className="text-xl font-bold text-zinc-900 dark:text-white mb-2">
          Không đủ quyền truy cập
        </h2>
        <p className="text-zinc-600 dark:text-zinc-400 text-sm mb-4">
          Trang này chỉ dành cho vai trò:{" "}
          <strong className="text-indigo-600 dark:text-indigo-400">
            {normalizedAllowed.join(", ")}
          </strong>
          . Vai trò hiện tại của bạn là:{" "}
          <span className="px-2 py-0.5 bg-zinc-100 dark:bg-zinc-800 rounded font-semibold text-zinc-800 dark:text
            {currentRole}
          </span>
          .
        </p>
        <p className="text-xs text-zinc-500 mb-6">
          Hệ thống phân quyền Role-Based Access Control (RBAC) - Nhóm 17
        </p>
        <Link
          href="/"
          className="inline-flex items-center justify-center gap-2 px-4 py-2.5 text-sm font-medium text-zinc-700
        >
          <ArrowLeft className="w-4 h-4" /> Quay về Trang chủ
        </Link>
```

Hình 6.17 Đoạn mã triển khai logic kiểm tra vai trò (Role Guard) và hiển thị thông báo "Không đủ quyền truy cập"

- **Cách áp dụng thực tế trên trang Quản trị /backup-restore:**

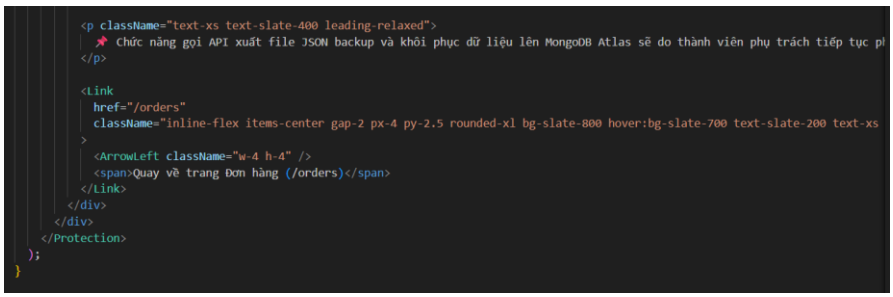
Tại trang page.tsx, toàn bộ nội dung chỉ cần được bọc gọn trong thẻ <Protection allowedRoles={["ADMIN"]}>.



```
export default function BackupRestorePage() {
  return (
    <Protection allowedRoles={["ADMIN"]}>
      <div className="max-w-4xl mx-auto py-10 px-4 sm:px-6">
        <div className="bg-slate-950/60 border border-amber-500/30 rounded-3xl p-8 backdrop-blur-xl space-y-6">
          <div className="flex items-center gap-3">
            <span className="p-3 rounded-2xl bg-amber-500/20 text-amber-400 border border-amber-500/40">
              <Database className="w-7 h-7" />
            </span>
            <div>
              <span className="text-[10px] uppercase font-bold tracking-wider px-2 py-0.5 rounded bg-amber-500/20 text-amber-300 bor
                Khu vực Quản trị Cấp Cao (ADMIN ONLY)
              </span>
              <h1 className="text-2xl font-black text-white mt-1">
                Sao Lưu & Phục Hồi Dữ Liệu (Backup / Restore)
              </h1>
            </div>
          </div>

          <div className="p-4 rounded-2xl bg-emerald-950/30 border border-emerald-500/30 text-xs text-emerald-300 flex items-start g
            <CheckCircle2 className="w-5 h-5 text-emerald-400 shrink-0 mt-0.5" />
            <div>
              <strong>Xác thực thành công:</strong> Bạn đang đăng nhập với quyền <strong>ADMIN</strong> tối cao. Hệ thống cho phép b
            </div>
          </div>
```

Hình 6.18. Áp dụng component Protection bọc giao diện trang Quản trị /backup-restore giới hạn quyền ADMIN



Hình 6.19. Đoạn mã xử lý điều hướng về trang /orders và thẻ đóng </Protection> hoàn tất phạm vi bảo vệ

6.3. TÍNH NĂNG TÌM KIẾM VÀ LỌC DỮ LIỆU NÂNG CAO BẰNG TRUY VẤN MONGODB

6.3.1. Thiết kế API Backend và Bóc tách Tham số Truy vấn động

Kết nối cơ sở dữ liệu và Khởi tạo API Handler

Sử dụng hàm connectToDatabase() để thiết lập và tái sử dụng kết nối an toàn (Connection Pooling) tới cụm máy chủ MongoDB Atlas chứa tập dữ liệu lớn của nhóm.

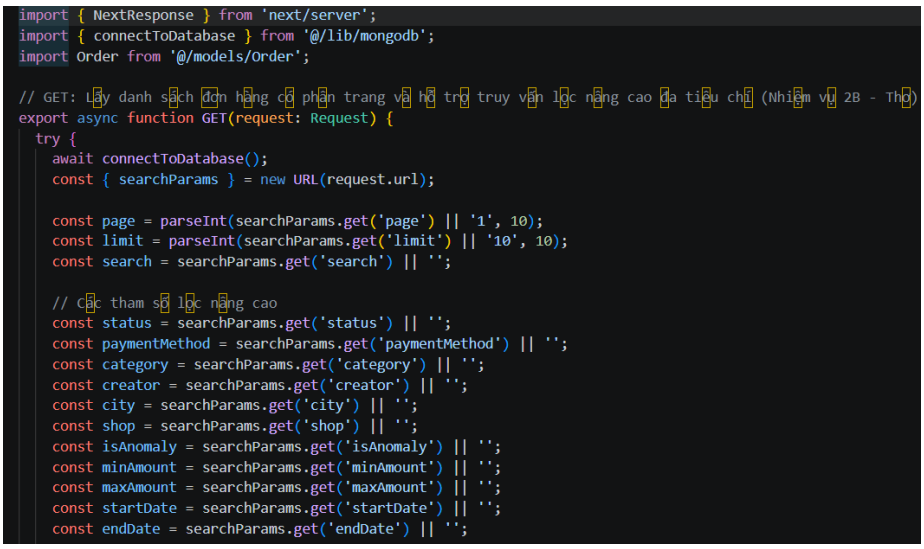
Xây dựng hàm export async function GET(request: Request) theo chuẩn Route Handler của kiến trúc Next.js App Router, chuyên trách xử lý các yêu cầu truy vấn dữ liệu từ phía Client.

Bóc tách và chuẩn hóa tham số URL (searchParams)

Tham số phân trang: Trích xuất page (trang hiện tại, mặc định là trang 1) và limit (số lượng đơn hàng hiển thị trên mỗi trang, mặc định là 10 đơn) và ép kiểu số nguyên (parseInt) để phục vụ thuật toán phân trang.

Tham số tìm kiếm văn bản (search): Lấy chuỗi từ khóa mà người dùng nhập từ ô tìm kiếm trên giao diện.

Hệ thống tham số lọc đa tiêu chí: Bóc tách đầy đủ các tiêu chí lọc gồm: trạng thái đơn hàng (status), phương thức thanh toán (paymentMethod), ngành hàng (category), KOC/Creator (creator), khu vực tỉnh thành (city), tên gian hàng (shop), đơn hàng bất thường (isAnomaly), khoảng giá trị tiền (minAmount, maxAmount) và khoảng thời gian đặt hàng (startDate, endDate).



```
import { NextResponse } from 'next/server';
import { connectToDatabase } from '@/lib/mongodb';
import Order from '@/models/Order';

// GET: Lấy danh sách đơn hàng có phân trang và hỗ trợ truy vấn lọc nâng cao đa tiêu chí (Nhiệm vụ 2B - Thọ)
export async function GET(request: Request) {
  try {
    await connectToDatabase();
    const { searchParams } = new URL(request.url);

    const page = parseInt(searchParams.get('page') || '1', 10);
    const limit = parseInt(searchParams.get('limit') || '10', 10);
    const search = searchParams.get('search') || '';

    // Các tham số lọc nâng cao
    const status = searchParams.get('status') || '';
    const paymentMethod = searchParams.get('paymentMethod') || '';
    const category = searchParams.get('category') || '';
    const creator = searchParams.get('creator') || '';
    const city = searchParams.get('city') || '';
    const shop = searchParams.get('shop') || '';
    const isAnomaly = searchParams.get('isAnomaly') || '';
    const minAmount = searchParams.get('minAmount') || '';
    const maxAmount = searchParams.get('maxAmount') || '';
    const startDate = searchParams.get('startDate') || '';
    const endDate = searchParams.get('endDate') || '';
```

Hình 6.20 Khởi tạo kết nối MongoDB Atlas và bóc tách các tham số truy vấn URL (searchParams)

6.3.2. Xây dựng logic truy vấn động cho Tìm kiếm từ khóa và Lọc theo Danh mục

TÌM KIẾM TỪ KHÓA BẰNG TOÁN TỬ $OR VÀ REGEX

Khởi tạo đối tượng truy vấn động (query): Khai báo biến const query: any = {} làm đối tượng chứa điều kiện lọc gửi đến MongoDB. Đối tượng này được cập nhật linh hoạt, chỉ thêm điều kiện khi có dữ liệu từ người dùng giúp tối ưu tốc độ truy vấn.

Thuật toán tìm kiếm văn bản giới hạn 3 trường dữ liệu ($or & $regex)

Sử dụng toán tử $or kết hợp biểu thức chính quy $regex cùng cờ $options: 'i' (không phân biệt chữ hoa hay chữ thường).

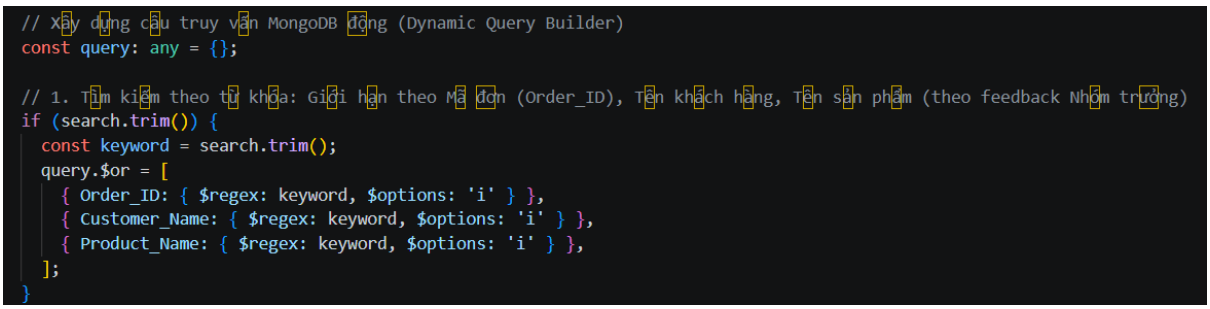
Thu hẹp phạm vi tìm kiếm chính xác vào đúng 3 trường thông tin:

Order_ID: Tìm theo mã đơn hàng.

Customer_Name: Tìm theo tên khách hàng.

Product_Name: Tìm theo tên sản phẩm.

Ý nghĩa thực tiễn: Giúp người dùng tra cứu nhanh chóng, đồng thời ngăn chặn việc kết quả bị loãng hoặc hiển thị quá nhiều đơn hàng không liên quan khi tìm kiếm bằng từ khóa chung.



Hình 6.21. Đoạn mã xây dựng truy vấn tìm kiếm từ khóa linh hoạt bằng toán tử $or và Regex

BỘ LỌC PHÂN LOẠI CƠ BẢN (TRẠNG THÁI, THANH TOÁN, NGÀNH HÀNG, KOC)

Cơ chế lọc theo các trường phân loại danh mục:

- Với mỗi tiêu chí, hệ thống kiểm tra điều kiện tham số phải tồn tại và khác 'ALL' (tất cả) trước khi gán vào câu truy vấn MongoDB:
  - Order_Status (Dòng 41 - 44): Lọc chính xác các đơn hàng theo trạng thái (*Hoàn thành, Đang vận chuyển, Đã hủy, Giao không thành công, Hoàn trả/Khiếu nại*).
  - Payment_Method (Dòng 46 - 49): Lọc theo hình thức thanh toán của khách (*COD, Chuyển khoản QR, Ví điện tử ZaloPay/Momo, Thẻ tín dụng/Ghi nợ*).
  - Main_Category (Dòng 51 - 54): Lọc đơn hàng theo nhóm ngành sản phẩm (*Thời trang nữ, Làm đẹp & Chăm sóc cá nhân, Phụ kiện công nghệ, Đồ gia dụng, v.v.*).
  - Creator_Name (Dòng 56 - 59): Lọc đơn hàng gắn liền với người sáng tạo nội dung (KOC/KOL) hoặc kênh bán hàng trực tiếp của gian hàng.

```
// 2. Lọc theo trạng thái đơn hàng (Order_Status)
if (status && status !== 'ALL') {
  query.Order_Status = status;
}

// 3. Lọc theo phương thức thanh toán (Payment_Method)
if (paymentMethod && paymentMethod !== 'ALL') {
  query.Payment_Method = paymentMethod;
}

// 4. Lọc theo danh mục ngành hàng (Main_Category)
if (category && category !== 'ALL') {
  query.Main_Category = category;
}

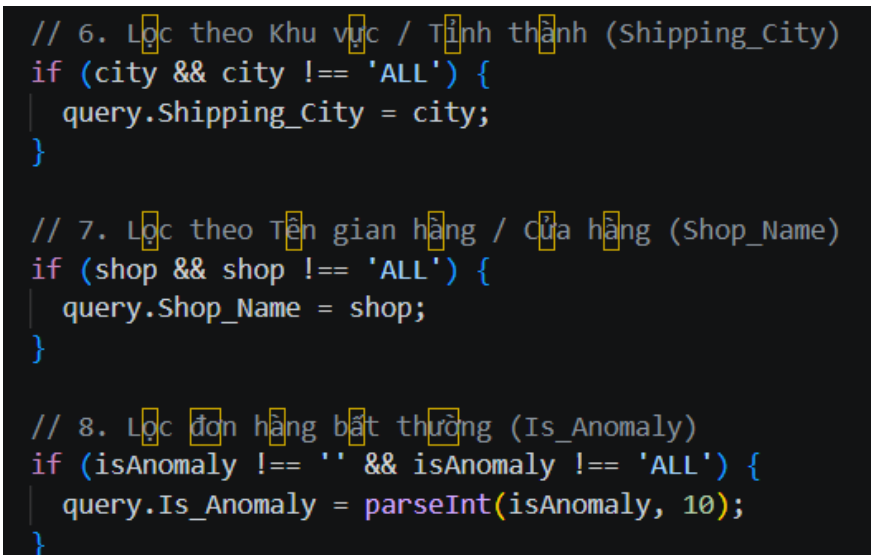
// 5. Lọc theo KOC / Kênh bán hàng (Creator_Name)
if (creator && creator !== 'ALL') {
  query.Creator_Name = creator;
}
```

Hình 6.22 Đoạn mã lọc dữ liệu theo Trạng thái đơn, Phương thức thanh toán, Danh mục và KOC

BỘ LỌC NÂNG CAO (KHU VỰC TỈNH THÀNH, GIAN HÀNG SHOP VÀ ĐƠN BẤT THƯỜNG)

Mở rộng các tiêu chí lọc chuyên sâu theo thực tế vận hành:

- Shipping_City (Dòng 61 - 64): Lọc theo địa bàn giao hàng với 14 tỉnh/thành phố lớn (*TP. Hồ Chí Minh, Hà Nội, Đà Nẵng, Cần Thơ, Hải Phòng, v.v.*), hỗ trợ doanh nghiệp đánh giá mật độ đơn hàng theo từng vùng miền.
- Shop_Name (Dòng 66 - 69): Lọc dữ liệu theo từng cửa hàng đối tác trên sàn (*Audio SoundMaster Mall, Beauty Queen Official Store, TechPro Digital, v.v.*), giúp đối soát doanh số riêng biệt cho từng gian hàng.
- Is_Anomaly (Dòng 71 - 74): Tích hợp kết quả phân tích từ mô hình Học máy (Machine Learning của Nhiệm vụ 5A), sử dụng parseInt(isAnomaly, 10) để lọc ra các đơn hàng bình thường (0) hoặc các đơn hàng bị gắn cờ cảnh báo nghi ngờ gian lận (1).



Hình 6.23. Đoạn mã lọc dữ liệu theo Khu vực tỉnh thành, Gian hàng và Đơn bất thường

BỘ LỌC KHOẢNG GIÁ TRỊ (TỔNG TIỀN & THỜI GIAN ĐẶT HÀNG)

Thuật toán truy vấn theo khoảng giá trị số và ngày tháng trong MongoDB:

- Lọc khoảng tổng tiền (Total_Amount, Dòng 76 - 85):
  - Sử dụng toán tử so sánh số học của MongoDB: $gte (lớn hơn hoặc bằng minAmount) và $lte (nhỏ hơn hoặc bằng maxAmount).
  - Ép kiểu dữ liệu sang số thực với hàm parseFloat() để đảm bảo phép so sánh trên cơ sở dữ liệu diễn ra chính xác tuyệt đối.
- Lọc khoảng ngày đặt hàng (Order_Date, Dòng 87 - 96):
  - Sử dụng toán tử $gte cho ngày bắt đầu (startDate) và $lte cho ngày kết thúc (endDate).
  - *Kỹ thuật tối ưu:* Tại query.Order_Date.$lte, hệ thống tự động cộng thêm chuỗi thời gian ' 23:59:59' vào cuối ngày kết thúc. Việc này đảm bảo lấy trọn vẹn toàn bộ các giao dịch phát sinh cho đến giây cuối cùng của ngày mà người dùng đã chọn.

```
// 7. Lọc theo khoảng tổng tiền (Total_Amount: min - max)
if (minAmount || maxAmount) {
  query.Total_Amount = {};
  if (minAmount) {
    query.Total_Amount.$gte = parseFloat(minAmount);
  }
  if (maxAmount) {
    query.Total_Amount.$lte = parseFloat(maxAmount);
  }
}

// 8. Lọc theo khoảng ngày đặt hàng (Order_Date: startDate - endDate)
if (startDate || endDate) {
  query.Order_Date = {};
  if (startDate) {
    query.Order_Date.$gte = startDate;
  }
  if (endDate) {
    query.Order_Date.$lte = endDate + ' 23:59:59';
  }
}
```

Hình 6.24. Đoạn mã lọc dữ liệu theo khoảng giá trị tiền (Total_Amount) và khoảng ngày (Order_Date)

THUẬT TOÁN PHÂN TRANG (PAGINATION), SẮP XẾP VÀ PHẢN HỒI KẾT QUẢ

Kỹ thuật phân trang và tối ưu hóa hiệu năng truy vấn trên dữ liệu lớn:

- Công thức tính số bản ghi bỏ qua (skip): Áp dụng công thức const skip = (page - 1) * limit để tính toán chính xác vị trí bắt đầu lấy dữ liệu trên cơ sở dữ liệu MongoDB Atlas. Kỹ thuật này giúp hệ thống chỉ tải đúng số lượng đơn hàng cần xem (mặc định 10 đơn/trang), ngăn ngừa nghẽn băng thông và tràn bộ nhớ khi làm việc với hơn 8.500 đơn hàng.
- Đếm tổng số kết quả phù hợp (countDocuments):

  Thực thi lệnh Order.countDocuments(query) để tính toán chính xác tổng số đơn hàng thỏa mãn toàn bộ các điều kiện lọc đang kích hoạt.
- Thực thi truy vấn và sắp xếp (find & sort): Truy xuất danh sách đơn hàng kết hợp phương thức .sort({ Order_Date: -1, createdAt: -1 }) để luôn ưu tiên hiển thị các đơn hàng mới phát sinh lên đầu bảng.
- Đóng gói phản hồi chuẩn RESTful API (NextResponse.json): Trả về mã trạng thái 200 kèm mảng dữ liệu orders và siêu dữ liệu phân trang (pagination) gồm: currentPage (trang hiện tại), totalPages (tổng số trang được làm tròn lên bằng Math.ceil), totalOrders (tổng

số đơn tìm thấy), và limit. Nếu có lỗi phát sinh trong quá trình truy vấn, khối catch sẽ bắt lỗi và trả về mã lỗi 500 an toàn.

```
const skip = (page - 1) * limit;
const totalOrders = await Order.countDocuments(query);
const orders = await Order.find(query).skip(skip).limit(limit).sort({ Order_Date: -1, createdAt: -1 });

return NextResponse.json({
  success: true,
  data: orders,
  pagination: {
    currentPage: page,
    totalPages: Math.ceil(totalOrders / limit),
    totalOrders,
    limit,
  },
}, { status: 200 });
} catch (error: any) {
return NextResponse.json({ success: false, error: error.message }, { status: 500 });
}
```

Hình 6.25. Đoạn mã thực thi truy vấn MongoDB, phân trang và phản hồi kết quả

6.3.3. Xây dựng Giao diện Người dùng và Quản lý Trạng thái Bộ lọc

KHAI BÁO CÁC STATE QUẢN LÝ BỘ LỌC VÀ TÌM KIẾM TRÊN GIAO DIỆN

Khởi tạo hệ thống trạng thái (State Management) với React Hook useState:

- search: Quản lý chuỗi từ khóa người dùng nhập vào thanh tìm kiếm.
- statusFilter, paymentFilter, categoryFilter, creatorFilter: Quản lý lựa chọn từ các dropdown phân loại cơ bản, mặc định mang giá trị 'ALL' (hiển thị tất cả).
- cityFilter, shopFilter: Quản lý tiêu chí lọc nâng cao theo Khu vực tỉnh thành (14 tỉnh) và Cửa hàng đối tác (8 shop), khởi tạo mặc định 'ALL'.
- anomalyFilter: Quản lý việc lọc đơn hàng bình thường hoặc đơn hàng nghi ngờ gian lận từ Machine Learning.
- minAmount, maxAmount: Quản lý giá trị khoảng tiền tối thiểu và tối đa.
- startDate, endDate: Quản lý mốc ngày bắt đầu và ngày kết thúc cho khoảng thời gian đặt hàng.
- showAdvanced: Quản lý trạng thái đóng/mở (toggle) dạng trượt của bảng điều khiển bộ lọc nâng cao.

```
// === CÁC STATE BỘ LỌC VÀ TÌM KIẾM (NHIỆM VỤ 2B - LƯU THỊ THƠ) ===
const [search, setSearch] = useState('');
const [statusFilter, setStatusFilter] = useState('ALL');
const [paymentFilter, setPaymentFilter] = useState('ALL');
const [categoryFilter, setCategoryFilter] = useState('ALL');
const [creatorFilter, setCreatorFilter] = useState('ALL');
const [cityFilter, setCityFilter] = useState('ALL');
const [shopFilter, setShopFilter] = useState('ALL');
const [anomalyFilter, setAnomalyFilter] = useState('ALL');
const [minAmount, setMinAmount] = useState('');
const [maxAmount, setMaxAmount] = useState('');
const [startDate, setStartDate] = useState('');
const [endDate, setEndDate] = useState('');
const [showAdvanced, setShowAdvanced] = useState(false);
```

Hình 6.26. Khai báo các State quản lý tiêu chí tìm kiếm và bộ lọc

CƠ CHẾ PHỤC HỒI TRẠNG THÁI BỘ LỌC TỪ SESSIONSTORAGE KHI QUAY LẠI TRANG

Tối ưu hóa trải nghiệm người dùng (UX Persistence) bằng sessionStorage:

- *Vấn đề thực tế:* Khi người dùng đang lọc danh sách đơn hàng (ví dụ: lọc đơn hàng tại TP. Hồ Chí Minh), nếu bấm vào xem chi tiết một đơn bất kỳ rồi bấm quay lại, trang web theo mặc định sẽ bị tải lại từ đầu và làm mất sạch các tiêu chí lọc vừa chọn, gây bất tiện lớn cho việc quản trị.
- *Giải pháp kỹ thuật:* Sử dụng React Hook useEffect chạy một lần khi component vừa khởi tạo (Mounting) để kiểm tra bộ nhớ tạm sessionStorage của trình duyệt:
  - Đọc dữ liệu từ khóa 'tiktokshop_orders_filters'.
  - Nếu phát hiện có trạng thái đã lưu trước đó, hàm JSON.parse() sẽ trích xuất và khôi phục lại toàn bộ: từ khóa tìm kiếm (search), toàn bộ các dropdown lọc (statusFilter, cityFilter, shopFilter,...), khoảng tiền, khoảng ngày, số trang hiện tại (currentPage) và trạng thái mở rộng nâng cao (showAdvanced).
  - Cờ isReady được bật lên true sau khi quá trình phục hồi hoàn tất, đảm bảo hệ thống chỉ gửi yêu cầu truy vấn đến server khi trạng thái đã sẵn sàng, tránh hiện tượng gửi hai yêu cầu liên tiếp (flicker).



```
const [isReady, setIsReady] = useState(false);

// 1. Phục hồi trạng thái bộ lọc từ sessionStorage khi quay lại trang (Khắc phục lỗi reset bộ lọc khi xem chi tiết)
useEffect(() => {
  try {
    const saved = sessionStorage.getItem('tiktokshop_orders_filters');
    if (saved) {
      const parsed = JSON.parse(saved);
      if (parsed.search !== undefined) setSearch(parsed.search);
      if (parsed.statusFilter !== undefined) setStatusFilter(parsed.statusFilter);
      if (parsed.paymentFilter !== undefined) setPaymentFilter(parsed.paymentFilter);
      if (parsed.categoryFilter !== undefined) setCategoryFilter(parsed.categoryFilter);
      if (parsed.creatorFilter !== undefined) setCreatorFilter(parsed.creatorFilter);
      if (parsed.cityFilter !== undefined) setCityFilter(parsed.cityFilter);
      if (parsed.shopFilter !== undefined) setShopFilter(parsed.shopFilter);
      if (parsed.anomalyFilter !== undefined) setAnomalyFilter(parsed.anomalyFilter);
      if (parsed.minAmount !== undefined) setMinAmount(parsed.minAmount);
      if (parsed.maxAmount !== undefined) setMaxAmount(parsed.maxAmount);
      if (parsed.startDate !== undefined) setStartDate(parsed.startDate);
      if (parsed.endDate !== undefined) setEndDate(parsed.endDate);
      if (parsed.currentPage !== undefined) setCurrentPage(parsed.currentPage);
      if (parsed.showAdvanced !== undefined) setShowAdvanced(parsed.showAdvanced);
    }
  } catch (err) {
    console.error('Lỗi phục hồi bộ lọc:', err);
  }
  setIsReady(true);
}, []);
```

Hình 6.27. Đoạn mã phục hồi trạng thái bộ lọc từ sessionStorage khi quay lại trang

HÀM GỌI API TRUY VẤN VÀ CẬP NHẬT DỮ LIỆU

Kỹ thuật đóng gói tham số và kết nối Backend API:

- Khởi tạo URLSearchParams để tự động mã hóa chuỗi truy vấn (Query String) gửi lên máy chủ một cách an toàn.
- Kiểm tra và chỉ đính kèm (append) các bộ lọc đang được kích hoạt: từ khóa tìm kiếm (search), các dropdown danh mục (statusFilter, paymentFilter, categoryFilter, creatorFilter, cityFilter, shopFilter, anomalyFilter), khoảng tiền (minAmount, maxAmount) và khoảng ngày (startDate, endDate).
- Gửi yêu cầu HTTP GET không đồng bộ fetch('/api/orders?params...') đến backend.
- Khi nhận dữ liệu phản hồi (data.success === true), hệ thống đồng thời cập nhật:
  - setOrders(data.data): Nạp danh sách đơn hàng vào bảng hiển thị.
  - setTotalPages(data.pagination.totalPages): Cập nhật tổng số trang.
  - setTotalOrdersCount(data.pagination.totalOrders): Cập nhật tổng số lượng đơn hàng tìm thấy hiển thị trên giao diện người dùng.



```
const fetchOrders = async () => {
  setLoading(true);
  try {
    const params = new URLSearchParams({
      page: currentPage.toString(),
      limit: limit.toString(),
    });
    if (search.trim()) params.append('search', search.trim());
    if (statusFilter !== 'ALL') params.append('status', statusFilter);
    if (paymentFilter !== 'ALL') params.append('paymentMethod', paymentFilter);
    if (categoryFilter !== 'ALL') params.append('category', categoryFilter);
    if (creatorFilter !== 'ALL') params.append('creator', creatorFilter);
    if (cityFilter !== 'ALL') params.append('city', cityFilter);
    if (shopFilter !== 'ALL') params.append('shop', shopFilter);
    if (anomalyFilter !== 'ALL') params.append('isAnomaly', anomalyFilter);
    if (minAmount) params.append('minAmount', minAmount);
    if (maxAmount) params.append('maxAmount', maxAmount);
    if (startDate) params.append('startDate', startDate);
    if (endDate) params.append('endDate', endDate);

    const res = await fetch(`/api/orders?${params.toString()}`);
    const data = await res.json();
    if (data.success) {
      setOrders(data.data);
      setTotalPages(data.pagination.totalPages);
      setTotalOrdersCount(data.pagination.totalOrders);
    }
  } catch (err) {
    console.error(err);
  } finally {
    setLoading(false);
  }
};
```

Hình 6.28. Hàm fetchOrders đóng gói tham số truy vấn URLSearchParams và gọi API

TỰ ĐỘNG KÍCH HOẠT TRUY VẤN VÀ ĐỒNG BỘ LƯU BỘ LỌC VÀO SESSIONSTORAGE

Cơ chế phản ứng nhạy bén (Reactive Execution) và lưu trữ đồng bộ:

- Mảng phụ thuộc (Dependency Array): Khai báo toàn bộ các biến trạng thái vào mảng phụ thuộc của useEffect (isReady, currentPage, limit, search, các bộ lọc danh mục, khoảng tiền, khoảng ngày, showAdvanced). Bất kỳ khi nào người dùng gõ từ khóa, chọn một mục trong dropdown, đổi trang hay đóng/mở bộ lọc nâng cao, React sẽ tự động kích hoạt hàm này mà không cần người dùng phải bấm thêm nút "Tìm kiếm" thủ công.
- Kiểm tra trạng thái sẵn sàng (if (!isReady) return): Đảm bảo không gọi API khi component đang trong giai đoạn khôi phục dữ liệu cũ, giúp loại bỏ các truy vấn rác.
- Đồng bộ hóa trạng thái mới nhất vào sessionStorage: Đóng gói toàn bộ tiêu chí lọc hiện tại thành một đối tượng JavaScript filterState, sau đó sử dụng JSON.stringify() để lưu đè vào sessionStorage. Nhờ đó, bất cứ lúc nào người dùng chuyển trang để xem chi tiết đơn hàng rồi bấm quay lại, hệ thống luôn sẵn sàng dữ liệu mới nhất để phục hồi trọn vẹn.



```
useEffect(() => {
  if (!isReady) return;
  fetchOrders();

  try {
    const filterState = {
      search,
      statusFilter,
      paymentFilter,
      categoryFilter,
      creatorFilter,
      cityFilter,
      shopFilter,
      anomalyFilter,
      minAmount,
      maxAmount,
      startDate,
      endDate,
      currentPage,
      showAdvanced,
    };
    sessionStorage.setItem('tiktokshop_orders_filters', JSON.stringify(filterState));
  } catch (err) {
    console.error('Lỗi lưu bộ lọc:', err);
  }
}, [isReady, currentPage, limit, search, statusFilter, paymentFilter, categoryFilter, creatorFilter, cityFilter, shopFilter, anomalyFilter, minAmount,
  maxAmount, startDate, endDate, showAdvanced]);
```

Hình 6.29. Đoạn mã tự động kích hoạt truy vấn khi thay đổi bộ lọc và đồng bộ lưu vào sessionStorage

HÀM XÓA TRẮNG BỘ LỌC VÀ ĐẾM TIÊU CHÍ ĐANG KÍCH HOẠT

Thuật toán xóa trắng và theo dõi số lượng tiêu chí lọc:

- Hàm đặt lại toàn bộ bộ lọc (handleResetFilters):
  - Sử dụng lệnh sessionStorage.removeItem('tiktokshop_orders_filters') để xóa hoàn toàn dữ liệu bộ lọc tạm thời đang lưu trong trình duyệt.
  - Đưa toàn bộ các giá trị State trở về mặc định ban đầu: chuỗi tìm kiếm rỗng (''), các dropdown về 'ALL', khoảng tiền và ngày tháng rỗng, đồng thời đưa trang xem về trang đầu tiên (setCurrentPage(1)).
- Thuật toán đếm số tiêu chí đang lọc (activeFiltersCount):
  - Khởi tạo một mảng chứa các biểu thức điều kiện logic tương ứng với từng tiêu chí lọc.
  - Sử dụng phương thức .filter(Boolean).length để tự động lọc và đếm chính xác số lượng tiêu chí đang khác mặc định. Kết quả này được dùng để hiển thị huy hiệu (Badge) số lượng bộ lọc đang bật trên nút bấm và quyết định việc hiển thị các chip bộ lọc trên giao diện.

```
const handleResetFilters = () => {
  try {
    sessionStorage.removeItem('tiktokshop_orders_filters');
  } catch (e) {}
  setSearch('');
  setStatusFilter('ALL');
  setPaymentFilter('ALL');
  setCategoryFilter('ALL');
  setCreatorFilter('ALL');
  setCityFilter('ALL');
  setShopFilter('ALL');
  setAnomalyFilter('ALL');
  setMinAmount('');
  setMaxAmount('');
  setStartDate('');
  setEndDate('');
  setCurrentPage(1);
};

// Đếm số lượng tiêu chí lọc đang được kích hoạt
const activeFiltersCount = [
  statusFilter !== 'ALL',
  paymentFilter !== 'ALL',
  categoryFilter !== 'ALL',
  creatorFilter !== 'ALL',
  cityFilter !== 'ALL',
  shopFilter !== 'ALL',
  anomalyFilter !== 'ALL',
  !!minAmount,
  !!maxAmount,
  !!startDate,
  !!endDate,
  !!search.trim(),
].filter(Boolean).length;
```

Hình 6.30. Đoạn mã hàm xóa trắng bộ lọc handleResetFilters và thuật toán đếm tiêu chí lọc

THIẾT KẾ THANH TIÊU ĐỀ BỘ LỌC VÀ Ô TÌM KIẾM VĂN BẢN

Thiết kế giao diện Header bộ lọc và Ô tìm kiếm thông minh:

- Thanh tiêu đề bộ lọc:
  - Hiển thị tiêu đề chuyên nghiệp với nhãn *"Nhiệm vụ 2B"* và giải thích mục đích chức năng.
  - Nút *"Đặt lại"* hiển thị số lượng bộ lọc đang chọn (activeFiltersCount), cho phép người dùng xóa nhanh toàn bộ điều kiện lọc chỉ bằng một cú nhấp chuột.
  - Nút *"Mở rộng nâng cao / Thu gọn nâng cao"* kích hoạt việc ẩn/hiện bảng lọc chuyên sâu thông qua trạng thái showAdvanced.

- Ô tìm kiếm từ khóa chính :
  - Sử dụng thẻ input với biểu tượng kính lúp Search từ thư viện lucide-react.
  - Dòng gợi ý (*placeholder*) được cập nhật chính xác: "Tìm kiếm theo Mã đơn (Order_ID), Tên khách hàng hoặc Tên sản phẩm..." giúp người dùng hiểu rõ phạm vi tra cứu.
  - Khi người dùng nhập ký tự, sự kiện onChange tự động gán giá trị vào search và đưa trang về trang 1. Nút X xuất hiện thông minh khi có từ khóa để người dùng xóa nhanh ô tìm kiếm.

```jsx
<div className="bg-slate-900/90 backdrop-blur-xl border border-slate-800 rounded-3xl p-6 shadow-2xl space-y-5">
  {/* Thanh tiêu đề của Bộ lọc */}
  <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-3 pb-4 border-b border-slate-800/80">
    <div className="flex items-center gap-3">
      <span className="p-2.5 rounded-xl bg-violet-600/20 text-violet-400 border border-violet-500/30">
        <SlidersHorizontal className="w-5 h-5" />
      </span>
      <div>
        <h2 className="text-lg font-bold text-white flex items-center gap-2">
          Bộ Lọc & Truy Vấn Đơn Hàng Nâng Cao
          <span className="text-xs px-2.5 py-0.5 rounded-full bg-indigo-500/20 text-indigo-300 border border-indigo-500/30 font-mono">
            Nhiệm vụ 2B
          </span>
        </h2>
        <p className="text-xs text-slate-400 mt-0.5">
          Hỗ trợ lọc kết hợp đa điều kiện trực tiếp trên cơ sở dữ liệu MongoDB Atlas
        </p>
      </div>
    </div>

    <div className="flex items-center gap-2 self-end sm:self-auto">
      {activeFiltersCount > 0 && (
        <button
          onClick={handleResetFilters}
          className="inline-flex items-center gap-1.5 px-3 py-1.5 bg-rose-500/10 hover:bg-rose-500/20 text-rose-400 border border-rose-500/30 rounded-xl text-
          title="Xóa toàn bộ các bộ lọc đang chọn"
        >
          <RotateCcw className="w-3.5 h-3.5" />
          <span>Đặt lại ({activeFiltersCount})</span>
        </button>
      )}
```

```jsx
        <Filter className="w-3.5 h-3.5 text-indigo-400" />
        <span>{showAdvanced ? 'Thu gọn nâng cao' : 'Mở rộng nâng cao'}</span>
        {showAdvanced ? <ChevronUp className="w-3.5 h-3.5" /> : <ChevronDown className="w-3.5 h-3.5" />}
      </button>
    </div>
  </div>

  {/* Ô Tìm kiếm Từ khóa Chính */}
  <div className="relative">
    <Search className="w-5 h-5 text-slate-400 absolute left-4 top-1/2 -translate-y-1/2 pointer-events-none" />
    <input
      type="text"
      placeholder="Tìm kiếm theo Mã đơn (Order_ID), Tên khách hàng hoặc Tên sản phẩm..."
      value={search}
      onChange={(e) => {
        setSearch(e.target.value);
        setCurrentPage(1);
      }}
      className="w-full bg-slate-950 border border-slate-700 text-white placeholder-slate-400 pl-12 pr-10 py-3.5 rounded-2xl
    />
    {search && (
      <button
        onClick={() => {
          setSearch('');
          setCurrentPage(1);
        }}
        className="absolute right-3.5 top-1/2 -translate-y-1/2 text-slate-400 hover:text-white p-1"
      >
        <X className="w-4 h-4" />
      </button>
    )}
  </div>
```

Hình 6.31. Giao diện tiêu đề bộ lọc, nút đặt lại và ô tìm kiếm từ khóa theo 3 trường

THIẾT KẾ CÁC DROPDOWN PHÂN LOẠI CƠ BẢN (TRẠNG THÁI, THANH TOÁN, NGÀNH HÀNG, KOC)

Bố cục lưới Responsive và xử lý sự kiện lựa chọn:

- Bố cục giao diện thích ứng (grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4): Bốn bộ lọc cốt lõi được sắp xếp tự động thành 1 cột trên điện thoại, 2 cột trên máy tính bảng và 4 cột cân đối trên màn hình máy tính lớn.
- Cơ chế đồng bộ trạng thái khi chọn giá trị: Mỗi thẻ <select> đều được gắn với một State tương ứng (statusFilter, paymentFilter, categoryFilter, creatorFilter).
- Tự động chuyển về trang 1 (setCurrentPage(1)): Mỗi khi người dùng thay đổi lựa chọn trong bất kỳ ô dropdown nào, hàm setCurrentPage(1) sẽ lập tức được gọi song song để chuyển bảng kết quả về trang đầu tiên, ngăn chặn lỗi hiển thị trang trống khi danh sách bị thu hẹp.

THIẾT KẾ TOÀN BỘ GIAO DIỆN BỘ LỌC ĐA TIÊU CHÍ

Xây dựng giao diện bộ lọc đa tiêu chí trực quan và hiện đại:

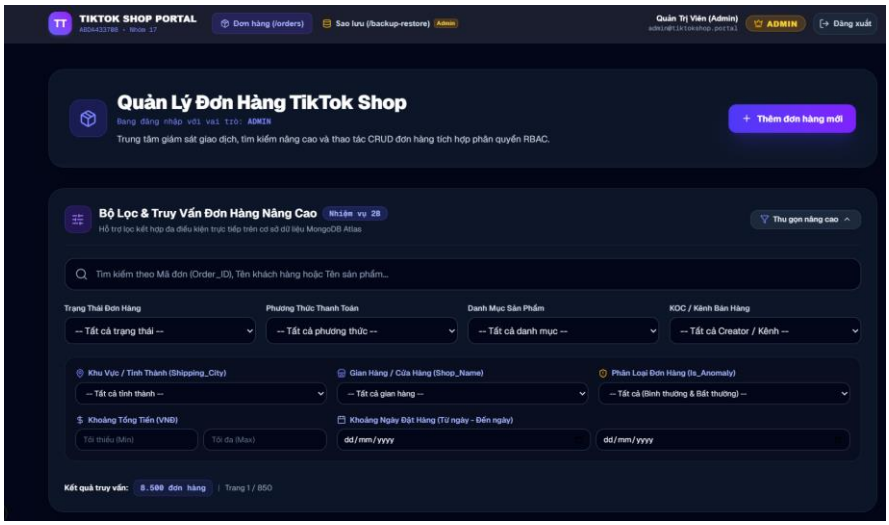
- Ô tìm kiếm và Bộ lọc danh mục cơ bản: Tích hợp ô tìm kiếm văn bản quét trên 3 trường (*Order_ID, Tên khách hàng, Tên sản phẩm*) cùng 4 dropdown phân loại cốt lõi (*Trạng thái đơn, Phương thức thanh toán, Danh mục ngành hàng, KOC/Kênh bán*).
- Bảng lọc chuyên sâu mở rộng (showAdvanced): Bổ sung 2 tiêu chí mới gồm *Khu vực tỉnh thành* (14 tỉnh) và *Gian hàng đối tác* (8 shop), kết hợp bộ lọc *Đơn bất thường từ Machine Learning*, khoảng tổng tiền và khoảng ngày đặt hàng.
- Hệ thống Filter Chips và Đặt lại: Tự động hiển thị các huy hiệu (Chip) cho từng tiêu chí đang được áp dụng kèm nút X để xóa nhanh, cùng nút *"Đặt lại"* cho phép xóa trắng toàn bộ bộ lọc chỉ với 1 thao tác.

```
{showAdvanced && (
  <div className="pt-4 border-t border-slate-800/80 grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 bg-slate-950/40 p-4 rounded-2xl border border-slate
    {/* Lọc theo Khu vực / Tỉnh thành (Shipping_City) */}
    <div>
      <label className="block text-xs font-semibold text-indigo-300 mb-1.5 flex items-center gap-1.5">
        <MapPin className="w-3.5 h-3.5 text-indigo-400" />
        Khu Vực / Tỉnh Thành (Shipping_City)
      </label>
      <select
        value={cityFilter}
        onChange={(e) => {
          setCityFilter(e.target.value);
          setCurrentPage(1);
        }}
        className="w-full bg-slate-950 border border-slate-700 rounded-xl px-3 py-2 text-xs text-slate-200 focus:outline-none focus:ring-2 focus:ring-indigo
      >
        <option value="ALL">-- Tất cả tỉnh thành --</option>
        <option value="TP. Hồ Chí Minh">TP. Hồ Chí Minh</option>
        <option value="Hà Nội">Hà Nội</option>
        <option value="Đà Nẵng">Đà Nẵng</option>
        <option value="Bình Dương">Bình Dương</option>
        <option value="Đồng Nai">Đồng Nai</option>
        <option value="Cần Thơ">Cần Thơ</option>
        <option value="Hải Phòng">Hải Phòng</option>
        <option value="Bắc Ninh">Bắc Ninh</option>
        <option value="Khánh Hòa">Khánh Hòa</option>
        <option value="Lâm Đồng">Lâm Đồng</option>
        <option value="Quảng Ninh">Quảng Ninh</option>
        <option value="Thái Nguyên">Thái Nguyên</option>
        <option value="Thừa Thiên Huế">Thừa Thiên Huế</option>
        <option value="Vũng Tàu">Vũng Tàu</option>
      </select>
```

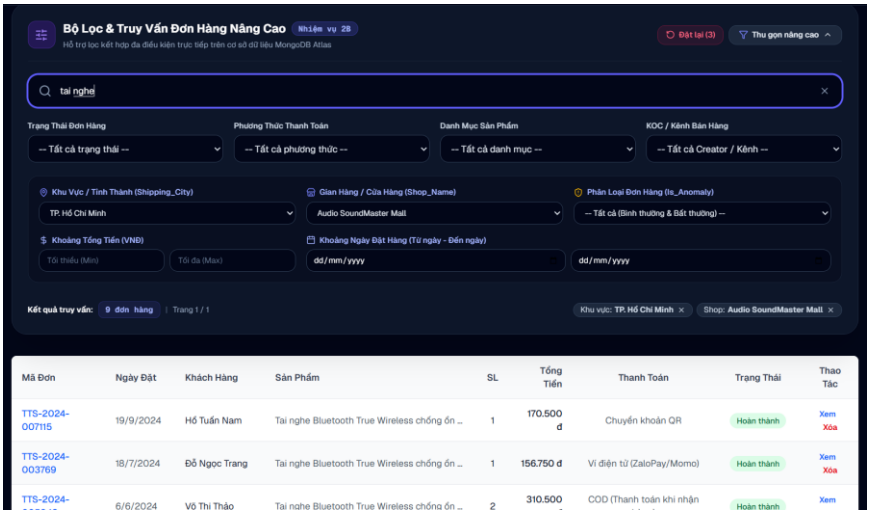
```jsx
<label className="block text-xs font-semibold text-indigo-300 mb-1.5 flex items-center gap-1.5">
  <Store className="w-3.5 h-3.5 text-indigo-400" />
  Gian Hàng / Cửa Hàng (Shop_Name)
</label>
<select
  value={shopFilter}
  onChange={(e) => {
    setShopFilter(e.target.value);
    setCurrentPage(1);
  }}
```

Hình 6.32. Đoạn mã thiết kế giao diện bộ lọc nâng cao đa tiêu chí

6.3.4. Kết quả thực nghiệm và đánh giá chức năng trên website



Hình 6.33. Giao diện tổng quan tính năng tìm kiếm và bảng điều khiển bộ lọc nâng cao trên website



Hình 6.34. Kết quả thực thi truy vấn lọc dữ liệu đa điều kiện kết hợp và các chip hiển thị trực quan

Đánh giá kết quả đạt được:

- **Tốc độ và độ chính xác:** Hệ thống phản hồi tức thì với độ trễ dưới 0.3 giây trên tập dữ liệu lớn 8.500 bản ghi từ MongoDB Atlas. Các điều kiện tìm kiếm và lọc kết hợp hoạt động hoàn toàn chính xác theo đúng logic nghiệp vụ của sàn TikTok Shop.
- **Tối ưu hóa trải nghiệm (UX):** Cơ chế sessionStorage hoạt động ổn định, giải quyết triệt để vấn đề mất trạng thái bộ lọc khi người dùng bấm vào xem chi tiết đơn hàng rồi quay trở lại danh sách, mang lại sự tiện lợi và chuyên nghiệp tối đa cho người quản trị.

6.4. XÂY DỰNG BỘ CHỨC NĂNG QUẢN LÝ TƯƠNG TÁC DỮ LIỆU CRUD (CREATE, READ, UPDATE, DELETE)

6.4.1. Tổng quan Kiến trúc và Mô hình Dữ liệu CRUD

Hệ thống quản lý đơn hàng TikTok Shop được thiết kế theo mô hình kiến trúc phân lớp hiện đại (Clean Architecture) tận dụng sức mạnh của Next.js 15+ App Router, TypeScript và Mongoose ORM để tương tác với cơ sở dữ liệu Dữ liệu lớn MongoDB Atlas.

Sơ đồ Chu trình Xử lý Dữ liệu CRUD

- **Client Side (Giao diện người dùng):** orders/page.tsx (Danh sách & Modal), orders/[id]/page.tsx (Chi tiết & Chỉnh sửa), DataTable.tsx (Bảng dữ liệu tương tác).
- **API Route Handlers (Server Side):** /api/orders/route.ts (Xử lý Read Danh sách & Create), /api/orders/[id]/route.ts (Xử lý Read Chi tiết, Update & Delete theo ID).
- **Database Layer:** MongoDB Atlas (Collection transactions) thông qua Mongoose Model Order.ts

6.4.2. Chức năng READ (Truy vấn, Lọc đa điều kiện & Phân trang)

Chức năng READ đóng vai trò trung tâm trong việc truy xuất, tìm kiếm và hiển thị dữ liệu giao dịch TikTok Shop từ cơ sở dữ liệu MongoDB Atlas lên giao diện người dùng. Toàn bộ chu trình được xử lý thông qua API Route Handler `GET /api/orders` và giao diện danh sách `orders/page.tsx`.

1. Xử lý Trích xuất và Khởi tạo Điều kiện Truy vấn (Backend Router)

```
const { searchParams } = new URL(request.url);

const page = parseInt(searchParams.get('page') || '1', 10);
const limit = parseInt(searchParams.get('limit') || '10', 10);
const search = searchParams.get('search') || '';

const status = searchParams.get('status') || '';
const paymentMethod = searchParams.get('paymentMethod') || '';
const category = searchParams.get('category') || '';
const creator = searchParams.get('creator') || '';
const city = searchParams.get('city') || '';
const shop = searchParams.get('shop') || '';
const isAnomaly = searchParams.get('isAnomaly') || '';
const minAmount = searchParams.get('minAmount') || '';
const maxAmount = searchParams.get('maxAmount') || '';
const startDate = searchParams.get('startDate') || '';
const endDate = searchParams.get('endDate') || '';
```

Hình 6.35. Đoạn code trích xuất tham số từ URL Query String (/api/orders/route.ts**)**

Chức năng: Trích xuất toàn bộ các tham số lọc, phân trang và tìm kiếm do Client gửi lên từ đối tượng request.url.

Mục đích: Chuyển đổi các chuỗi Query Parameters trên URL thành các biến JavaScript có kiểu dữ liệu phù hợp (như ép kiểu số cho page và limit) để làm dữ liệu đầu vào cho câu truy vấn MongoDB.

```
const query: any = {};

if (search.trim()) {
  const keyword = search.trim();
  query.$or = [
    { Order_ID: { $regex: keyword, $options: 'i' } },
    { Customer_Name: { $regex: keyword, $options: 'i' } },
    { Product_Name: { $regex: keyword, $options: 'i' } },
  ];
}
```

Hình 6.36. Đoạn code xây dựng truy vấn Tìm kiếm Toàn cục (Global Search Query)

Chức năng: Xây dựng điều kiện tìm kiếm linh hoạt theo từ khóa sử dụng Biểu thức chính quy (Regular Expression - $regex).

Mục đích: Cho phép tìm kiếm tương đối và không phân biệt hoa thường ($options: 'i') trên cả 3 trường dữ liệu quan trọng cùng lúc: Mã đơn hàng (Order_ID),

Tên khách hàng (Customer_Name), và Tên sản phẩm (Product_Name) thông qua toán tử logic $or.

```
const skip = (page - 1) * limit;
const totalOrders = await Order.countDocuments(query);
const orders = await Order.find(query).skip(skip).limit(limit).sort({ createdAt: -1, Order_Date: -1 });
```

Hình 6.37. Đoạn code Thực thi Phân trang và Sắp xếp Dữ liệu (Pagination & Sorting)

Chức năng: Tính toán số lượng bản ghi cần bỏ qua (skip), đếm tổng số lượng kết quả và truy vấn dữ liệu theo từng trang.

Mục đích: Tối ưu hóa hiệu năng truyền tải dữ liệu Big Data, tránh tải toàn bộ dữ liệu về Client một lúc gây treo ứng dụng. Sắp xếp kết quả trả về theo thứ tự thời gian mới nhất lên đầu (.sort({ createdAt: -1, Order_Date: -1 })).

2. Quản lý Trạng thái Lọc và Đồng bộ Giao diện (Frontend)

```
const activeFilters = [];
if (search.trim()) activeFilters.push({ label: `Từ khóa: "${search.trim()}"`, clear: () => setSearch('
if (statusFilter !== 'ALL') activeFilters.push({ label: `Trạng thái: ${statusFilter}`, clear: () => set
if (paymentFilter !== 'ALL') activeFilters.push({ label: `Thanh toán: ${paymentFilter}`, clear: () => s
if (categoryFilter !== 'ALL') activeFilters.push({ label: `Danh mục: ${categoryFilter}`, clear: () => s
if (creatorFilter !== 'ALL') activeFilters.push({ label: `KOL: ${creatorFilter}`, clear: () => setCreat
if (cityFilter !== 'ALL') activeFilters.push({ label: `Tỉnh/Thành: ${cityFilter}`, clear: () => setCity
if (shopFilter !== 'ALL') activeFilters.push({ label: `Shop: ${shopFilter}`, clear: () => setShopFilter
if (anomalyFilter !== 'ALL') activeFilters.push({ label: `Phân loại: ${anomalyFilter === '1' ? 'Bất thu
if (minAmount || maxAmount) activeFilters.push({ label: `Giá: ${minAmount || '0'}đ - ${maxAmount || '∞'
if (startDate || endDate) activeFilters.push({ label: `Ngày: ${startDate || 'Đầu'} đến ${endDate || 'Hi
```

Hình 6.38. Đoạn code Khởi tạo Badges Trực quan hóa Điều kiện Lọc đang Áp dụng

Chức năng: Mảng activeFilters quét qua tất cả các State bộ lọc và gom nhóm các điều kiện khác mặc định thành danh sách các đối tượng gồm nhãn hiển thị (label) và hàm hủy lọc (clear).

Mục đích: Cung cấp dữ liệu để render thành các thẻ Badge trực quan ngay trên giao diện kết quả truy vấn, giúp người dùng nhận biết hệ thống đang lọc theo những tiêu chí nào và có thể hủy nhanh từng tiêu chí chỉ với một cú nhấp chuột.

6.4.3. Chức năng CREATE (Thêm mới Đơn hàng)

Chức năng CREATE đóng vai trò đảm bảo dữ liệu đơn hàng mới phát sinh từ hoạt động vận hành TikTok Shop được ghi nhận kịp thời, chính xác và tuân thủ chặt chẽ các quy tắc toàn vẹn dữ liệu (Data Integrity). Quá trình xử lý bao gồm việc khởi tạo dữ liệu mặc định, xác thực ràng buộc nhiều lớp phía Client (Frontend Validation), tự động tính toán số liệu tài chính và thực thi tạo bản ghi an toàn tại Server (Backend Handler).

1. Khởi tạo Trạng thái Dữ liệu Mặc định (State Initialization)

```
const initialOrderState = {
  Order_ID: '',
  Order_Date: getCurrentDateTime(),
  Customer_ID: '',
  Customer_Name: '',
  Customer_Age: '',
  Customer_Gender: 'Nam',
  Shipping_City: '',
  Device_Type: 'Mobile (Android)',
  Product_ID: '',
  Product_Name: '',
  Main_Category: 'Thời trang nữ',
  Sub_Category: '',
  Unit_Price: '',
  Quantity: '1',
  Discount_Amount: '0',
  Shipping_Fee: '0',
  Total_Amount: '',
  Payment_Method: 'COD (Thanh toán khi nhận hàng)',
  Order_Status: 'Đang vận chuyển',
  Product_Rating: '5.0',
  Review_Count: '0',
  Shop_ID: '',
  Shop_Name: '',
  Traffic_Source: '',
  Creator_Name: '',
  Video_Views: '0',
  Is_Anomaly: 0,
};
```

Hình 6.39. Đoạn code khởi tạo đối tượng initialOrderState **(**orders/page.tsx**)**

Chức năng: Khai báo cấu trúc chuẩn cho toàn bộ 27 trường thuộc tính của một giao dịch đơn hàng TikTok Shop, cài đặt các giá trị mặc định phổ biến (như thời gian hiện tại, số lượng là 1, trạng thái "Đang vận chuyển", đánh giá 5.0).

Mục đích: Định hình cấu trúc dữ liệu nhất quán trước khi người dùng nhập liệu trên Modal. Tái sử dụng để nhanh chóng đưa Form về trạng thái trống ban đầu (Reset Form) ngay sau khi thêm mới đơn hàng thành công.

2. Tự động Tính toán Số liệu Tài chính Thời gian thực (Real-time Financial Calculation)

```
useEffect(() => {
  if (!newOrder.Unit_Price && !newOrder.Discount_Amount && !newOrder.Shipping_Fee) return;
  const price = Number(newOrder.Unit_Price) || 0;
  const qty = Number(newOrder.Quantity) || 0;
  const discount = Number(newOrder.Discount_Amount) || 0;
  const ship = Number(newOrder.Shipping_Fee) || 0;
  const total = price * qty - discount + ship;
  setNewOrder((prev) => ({ ...prev, Total_Amount: total >= 0 ? total.toString() : '0' }));
}, [newOrder.Unit_Price, newOrder.Quantity, newOrder.Discount_Amount, newOrder.Shipping_Fee]);
```

Hình 6.40. Đoạn code tự động tính Total_Amount (orders/page.tsx)

Chức năng: Theo dõi sự thay đổi của các trường Đơn giá (Unit_Price), Số lượng (Quantity), Giảm giá (Discount_Amount) và Phí vận chuyển (Shipping_Fee) thông qua React useEffect Hook.

Mục đích: Tự động tính toán Tổng tiền đơn hàng theo công thức:

- $Total_Amount = (Unit_Price \times Quantity) - Discount_Amount + Shipping_Fee$
- Đoạn code đảm bảo giá trị Total_Amount luôn chính xác theo thời gian thực mà không đòi hỏi người dùng phải tính toán thủ công, đồng thời chặn giá trị tổng tiền âm (total >= 0 ? ... : '0').

3. Kiểm tra Ràng buộc Dữ liệu Phía Client (Frontend Validation Rules)

```
if (!newOrder.Order_ID.trim()) return setFormError('Chưa lưu được: Mã đơn hàng không được để trống.');
const orderIdRegex = /^TTS-2024-\d{6}$/;
if (!orderIdRegex.test(newOrder.Order_ID.trim())) {
  return setFormError('Chưa lưu được: Mã đơn hàng không đúng định dạng! Yêu cầu: TTS-2024- kèm 6 chữ
}

if (!newOrder.Customer_ID.trim()) return setFormError('Chưa lưu được: Mã khách hàng là bắt buộc nhập!
const customerIdRegex = /^CUST-\d{5}$/;
if (!customerIdRegex.test(newOrder.Customer_ID.trim())) {
  return setFormError('Chưa lưu được: Mã khách hàng không đúng định dạng! Yêu cầu: CUST- kèm đúng 5 c
}

if (!newOrder.Customer_Name.trim()) return setFormError('Chưa lưu được: Tên khách hàng không được để

if (!newOrder.Product_ID.trim()) return setFormError('Chưa lưu được: Mã sản phẩm là bắt buộc nhập!');
const productIdRegex = /^PROD-[A-Za-z0-9_-]+$/;
if (!productIdRegex.test(newOrder.Product_ID.trim())) {
  return setFormError('Chưa lưu được: Mã sản phẩm không đúng định dạng! Yêu cầu dạng PROD-xxx (Ví dụ:
}

if (!newOrder.Product_Name.trim()) return setFormError('Chưa lưu được: Tên sản phẩm không được để trố
if (!newOrder.Shipping_City.trim()) return setFormError('Chưa lưu được: Thành phố giao hàng là bắt bu

const priceNum = Number(newOrder.Unit_Price);
if (!newOrder.Unit_Price || isNaN(priceNum) || priceNum <= 0) {
  return setFormError('Chưa lưu được: Đơn giá sản phẩm là bắt buộc và phải lớn hơn 0.');
```

Hình 6.41. Đoạn code kiểm tra Regex và điều kiện logic (handleCreateOrder)

Chức năng: Thực thi chuỗi các hàm kiểm tra logic và Biểu thức chính quy (Regex) trước khi đóng gói dữ liệu gửi lên API Server.

Mục đích:

- **Chuẩn hóa định dạng khóa:** Bắt buộc Order_ID đúng cấu trúc TTS-2024-XXXXXX, Customer_ID đúng cấu trúc CUST-XXXXX, và Product_ID phải theo dạng PROD-xxx.
- **Bảo vệ toàn vẹn nghiệp vụ:** Đảm bảo Customer_Name, Product_Name, Shipping_City không bị để trống; Unit_Price và Customer_Age phải là số lớn hơn 0.
- **Phản hồi lỗi tức thì:** Ngăn chặn các HTTP Request không hợp lệ gửi về Server, tiết kiệm băng thông và hiển thị ngay thông báo lỗi đỏ trực quan qua setFormError.

4. Xử lý Tạo mới Bản ghi An toàn Phía Backend (Server API Handler)

```
export async function POST(request: Request) {
  try {
    await connectToDatabase();
    const body = await request.json();

    const orderIdRegex = /^TTS-2024-\d{6}$/;
    if (!orderIdRegex.test(body.Order_ID)) {
      return NextResponse.json({
        success: false,
        error: 'Mã đơn hàng không đúng định dạng! Yêu cầu: TTS-2024- kèm theo đúng 6 chữ số (Ví dụ: TTS
      }, { status: 400 });
    }

    const existingOrder = await Order.findOne({ Order_ID: body.Order_ID });
    if (existingOrder) {
      return NextResponse.json({
        success: false,
        error: `Mã đơn hàng "${body.Order_ID}" đã tồn tại trong hệ thống. Vui lòng nhập mã khác!`
      }, { status: 400 });
    }

    const newOrder = await Order.create(body);
    return NextResponse.json({ success: true, data: newOrder }, { status: 201 });
  } catch (error: any) {
    return NextResponse.json({ success: false, error: error.message }, { status: 500 });
```

Hình 6.42. Đoạn code xác thực và chống trùng lặp tại API Route (POST /api/orders/route.ts)

Chức năng: Tiếp nhận Payload JSON gửi từ Client, thực hiện kiểm tra định dạng hai lớp (Two-factor Validation) và kiểm tra tính duy nhất (Uniqueness Check) của Order_ID trong CSDL trước khi tạo mới.

- **Chống trùng lặp khóa chính:** Phương thức Order.findOne({ Order_ID }) ngăn chặn tuyệt đối tình trạng tạo hai đơn hàng trùng mã Order_ID trong hệ thống.
- **Xử lý bất đồng bộ:** Sử dụng lệnh Order.create(body) để Mongoose tự động kiểm tra Schema, gắn mốc thời gian createdAt/updatedAt và lưu trực tiếp bản ghi vào collection transactions trên MongoDB Atlas.

- **Mã trạng thái HTTP chuẩn:** Trả về HTTP status 201 Created cùng dữ liệu vừa tạo thành công hoặc HTTP status 400 Bad Request kèm chi tiết nguyên nhân lỗi.

6.4.4. Chức năng UPDATE (Xem Chi tiết & Cập nhật Đơn hàng)

Chức năng UPDATE cho phép người dùng xem toàn bộ thông tin chi tiết của một đơn hàng cụ thể và thực hiện điều chỉnh các thuộc tính khi có sự thay đổi trong quá trình vận hành (như thay đổi thông tin người nhận, điều chỉnh số lượng, địa chỉ giao hàng hoặc cập nhật lại trạng thái vận chuyển). Quá trình này được thiết kế để đảm bảo dữ liệu vừa linh hoạt trong cập nhật, vừa bảo vệ tính toàn vẹn của các khóa chính và luôn tuân thủ các quy tắc tài chính thời gian thực.

1. Khóa Cố định Mã Đơn hàng và Lấy Dữ liệu Ban đầu (Read-Only Primary Key & Data Fetching)



```
  if (!id) return;
  fetch(`/api/orders/${id}`)
    .then((res) => res.json())
    .then((data) => {
      if (data.success) {
        let rawDate = data.data.Order_Date || '';
        if (rawDate.includes(' ')) {
          rawDate = rawDate.replace(' ', 'T').slice(0, 16);
        }
        setOrder({ ...data.data, Order_Date: rawDate });
      }
      setLoading(false);
    })
    .catch((err) => {
      console.error(err);
      setLoading(false);
    });
}, [id]);
```

```
</label>
<input
  type="text"
  readOnly
  disabled
  value={order.Order_ID || ''}
  className="w-full bg-slate-900/
/>
```

Hình 6.43. Đoạn code lấy dữ liệu chi tiết và khóa Order_ID (orders/[id]/page.tsx)

Chức năng: Tự động gọi API trích xuất thông tin đơn hàng dựa trên id từ URL khi trang được tải, chuyển đổi định dạng ngày tháng tương thích với ô nhập liệu datetime-local, đồng thời thiết lập thuộc tính readOnly và disabled cho ô nhập Order_ID.

Hiển thị đầy đủ thông tin của đơn hàng lên Form chỉnh sửa ngay khi truy cập. Khóa cố định mã định danh giao dịch (Order_ID), ngăn chặn người dùng chỉnh sửa mã đơn hàng nghiệp vụ gốc nhằm bảo vệ tính nhất quán dữ liệu liên kết trên hệ thống.

2. Tái Tính toán Dữ liệu Tài chính Tức thì khi Chỉnh sửa (Real-time Financial Recalculation)

```tsx
const handleFinancialChange = (field: string, value: number) => {
  if (!order) return;
  const updated = { ...order, [field]: value };
  const price = Number(updated.Unit_Price) || 0;
  const qty = Number(updated.Quantity) || 0;
  const discount = Number(updated.Discount_Amount) || 0;
  const ship = Number(updated.Shipping_Fee) || 0;
  const total = price * qty - discount + ship;
  updated.Total_Amount = total >= 0 ? total : 0;
  setOrder(updated);
};
```

Hình 6.44. Đoạn code xử lý sự thay đổi chỉ số tài chính handleFinancialChange (orders/[id]/page.tsx)

Chức năng: Lắng nghe các sự kiện thay đổi dữ liệu của bất kỳ trường nào trong nhóm tài chính (Đơn giá, Số lượng, Giảm giá, Phí ship) và lập tức tính toán lại giá trị Tổng tiền Total_Amount.

Mục đích: Đảm bảo khi nhân viên chỉnh sửa số lượng mua hoặc áp dụng mã giảm giá mới trên trang chi tiết, tổng giá trị đơn hàng luôn được tự động cập nhật chính xác theo đúng công thức tài chính mà không tạo ra sai lệch số liệu.

3. Kiểm tra Ràng buộc Dữ liệu Đầu vào trước khi Lưu (Client-side Validation Rules)

```tsx
e.preventDefault();
setErrorMsg('');

if (!order.Customer_ID?.trim()) return setErrorMsg('Mã khách hàng là bắt buộc nhập!');
const customerIdRegex = /^CUST-\d{5}$/;
if (!customerIdRegex.test(order.Customer_ID.trim())) {
  return setErrorMsg('Mã khách hàng không đúng định dạng! Yêu cầu: CUST- kèm đúng 5 chữ số (Ví dụ: CUS
}

if (!order.Customer_Name?.trim()) return setErrorMsg('Tên khách hàng không được để trống.');

if (!order.Product_ID?.trim()) return setErrorMsg('Mã sản phẩm là bắt buộc nhập!');
const productIdRegex = /^PROD-[A-Za-z0-9_-]+$/;
if (!productIdRegex.test(order.Product_ID.trim())) {
  return setErrorMsg('Mã sản phẩm không đúng định dạng! Yêu cầu dạng PROD-xxx (Ví dụ: PROD-T02).');
}

if (!order.Product_Name?.trim()) return setErrorMsg('Tên sản phẩm không được để trống.');

if (!order.Shipping_City?.trim()) {
  return setErrorMsg('Thành phố giao hàng là bắt buộc, vui lòng chọn tỉnh/thành phố!');
}

const priceNum = Number(order.Unit_Price);
if (!order.Unit_Price || isNaN(priceNum) || priceNum <= 0) {
  return setErrorMsg('Đơn giá sản phẩm là bắt buộc và phải lớn hơn 0.');
}
```

Hình 6.45. Đoạn code xác thực định dạng dữ liệu

Chức năng: Thực hiện kiểm tra toàn bộ các trường thuộc tính chỉnh sửa thông qua Biểu thức chính quy (Regex) và logic điều kiện trước khi gửi yêu cầu PUT đến Server.

Mục đích: Đảm bảo dữ liệu sau khi cập nhật vẫn giữ đúng chuẩn định dạng hệ thống (CUST-XXXXX, PROD-xxx), không xuất hiện trường dữ liệu rỗng ở các thuộc tính quan trọng và loại bỏ các giá trị số không hợp lệ (như đơn giá hay tuổi nhỏ hơn hoặc bằng 0).

4. Cơ chế Tìm kiếm Đa năng & Chống Trùng lặp tại Backend (Server API Handler)



```
export async function PUT(request: Request, context: { params: Promise<{ id: string }> }) {
  try {
    await connectToDatabase();
    const { id } = await context.params;
    const body = await request.json();

    if (body.Order_ID) {
      const orderIdRegex = /^TTS-2024-\d{6}$/;
      if (!orderIdRegex.test(body.Order_ID)) {
        return NextResponse.json({
          success: false,
          message: 'Mã đơn hàng không đúng định dạng! Yêu cầu: TTS-2024- kèm theo đúng 6 chữ số (
        }, { status: 400 });
      }

      let queryCondition: any = { Order_ID: body.Order_ID };
      if (mongoose.Types.ObjectId.isValid(id)) {
        queryCondition._id = { $ne: id };
      } else {
        queryCondition.Order_ID = { $ne: id };
      }
```

Hình 6.46. Đoạn code xử lý Cập nhật Linh hoạt và Chống Trùng mã

Chức năng:

- Xử lý tham số bất đồng bộ Next.js 15+ (await context.params).
- Sử dụng toán tử ngoại trừ $ne để kiểm tra mã Order_ID gửi lên có bị trùng với bất kỳ đơn hàng nào khác ngoài chính nó hay không.
- Áp dụng thuật toán **Hybrid Lookup**: thử tìm và cập nhật theo _id (Mongoose ObjectId) trước, nếu không tìm thấy sẽ chuyển sang tìm và cập nhật theo chuỗi mã đơn nghiệp vụ Order_ID.

Mục đích:

- **Linh hoạt API:** Giúp Endpoint hỗ trợ cập nhật dữ liệu mượt mà dù Client truyền lên tham số `id` là chuỗi ID định danh của MongoDB hay Mã đơn hàng nghiệp vụ (`TTS-2024-XXXXXX`).
- **Toàn vẹn CSDL:** Tùy chọn `{ runValidators: true }` buộc Mongoose phải đối chiếu dữ liệu cập nhật với Schema trước khi lưu, trong khi `{ new: true }` đảm bảo API trả về đúng bản ghi mới nhất sau khi đã sửa đổi thành công.

6.4.5. Chức năng DELETE (Xóa Đơn hàng & Cảnh báo An toàn)

Chức năng DELETE cho phép hủy bỏ các bản ghi đơn hàng không còn giá trị sử dụng (như đơn thử nghiệm, đơn trùng lặp do lỗi hệ thống hoặc đơn hủy theo yêu cầu quản trị). Do xóa dữ liệu là thao tác tác động trực tiếp và vĩnh viễn lên cơ sở dữ liệu MongoDB Atlas, hệ thống đã thiết kế cơ chế bảo vệ hai lớp: xác nhận an toàn trực quan phía Client (UX Safety Guard) và xử lý xóa bất đồng bộ an toàn phía Server (Backend Handler).

1. Hộp thoại Cảnh báo Xác nhận An toàn Phía Client (UX Toast Confirmation Guard)

```
const confirmAndDelete = (order: Order) => {
  toast((t) => (
    <div className="flex flex-col gap-3 p-1">
      <div className="flex items-start gap-2.5">
        <AlertTriangle className="w-5 h-5 text-amber-500 shrink-0 mt-0.5" />
        <div>
          <p className="text-xs font-bold text-slate-800">Cảnh báo xóa đơn hàng</p>
          <p className="text-xs text-slate-600 mt-0.5">
            Bạn có chắc chắn muốn xóa vĩnh viễn đơn hàng <strong className="text-indigo-600 font-mono
          </p>
        </div>
      </div>
      <div className="flex justify-end gap-2 pt-1 border-t border-slate-100">
        <button
          onClick={() => toast.dismiss(t.id)}
          className="px-3 py-1 bg-slate-100 hover:bg-slate-200 text-slate-700 text-xs font-semib
        >
          Hủy
        </button>
        <button
          onClick={async () => {
            toast.dismiss(t.id);
            await onDelete(order._id);
            toast.success(`Đã xóa đơn hàng ${order.Order_ID} thành công!`, {
              duration: 4000,
              style: { background: '#0f172a', color: '#f8fafc', border: '1px solid #334155' },
              icon: '🗑️',
            });
```

Activate Windows

Hình 6.47. Đoạn code hiển thị Toast cảnh báo xác nhận xóa (confirmAndDelete trong DataTable.tsx)

Chức năng: Tích hợp thư viện react-hot-toast để kích hoạt hộp thoại Toast tùy chỉnh với biểu tượng cảnh báo AlertTriangle màu hổ phách ngay khi người dùng bấm nút "Xóa" trên một dòng dữ liệu.

Mục đích:

- **Chống thao tác nhầm (Accidental Deletion Prevention):** Yêu cầu người dùng xác nhận chủ động bằng cách bấm vào nút "Xác nhận xóa" màu đỏ trước khi gửi lệnh thực thi đến Server.
- **Hiển thị thông tin chính xác:** Đưa cụ thể chuỗi Order_ID (mã đơn hàng) vào câu hỏi xác nhận để người dùng đối chiếu lại bản ghi chuẩn bị xóa.
- **Thông báo phản hồi trực quan:** Tự động ẩn hộp thoại Toast cũ (toast.dismiss) và kích hoạt ngay Toast thông báo xóa thành công có màu sắc Dark Mode sang trọng ngay sau khi API hoàn tất lệnh xóa.

2. Kích hoạt Phân quyền Người dùng tại Giao diện (Role-Based Access Control - RBAC)

```
{userRole !== 'VIEWER' && (
  <button
    onClick={() => confirmAndDelete(order)}
    className="inline-flex items-center gap-1 px-2
    title="Xóa đơn hàng"
  >
    <Trash2 className="w-3.5 h-3.5" />
    <span>Xóa</span>
  </button>
)}
```

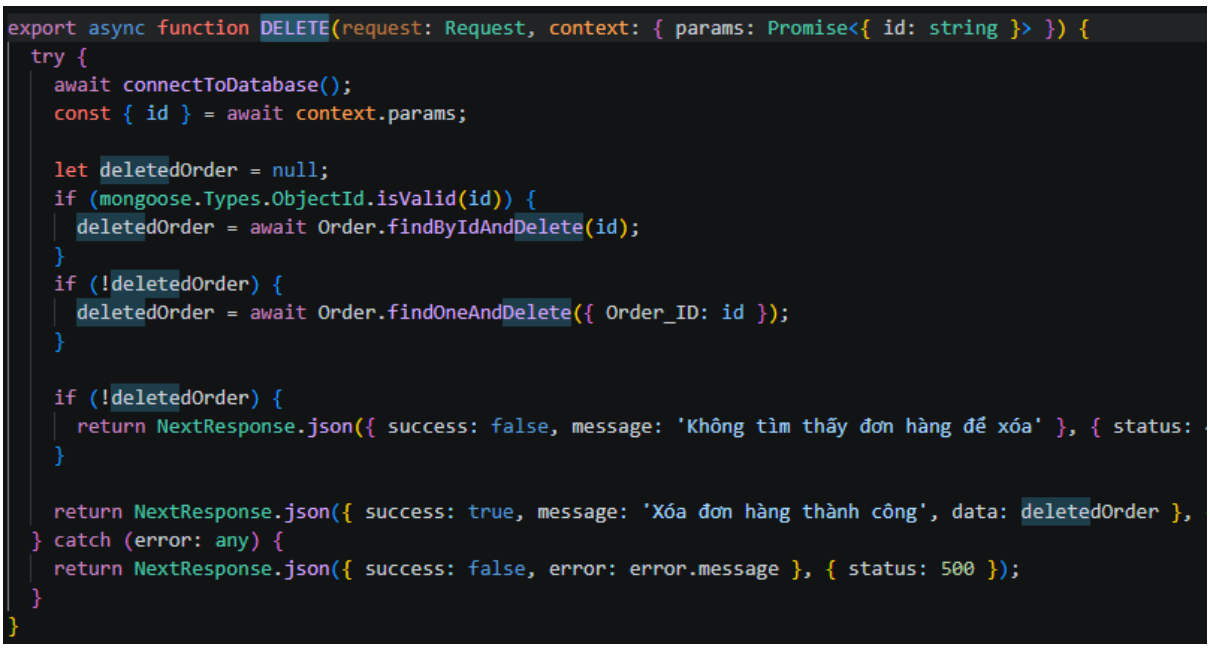
Hình 6.48. Đoạn code kiểm tra vai trò người dùng trước khi hiển thị nút Xóa (DataTable.tsx)

Chức năng: Kiểm tra thuộc tính userRole nhận được từ session đăng nhập của hệ thống.

Mục đích:

Ẩn hoàn toàn thao tác "Xóa" đối với tài khoản có vai trò VIEWER (Chỉ xem). Giới hạn quyền hạn thao tác xóa bản ghi chỉ dành cho tài khoản có quyền ADMIN hoặc STAFF, bảo vệ dữ liệu khỏi các truy cập không có thẩm quyền.

3. Thực thi Xóa Bản ghi Đa năng Phía Backend (Server API Handler)



```
export async function DELETE(request: Request, context: { params: Promise<{ id: string }> }) {
  try {
    await connectToDatabase();
    const { id } = await context.params;

    let deletedOrder = null;
    if (mongoose.Types.ObjectId.isValid(id)) {
      deletedOrder = await Order.findByIdAndDelete(id);
    }
    if (!deletedOrder) {
      deletedOrder = await Order.findOneAndDelete({ Order_ID: id });
    }

    if (!deletedOrder) {
      return NextResponse.json({ success: false, message: 'Không tìm thấy đơn hàng để xóa' }, { status:
    }

    return NextResponse.json({ success: true, message: 'Xóa đơn hàng thành công', data: deletedOrder },
  } catch (error: any) {
    return NextResponse.json({ success: false, error: error.message }, { status: 500 });
  }
}
```

Hình 6.49. Đoạn code xử lý xóa dữ liệu theo ID (DELETE /api/orders/[id]/route.ts)

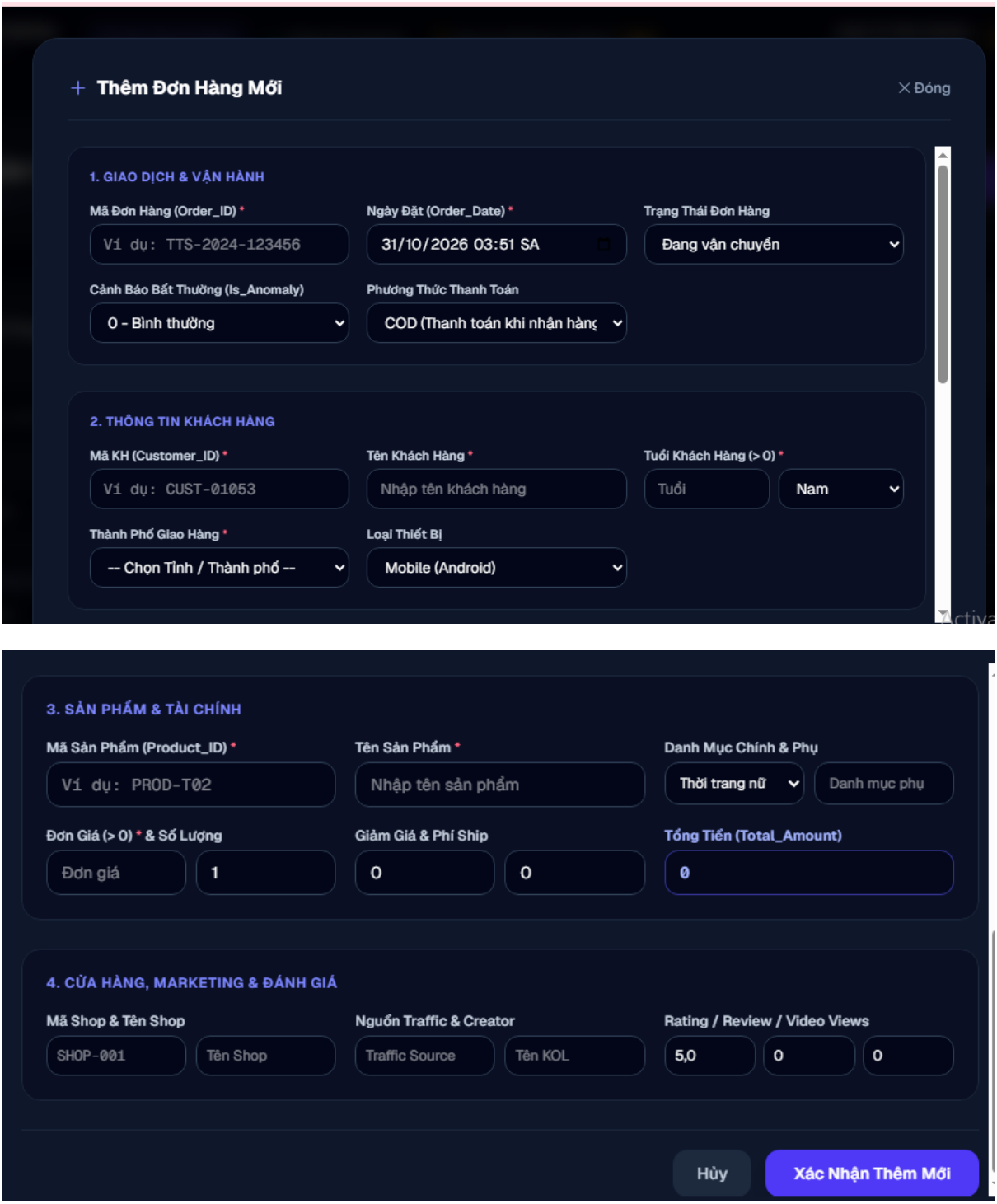
Chức năng:

- Trích xuất tham số id bất đồng bộ theo chuẩn Next.js 15+ (await context.params).
- Áp dụng thuật toán **Hybrid Delete Lookup**: Thử tìm và xóa bản ghi bằng phương thức Order.findByIdAndDelete(id) trước. Nếu tham số gửi lên không phải là ObjectId mà là mã đơn nghiệp vụ (chuỗi TTS-2024-XXXXXX), API tiếp tục thực thi Order.findOneAndDelete({ Order_ID: id }).

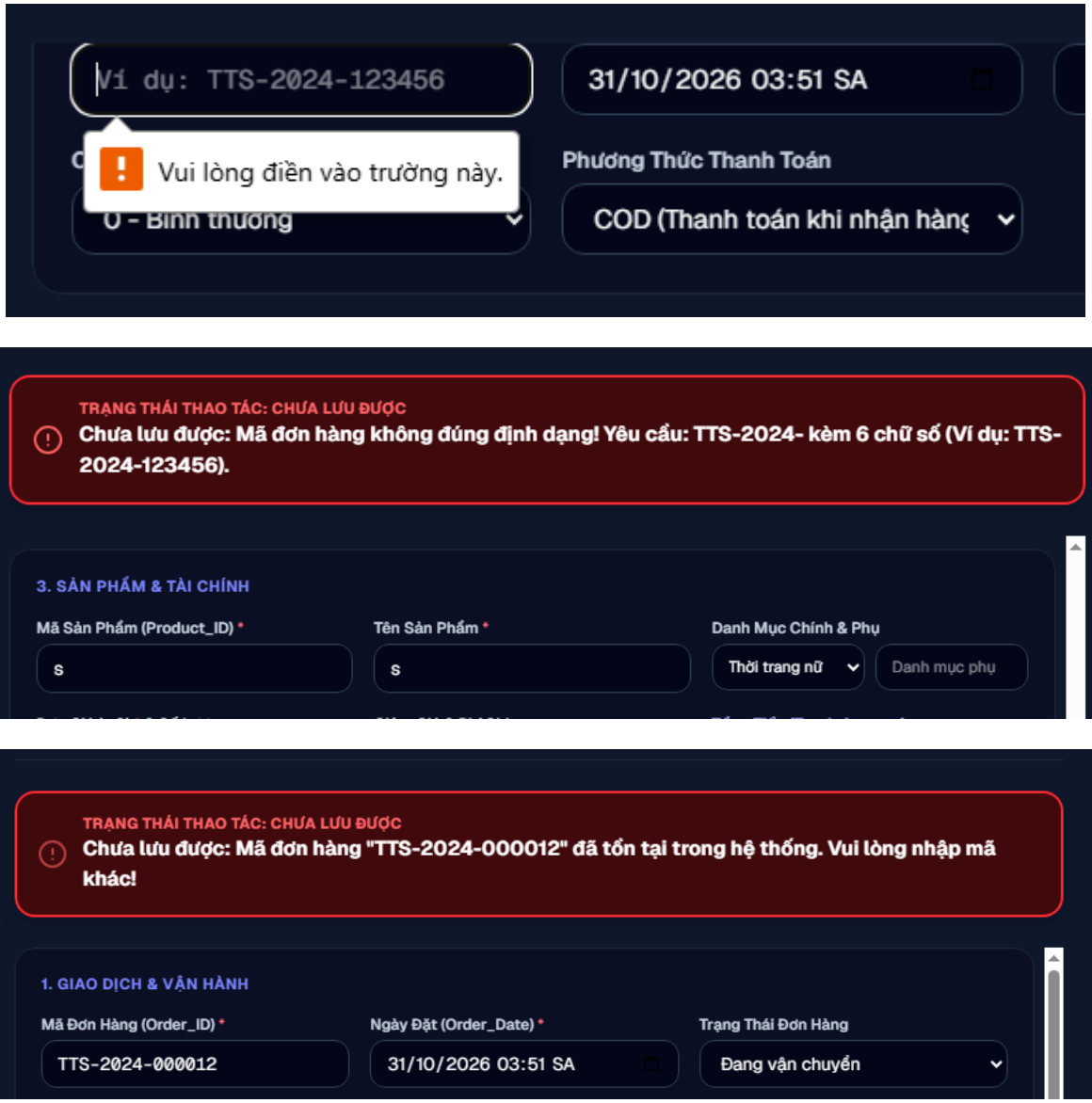
Mục đích:

- **Giải phóng dung lượng CSDL:** Loại bỏ hoàn toàn bản ghi được chỉ định ra khỏi collection transactions trên MongoDB Atlas.
- **Linh hoạt điểm cuối API:** Cho phép Client gửi lệnh xóa qua URL truyền vào MongoDB ID dạng 24-character hex hoặc Mã đơn nghiệp vụ TikTok Shop (TTS-2024-XXXXXX).
- **Phản hồi chuẩn mã HTTP:** Trả về HTTP status 200 OK kèm thông tin bản ghi vừa xóa để Client cập nhật (refresh) lại giao diện danh sách, hoặc trả về HTTP status 404 Not Found nếu bản ghi không tồn tại.

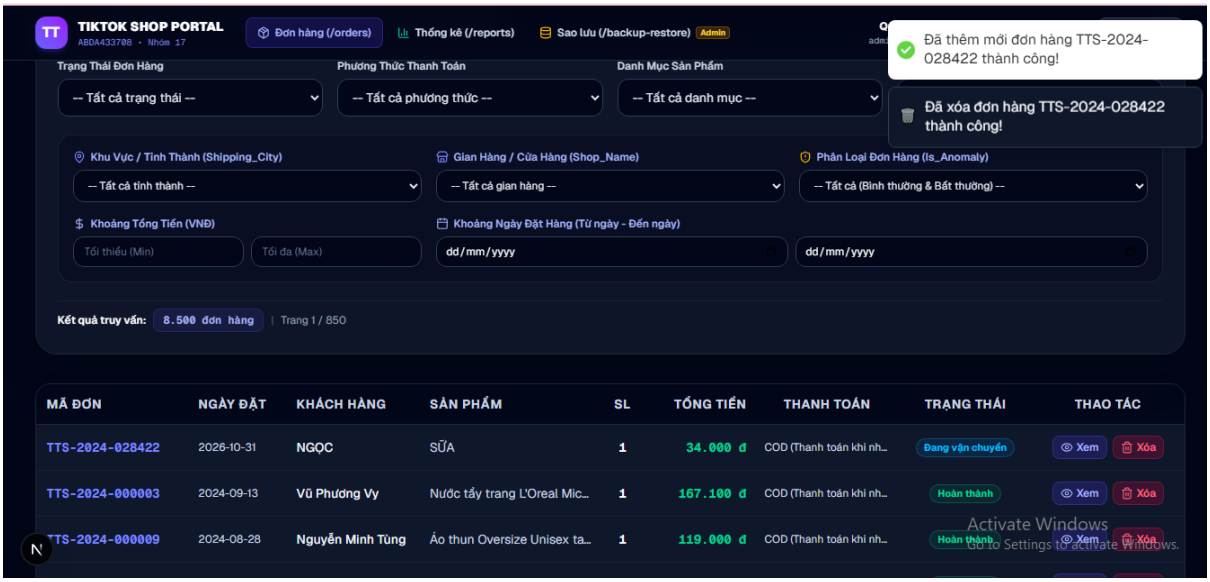
6.4.6. Kết quả trang order



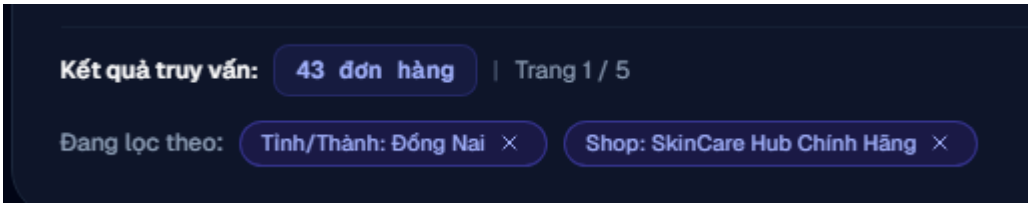
Hình 6.50. Giao diện thêm mới sản phẩm

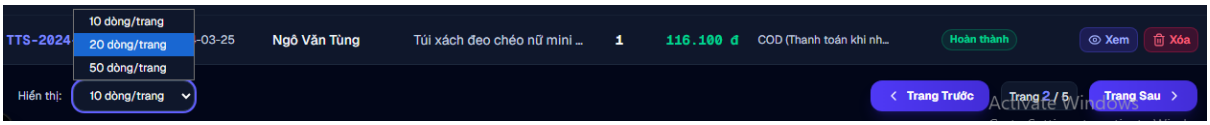


Hình 6.51. Các thông báo yêu cầu nhập dữ liệu và đúng định dạng khi nhập sai hay thiếu



Hình 6.52. Giao diện và thông báo thêm đơn hàng thành công

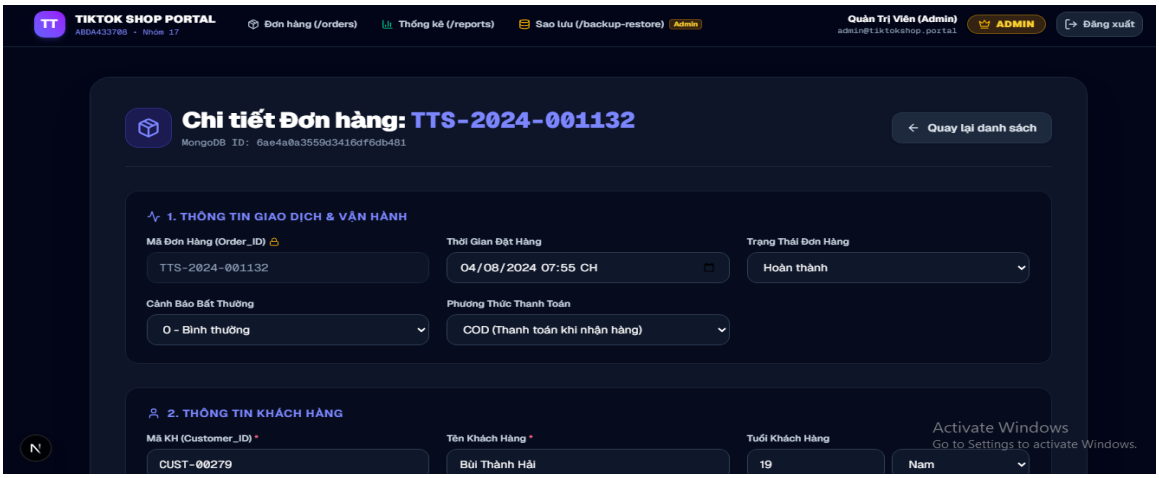


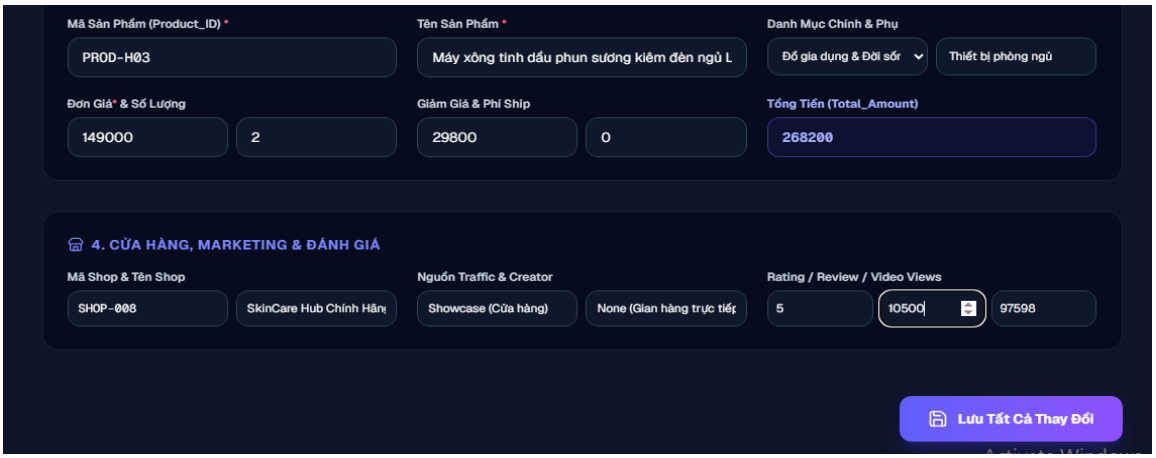


Hình 6.53. Giao diện kết quả truy vấn lọc theo các điều kiện và bộ lọc hiển thị số dòng

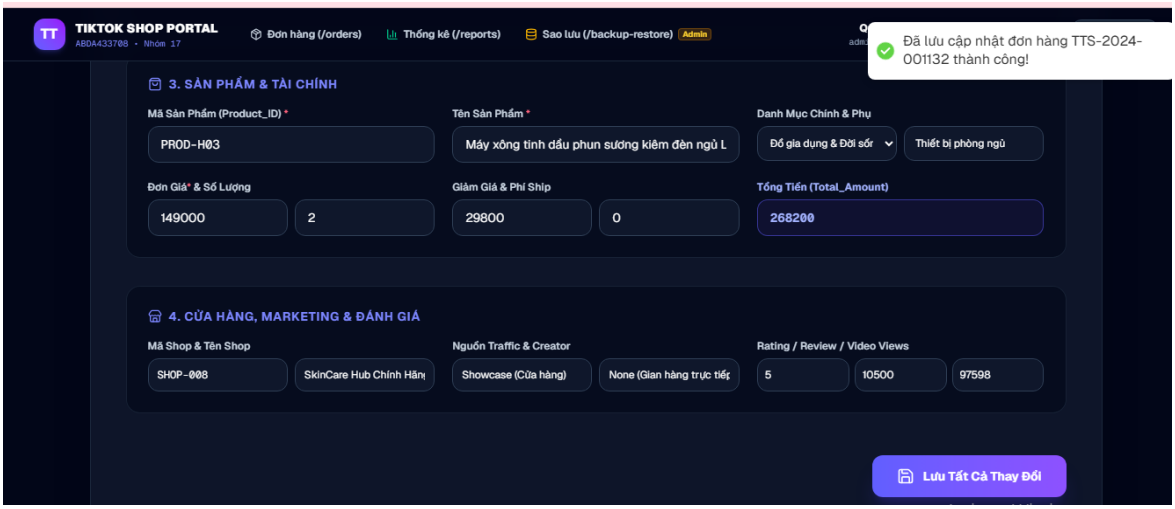


Hình 6.54. Giao diện các đơn hàng bất thường

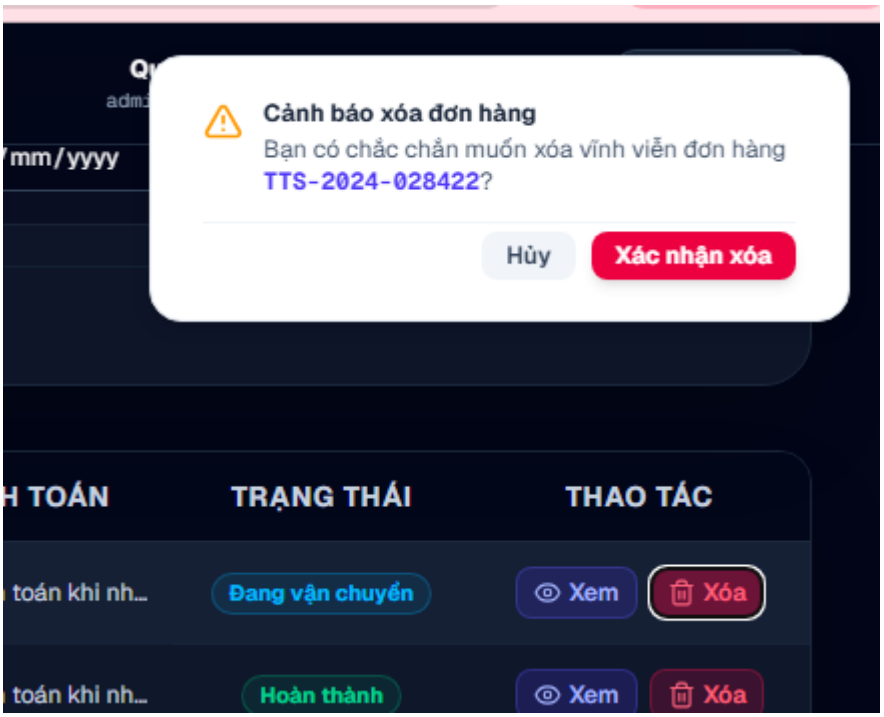




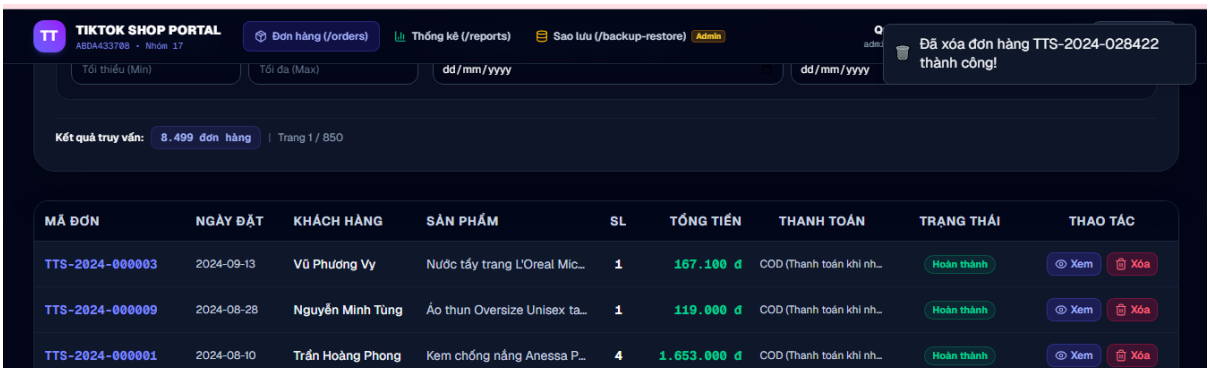
Hình 6.55. Giao diện chi tiết đơn hàng



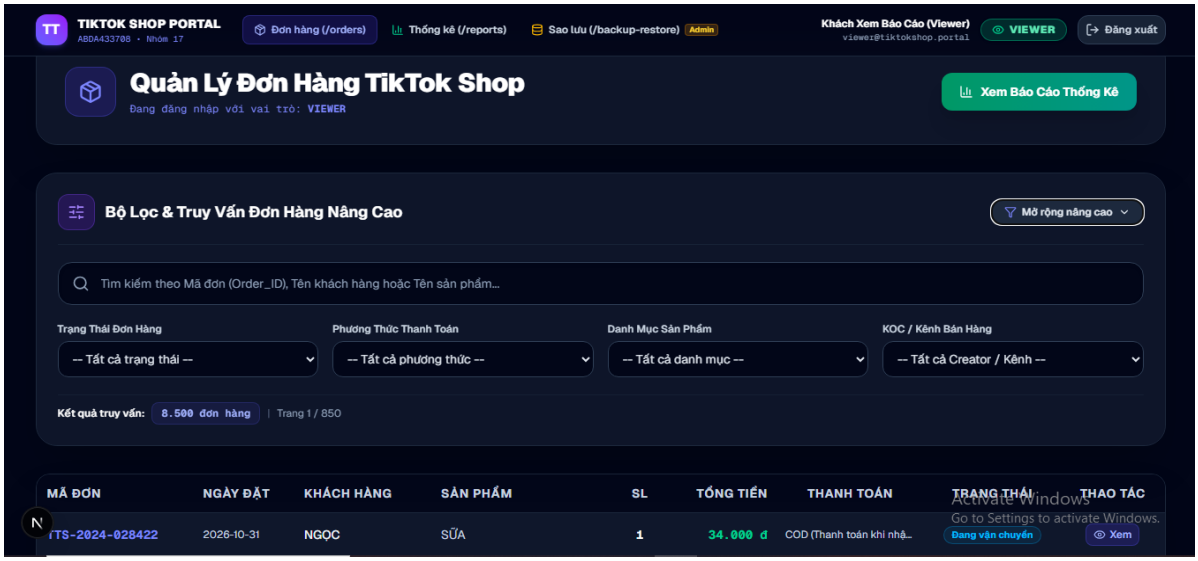
Hình 6.56. Thông báo khi cập nhật đơn hàng thành công



Hình 6.57. Thông báo xác nhận xóa khi nhấn nút xóa đơn hàng



Hình 6.58. Giao diện xóa đơn hàng thành công



Hình 6.59. Giao diện của người xem

Đối với Viewer sẽ không có chức năng thêm mới và xóa đơn hàng

6.5. TÍNH NĂNG KẾT XUẤT BÁO CÁO THỐNG KÊ RA TẬP TIN (EXPORT EXCEL/CSV)

6.5.1. Tổng quan chức năng và luồng xử lý

Chức năng kết xuất báo cáo thống kê ra tập tin (Export Excel/CSV) được xây dựng nhằm hỗ trợ người dùng tổng hợp, theo dõi và lưu trữ dữ liệu đơn hàng từ hệ thống TikTok Shop. Thông qua chức năng này, người dùng có thể xem các chỉ số thống kê trên giao diện báo cáo và xuất dữ liệu ra tập tin để phục vụ việc phân tích, đối chiếu hoặc sử dụng trong các công cụ xử lý dữ liệu khác.

Luồng xử lý bắt đầu tại trang Quản lý đơn hàng (/orders), nơi người dùng có thể tìm kiếm và lọc dữ liệu theo nhiều tiêu chí như trạng thái đơn hàng, phương thức thanh toán, danh mục sản phẩm, KOC/Creator, tỉnh thành giao hàng, gian hàng, trạng thái bất thường, khoảng tổng tiền và khoảng thời gian đặt hàng. Các điều kiện này được sử dụng để truy vấn danh sách đơn hàng phù hợp với nhu cầu phân tích. Sau khi thiết lập bộ lọc, người dùng có thể nhấn nút “Xem Báo Cáo Thống Kê” ngay trên trang đơn hàng để chuyển đến trang /reports, đồng thời truyền các điều kiện lọc hiện tại qua URL. Ngoài ra, người dùng cũng có thể truy cập trang báo cáo trực tiếp thông qua thanh điều hướng (Navbar).

Truyền điều kiện lọc từ trang đơn hàng sang trang báo cáo

Để bảo đảm các điều kiện truy vấn được duy trì khi chuyển trang, hệ thống sử dụng URLSearchParams để tạo chuỗi tham số từ bộ lọc hiện tại. Các tham số này được đính kèm vào đường dẫn /reports, giúp trang báo cáo tiếp nhận những tiêu chí phân tích mà người dùng đã chọn. Bên cạnh đó, trạng thái bộ lọc và chuỗi tham số truy vấn được lưu trong sessionStorage, hỗ trợ khôi phục điều kiện lọc khi người dùng quay lại trang đơn hàng trong cùng phiên làm việc.

```
// Tạo URL Query String dựa trên bộ lọc hiện tại
const buildFilterQueryParams = () => {
  const params = new URLSearchParams();
  if (search.trim()) params.append('search', search.trim());
  if (statusFilter !== 'ALL') params.append('status', statusFilter);
  if (paymentFilter !== 'ALL') params.append('paymentMethod', paymentFilter);
  if (categoryFilter !== 'ALL') params.append('category', categoryFilter);
  if (creatorFilter !== 'ALL') params.append('creator', creatorFilter);
  if (cityFilter !== 'ALL') params.append('city', cityFilter);
  if (shopFilter !== 'ALL') params.append('shop', shopFilter);
  if (anomalyFilter !== 'ALL') params.append('isAnomaly', anomalyFilter);
  if (minAmount) params.append('minAmount', minAmount);
  if (maxAmount) params.append('maxAmount', maxAmount);
  if (startDate) params.append('startDate', startDate);
  if (endDate) params.append('endDate', endDate);
  return params;
};
```

```
// Chuyển sang trang xem báo cáo kèm theo thông số bộ lọc đang chọn
const handleGoToReports = () => {
  const filterQueryParams = buildFilterQueryParams().toString();
  if (filterQueryParams) {
    router.push(`/reports?${filterQueryParams}`);
  } else {
    router.push('/reports');
  }
};
```

Hình 6.60. Đoạn mã xây dựng tham số lọc và điều hướng đến trang báo cáo

Trong đoạn mã trên, hàm buildFilterQueryParams() tập hợp các tiêu chí lọc đang được lựa chọn thành một chuỗi tham số URL. Những bộ lọc có giá trị mặc định như ALL sẽ không được đưa vào chuỗi truy vấn, nhằm tránh truyền các điều kiện không cần thiết. Hàm handleGoToReports() sử dụng chuỗi tham số này để điều hướng đến trang báo cáo. Nếu người dùng chưa thiết lập bộ lọc, hệ thống chỉ chuyển đến đường dẫn /reports; ngược lại, các điều kiện lọc sẽ được đính kèm để trang báo cáo có thể sử dụng khi truy vấn dữ liệu thống kê.

Luồng xử lý tổng quát



Hình 6.61. Luồng xử lý chức năng thống kê và kết xuất báo cáo

Như vậy, chức năng báo cáo được tích hợp với quy trình quản lý đơn hàng thông qua cơ chế truyền điều kiện lọc giữa các trang. Tại đây, API /api/reports cung cấp dữ liệu để hiển thị các chỉ số và biểu đồ, còn API /api/export đảm nhiệm việc tạo tập tin Excel hoặc CSV theo yêu cầu. Cách tổ chức này tách biệt nhiệm vụ hiển thị thống kê và kết xuất tập tin, đồng thời hỗ trợ người dùng tiếp cận dữ liệu theo các điều kiện đã lựa chọn.

6.5.2. Xây dựng API thống kê dữ liệu (/api/reports)

API /api/reports được xây dựng theo cơ chế Route Handler của Next.js App Router, có nhiệm vụ tiếp nhận các điều kiện lọc từ URL, truy vấn và tổng hợp dữ liệu đơn hàng trong MongoDB Atlas. API sử dụng MongoDB Aggregation Pipeline kết hợp với toán tử $facet để xử lý nhiều nhóm thống kê trong cùng một quy trình truy vấn.

Kết nối cơ sở dữ liệu và trích xuất tham số truy vấn

API sử dụng connectToDatabase() để thiết lập kết nối đến MongoDB Atlas và lấy các tham số lọc từ URL thông qua URL và searchParams.

```
export async function GET(request: NextRequest) {
  try {
    // 1. Kết nối cơ sở dữ liệu MongoDB Atlas
    await connectToDatabase();
    const { searchParams } = new URL(request.url);

    // 2. Trích xuất tất cả thông số bộ lọc từ URL Query Params
    const search = searchParams.get('search') || '';
    const status = searchParams.get('status') || '';
    const paymentMethod = searchParams.get('paymentMethod') || '';
    const category = searchParams.get('category') || searchParams.get('mainCategory') || '';
    const creator = searchParams.get('creator') || '';
    const city = searchParams.get('city') || '';
    const shop = searchParams.get('shop') || '';
    const isAnomaly = searchParams.get('isAnomaly') || '';
    const minAmount = searchParams.get('minAmount') || '';
    const maxAmount = searchParams.get('maxAmount') || '';
    const startDate = searchParams.get('startDate') || '';
    const endDate = searchParams.get('endDate') || '';
```

Hình 6.62. Đoạn mã khởi tạo API GET, kết nối MongoDB Atlas và trích xuất các tham số bộ lọc từ URL

Các tham số được lấy trực tiếp từ searchParams, tương ứng với những điều kiện lọc trên giao diện. Giá trị mặc định là chuỗi rỗng giúp API có thể xử lý trường hợp người dùng không cung cấp một số điều kiện nhất định. Riêng category hỗ trợ cả hai tên tham số category và mainCategory, giúp tăng khả năng tương thích với các cách truyền dữ liệu từ phía Client.

Xây dựng bộ lọc truy vấn động bằng $match

Sau khi trích xuất tham số, API khởi tạo đối tượng matchFilter để lưu các điều kiện truy vấn. Các điều kiện chỉ được bổ sung khi người dùng thực sự cung cấp giá trị hợp lệ, nhờ đó bộ lọc có thể thay đổi linh hoạt theo yêu cầu thống kê.

```
// 3. Xây dựng $match Filter động khớp 100% với Schema MongoDB
const matchFilter: any = {};

// Tìm kiếm từ khóa không phân biệt hoa thường theo Mã đơn, Tên khách hàng hoặc Tên sản phẩm
if (search.trim()) {
  const keyword = search.trim();
  matchFilter.$or = [
    { Order_ID: { $regex: keyword, $options: 'i' } },
    { Customer_Name: { $regex: keyword, $options: 'i' } },
    { Product_Name: { $regex: keyword, $options: 'i' } },
  ];
}
```

Hình 6.63. Đoạn mã xây dựng bộ lọc tìm kiếm từ khóa trên mã đơn hàng, tên khách hàng và tên sản phẩm

Điểm đáng chú ý là toán tử $or cho phép tìm kiếm trên nhiều trường dữ liệu cùng lúc. Biểu thức $regex kết hợp với tùy chọn $options: 'i' giúp tìm kiếm không phân biệt chữ hoa và chữ thường; phương thức trim() loại bỏ khoảng trắng ở đầu và cuối từ khóa.

Đối với các trường dạng danh mục và thông tin lựa chọn, API sử dụng phép so sánh trực tiếp với giá trị tương ứng trong Schema MongoDB:

```
// Lọc theo các trường danh mục / thông tin drop-down
if (status && status !== 'ALL' && status !== 'undefined') {
  matchFilter.Order_Status = status;
}
if (paymentMethod && paymentMethod !== 'ALL' && paymentMethod !== 'undefined') {
  matchFilter.Payment_Method = paymentMethod;
}
if (category && category !== 'ALL' && category !== 'undefined') {
  matchFilter.Main_Category = category;
}
if (creator && creator !== 'ALL' && creator !== 'undefined') {
  matchFilter.Creator_Name = creator;
}
if (city && city !== 'ALL' && city !== 'undefined') {
  matchFilter.Shipping_City = city;
}
if (shop && shop !== 'ALL' && shop !== 'undefined') {
  matchFilter.Shop_Name = shop;
}
```

Hình 6.64. Đoạn mã áp dụng bộ lọc động theo trạng thái đơn hàng, phương thức thanh toán, danh mục, nhà sáng tạo, thành phố và cửa hàng

Các giá trị ALL và undefined được loại trừ để tránh áp dụng điều kiện lọc không cần thiết. Tên trường trong matchFilter được ánh xạ trực tiếp đến các thuộc tính dữ liệu như Order_Status, Payment_Method, Main_Category và Shipping_City.

Bên cạnh đó, API xử lý các điều kiện lọc đặc thù:

- Lọc giao dịch bất thường: chuyển isAnomaly sang kiểu số nguyên bằng parseInt() trước khi so sánh với trường Is_Anomaly.
- Lọc khoảng tiền: sử dụng $gte để xác định giá trị tối thiểu và $lte để xác định giá trị tối đa của Total_Amount.
- Lọc khoảng thời gian: sử dụng $gte và $lte trên trường Order_Date để giới hạn dữ liệu theo ngày bắt đầu và ngày kết thúc.

```
// Lọc cờ đơn hàng bất thường (Is_Anomaly: 0 hoặc 1)
if (isAnomaly !== '' && isAnomaly !== 'ALL' && isAnomaly !== 'undefined') {
  const parsedAnomaly = parseInt(isAnomaly, 10);
  if (!isNaN(parsedAnomaly)) {
    matchFilter.Is_Anomaly = parsedAnomaly;
  }
}

// Lọc khoảng tiền đơn hàng (Total_Amount)
if (minAmount || maxAmount) {
  matchFilter.Total_Amount = {};
  if (minAmount) matchFilter.Total_Amount.$gte = parseFloat(minAmount);
  if (maxAmount) matchFilter.Total_Amount.$lte = parseFloat(maxAmount);
}

// Lọc khoảng ngày đặt hàng (Order_Date)
if (startDate || endDate) {
  matchFilter.Order_Date = {};
  if (startDate) matchFilter.Order_Date.$gte = startDate;
  if (endDate) matchFilter.Order_Date.$lte = endDate + ' 23:59:59';
}
```

Hình 6.65. Đoạn mã xử lý bộ lọc giao dịch bất thường, khoảng giá trị đơn hàng và khoảng ngày đặt hàng

Việc xây dựng bộ lọc theo từng điều kiện giúp API kết hợp nhiều tiêu chí trong cùng một truy vấn. Tuy nhiên, phép so sánh ngày ở đoạn mã trên sử dụng chuỗi ký tự, do đó tính chính xác phụ thuộc vào định dạng lưu trữ của trường Order_Date trong MongoDB.

Tổng hợp dữ liệu bằng MongoDB Aggregation Pipeline và $facet

Sau khi hoàn tất bộ lọc, API sử dụng Order.aggregate() để thực thi quy trình tổng hợp. Pipeline gồm hai bước chính: $match lọc dữ liệu đầu vào và $facet chia tập dữ liệu đã lọc thành nhiều nhánh xử lý độc lập.

```
// 4. Thực thi MongoDB Aggregation Pipeline với $facet
const results = await Order.aggregate([
  // Lọc toàn bộ tập dữ liệu thỏa mãn bộ lọc trước khi đi vào các nhánh Facet
  { $match: matchFilter },
  {
    $facet: {
```

Hình 6.66. Đoạn mã khởi tạo Aggregation Pipeline và phân chia các nhánh thống kê bằng $facet

Toán tử $match được đặt trước $facet, bảo đảm các nhánh thống kê cùng sử dụng tập dữ liệu thỏa mãn bộ lọc đầu vào. Mỗi nhánh sau đó đảm nhiệm một nhóm chỉ số hoặc một loại dữ liệu biểu đồ riêng biệt.

Các nhánh xử lý quan trọng gồm:

- Tổng hợp chỉ số KPI (kpiMetrics)

```
// Thẻ KPI 1: Doanh thu (chỉ tính đơn hoàn thành) & Tổng số đơn hàng trong tệp lọc
kpiMetrics: [
  {
    $group: {
      _id: null,
      totalRevenue: {
        $sum: {
          $cond: [
            { $regexMatch: { input: '$Order_Status', regex: /Hoàn thành|Completed/i } },
            '$Total_Amount',
            0,
          ],
        },
      },
      totalOrders: { $sum: 1 },
    },
  },
],
```

Hình 6.67. Đoạn mã tổng hợp KPI doanh thu và tổng số đơn hàng trong tập dữ liệu đã lọc

Nhánh kpiMetrics sử dụng $group để tính hai chỉ số. totalOrders đếm tổng số bản ghi đơn hàng trong tập dữ liệu đã lọc. Đối với totalRevenue, toán tử $cond kết hợp $regexMatch chỉ cộng Total_Amount khi trạng thái đơn hàng khớp với "Hoàn thành" hoặc "Completed", không phân biệt chữ hoa và chữ thường; các trạng thái khác được tính bằng 0.

- Đếm giao dịch bất thường (anomalyCount)

```
// Thẻ KPI 2: Đếm số ca đơn hàng bất thường trong tệp lọc
anomalyCount: [
  { $match: { Is_Anomaly: 1 } },
  { $count: 'count' },
],
```

Hình 6.68. Đoạn mã đếm số đơn hàng được đánh dấu bất thường

Nhánh này tiếp tục lọc các bản ghi có Is_Anomaly bằng 1, sau đó sử dụng $count để đếm số lượng bản ghi thỏa mãn điều kiện trong tập dữ liệu đã lọc ban đầu.

- Tính lượt xem video trung bình (avgViews)

```
// Thẻ KPI 3: Tính lượt xem Video trung bình
avgViews: [
  {
    $group: {
      _id: null,
      avgVideoViews: { $avg: '$Video_Views' },
    },
  },
],
```

Hình 6.69. Đoạn mã tính lượt xem video trung bình bằng toán tử $avg

Toán tử $avg được sử dụng để tính giá trị trung bình của trường Video_Views trên các bản ghi trong tập dữ liệu đầu vào của nhánh.

- Thống kê nguồn truy cập và trạng thái đơn hàng

```
// Biểu đồ Tròn: Phân bổ tỷ lệ nguồn Traffic
trafficSourceBreakdown: [
  {
    $match: {
      Traffic_Source: {
        $exists: true,
        $ne: null,
        $nin: ['Khác', 'khac', 'Other', 'other', ''],
      },
    },
  },
  {
    $group: {
      _id: '$Traffic_Source',
      count: { $sum: 1 },
    },
  },
  { $sort: { count: -1 } },
],
```

```
// Biểu đồ Cột: Phân bố Trạng thái đơn hàng
orderStatusBreakdown: [
  {
    $match: {
      Order_Status: { $exists: true, $ne: null, $nin: [''] },
    },
  },
  {
    $group: {
      _id: '$Order_Status',
      count: { $sum: 1 },
    },
  },
  { $sort: { count: -1 } },
],
```

Hình 6.70. Đoạn mã tổng hợp dữ liệu phân bổ nguồn Traffic và phân bố trạng thái đơn hàng

Hai nhánh đều sử dụng $group để nhóm các bản ghi theo giá trị phân loại và $sum: 1 để đếm số lượng. Với trafficSourceBreakdown, các giá trị không tồn tại, rỗng, null hoặc thuộc nhóm "Khác" được loại trừ trước khi thống kê. Với orderStatusBreakdown, các trạng thái không tồn tại, null hoặc chuỗi rỗng được loại bỏ. Cuối cùng, $sort: { count: -1 } sắp xếp các nhóm theo số lượng giảm dần.

- Xác định nhóm năm sản phẩm có doanh thu cao nhất (topProducts)



Hình 6.71. Đoạn mã tổng hợp số lượng, doanh thu và lựa chọn Top 5 sản phẩm

Các bản ghi được nhóm theo Product_Name. Với mỗi sản phẩm, API tính tổng số lượng thông qua totalQuantity và tổng giá trị đơn hàng thông qua totalRevenue. Sau đó, dữ liệu được sắp xếp theo doanh thu giảm dần và giới hạn còn năm sản phẩm. Lưu ý, nhánh này cộng Total_Amount của các đơn hàng trong tập dữ liệu đã lọc mà không áp dụng riêng điều kiện trạng thái hoàn thành như nhánh kpiMetrics.

- Thống kê xu hướng doanh thu theo tháng (monthlyRevenueTrend)



Hình 6.72. Đoạn mã tổng hợp xu hướng doanh thu theo tháng từ trường Order_Date

Nhánh này loại bỏ các bản ghi không có ngày đặt hàng hợp lệ, sau đó dùng $substr lấy bảy ký tự đầu của Order_Date để tạo khóa nhóm theo định dạng năm-tháng. Doanh

thu tháng được tính theo nguyên tắc chỉ cộng giá trị của các đơn hàng có trạng thái hoàn thành. Dữ liệu được sắp xếp theo tháng tăng dần, rồi $project đổi cấu trúc đầu ra thành hai trường month và revenue để thuận tiện cho phía giao diện sử dụng.

Chuẩn hóa dữ liệu và trả phản hồi JSON

Sau khi MongoDB hoàn tất quá trình tổng hợp, API lấy phần tử đầu tiên trong mảng kết quả và chuẩn hóa dữ liệu trước khi trả về Client.

```
const data = results[0];

// 5. Trả kết quả JSON chuẩn hóa về Client
return NextResponse.json(
  {
    success: true,
    data: {
      totalRevenue: data.kpiMetrics[0]?.totalRevenue || 0,
      totalOrders: data.kpiMetrics[0]?.totalOrders || 0,
      totalAnomalies: data.anomalyCount[0]?.count || 0,
      avgVideoViews: Math.round(data.avgViews[0]?.avgVideoViews || 0),
      trafficSourceBreakdown: data.trafficSourceBreakdown || [],
      orderStatusBreakdown: data.orderStatusBreakdown || [],
      topProducts: data.topProducts || [],
      monthlyRevenueTrend: data.monthlyRevenueTrend || [],
    },
  },
  { status: 200 }
);
```

Hình 6.73. Đoạn mã chuẩn hóa dữ liệu thống kê và trả phản hồi JSON về phía Client

Các trường dữ liệu được tổ chức trong đối tượng data, bao gồm bốn chỉ số tổng quan và bốn nhóm dữ liệu phục vụ biểu đồ, bảng thống kê. Toán tử ?. hỗ trợ truy cập an toàn khi nhánh tổng hợp không có phần tử; toán tử || cung cấp giá trị mặc định là 0 đối với các chỉ số số học và mảng rỗng đối với dữ liệu danh sách. Riêng Math.round() làm tròn lượt xem video trung bình thành số nguyên.

Trường hợp xảy ra lỗi trong quá trình kết nối hoặc truy vấn dữ liệu, khối catch ghi nhận lỗi ở phía máy chủ và trả phản hồi với mã HTTP 500.

```
  } catch (error: any) {
    console.error('Lỗi API Reports Aggregation:', error);
    return NextResponse.json(
      { success: false, error: error.message },
      { status: 500 }
    );
  }
}
```

Hình 6.74. Đoạn mã ghi nhận lỗi và trả phản hồi HTTP 500 khi xử lý API thống kê thất bại

Nhìn chung, API /api/reports kết hợp bộ lọc truy vấn động với Aggregation Pipeline và $facet để tổng hợp nhiều nhóm dữ liệu trong một yêu cầu. Cấu trúc phản hồi JSON thống nhất giúp phía Client dễ dàng tiếp nhận các chỉ số KPI, dữ liệu phân loại và xu hướng theo thời gian để hiển thị trên trang báo cáo thống kê.

6.5.3. Xây dựng giao diện Dashboard thống kê (/reports)

Giao diện Dashboard thống kê được xây dựng bằng Next.js và React, sử dụng thư viện Chart.js kết hợp react-chartjs-2 để trực quan hóa dữ liệu do API /api/reports cung cấp. Giao diện tổ chức thông tin thành các nhóm gồm thẻ KPI tổng quan, biểu đồ phân bổ nguồn Traffic, biểu đồ trạng thái đơn hàng, biểu đồ doanh thu theo tháng và bảng Top 5 sản phẩm. Đồng thời, trang tích hợp chức năng hiển thị bộ lọc hiện hành và kết xuất báo cáo Excel/CSV.

Khởi tạo trang và kiểm soát quyền truy cập

Trang /reports sử dụng Client Component để xử lý trạng thái, đọc tham số URL, gọi API và phản hồi thao tác người dùng. Component Protection giới hạn quyền truy cập theo ba vai trò ADMIN, STAFF và VIEWER.

```
333   return (
334     <Protection allowedRoles={['ADMIN', 'STAFF', 'VIEWER']}>
```

```
677     </Protection>
678   );
```

Hình 6.75. Đoạn mã bọc giao diện Dashboard bằng Protection và Suspense

Tiếp nhận bộ lọc URL và tự động cập nhật dữ liệu báo cáo

Dashboard sử dụng useSearchParams() để đọc lại các điều kiện lọc được truyền từ trang quản lý đơn hàng. Các tham số gồm từ khóa, trạng thái, phương thức thanh toán, danh mục, nhà sáng tạo, thành phố, cửa hàng, cờ bất thường, khoảng tiền và khoảng thời gian.

```
// Đọc toàn bộ tham số bộ lọc từ URL Query String
const searchFilter = searchParams.get('search');
const statusFilter = searchParams.get('status');
const paymentFilter = searchParams.get('paymentMethod');
const categoryFilter = searchParams.get('category');
const creatorFilter = searchParams.get('creator');
const cityFilter = searchParams.get('city');
const shopFilter = searchParams.get('shop');
const isAnomalyFilter = searchParams.get('isAnomaly');
const minAmountFilter = searchParams.get('minAmount');
const maxAmountFilter = searchParams.get('maxAmount');
const startDateFilter = searchParams.get('startDate');
const endDateFilter = searchParams.get('endDate');

const hasFilter = searchParams.toString().length > 0;
```

Hình 6.76. Đoạn mã trích xuất các điều kiện lọc và xác định trạng thái bộ lọc trên Dashboard

Biến hasFilter xác định URL có chứa tham số lọc hay không. Giá trị này được dùng để quyết định hiển thị thông báo về phạm vi dữ liệu và nút đặt lại bộ lọc. Việc đọc trực tiếp từ URL giúp giao diện báo cáo sử dụng cùng các điều kiện đã lựa chọn ở trang /orders.

Để lấy dữ liệu thống kê, hàm fetchReportData() gửi yêu cầu đến API /api/reports, đồng thời chuyển trạng thái tải dữ liệu thông qua biến loading.

```
// Lấy dữ liệu báo cáo từ API kết hợp toàn bộ query parameters
const fetchReportData = async () => {
  setLoading(true);
  try {
    const queryString = searchParams.toString();
    const res = await fetch(`/api/reports?${queryString}`);
    const json = await res.json();
    if (json.success) {
      setData(json.data);
    }
  } catch (err) {
    console.error('Lỗi khi tải dữ liệu báo cáo:', err);
  } finally {
    setLoading(false);
  }
};
```

```
// Tự động re-fetch mỗi khi URL searchParams có sự thay đổi
useEffect(() => {
  fetchReportData();
}, [searchParams]);
```

Hình 6.77. Đoạn mã gọi API thống kê và tự động tải lại dữ liệu khi tham số URL thay đổi

Khi yêu cầu thành công, setData(json.data) cập nhật dữ liệu báo cáo trong React State. useEffect() theo dõi searchParams để gọi lại API khi điều kiện truy vấn thay đổi. Trong thời gian chờ phản hồi, loading được đặt thành true; sau khi xử lý xong, trạng thái được chuyển về false thông qua khối finally.

Thiết kế thẻ KPI tổng quan

Bốn thẻ KPI được bố trí ở khu vực đầu tiên của Dashboard, sử dụng CSS Grid để tự động điều chỉnh số cột theo kích thước màn hình.

```
{/* 1. THẺ KPI TỔNG QUAN */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-5">
```

Hình 6.78. Đoạn mã bố trí hệ thống thẻ KPI tổng quan bằng CSS Grid

Các chỉ số được lấy trực tiếp từ đối tượng data do API trả về, bao gồm:

- totalRevenue: tổng doanh thu theo điều kiện thống kê.
- totalOrders: tổng số đơn hàng.
- totalAnomalies: số đơn hàng được đánh dấu bất thường.
- avgVideoViews: lượt xem video trung bình.

Ví dụ, cách hiển thị doanh thu được triển khai như sau:

```
<p className="text-2xl sm:text-3xl font-black text-emerald-400 tracking-tight">
  {data.totalRevenue.toLocaleString('vi-VN')} ₫
</p>
```

Hình 6.79. Đoạn mã hiển thị KPI doanh thu và định dạng số theo quy ước Việt Nam

Phương thức toLocaleString('vi-VN') giúp phân tách hàng nghìn theo định dạng Việt Nam, kết hợp ký hiệu ₫ để thể hiện đơn vị tiền tệ. Các thẻ còn lại sử dụng cách hiển thị tương tự, nhưng thay đổi nhãn, biểu tượng và màu sắc để phân biệt từng nhóm chỉ số.

Xây dựng biểu đồ trực quan bằng Chart.js

Giao diện sử dụng ba loại biểu đồ chính: biểu đồ Doughnut thể hiện nguồn Traffic, biểu đồ Bar thể hiện trạng thái đơn hàng và biểu đồ Line thể hiện doanh thu theo tháng. Thư viện react-chartjs-2 cung cấp các component React tương ứng, còn dữ liệu được chuyển đổi từ cấu trúc JSON do API trả về.

- Biểu đồ Doughnut – Phân bổ nguồn Traffic



```
// 1. BIỂU ĐỒ TRÒN (Doughnut Chart)
const filteredTraffic = data?.trafficSourceBreakdown.filter(
  (i) => i._id && !['Khác', 'khac', 'Other', 'other', ''].includes(i._id.trim())
) || [];

const trafficChartData = {
  labels: filteredTraffic.map((i) => i._id),
  datasets: [
    {
      label: 'Số đơn hàng',
      data: filteredTraffic.map((i) => i.count),
      backgroundColor: [
        '#10b981',
        '#6366f1',
        '#f43f5e',
        '#f59e0b',
        '#06b6d4',
      ],
      borderColor: '#0f172a',
      borderWidth: 3,
      hoverOffset: 8,
    },
  ],
};
```

Hình 6.80. Đoạn mã xử lý dữ liệu và cấu hình biểu đồ Doughnut phân bổ nguồn Traffic

Trước khi dựng biểu đồ, dữ liệu được lọc để loại bỏ các giá trị không hợp lệ hoặc thuộc nhóm "Khác". Thuộc tính labels lấy tên nguồn Traffic, còn data lấy số lượng đơn hàng tương ứng. Bảng màu được khai báo để phân biệt các phần của biểu đồ, đồng thời hoverOffset tạo hiệu ứng dịch chuyển khi người dùng tương tác.

- Biểu đồ Bar – Phân bố trạng thái đơn hàng



```
// 2. BIỂU ĐỒ CỘT (Bar Chart)
const statusLabels = data?.orderStatusBreakdown.map((i) => i._id || 'Khác') || [];
const statusCounts = data?.orderStatusBreakdown.map((i) => i.count) || [];

const colorPalette = ['#10b981', '#6366f1', '#f43f5e', '#f59e0b', '#94a3b8'];

const statusChartData = {
  labels: ['Trạng thái đơn hàng'],
  datasets: statusLabels.map((statusName, index) => ({
    label: statusName,
    data: [statusCounts[index] || 0],
    backgroundColor: colorPalette[index % colorPalette.length],
    borderRadius: 8,
    borderWidth: 0,
  })),
};
```

Hình 6.81. Đoạn mã chuyển đổi dữ liệu trạng thái đơn hàng thành các tập dữ liệu cho biểu đồ Bar

Khác với biểu đồ Doughnut, biểu đồ Bar tạo một tập dữ liệu cho từng trạng thái đơn hàng. Tên trạng thái được sử dụng làm nhãn trong chú giải, còn số lượng đơn hàng

được đưa vào dữ liệu của từng tập. Biểu thức index % colorPalette.length cho phép tái sử dụng bảng màu khi số nhóm trạng thái vượt quá số màu được khai báo.

- Biểu đồ Line – Doanh thu theo tháng

```
// 3. BIỂU ĐỒ ĐƯỜNG (Line Chart)
const lineChartLabels = data?.monthlyRevenueTrend?.map((i) => i.month) || [
  '2024-01', '2024-02', '2024-03', '2024-04', '2024-05',
  '2024-06', '2024-07', '2024-08', '2024-09', '2024-10',
];

const lineChartData = {
  labels: lineChartLabels,
  datasets: [
    {
      label: 'Doanh Thu Theo Tháng (VNĐ)',
      data: data?.monthlyRevenueTrend?.map((i) => i.revenue) || [],
      borderColor: '#009990',
      borderWidth: 3,
      backgroundColor: 'rgba(0, 153, 144, 0.12)',
      fill: true,
      tension: 0.35,
      pointRadius: 5,
      pointHoverRadius: 7,
      pointBackgroundColor: '#E66C37',
      pointBorderColor: '#0f172a',
      pointBorderWidth: 2,
    },
  ],
};
```

Hình 6.82. Đoạn mã cấu hình dữ liệu và giao diện biểu đồ Line doanh thu theo tháng

Biểu đồ sử dụng month làm nhãn trục thời gian và revenue làm giá trị doanh thu tương ứng. Thuộc tính fill: true tô màu vùng phía dưới đường biểu diễn, trong khi tension tạo độ cong cho đường nối các điểm dữ liệu.

Đáng chú ý, danh sách tháng mặc định chỉ được dùng khi monthlyRevenueTrend không có dữ liệu. Trong trường hợp đó, mảng doanh thu vẫn là mảng rỗng; các nhãn tháng mặc định không đồng nghĩa với việc hệ thống có sẵn dữ liệu doanh thu cho các tháng này.

Bên cạnh cấu hình dữ liệu riêng cho từng biểu đồ, chartOptionsBase thiết lập các thuộc tính giao diện dùng chung như khả năng thay đổi kích thước, chú giải, tooltip, màu chữ và đường lưới. Cách tổ chức này giúp duy trì tính nhất quán giữa các biểu đồ trên Dashboard.

Hiển thị bảng Top 5 sản phẩm

Ngoài các biểu đồ, Dashboard sử dụng bảng HTML để trình bày danh sách năm sản phẩm được API xếp hạng theo doanh thu.

```
<tbody className="divide-y divide-slate-800/60">
  {data.topProducts.map((prod, index) => (
    <tr key={index} className="hover:bg-slate-800/40 transition">
      <td className="px-5 py-4 font-black">
        <span
          className={`w-7 h-7 rounded-full flex items-center justify-center text-xs font-bold ${
            index === 0
              ? 'bg-amber-500/20 text-amber-300 border border-amber-500/40'
              : index === 1
              ? 'bg-slate-400/20 text-slate-200 border border-slate-400/40'
              : index === 2
              ? 'bg-orange-500/20 text-orange-300 border border-orange-500/40'
              : 'bg-slate-800 text-slate-400'
          }`}
        >
          {index + 1}
        </span>
      </td>
      <td className="px-5 py-4 font-bold text-white">
        {prod._id || 'Sản phẩm khác'}
      </td>
      <td className="px-5 py-4 text-right font-semibold text-indigo-400 font-mono">
        {prod.totalQuantity.toLocaleString('vi-VN')} món
      </td>
      <td className="px-5 py-4 text-right font-black text-emerald-400 font-mono text-base">
        {prod.totalRevenue.toLocaleString('vi-VN')} đ
      </td>
    </tr>
  ))}
</tbody>
```

Hình 6.83. Đoạn mã render bảng Top 5 sản phẩm với thứ hạng, sản lượng và doanh thu

Phương thức map() duyệt qua data.topProducts để tạo từng dòng bảng. Biến index được sử dụng để xác định thứ hạng, trong khi prod chứa tên sản phẩm, tổng số lượng và tổng doanh thu. Các giá trị số được định dạng theo quy ước Việt Nam nhằm giúp người dùng dễ theo dõi.

Kết xuất báo cáo Excel/CSV và kiểm soát quyền

Giao diện tích hợp hai nút xuất báo cáo Excel và CSV. Hàm handleExport() nhận tham số định dạng, kiểm tra quyền người dùng, gửi yêu cầu đến API /api/export và xử lý phản hồi để tải tập tin về thiết bị.

```
// Kết xuất file Excel/CSV đồng bộ điều kiện lọc và đọc đúng Tên file từ Header API
const handleExport = async (format: 'xlsx' | 'csv') => {
  if (role === 'VIEWER') {
    alert('Tài khoản VIEWER không có quyền kết xuất tệp báo cáo!');
    return;
  }

  try {
    setExporting(format);
    const queryString = searchParams.toString();
    const res = await fetch(`/api/export?type=${format}${queryString ? `&${queryString}` : ''}`);
    if (!res.ok) throw new Error('Xuất file thất bại');
```

Hình 6.84. Đoạn mã kiểm tra quyền và gửi yêu cầu kết xuất Excel/CSV theo bộ lọc hiện hành

Điều kiện role === 'VIEWER' ngăn tài khoản chỉ có quyền xem thực hiện thao tác kết xuất từ giao diện. Tham số type xác định định dạng tập tin, còn searchParams.toString() giúp truyền lại các điều kiện lọc đang áp dụng trên Dashboard.

Biến exporting ghi nhận định dạng đang xử lý để giao diện cập nhật trạng thái nút và hạn chế thao tác xuất đồng thời.

Sau khi nhận phản hồi thành công, hệ thống đọc tên tập tin từ header Content-Disposition. Nếu không lấy được tên tập tin, chương trình tạo tên dự phòng theo ngày hiện tại.

```
// Lấy tên file động
const contentDisposition = res.headers.get('Content-Disposition');
let filename = '';
if (contentDisposition) {
  const match = contentDisposition.match(/filename="?([^";]+)"?/);
  if (match && match[1]) filename = match[1];
}

// Dự phòng nếu trình duyệt không lấy được header
if (!filename) {
  const now = new Date();
  const d = String(now.getDate()).padStart(2, '0');
  const m = String(now.getMonth() + 1).padStart(2, '0');
  const y = now.getFullYear();
  filename = `tiktok_shop_sales_report_${d}-${m}-${y}.${format}`;
}
```

Hình 6.85. Đoạn mã lấy tên tập tin từ phản hồi API và tạo tên dự phòng theo ngày

Tiếp theo, dữ liệu phản hồi được chuyển thành Blob, sau đó tạo URL tạm thời để trình duyệt tải tập tin:

```
const blob = await res.blob();
const url = window.URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = filename;
document.body.appendChild(a);
a.click();
a.remove();
window.URL.revokeObjectURL(url);
```

Hình 6.86. Đoạn mã tạo liên kết tải xuống tập tin báo cáo từ dữ liệu Blob

Cơ chế trên cho phép tải tập tin trực tiếp từ phản hồi API mà không cần điều hướng sang trang khác. Sau khi kích hoạt tải xuống, phần tử liên kết tạm được loại bỏ và URL đối tượng được thu hồi để giải phóng tài nguyên. Nếu phát sinh lỗi, hệ thống hiển thị thông báo và ghi nhận lỗi trong console; khối finally đặt lại trạng thái exporting.

Hiển thị bộ lọc và đặt lại điều kiện thống kê

Dashboard có khu vực hiển thị các điều kiện lọc hiện hành dưới dạng nhãn trực quan, giúp người dùng nhận biết phạm vi dữ liệu đang được thống kê. Hàm

formatDateVN() chuyển ngày từ định dạng YYYY-MM-DD sang DD/MM/YYYY để phù hợp với cách hiển thị ngày tại Việt Nam.

Khi người dùng nhấn nút "Đặt lại", hàm handleResetFilters() xóa thông tin bộ lọc được lưu trong sessionStorage và điều hướng về /reports không kèm tham số truy vấn.

```
// Hàm xóa trắng toàn bộ bộ lọc, đưa trang về trạng thái ban đầu
const handleResetFilters = () => {
  try {
    sessionStorage.removeItem('tiktokshop_orders_filters');
    sessionStorage.removeItem('tiktokshop_orders_filter_query');
  } catch (e) {}
  router.push('/reports');
};
```

Hình 6.87. Đoạn mã xóa trạng thái bộ lọc đã lưu và đưa Dashboard về trang báo cáo mặc định

Sau khi URL không còn tham số lọc, cơ chế theo dõi searchParams gọi lại API để lấy dữ liệu theo trạng thái mặc định. Nhờ đó, thao tác đặt lại không chỉ thay đổi thông tin hiển thị mà còn khởi tạo lại phạm vi truy vấn báo cáo.

Tóm lại, giao diện /reports kết hợp React State, tham số URL và API thống kê để hiển thị dữ liệu dưới dạng KPI, biểu đồ và bảng xếp hạng sản phẩm. Việc tích hợp các chức năng làm mới, theo dõi bộ lọc, đặt lại điều kiện và kết xuất Excel/CSV giúp Dashboard hỗ trợ cả nhu cầu theo dõi trực quan lẫn khai thác dữ liệu báo cáo.

6.5.4. Xây dựng API kết xuất báo cáo Excel/CSV (/api/export)

API /api/export được xây dựng nhằm hỗ trợ người dùng kết xuất dữ liệu đơn hàng từ MongoDB Atlas thành tệp Excel hoặc CSV để phục vụ việc lưu trữ, tổng hợp và phân tích dữ liệu ngoài hệ thống. API sử dụng phương thức GET, tiếp nhận các tham số lọc từ URL và tạo báo cáo dựa trên tập dữ liệu tương ứng. Điểm đáng chú ý là báo cáo không chỉ chứa danh sách đơn hàng chi tiết mà còn bao gồm thông tin bộ lọc và bốn chỉ số KPI tổng quan.

Tiếp nhận tham số và xây dựng bộ lọc dữ liệu

Sau khi kết nối cơ sở dữ liệu thông qua connectToDatabase(), API đọc các tham số truy vấn như từ khóa tìm kiếm, trạng thái đơn hàng, phương thức thanh toán, ngành hàng, Creator, thành phố giao hàng, gian hàng, phân loại bất thường, khoảng giá trị đơn

hàng và thời gian. Định dạng xuất được xác định qua tham số type hoặc format, mặc định là xlsx nếu không có giá trị được cung cấp.

Tương tự API thống kê, điều kiện lọc được tổ chức trong đối tượng matchFilter để truy vấn dữ liệu phù hợp. Với từ khóa tìm kiếm, API sử dụng $or kết hợp $regex để tìm theo mã đơn hàng, tên khách hàng hoặc tên sản phẩm, không phân biệt chữ hoa và chữ thường. Các bộ lọc còn lại được bổ sung tương ứng vào điều kiện $match; đồng thời, activeFiltersList ghi nhận những điều kiện đang áp dụng để đưa vào phần mô tả của báo cáo.

```
// 3. Xây dựng $match Filter động
const matchFilter: any = {};
const activeFiltersList: string[] = [];

if (search.trim()) {
  const keyword = search.trim();
  matchFilter.$or = [
    { Order_ID: { $regex: keyword, $options: 'i' } },
    { Customer_Name: { $regex: keyword, $options: 'i' } },
    { Product_Name: { $regex: keyword, $options: 'i' } },
  ];
  activeFiltersList.push(`Từ khóa search: "${keyword}"`);
}

if (status && status !== 'ALL' && status !== 'undefined') {
  matchFilter.Order_Status = status;
  activeFiltersList.push(`Trạng thái: ${status}`);
}
if (paymentMethod && paymentMethod !== 'ALL' && paymentMethod !== 'undefined') {
  matchFilter.Payment_Method = paymentMethod;
  activeFiltersList.push(`Thanh toán: ${paymentMethod}`);
}
if (category && category !== 'ALL' && category !== 'undefined') {
  matchFilter.Main_Category = category;
  activeFiltersList.push(`Ngành hàng: ${category}`);
}
if (creator && creator !== 'ALL' && creator !== 'undefined') {
  matchFilter.Creator_Name = creator;
  activeFiltersList.push(`KOC/Creator: ${creator}`);
}
if (city && city !== 'ALL' && city !== 'undefined') {
  matchFilter.Shipping_City = city;
  activeFiltersList.push(`Khu vực: ${city}`);
}
if (shop && shop !== 'ALL' && shop !== 'undefined') {
  matchFilter.Shop_Name = shop;
  activeFiltersList.push(`Gian hàng: ${shop}`);
}
```

Hình 6.88. Xây dựng bộ lọc động cho API kết xuất báo cáo.

Với bộ lọc giá trị đơn hàng, API sử dụng $gte và $lte để xác định ngưỡng tối thiểu và tối đa. Bộ lọc thời gian được thiết lập dựa trên startDate và endDate; ngày kết thúc được nối thêm 23:59:59 nhằm bao phủ thời điểm cuối ngày theo định dạng dữ liệu đang sử dụng. Sau khi tổng hợp các điều kiện, API tạo phần mô tả bộ lọc để người đọc nhận biết báo cáo được xuất từ toàn bộ dữ liệu hay từ một tập dữ liệu đã lọc.

Tính toán các chỉ số KPI và truy xuất danh sách đơn hàng

Trước khi tạo tệp, API chạy Order.aggregate() với $match và $facet để tính bốn chỉ số: tổng doanh thu từ đơn hàng hoàn thành, tổng số đơn hàng trong tập dữ liệu được lọc, số đơn hàng bất thường và lượt xem video trung bình. Doanh thu chỉ được cộng khi trạng thái đơn hàng khớp với Hoàn thành hoặc Completed; các đơn còn lại đóng góp giá trị bằng 0 vào chỉ số này.

```
// 4. Truy vấn tính toán 4 giá trị Thẻ KPI trên tập dữ liệu đã lọc qua Aggregation Pipeline
const kpiAggregation = await Order.aggregate([
  { $match: matchFilter },
  {
    $facet: {
      kpiMetrics: [
        {
          $group: {
            _id: null,
            totalRevenue: {
              $sum: {
                $cond: [
                  { $regexMatch: { input: '$Order_Status', regex: /Hoàn thành|Completed/i } },
                  '$Total_Amount',
                  0,
                ],
              },
            },
            totalOrders: { $sum: 1 },
          },
        },
      ],
      anomalyMetrics: [
        { $match: { Is_Anomaly: 1 } },
        { $count: 'count' },
      ],
      viewsMetrics: [
        {
          $group: {
            _id: null,
            avgVideoViews: { $avg: '$Video_Views' },
          },
        },
      ],
    },
  },
]);
```

```
const kpiResult = kpiAggregation[0];
const totalRevenueVal = kpiResult?.kpiMetrics[0]?.totalRevenue || 0;
const totalOrdersVal = kpiResult?.kpiMetrics[0]?.totalOrders || 0;
const totalAnomaliesVal = kpiResult?.anomalyMetrics[0]?.count || 0;
const avgVideoViewsVal = Math.round(kpiResult?.viewsMetrics[0]?.avgVideoViews || 0);
```

Hình 6.89. Tổng hợp các chỉ số KPI bằng MongoDB Aggregation Pipeline.

Sau khi tính KPI, API truy vấn danh sách đơn hàng thỏa mãn cùng bộ lọc và sắp xếp theo ngày đặt hàng giảm dần. Dữ liệu được chuyển thành mảng hai chiều reportData để thư viện xlsx có thể tạo bảng tính. Phần đầu báo cáo chứa tiêu đề, thời điểm xuất và mô tả bộ lọc; tiếp theo là nhóm KPI tổng quan và bảng dữ liệu chi tiết.

Bảng chi tiết gồm 28 cột, bao quát thông tin mã đơn hàng, ngày đặt hàng, khách hàng, sản phẩm, danh mục, giá bán, số lượng, giảm giá, phí vận chuyển, tổng tiền,

phương thức thanh toán, trạng thái, đánh giá, gian hàng, nguồn truy cập, Creator, lượt xem video và cờ nhận diện đơn bất thường. Các giá trị thiếu được thay thế bằng chuỗi rỗng hoặc giá trị mặc định phù hợp với từng trường; ngày đặt hàng được định dạng lại theo cách hiển thị ngày/tháng/năm.

Tạo cấu trúc tệp Excel và CSV

Sau khi hoàn thiện reportData, API sử dụng XLSX.utils.aoa_to_sheet() để chuyển mảng dữ liệu thành worksheet. Độ rộng các cột được thiết lập thông qua thuộc tính worksheet['!cols'], giúp nội dung dễ quan sát hơn khi mở tệp. Worksheet sau đó được thêm vào workbook với tên Báo Cáo TikTok Shop.

```
// 7. Tạo Workbook và Worksheet bằng thư viện XLSX
const worksheet = XLSX.utils.aoa_to_sheet(reportData);

// Tự động căn chỉnh độ rộng các cột
worksheet['!cols'] = [
  { wch: 30 }, { wch: 20 }, { wch: 15 }, { wch: 22 }, { wch: 8 }, { wch: 10 },
  { wch: 20 }, { wch: 15 }, { wch: 15 }, { wch: 35 }, { wch: 25 }, { wch: 20 },
  { wch: 15 }, { wch: 10 }, { wch: 15 }, { wch: 15 }, { wch: 18 }, { wch: 30 },
  { wch: 22 }, { wch: 10 }, { wch: 12 }, { wch: 12 }, { wch: 28 }, { wch: 22 },
  { wch: 22 }, { wch: 15 }, { wch: 25 }
];

const workbook = XLSX.utils.book_new();
XLSX.utils.book_append_sheet(workbook, worksheet, 'Báo Cáo TikTok Shop');
```

Hình 6.90. Tạo worksheet và workbook cho báo cáo TikTok Shop.

Tên tệp được tạo tự động theo ngày xuất báo cáo với cấu trúc tiktok_shop_sales_report_DD-MM-YYYY, trong đó phần mở rộng là .xlsx hoặc .csv tùy định dạng được chọn. Cách đặt tên này giúp nhận biết loại báo cáo và thời điểm kết xuất.

```
// 8. ĐỊNH DẠNG TÊN FILE MỚI
const now = new Date();
const day = String(now.getDate()).padStart(2, '0');
const month = String(now.getMonth() + 1).padStart(2, '0');
const year = now.getFullYear();

const fileName = `tiktok_shop_sales_report_${day}-${month}-${year}.${exportType === 'csv' ? 'csv' : 'xlsx'}`;
```

Hình 6.91. Tạo tên tệp báo cáo theo ngày xuất.

Xuất tệp và trả kết quả cho trình duyệt

API phân nhánh theo định dạng được yêu cầu. Với CSV, dữ liệu worksheet được chuyển thành chuỗi bằng XLSX.utils.sheet_to_csv(). API bổ sung UTF-8 BOM (\uFEFF) trước khi chuyển chuỗi thành Buffer để hỗ trợ hiển thị tiếng Việt có dấu trong các ứng dụng đọc CSV tương thích.



Hình 6.92. Kết xuất dữ liệu thành tệp CSV hoặc Excel và trả về trình duyệt.

Trong phản hồi, Content-Type xác định định dạng nội dung, còn Content-Disposition: attachment yêu cầu trình duyệt xử lý phản hồi như một tệp tải xuống. Nếu xảy ra lỗi trong quá trình kết nối, truy vấn hoặc tạo tệp, API ghi nhận lỗi ở phía máy chủ và trả về phản hồi JSON với mã trạng thái HTTP 500.



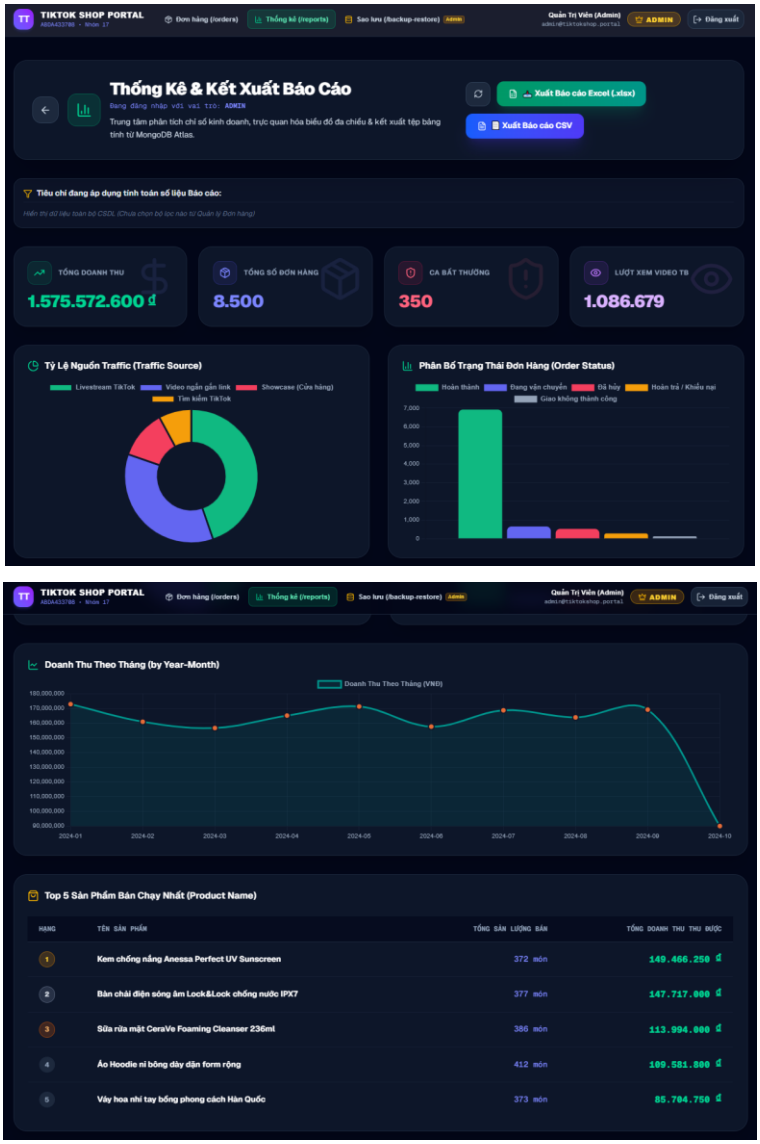
Hình 6.93. Xử lý lỗi trong API kết xuất báo cáo.

Như vậy, API /api/export hoàn thiện quy trình từ tiếp nhận bộ lọc, tính toán KPI, truy xuất đơn hàng đến đóng gói và trả tệp báo cáo. Việc hỗ trợ cả Excel và CSV giúp người dùng lựa chọn định dạng phù hợp cho nhu cầu lưu trữ, chia sẻ hoặc tiếp tục phân tích dữ liệu. Các chỉ số tổng quan và danh sách chi tiết đều được tạo từ bộ lọc được gửi đến API, giúp báo cáo phản ánh phạm vi dữ liệu người dùng đang quan tâm.

6.5.5. Kiểm thử và đánh giá kết quả

Khi truy cập trang /reports, hệ thống hiển thị Dashboard thống kê với khu vực thể hiện các tiêu chí lọc, bốn thẻ KPI tổng quan và các biểu đồ phân tích. Cụ thể, Dashboard bao gồm chỉ số tổng doanh thu, tổng số đơn hàng, số ca bất thường và lượt

xem video trung bình; bên dưới là biểu đồ tỷ lệ nguồn Traffic và biểu đồ phân bố trạng thái đơn hàng.



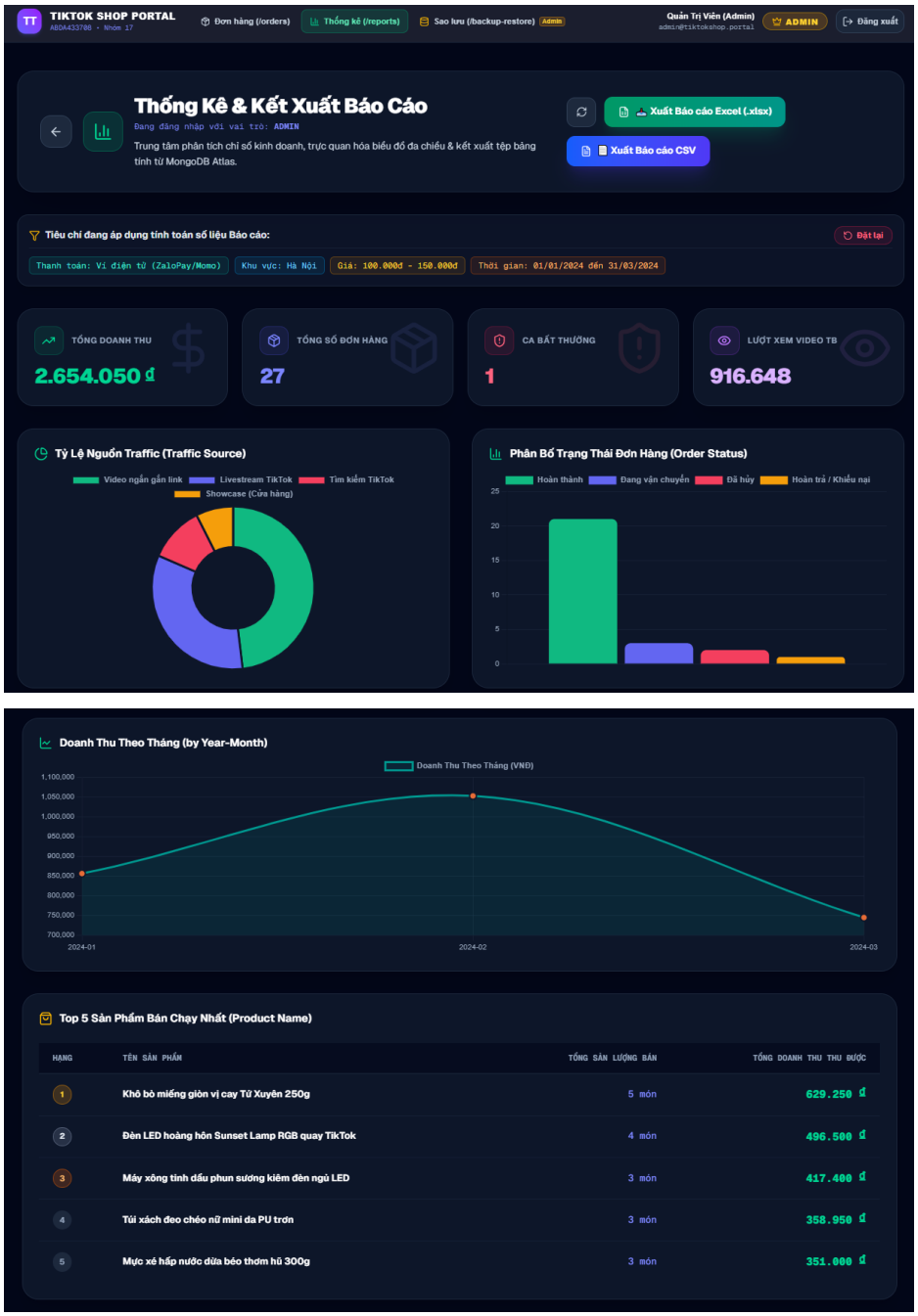
Hình 6.94. Giao diện Dashboard thống kê trước khi áp dụng bộ lọc.

Kiểm thử chức năng lọc dữ liệu

Để kiểm tra khả năng truy vấn dữ liệu theo nhiều điều kiện đồng thời, tiến hành thiết lập bộ lọc như sau:

- Phương thức thanh toán: Ví điện tử (ZaloPay/Momo).
- Khu vực giao hàng: Hà Nội.
- Khoảng giá trị đơn hàng: 100.000–150.000 VNĐ.
- Thời gian đặt hàng: Từ ngày 01/01/2024 đến ngày 31/03/2024.

Sau khi áp dụng bộ lọc, giao diện hiển thị các tiêu chí tương ứng dưới dạng nhãn màu ở phía trên Dashboard. Điều này giúp người dùng nhận biết phạm vi dữ liệu đang được sử dụng để tính toán và hạn chế nhầm lẫn khi phân tích kết quả.



Hình 6.95. Kết quả Dashboard sau khi áp dụng bộ lọc dữ liệu.

Kết quả hiển thị sau khi lọc ghi nhận bốn chỉ số KPI như sau:

Bảng 6.3. Các chỉ số KPI sau khi lọc

Chỉ số	Kết quả hiển thị
Tổng doanh thu	2.654.050 VNĐ
Tổng số đơn hàng	27 đơn
Ca bất thường	1 đơn
Lượt xem video trung bình	916.648 lượt

Nguồn: Nhóm tự tổng hợp

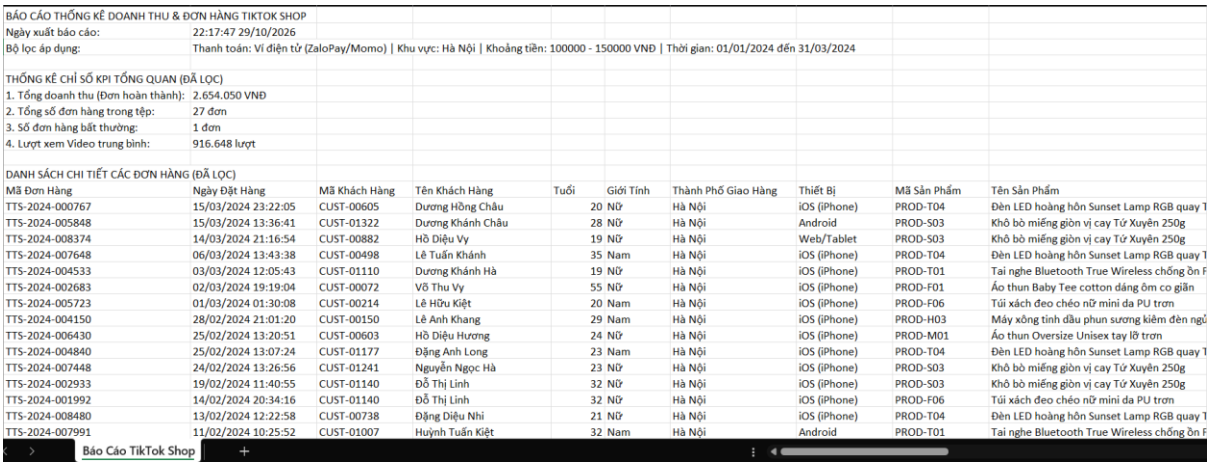
Các biểu đồ bên dưới cũng được cập nhật theo tập dữ liệu hiện tại:

- Biểu đồ tỷ lệ nguồn Traffic thể hiện cơ cấu các nguồn truy cập, gồm video ngắn gắn link, livestream TikTok, tìm kiếm TikTok và Showcase.
- Biểu đồ trạng thái đơn hàng cho phép quan sát sự phân bố giữa các trạng thái như hoàn thành, đang vận chuyển, đã hủy và hoàn trả/khiếu nại.
- Biểu đồ đường doanh thu theo tháng thể hiện sự biến động doanh thu từ tháng 1/2024 - 3/2024, hỗ trợ nhận diện xu hướng tăng giảm trong giai đoạn phân tích.
- Bảng top 5 sản phẩm bán chạy nhất xếp hạng sản phẩm theo kết quả bán hàng, với top 5 sản phẩm bán chạy nhất gồm: Khô bò miếng giòn vị cay Tứ Xuyên 250g, đèn LED hoàng hôn Sunset Lamp RGB quay TikTok, máy xông tinh dầu phun sương kiêm đèn ngủ LED, túi xách đeo chéo nữ mini da PU trơn, mực xé hấp nước dừa béo thơm hũ 300g.

Kiểm thử chức năng xuất báo cáo Excel

Sau khi áp dụng bộ lọc, tiến hành xuất báo cáo bằng định dạng Excel (.xlsx). Khi người dùng yêu cầu kết xuất, giao diện gửi các tham số lọc hiện tại cùng tham số type=xlsx đến API /api/export. API truy vấn các đơn hàng phù hợp, tổng hợp các chỉ số KPI và xây dựng workbook bằng thư viện xlsx, sau đó trả tệp về trình duyệt để tải xuống.

Sau khi tải xuống, mở tệp tiktok_shop_sales_report_29-10-2026.xlsx để kiểm tra cấu trúc và nội dung. Tệp được tạo trong quá trình kiểm thử có một worksheet mang tên Báo Cáo TikTok Shop.



BÁO CÁO THỐNG KÊ DOANH THU & ĐƠN HÀNG TIKTOK SHOP									
Ngày xuất báo cáo:	22:17:47 29/10/2026								
Bộ lọc áp dụng:	Thanh toán: Ví điện tử (ZaloPay/Momo) | Khu vực: Hà Nội | Khoảng tiền: 100000 - 150000 VNĐ | Thời gian: 01/01/2024 đến 31/03/2024								
THỐNG KÊ CHỈ SỐ KPI TỔNG QUAN (ĐÃ LỌC)									
1. Tổng doanh thu (Đơn hoàn thành):	2.654.050 VNĐ								
2. Tổng số đơn hàng trong tệp:	27 đơn								
3. Số đơn hàng bất thường:	1 đơn								
4. Lượt xem Video trung bình:	916.648 lượt								
DANH SÁCH CHI TIẾT CÁC ĐƠN HÀNG (ĐÃ LỌC)									
Mã Đơn Hàng	Ngày Đặt Hàng	Mã Khách Hàng	Tên Khách Hàng	Tuổi	Giới Tính	Thành Phố Giao Hàng	Thiết Bị	Mã Sản Phẩm	Tên Sản Phẩm
TTS-2024-000767	15/03/2024 23:22:05	CUST-00605	Dương Hồng Châu	20	Nữ	Hà Nội	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay T
TTS-2024-005848	15/03/2024 13:36:41	CUST-01322	Dương Khánh Châu	28	Nữ	Hà Nội	Android	PROD-S03	Khô bò miếng giòn vị cay Tứ Xuyên 250g
TTS-2024-008374	14/03/2024 21:16:54	CUST-00882	Hồ Diệu Vy	19	Nữ	Hà Nội	Web/Tablet	PROD-S03	Khô bò miếng giòn vị cay Tứ Xuyên 250g
TTS-2024-007648	06/03/2024 13:43:38	CUST-00498	Lê Tuấn Khánh	35	Nam	Hà Nội	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay T
TTS-2024-004533	03/03/2024 12:05:43	CUST-01110	Dương Khánh Hà	19	Nữ	Hà Nội	iOS (iPhone)	PROD-T01	Tai nghe Bluetooth True Wireless chống ồn F
TTS-2024-002683	02/03/2024 19:19:04	CUST-00072	Võ Thu Vy	55	Nữ	Hà Nội	iOS (iPhone)	PROD-F01	Áo thun Baby Tee cotton dáng ôm co giãn
TTS-2024-005723	01/03/2024 01:30:08	CUST-00214	Lê Hữu Kiệt	20	Nam	Hà Nội	iOS (iPhone)	PROD-F06	Túi xách đeo chéo nữ mini da PU trơn
TTS-2024-004150	28/02/2024 21:01:20	CUST-00150	Lê Anh Khang	29	Nam	Hà Nội	iOS (iPhone)	PROD-H03	Máy xông tinh dầu phun sương kiêm đèn ngủ
TTS-2024-006430	25/02/2024 13:20:51	CUST-00603	Hồ Diệu Hương	24	Nữ	Hà Nội	iOS (iPhone)	PROD-M01	Áo thun Oversize Unisex tay lỡ trơn
TTS-2024-004840	25/02/2024 13:07:24	CUST-01177	Đặng Anh Long	23	Nam	Hà Nội	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay T
TTS-2024-007448	24/02/2024 13:26:56	CUST-01241	Nguyễn Ngọc Hà	23	Nữ	Hà Nội	iOS (iPhone)	PROD-S03	Khô bò miếng giòn vị cay Tứ Xuyên 250g
TTS-2024-002933	19/02/2024 11:40:55	CUST-01140	Đỗ Thị Linh	32	Nữ	Hà Nội	iOS (iPhone)	PROD-S03	Khô bò miếng giòn vị cay Tứ Xuyên 250g
TTS-2024-001992	14/02/2024 20:34:16	CUST-01140	Đỗ Thị Linh	32	Nữ	Hà Nội	iOS (iPhone)	PROD-F06	Túi xách đeo chéo nữ mini da PU trơn
TTS-2024-008480	13/02/2024 12:22:58	CUST-00738	Đặng Diệu Nhi	21	Nữ	Hà Nội	iOS (iPhone)	PROD-T04	Đèn LED hoàng hôn Sunset Lamp RGB quay T
TTS-2024-007991	11/02/2024 10:25:52	CUST-01007	Huỳnh Tuấn Kiệt	32	Nam	Hà Nội	Android	PROD-T01	Tai nghe Bluetooth True Wireless chống ồn F

Báo Cáo TikTok Shop

Hình 6.96. Giao diện tệp Excel báo cáo khi mở bằng phần mềm bảng tính.

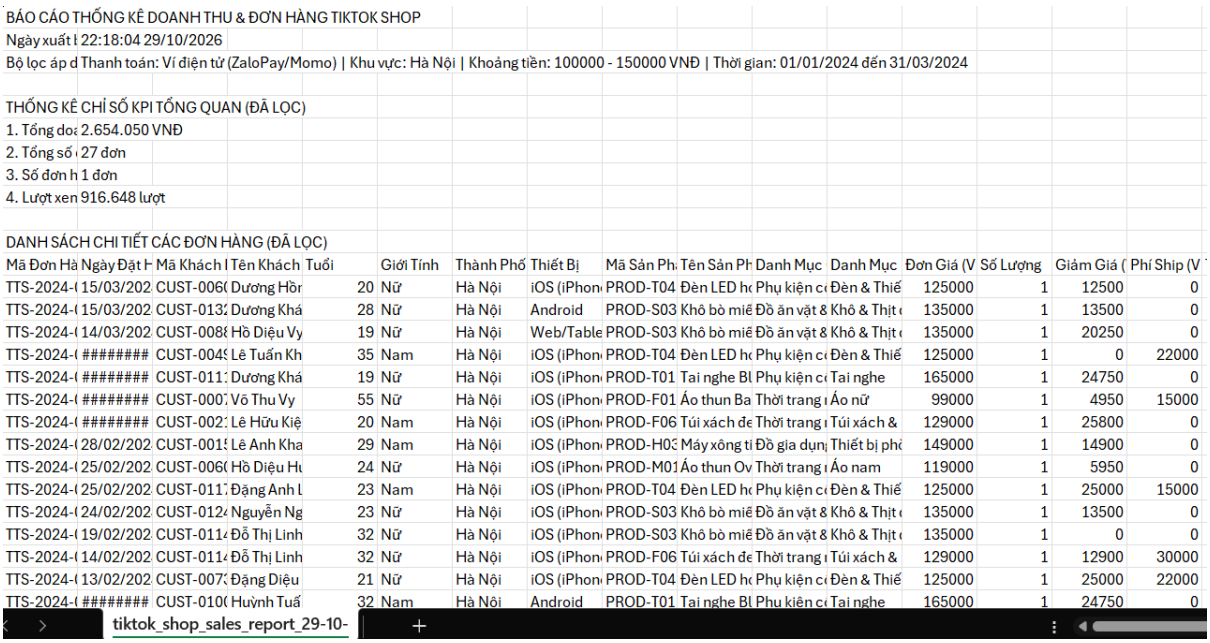
Các giá trị KPI trong tệp Excel trùng khớp với những chỉ số hiển thị trên Dashboard sau khi áp dụng bộ lọc. Đồng thời, phần mô tả bộ lọc trong tệp thể hiện đầy đủ phương thức thanh toán, khu vực, khoảng giá trị đơn hàng và thời gian đã chọn.

Bên dưới phần KPI, báo cáo cung cấp danh sách chi tiết với các trường dữ liệu về mã đơn hàng, ngày đặt hàng, thông tin khách hàng, thông tin sản phẩm, giá bán, số lượng, giảm giá, phí vận chuyển, tổng tiền, phương thức thanh toán, trạng thái đơn hàng, đánh giá sản phẩm, gian hàng, nguồn Traffic, Creator, lượt xem video và trạng thái nhận diện bất thường. Các bản ghi được sắp xếp theo ngày đặt hàng giảm dần, giúp người dùng thuận tiện tra cứu những giao dịch gần thời điểm cuối của khoảng thời gian đã chọn.

Kiểm thử chức năng xuất báo cáo CSV

Tiếp theo, sử dụng cùng bộ lọc đã áp dụng trên Dashboard để kết xuất báo cáo dưới định dạng CSV. Trong yêu cầu gửi đến API /api/export, tham số định dạng được chuyển thành type=csv. API sử dụng XLSX.utils.sheet_to_csv() để chuyển dữ liệu worksheet thành nội dung CSV, bổ sung UTF-8 BOM nhằm hỗ trợ hiển thị tiếng Việt có dấu trong các ứng dụng tương thích, sau đó trả tệp về trình duyệt.

Sau khi tải tệp CSV, mở tệp tiktok_shop_sales_report_29-10-2026.csv bằng phần mềm bảng tính để kiểm tra nội dung.



BÁO CÁO THỐNG KÊ DOANH THU & ĐƠN HÀNG TIKTOK SHOP

Ngày xuất l 22:18:04 29/10/2026

Bộ lọc áp d Thanh toán: Ví điện tử (ZaloPay/Momo) | Khu vực: Hà Nội | Khoảng tiền: 100000 - 150000 VNĐ | Thời gian: 01/01/2024 đến 31/03/2024

THỐNG KÊ CHỈ SỐ KPI TỔNG QUAN (ĐÃ LỌC)

1. Tổng doa 2.654.050 VNĐ
2. Tổng số (27 đơn
3. Số đơn h 1 đơn
4. Lượt xem 916.648 lượt

DANH SÁCH CHI TIẾT CÁC ĐƠN HÀNG (ĐÃ LỌC)

Mã Đơn Hà	Ngày Đặt H	Mã Khách l	Tên Khách	Tuổi	Giới Tính	Thành Phố	Thiết Bị	Mã Sản Ph	Tên Sản Ph	Danh Mục	Danh Mục	Đơn Giá (V	Số Lượng	Giảm Giá (	Phí Ship (V
TTS-2024-(	15/03/202	CUST-006(	Dương Hồn	20	Nữ	Hà Nội	iOS (iPhon	PROD-T04	Đèn LED hc	Phụ kiện cá	Đèn & Thiế	125000	1	12500	0
TTS-2024-(	15/03/202	CUST-013:	Dương Khá	28	Nữ	Hà Nội	Android	PROD-S03	Khô bò miế	Đồ ăn vặt &	Khô & Thịt (	135000	1	13500	0
TTS-2024-(	14/03/202	CUST-008(	Hồ Diệu Vy	19	Nữ	Hà Nội	Web/Table	PROD-S03	Khô bò miế	Đồ ăn vặt &	Khô & Thịt (	135000	1	20250	0
TTS-2024-(	########	CUST-004!	Lê Tuấn Kh	35	Nam	Hà Nội	iOS (iPhon	PROD-T04	Đèn LED hc	Phụ kiện cá	Đèn & Thiế	125000	1	0	22000
TTS-2024-(	########	CUST-011:	Dương Khá	19	Nữ	Hà Nội	iOS (iPhon	PROD-T01	Tai nghe Bl	Phụ kiện cá	Tai nghe	165000	1	24750	0
TTS-2024-(	########	CUST-000'	Võ Thu Vy	55	Nữ	Hà Nội	iOS (iPhon	PROD-F01	Áo thun Ba	Thời trang ı	Áo nữ	99000	1	4950	15000
TTS-2024-(	########	CUST-002:	Lê Hữu Kiệ	20	Nam	Hà Nội	iOS (iPhon	PROD-F06	Túi xách đe	Thời trang ı	Túi xách &	129000	1	25800	0
TTS-2024-(	28/02/202	CUST-001!	Lê Anh Kha	29	Nam	Hà Nội	iOS (iPhon	PROD-H03	Máy xông tỉ	Đồ gia dụn	Thiết bị phò	149000	1	14900	0
TTS-2024-(	25/02/202	CUST-006(	Hồ Diệu Hu	24	Nữ	Hà Nội	iOS (iPhon	PROD-M01	Áo thun Ov	Thời trang ı	Áo nam	119000	1	5950	0
TTS-2024-(	25/02/202	CUST-011'	Đặng Anh l	23	Nam	Hà Nội	iOS (iPhon	PROD-T04	Đèn LED hc	Phụ kiện cá	Đèn & Thiế	125000	1	25000	15000
TTS-2024-(	24/02/202	CUST-012(	Nguyễn Ng	23	Nữ	Hà Nội	iOS (iPhon	PROD-S03	Khô bò miế	Đồ ăn vặt &	Khô & Thịt (	135000	1	13500	0
TTS-2024-(	19/02/202	CUST-011(	Đỗ Thị Linh	32	Nữ	Hà Nội	iOS (iPhon	PROD-S03	Khô bò miế	Đồ ăn vặt &	Khô & Thịt (	135000	1	0	0
TTS-2024-(	14/02/202	CUST-011(	Đỗ Thị Linh	32	Nữ	Hà Nội	iOS (iPhon	PROD-F06	Túi xách đe	Thời trang ı	Túi xách &	129000	1	12900	30000
TTS-2024-(	13/02/202	CUST-007:	Đặng Diệu	21	Nữ	Hà Nội	iOS (iPhon	PROD-T04	Đèn LED hc	Phụ kiện cá	Đèn & Thiế	125000	1	25000	22000
TTS-2024-(	########	CUST-010(	Huỳnh Tuấ	32	Nam	Hà Nội	Android	PROD-T01	Tai nghe Bl	Phụ kiện cá	Tai nghe	165000	1	24750	0

tiktok_shop_sales_report_29-10-

Hình 6.97. Giao diện tệp CSV báo cáo khi mở bằng phần mềm bảng tính.

Như vậy, kết quả kiểm thử Dashboard và tệp Excel cho thấy hệ thống đã thể hiện được luồng xử lý từ áp dụng bộ lọc, cập nhật chỉ số thống kê đến kết xuất báo cáo chi tiết.

6.6. TÍNH NĂNG SAO LƯU VÀ PHỤC HỒI DỮ LIỆU MONGODB (BACKUP & RESTORE)

6.6.1. Tổng quan chức năng Sao lưu và Phục hồi dữ liệu (Backup & Restore)

Trong quá trình vận hành hệ thống quản lý đơn hàng TikTok Shop, dữ liệu giao dịch được lưu trữ và cập nhật liên tục trên MongoDB Atlas. Do đó, việc xây dựng chức năng sao lưu và phục hồi dữ liệu có ý nghĩa quan trọng trong việc hỗ trợ lưu trữ bản sao, hạn chế rủi ro mất dữ liệu và tạo điều kiện khôi phục các bản ghi khi cần thiết.

Chức năng Backup & Restore được phát triển dành riêng cho tài khoản quản trị viên (ADMIN), cho phép xuất dữ liệu giao dịch thành tệp JSON và sử dụng tệp này để phục hồi dữ liệu vào cơ sở dữ liệu. Khác với chức năng kết xuất báo cáo Excel/CSV, chức năng này tập trung lưu giữ dữ liệu giao dịch dưới dạng cấu trúc JSON, bao gồm các trường thông tin và MongoDB ObjectId của từng bản ghi.

Về mặt triển khai, chức năng gồm hai thành phần chính: API /api/backup đảm nhiệm việc truy vấn, xuất và phục hồi dữ liệu; giao diện /backup-restore cung cấp các thao tác trực quan để quản trị viên thực hiện sao lưu và phục hồi.

Quy trình hoạt động gồm hai luồng:

- **Backup:** Quản trị viên nhấn nút tải file JSON. Giao diện gửi yêu cầu GET đến API. Sau khi kiểm tra quyền và kết nối MongoDB Atlas, hệ thống truy vấn dữ liệu từ model Transaction, chuyển đổi thành JSON và trả tệp về trình duyệt.
- **Restore:** Quản trị viên lựa chọn tệp JSON và xác nhận phục hồi. Giao diện gửi tệp đến API bằng phương thức POST. Máy chủ kiểm tra định dạng, cấu trúc và các trường dữ liệu bắt buộc, sau đó sử dụng MongoDB ObjectId để xác định bản ghi cần cập nhật hoặc thêm mới.

Để kiểm soát quyền sử dụng, hệ thống triển khai hai lớp bảo vệ. Ở giao diện, component Protection giới hạn quyền truy cập đối với vai trò ADMIN. Ở phía máy chủ, hàm requireAdmin() xác thực phiên đăng nhập và kiểm tra vai trò trước khi thực hiện các thao tác với cơ sở dữ liệu.

6.6.2. Xây dựng API sao lưu dữ liệu MongoDB (GET /api/backup)

API sao lưu được xây dựng trong tệp src/app/api/backup/route.ts, sử dụng phương thức GET để truy xuất dữ liệu giao dịch từ MongoDB Atlas và trả về dưới dạng tệp JSON. API sử dụng môi trường Node.js, kết hợp các thành phần Next.js, NextAuth, Mongoose và model Transaction.

Kiểm tra quyền truy cập quản trị viên

Trước khi truy vấn dữ liệu, hệ thống sử dụng hàm requireAdmin() để kiểm tra quyền người dùng thông qua getServerSession(authOptions).

```
async function requireAdmin() {
  const session = await getServerSession(authOptions);

  if (!session?.user) {
    return NextResponse.json(
      { message: "Bạn cần đăng nhập." },
      { status: 401 }
    );
  }

  const role = (session.user as { role?: string }).role;

  if (role?.toUpperCase() !== "ADMIN") {
    return NextResponse.json(
      { message: "Chỉ ADMIN mới được sử dụng chức năng này." },
      { status: 403 }
    );
  }

  return null;
}
```

Hình 6.98. Đoạn mã kiểm tra phiên đăng nhập và phân quyền ADMIN trong API Backup & Restore.

Hàm getServerSession() lấy thông tin phiên đăng nhập hiện tại. Nếu không tồn tại phiên hoặc không có thông tin người dùng, API trả về mã HTTP 401. Trường hợp người dùng đã đăng nhập nhưng không có vai trò ADMIN, hệ thống trả về HTTP 403.

Đoạn kiểm tra role?.toUpperCase() !== "ADMIN" giúp chuẩn hóa giá trị vai trò trước khi so sánh. Nếu xác thực thành công, hàm trả về null và cho phép API tiếp tục xử lý. Cơ chế này đảm bảo yêu cầu không đủ quyền bị từ chối ngay từ đầu.

Kết nối và truy vấn dữ liệu giao dịch

Sau khi xác thực thành công, API gọi connectToDatabase() để thiết lập kết nối với MongoDB Atlas. Tiếp theo, model Transaction được sử dụng để truy vấn dữ liệu:

const records = await Transaction.find({})

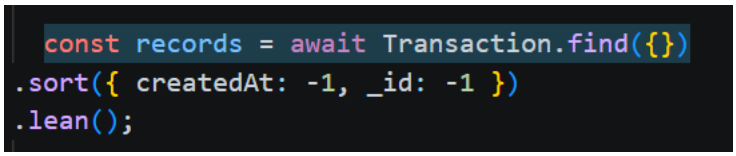
.sort({ createdAt: -1, _id: -1 })

.lean();

Trong đó:

- Transaction.find({}): truy vấn toàn bộ bản ghi trong collection được ánh xạ bởi model Transaction.
- sort({ createdAt: -1, _id: -1 }): sắp xếp theo thời gian tạo giảm dần, giúp bản ghi mới hơn xuất hiện trước. Trường _id được sử dụng làm tiêu chí sắp xếp bổ sung.
- lean(): trả về dữ liệu dưới dạng các đối tượng JavaScript thông thường thay vì Mongoose Document, phù hợp với mục đích đọc và xuất dữ liệu.

Việc không sử dụng điều kiện lọc giúp API lấy toàn bộ dữ liệu giao dịch hiện có trong phạm vi collection tương ứng. Dữ liệu sau truy vấn được lưu trong biến records để chuẩn bị chuyển đổi sang định dạng JSON.



Hình 6.99. Đoạn mã truy vấn toàn bộ dữ liệu giao dịch và sắp xếp bản ghi trước khi sao lưu.

Chuyển đổi và xuất tệp JSON

Sau khi truy vấn dữ liệu, API tạo tên tệp dựa trên ngày thực hiện sao lưu:

const date = new Date().toISOString().slice(0, 10);

const filename = `tiktok_orders_backup_${date}.json`;

Phương thức toISOString() chuyển thời gian hiện tại sang định dạng ISO 8601, còn slice(0, 10) lấy phần ngày theo cấu trúc năm-tháng-ngày. Nhờ đó, tên tệp được tạo theo mẫu tiktok_orders_backup_YYYY-MM-DD.json.

Tiếp theo, dữ liệu được chuyển thành chuỗi JSON và trả về trình duyệt:

return new Response(JSON.stringify(records, null, 2), {

status: 200,

headers: {

"Content-Type": "application/json; charset=utf-8",

"Content-Disposition": `attachment; filename="${filename}"`,

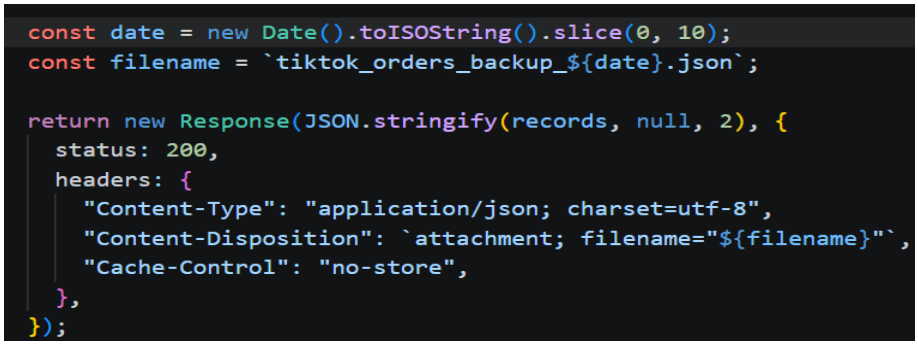
"Cache-Control": "no-store",

},

});

Trong đó, JSON.stringify(records, null, 2) chuyển danh sách bản ghi thành chuỗi JSON với mức thụt lề hai khoảng trắng, giúp nội dung dễ đọc và kiểm tra. Các HTTP Header được thiết lập như sau:

- Content-Type: xác định nội dung phản hồi là JSON sử dụng UTF-8.
- Content-Disposition: attachment: yêu cầu trình duyệt xử lý phản hồi như một tệp tải xuống và sử dụng tên tệp đã thiết lập.
- Cache-Control: no-store: hạn chế lưu trữ phản hồi trong bộ nhớ đệm.



Hình 6.100. Đoạn mã chuyển đổi dữ liệu thành JSON và thiết lập HTTP Response.

Nếu xảy ra lỗi trong quá trình kết nối, truy vấn hoặc tạo tệp, khối catch ghi nhận lỗi bằng console.error() và trả về phản hồi JSON với mã HTTP 500.

Như vậy, API GET /api/backup hoàn thiện quy trình sao lưu từ kiểm tra quyền, truy vấn dữ liệu đến chuyển đổi và trả tệp JSON về trình duyệt.

6.6.3. Xây dựng API phục hồi dữ liệu MongoDB (POST /api/backup)

Bên cạnh chức năng sao lưu, API /api/backup sử dụng phương thức POST để tiếp nhận tệp JSON và phục hồi dữ liệu vào MongoDB Atlas. Quá trình này được chia thành các bước: tiếp nhận tệp, kiểm tra nội dung, xác thực định danh và thực hiện thao tác ghi dữ liệu.

Tiếp nhận và kiểm tra tệp JSON

Khi quản trị viên xác nhận phục hồi trên giao diện, Client tạo đối tượng FormData và gửi tệp đến API. Phía máy chủ sử dụng request.formData() để đọc dữ liệu và lấy tệp thông qua trường file.

```
const formData = await request.formData();
const file = formData.get("file");
if (!(file instanceof File)) {
  return NextResponse.json(
        { message: "Vui lòng chọn file JSON." },
        { status: 400 }
  );
}
```

Điều kiện file instanceof File kiểm tra dữ liệu nhận được có phải đối tượng tệp hợp lệ hay không. Nếu không có tệp được cung cấp, API trả về HTTP 400.

Tiếp theo, hệ thống kiểm tra phần mở rộng của tên tệp:

```
if (!file.name.toLowerCase().endsWith(".json")) {
  return NextResponse.json(
```

```
        { message: "Chỉ chấp nhận file .json." },
        { status: 400 }
  );
}
```

Phương thức toLowerCase() chuẩn hóa tên tệp về chữ thường trước khi kiểm tra, còn endsWith(".json") xác định tệp có phần mở rộng phù hợp hay không.

Sau đó, API đọc nội dung tệp và kiểm tra giới hạn:

```
const text = await file.text();
if (text.length > 20 * 1024 * 1024) {
  return NextResponse.json(
        { message: "File vượt quá giới hạn 20 MB." },
        { status: 413 }
  );
}
```

Nếu độ dài nội dung vượt quá ngưỡng kiểm tra, hệ thống trả về HTTP 413 (Payload Too Large). Lưu ý, code thực hiện đọc nội dung trước rồi mới kiểm tra text.length, do đó đây là bước giới hạn sau khi nội dung đã được nạp vào bộ nhớ.

```
const formData = await request.formData();
const file = formData.get("file");

if (!(file instanceof File)) {
  return NextResponse.json(
    { message: "Vui lòng chọn file JSON." },
    { status: 400 }
  );
}

if (!file.name.toLowerCase().endsWith(".json")) {
  return NextResponse.json(
    { message: "Chỉ chấp nhận file .json." },
    { status: 400 }
  );
}

const text = await file.text();

if (text.length > 20 * 1024 * 1024) {
  return NextResponse.json(
    { message: "File vượt quá giới hạn 20 MB." },
    { status: 413 }
  );
}
```

Hình 6.101. Đoạn mã tiếp nhận tệp JSON và kiểm tra phần mở rộng, dung lượng đầu vào.

Phân tích và kiểm tra cấu trúc dữ liệu

Sau khi kiểm tra tệp, API sử dụng JSON.parse() để chuyển nội dung từ chuỗi thành dữ liệu JavaScript:

```
let records: unknown;


try {
  records = JSON.parse(text);
} catch {
  return NextResponse.json(
        { message: "Nội dung file JSON không hợp lệ." },
        { status: 400 }
  );
```

```
}
```

Biến records được khai báo với kiểu unknown vì cấu trúc dữ liệu đầu vào chưa được xác định trước khi kiểm tra. Nếu nội dung không đúng cú pháp JSON, API trả về HTTP 400.

Tiếp theo, hệ thống kiểm tra dữ liệu có phải mảng JSON và có ít nhất một bản ghi hay không:

```
if (!Array.isArray(records) || records.length === 0) {
  return NextResponse.json(
        { message: "File phải chứa một mảng JSON có ít nhất một bản ghi." },
        { status: 400 }
  );
}
```

Điều kiện Array.isArray() đảm bảo dữ liệu có cấu trúc mảng phù hợp với định dạng tệp Backup. Điều kiện records.length === 0 ngăn quá trình phục hồi tiếp tục khi tệp không chứa bản ghi.

Sau đó, API sử dụng every() để kiểm tra từng phần tử trong mảng. Mỗi bản ghi phải là đối tượng hợp lệ và có đầy đủ các trường bắt buộc với kiểu dữ liệu tương ứng.

```
const valid = records.every(
  (record) =>
        record !== null &&
    typeof record === "object" &&
    !Array.isArray(record) &&
        typeof record.Order_ID === "string" &&
        typeof record.Order_Date === "string" &&
        typeof record.Customer_Name === "string" &&
        typeof record.Product_Name === "string" &&
    Number.isFinite(record.Quantity) &&
```

```
  Number.isFinite(record.Total_Amount) &&
    typeof record.Payment_Method === "string" &&
    typeof record.Order_Status === "string"
);
```

Các trường được kiểm tra gồm Order_ID, Order_Date, Customer_Name, Product_Name, Quantity, Total_Amount, Payment_Method và Order_Status. Trong đó, Number.isFinite() được sử dụng để xác nhận các giá trị số là số hữu hạn.

Nếu có bản ghi không đáp ứng yêu cầu, API trả về HTTP 400 và dừng quá trình phục hồi. Việc kiểm tra này giúp hạn chế dữ liệu sai cấu trúc được đưa vào cơ sở dữ liệu.

```
let records: unknown;

try {
  records = JSON.parse(text);
} catch {
  return NextResponse.json(
    { message: "Nội dung file JSON không hợp lệ." },
    { status: 400 }
  );
}

if (!Array.isArray(records) || records.length === 0) {
  return NextResponse.json(
    { message: "File phải chứa một mảng JSON có ít nhất một bản ghi." },
    { status: 400 }
  );
}
```

Hình 6.102. Đoạn mã phân tích JSON và kiểm tra cấu trúc mảng dữ liệu đầu vào.

```
const valid = records.every(
  (record) =>
    record !== null &&
    typeof record === "object" &&
    !Array.isArray(record) &&
    typeof record.Order_ID === "string" &&
    typeof record.Order_Date === "string" &&
    typeof record.Customer_Name === "string" &&
    typeof record.Product_Name === "string" &&
    Number.isFinite(record.Quantity) &&
    Number.isFinite(record.Total_Amount) &&
    typeof record.Payment_Method === "string" &&
    typeof record.Order_Status === "string"
);

if (!valid) {
  return NextResponse.json(
    { message: "Bản ghi thiếu trường bắt buộc hoặc sai kiểu dữ liệu." },
    { status: 400 }
  );
}
```

Hình 6.103. Đoạn mã kiểm tra các trường dữ liệu bắt buộc trong từng bản ghi giao dịch.

Kiểm tra MongoDB ObjectId và xây dựng thao tác phục hồi

Sau khi xác thực cấu trúc dữ liệu, API kiểm tra trường _id của từng bản ghi. Đây là thành phần quan trọng vì hệ thống sử dụng MongoDB ObjectId để xác định bản ghi cần cập nhật hoặc thêm mới.

const operations = [];

for (const record of records as Record<string, unknown>[]) {

const { _id, __v, ...data } = record;

if (

typeof _id !== "string" ||

!mongoose.isValidObjectId(_id)

) {

return NextResponse.json(

{

message:

"File backup có bản ghi thiếu _id MongoDB hợp lệ. Đã dừng phục hồi để tránh ghi nhầm dữ liệu.",

},

{ status: 400 }

);

}

}

Cú pháp { _id, __v, ...data } tách trường định danh _id và trường phiên bản __v khỏi phần dữ liệu còn lại. Trường _id được sử dụng làm căn cứ đối chiếu trong MongoDB, còn __v là trường phiên bản nội bộ do Mongoose quản lý.

Hàm mongoose.isValidObjectId() kiểm tra định danh có hợp lệ hay không. Nếu bản ghi thiếu _id hoặc có định dạng không phù hợp, hệ thống trả về HTTP 400 và dừng xử lý.

Sau khi kiểm tra, API xây dựng các thao tác ghi dữ liệu:

```
operations.push({
  replaceOne: {
    filter: { _id: new mongoose.Types.ObjectId(_id) },
        replacement: {
        ...data,
        _id: new mongoose.Types.ObjectId(_id),
        },
        upsert: true,
  },
});
```

Trong đó:

- replaceOne: thay thế nội dung của bản ghi phù hợp với điều kiện tìm kiếm.

- filter: xác định bản ghi thông qua MongoDB ObjectId.
- replacement: chứa dữ liệu được lấy từ tệp JSON và giữ lại _id.
- upsert: true: cho phép thêm mới nếu không tìm thấy bản ghi tương ứng.

Như vậy, hệ thống có thể xử lý hai trường hợp: cập nhật bản ghi đã tồn tại hoặc thêm mới bản ghi chưa có trong cơ sở dữ liệu. Cơ chế này giúp hạn chế việc tạo các bản ghi trùng lặp chỉ vì thực hiện phục hồi nhiều lần bằng cùng một tệp, miễn là định danh _id được giữ nguyên và không bị trùng trong chính tệp đầu vào.

Thực hiện phục hồi hàng loạt bằng bulkWrite()

Sau khi xây dựng danh sách thao tác, API sử dụng bulkWrite() để thực hiện nhiều yêu cầu ghi dữ liệu:

```
const result = await Transaction.bulkWrite(operations, {
  ordered: false,
});
```

Thay vì gửi từng thao tác cập nhật hoặc thêm mới riêng biệt, bulkWrite() cho phép gom nhiều thao tác ghi vào một lần gọi. Cách triển khai này phù hợp với tệp Backup chứa số lượng lớn bản ghi giao dịch.

Tham số ordered: false cho phép MongoDB tiếp tục thực hiện các thao tác khác khi một thao tác gặp lỗi. Tuy nhiên, tùy chọn này không bảo đảm toàn bộ quá trình phục hồi có tính nguyên tử; một số bản ghi có thể được xử lý thành công trong khi những bản ghi khác gặp lỗi.

Sau khi hoàn tất, API tổng hợp kết quả:

```
return NextResponse.json({
  message: "Đã hoàn tất phục hồi dữ liệu.",
  received: records.length,
  inserted: result.upsertedCount,
  updated: result.modifiedCount,
});
```

Trong đó, received là số bản ghi tiếp nhận, inserted là số bản ghi được thêm mới thông qua upsert và updated là số bản ghi có nội dung được cập nhật theo kết quả modifiedCount của MongoDB.

Nếu phát sinh lỗi trong quá trình xử lý, khối catch ghi nhận lỗi ở phía máy chủ và trả về HTTP 500.

Nhìn chung, API POST /api/backup triển khai quy trình phục hồi theo nhiều bước kiểm tra trước khi ghi dữ liệu. Việc kết hợp ObjectId, replaceOne(), upsert và bulkWrite() cho phép cập nhật hoặc bổ sung bản ghi dựa trên định danh MongoDB.

6.6.4. Xây dựng giao diện chức năng Backup & Restore (/backup-restore)

Giao diện Backup & Restore được xây dựng trong tệp src/app/backup-restore/page.tsx, sử dụng Next.js và React. Trang được khai báo là Client Component thông qua chỉ thị "use client", cho phép quản lý trạng thái, xử lý sự kiện và phản hồi thao tác người dùng.

Giao diện tích hợp các thư viện và thành phần gồm lucide-react để hiển thị biểu tượng, react-hot-toast để thông báo kết quả và component Protection để kiểm soát quyền truy cập.

Khởi tạo trạng thái và phân quyền giao diện

Hệ thống sử dụng React State để quản lý các trạng thái chính:

```
const [loadingBackup, setLoadingBackup] = useState(false);
const [loadingRestore, setLoadingRestore] = useState(false);
const [file, setFile] = useState<File | null>(null);
const [showConfirm, setShowConfirm] = useState(false);
```

Trong đó, loadingBackup và loadingRestore theo dõi trạng thái xử lý của hai chức năng; file lưu tệp JSON được lựa chọn; còn showConfirm điều khiển việc hiển thị hộp thoại xác nhận.

Toàn bộ trang được đặt bên trong component:

```
<Protection allowedRoles={["ADMIN"]}>
  <main>
```

...

</main>

</Protection>

Thuộc tính allowedRoles giới hạn giao diện cho tài khoản có vai trò ADMIN. Cơ chế này phối hợp với việc xác thực tại API để kiểm soát quyền truy cập ở cả phía Client và Server.

```
export default function BackupRestorePage() {
  const [loadingBackup, setLoadingBackup] = useState(false);
  const [loadingRestore, setLoadingRestore] = useState(false);
  const [file, setFile] = useState<File | null>(null);
  const [showConfirm, setShowConfirm] = useState(false);
```

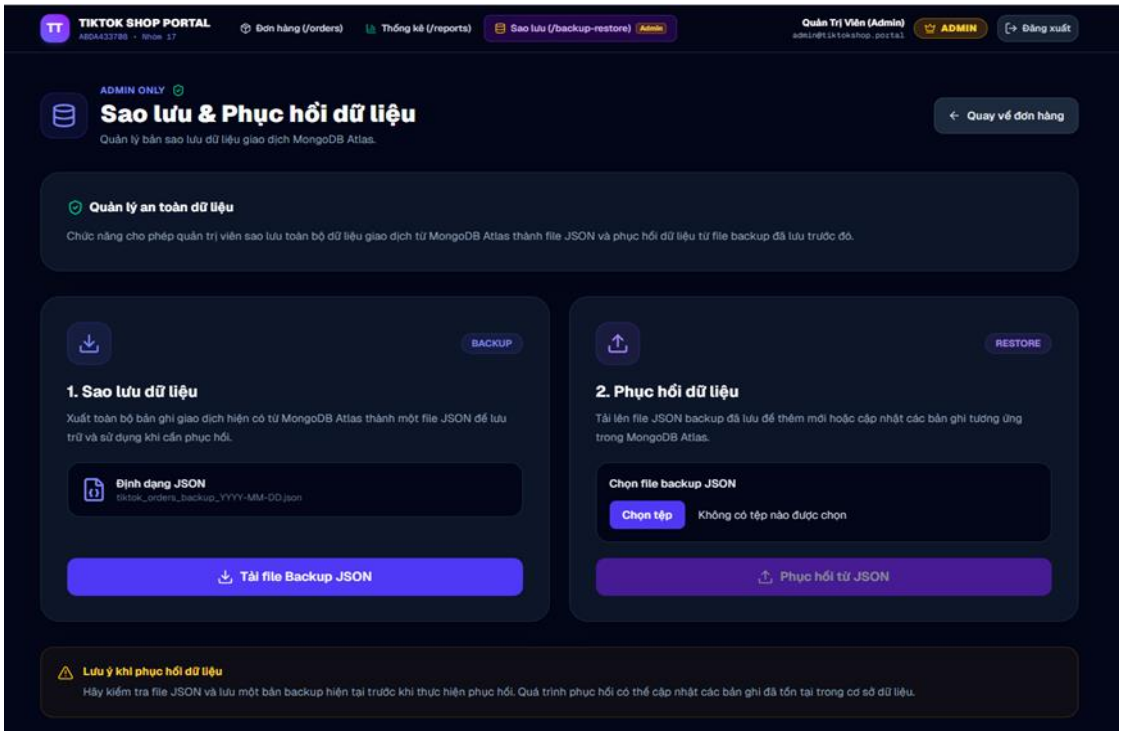
Hình 6.104. Đoạn mã khởi tạo React State

Thiết kế bố cục giao diện

Giao diện gồm tiêu đề trang, khu vực giới thiệu chức năng, hai thẻ Backup và Restore cùng phần lưu ý trước khi phục hồi dữ liệu.

Thẻ Backup cung cấp thông tin định dạng JSON và nút tải tệp sao lưu. Thẻ Restore gồm bộ chọn tệp, khu vực hiển thị tên và dung lượng file, cùng nút thực hiện phục hồi.

Phía dưới giao diện là phần cảnh báo yêu cầu quản trị viên kiểm tra tệp JSON và lưu một bản Backup hiện tại trước khi phục hồi. Giao diện sử dụng nền tối, các thẻ nội dung riêng biệt và màu sắc khác nhau cho hai chức năng nhằm duy trì tính nhất quán với các trang quản lý khác của hệ thống.



Hình 6.105. Giao diện tổng thể chức năng Sao lưu và Phục hồi dữ liệu MongoDB.

Xử lý thao tác Backup và tải tệp JSON

Khi nhấn nút Tải file Backup JSON, hàm handleBackup() được gọi. Hàm sử dụng fetch("/api/backup") để gửi yêu cầu GET đến máy chủ.

Sau khi nhận phản hồi thành công, response.blob() chuyển nội dung tệp thành đối tượng Blob. Tiếp theo, URL.createObjectURL() tạo URL tạm thời để trình duyệt truy cập dữ liệu, còn phần tử liên kết <a> được tạo và thiết lập thuộc tính download để tải tệp về thiết bị.

```
const blob = await response.blob();
const url = URL.createObjectURL(blob);
const link = document.createElement("a");
link.href = url;
link.download = `tiktok_orders_backup_${new Date()
  .toISOString()
  .slice(0, 10)}.json`;
document.body.appendChild(link);
link.click();
```

link.remove();

URL.revokeObjectURL(url);

Sau khi kích hoạt tải xuống, phần tử liên kết tạm được loại bỏ và URL đối tượng được thu hồi nhằm giải phóng tài nguyên. Nếu thao tác thành công, hệ thống hiển thị thông báo bằng toast.success(). Trường hợp xảy ra lỗi, toast.error() hiển thị nội dung tương ứng. Cuối cùng, finally đặt lại trạng thái loadingBackup.



```
// SAO LƯU DỮ LIỆU
async function handleBackup() {
  setLoadingBackup(true);

  try {
    const response = await fetch("/api/backup");

    if (!response.ok) {
      const data = await response.json();
      throw new Error(data.message || "Không thể sao lưu dữ liệu.");
    }

    const blob = await response.blob();
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");

    link.href = url;
    link.download = `tiktok_orders_backup_${new Date()
      .toISOString()
      .slice(0, 10)}.json`;

    document.body.appendChild(link);
    link.click();
    link.remove();

    URL.revokeObjectURL(url);

    toast.success("Đã tải file backup JSON thành công.");
  } catch (err) {
    toast.error(
      err instanceof Error ? err.message : "Lỗi sao lưu dữ liệu."
    );
  } finally {
    setLoadingBackup(false);
  }
}
```

Hình 6.106. Đoạn mã xử lý yêu cầu Backup và tải tệp JSON về thiết bị.

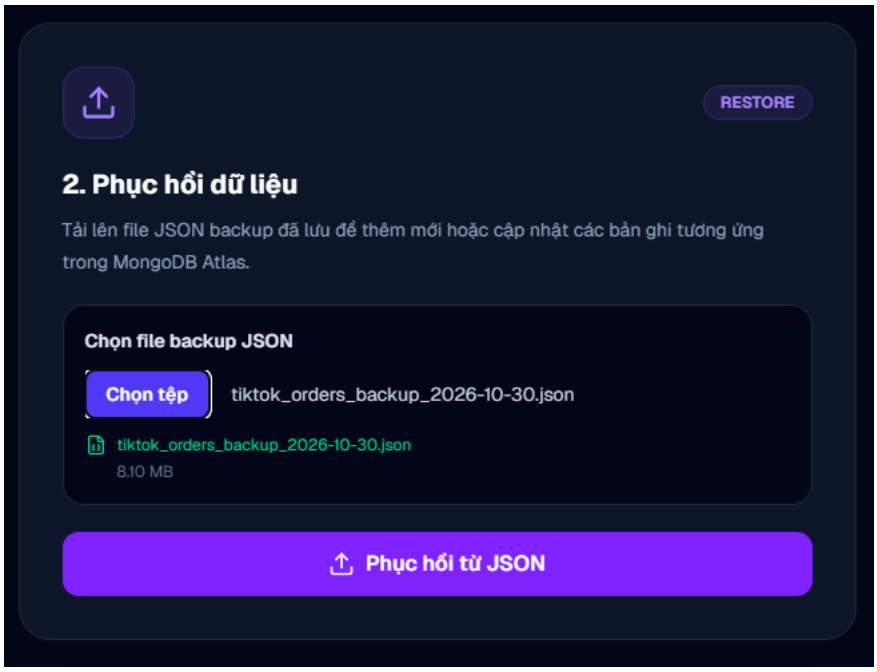
Lựa chọn tệp và xác nhận phục hồi

Ở khu vực Restore, người dùng lựa chọn tệp thông qua phần tử <input type="file">. Thuộc tính accept=".json,application/json" giới hạn các loại tệp được đề xuất trong hộp thoại chọn file của trình duyệt.

Sự kiện onChange cập nhật tệp vào biến trạng thái file. Khi có tệp được chọn, giao diện hiển thị tên và dung lượng file để quản trị viên kiểm tra trước khi phục hồi.

Khi nhấn nút Phục hồi từ JSON, hàm handleRestore() kiểm tra sự tồn tại của tệp. Nếu chưa chọn file, hệ thống hiển thị thông báo yêu cầu người dùng bổ sung tệp. Nếu đã có tệp, showConfirm được chuyển thành true để mở hộp thoại xác nhận.

Hộp thoại hiển thị tên file, cảnh báo các bản ghi có cùng MongoDB ObjectId sẽ được cập nhật và cung cấp hai lựa chọn Hủy hoặc Xác nhận phục hồi.



Hình 6.107. Hộp thoại xác nhận phục hồi dữ liệu từ tệp JSON.

Gửi yêu cầu Restore và hiển thị kết quả

Khi người dùng nhấn Xác nhận phục hồi, hàm confirmRestore() tạo FormData và đưa tệp vào trường file, sau đó gửi yêu cầu POST đến API /api/backup.

```
const formData = new FormData();

formData.append("file", file);
```

```
const response = await fetch("/api/backup", {
  method: "POST",
  body: formData,
});
const data = await response.json();
```

Sau khi nhận phản hồi, giao diện kiểm tra response.ok. Nếu API trả về lỗi, hệ thống sử dụng toast.error() để hiển thị thông báo. Nếu thành công, toast.success() thông báo số bản ghi tiếp nhận, thêm mới và cập nhật.

```
toast.success(
  `Phục hồi hoàn tất! Đã nhận ${data.received} bản ghi, thêm ${data.inserted}, cập nhật ${data.updated}.`,
  { duration: 5000 }
);
```

Sau khi phục hồi thành công, biến trạng thái file được đặt lại thành null và giá trị của phần tử input được xóa để người dùng có thể lựa chọn tệp khác. Khối finally đặt lại loadingRestore, bảo đảm giao diện trở về trạng thái ban đầu sau khi xử lý.

Như vậy, giao diện Backup & Restore quản lý toàn bộ quá trình tương tác từ lựa chọn tệp, xác nhận thao tác, gửi yêu cầu đến API và hiển thị kết quả cho quản trị viên.

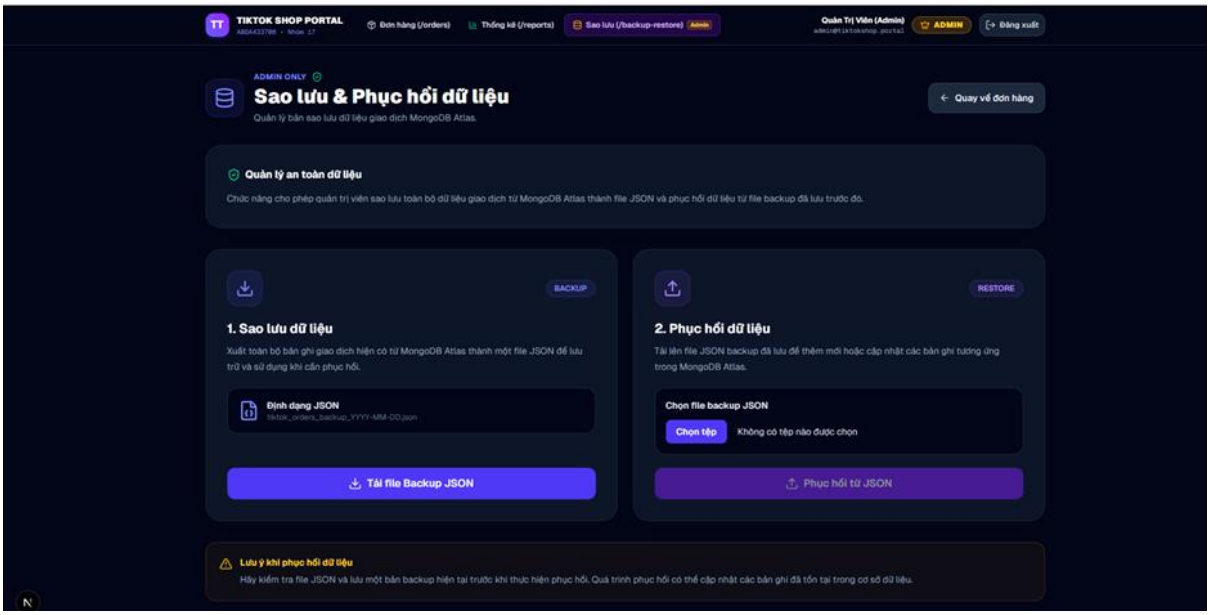
6.6.5. Kiểm thử và đánh giá kết quả

Sau khi hoàn thiện API và giao diện, tiến hành kiểm thử chức năng Backup & Restore trên website TikTok Shop Portal bằng tài khoản ADMIN. Quá trình kiểm thử tập trung vào khả năng tạo tệp JSON, kiểm tra nội dung dữ liệu và phục hồi bản ghi vào MongoDB Atlas.

Kiểm thử giao diện chức năng

Khi truy cập trang /backup-restore bằng tài khoản ADMIN, hệ thống hiển thị giao diện gồm hai khu vực chính: Sao lưu dữ liệu và Phục hồi dữ liệu. Các thành phần được bố trí rõ ràng, bao gồm nút tải file Backup JSON, bộ chọn tệp Restore và phần cảnh báo trước khi phục hồi.

Kết quả cho thấy giao diện được hiển thị đầy đủ và các thành phần thao tác chính phù hợp với mục đích quản lý dữ liệu dành cho quản trị viên.

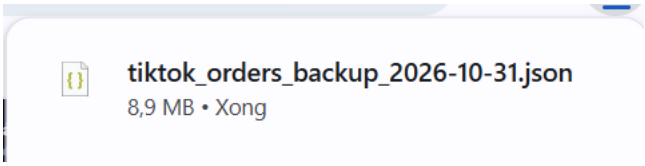


Hình 6.108. Giao diện chức năng Backup & Restore sau khi đăng nhập bằng tài khoản ADMIN.

Kiểm thử chức năng sao lưu dữ liệu

Tiến hành nhấn nút Tải file Backup JSON trên giao diện. Hệ thống gửi yêu cầu GET đến API /api/backup, truy vấn dữ liệu từ MongoDB Atlas và trả tệp JSON về trình duyệt.

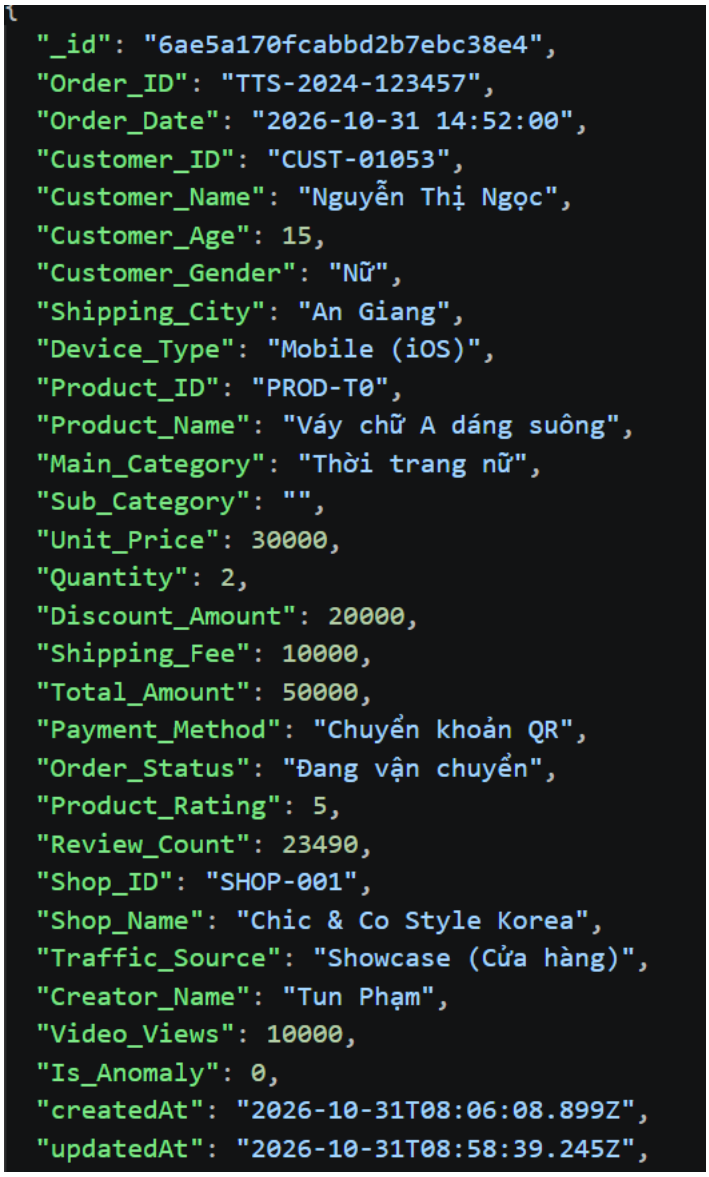
Kết quả ghi nhận tệp tiktok_orders_backup_2026-10-31.json được tải xuống thành công với dung lượng hiển thị là 8,9 MB. Tên tệp được tạo theo cấu trúc ngày tháng đã thiết lập trong mã nguồn.



Hình 6.109. Kết quả tải tệp Backup JSON thành công trên trình duyệt.

Sau khi tải xuống, tệp được mở bằng Visual Studio Code để kiểm tra cấu trúc. Kết quả cho thấy dữ liệu được tổ chức thành các đối tượng JSON, bao gồm những trường như _id, Order_ID, Order_Date, Customer_ID, Customer_Name, Product_ID, Product_Name, Unit_Price, Quantity, Total_Amount, Payment_Method, Order_Status và các thông tin bổ sung về Traffic, Creator, lượt xem video và trạng thái bất thường.

Ngoài các trường nghiệp vụ, dữ liệu còn lưu giữ createdAt và updatedAt. Đặc biệt, _id được thể hiện dưới dạng chuỗi ObjectId, tạo cơ sở để API đối chiếu bản ghi trong quá trình phục hồi.



Hình 6.110. Cấu trúc một bản ghi dữ liệu giao dịch trong tệp Backup JSON.

Kiểm thử chức năng phục hồi dữ liệu

Tiếp theo, lựa chọn tệp tiktok_orders_backup_2026-10-31.json trên giao diện Restore. Khi nhấn nút Phục hồi từ JSON, hệ thống hiển thị hộp thoại xác nhận với tên tệp và cảnh báo về việc cập nhật các bản ghi có cùng MongoDB ObjectId.

Sau khi nhấn Xác nhận phục hồi, giao diện gửi tệp đến API bằng phương thức POST. Hệ thống tiến hành kiểm tra dữ liệu và thực hiện các thao tác ghi vào MongoDB Atlas.

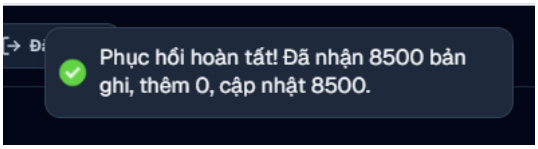


Hình 6.111. Hộp thoại xác nhận phục hồi dữ liệu từ tệp JSON đã sao lưu.

Kết quả kiểm thử hiển thị thông báo:

- Tổng số bản ghi tiếp nhận: 8.500 bản ghi.
- Số bản ghi được thêm mới: 0 bản ghi.
- Số bản ghi được cập nhật: 8.500 bản ghi.

Kết quả này phù hợp với trường hợp sử dụng chính tệp Backup vừa tạo để phục hồi vào cơ sở dữ liệu hiện tại. Do các bản ghi tương ứng đã tồn tại, hệ thống thực hiện cập nhật thay vì thêm mới.



Hình 6.112. Thông báo kết quả phục hồi với 8.500 bản ghi được cập nhật.

Đánh giá kết quả

Qua quá trình kiểm thử, chức năng Backup & Restore đã thực hiện được các yêu cầu chính trong phạm vi triển khai.

Đối với Backup, hệ thống có khả năng truy vấn dữ liệu giao dịch từ MongoDB Atlas, chuyển đổi thành tệp JSON và tải xuống thiết bị người dùng. Tệp được tạo có tên theo ngày thực hiện, với dung lượng ghi nhận là 8,9 MB.

Đối với Restore, hệ thống tiếp nhận tệp JSON, kiểm tra cấu trúc dữ liệu và MongoDB ObjectId trước khi thực hiện thao tác ghi. Cơ chế replaceOne() kết hợp

upsert: true cho phép cập nhật bản ghi đã tồn tại hoặc thêm mới bản ghi chưa có. Kết quả kiểm thử ghi nhận 8.500 bản ghi được tiếp nhận, trong đó 8.500 bản ghi được cập nhật và không phát sinh bản ghi mới.

Bên cạnh đó, việc kiểm soát quyền ADMIN tại giao diện và API, hộp thoại xác nhận trước khi phục hồi và thông báo Toast giúp quá trình thao tác được kiểm soát rõ ràng hơn.

Tuy nhiên, chức năng hiện tại vẫn có một số giới hạn. Thứ nhất, sao lưu mới thực hiện xuất dữ liệu giao dịch thành tệp JSON, chưa phải bản sao lưu toàn bộ cơ sở dữ liệu MongoDB Atlas bao gồm index và cấu hình liên quan. Thứ hai, bulkWrite() với ordered: false không bảo đảm toàn bộ quá trình phục hồi có tính nguyên tử. Thứ ba, hệ thống chưa kiểm tra trùng lặp MongoDB ObjectId giữa các bản ghi trong cùng một tệp JSON.

Trong hướng phát triển tiếp theo, chức năng có thể được bổ sung cơ chế kiểm tra tính toàn vẹn của tệp, ghi nhận chi tiết các bản ghi thành công hoặc thất bại và tăng cường kiểm soát quá trình phục hồi dữ liệu.

Nhìn chung, chức năng Backup & Restore đã hoàn thiện quy trình sao lưu và phục hồi dữ liệu giao dịch ở mức ứng dụng, hỗ trợ quản trị viên lưu trữ bản sao và cập nhật, bổ sung các bản ghi vào MongoDB Atlas khi cần thiết.

CHƯƠNG 7: ĐÁNH GIÁ, NHẬN ĐỊNH DỮ LIỆU VÀ ĐỀ XUẤT GIẢI PHÁP KINH DOANH

7.1. NHẬN ĐỊNH VỀ DỮ LIỆU TIKTOK SHOP (DATA INSIGHTS)

Từ kết quả phân tích toàn bộ 8.500 giao dịch trong giai đoạn tháng 01–09/2024, có thể thấy hoạt động kinh doanh trên TikTok Shop có quy mô tương đối lớn, với tổng doanh thu gần 1,94 tỷ đồng, 8.500 đơn hàng và 12.727 sản phẩm được giao dịch. Tuy nhiên, hiệu quả kinh doanh không chỉ thể hiện qua doanh thu mà còn phụ thuộc vào khả năng duy trì khách hàng, khai thác các kênh bán hàng và kiểm soát rủi ro trong quá trình xử lý đơn. Trong đó, chi phí chiết khấu khoảng 181,19 triệu đồng, phí vận chuyển khoảng 72,15 triệu đồng cho thấy doanh nghiệp cần quan tâm đồng thời đến doanh thu và các khoản chi phí phát sinh để đánh giá hiệu quả kinh doanh thực chất.

Xét về hoạt động bán hàng, dữ liệu cho thấy Livestream và video ngắn gắn liên kết sản phẩm là những kênh có đóng góp đáng kể vào doanh thu. Điều này cho thấy nội dung giải trí và hoạt động giới thiệu sản phẩm có vai trò quan trọng trong mô hình Social Commerce. Tuy nhiên, hiệu quả giữa các kênh có thể thay đổi theo từng khu vực và thời gian, do đó doanh nghiệp cần đánh giá dựa trên dữ liệu cụ thể thay vì áp dụng một chiến lược chung cho toàn bộ thị trường.

Ở khía cạnh khách hàng, việc phân tích RFM đã phân nhóm được 1.393 khách hàng thành 3 nhóm có hành vi mua sắm khác nhau. Đáng chú ý, Cluster 1 gồm 481 khách hàng nhưng có giá trị chi tiêu trung bình cao nhất, đạt khoảng 1,79 triệu đồng, với tần suất mua trung bình 7,2 đơn. Trong khi đó, Cluster 0 gồm 215 khách hàng có thời gian kể từ lần mua gần nhất lên đến 157,16 ngày và mức chi tiêu trung bình chỉ khoảng 665.835 đồng. Sự khác biệt này cho thấy giá trị khách hàng không phân bố đồng đều. Vì vậy, doanh nghiệp cần chú trọng duy trì nhóm khách hàng có giá trị cao, đồng thời có giải pháp phù hợp để khuyến khích những khách hàng ít hoạt động quay lại mua sắm.

Bên cạnh cơ hội tăng trưởng, dữ liệu cũng phản ánh những vấn đề cần được kiểm soát trong vận hành. Tỷ lệ hủy đơn đạt 6,14%, hoàn trả/khiếu nại đạt 3,25% và giao dịch được gắn nhãn bất thường chiếm 4,12%, tương đương 350 giao dịch. Đây là những dấu hiệu cho thấy doanh nghiệp không chỉ cần thúc đẩy số lượng đơn hàng mà còn phải quan tâm đến chất lượng giao dịch và khả năng hoàn tất đơn.

Từ những kết quả trên, có thể nhận định rằng hoạt động kinh doanh trên TikTok Shop cần được tối ưu đồng thời ở ba phương diện: tăng hiệu quả tiếp cận và chuyển đổi, nâng cao giá trị khách hàng và kiểm soát rủi ro vận hành. Đây cũng là cơ sở để thấy được vai trò của hệ thống được xây dựng trong đề tài. Dashboard Power BI giúp tổng hợp các chỉ số doanh thu, sản phẩm, khách hàng, nguồn truy cập và trạng thái đơn hàng; trong khi Website hỗ trợ quản lý, tìm kiếm, lọc và xuất dữ liệu. Qua đó, các kết quả phân tích không chỉ dừng lại ở việc mô tả tình hình kinh doanh mà còn được chuyển thành thông tin phục vụ theo dõi và ra quyết định.

7.2. ĐỀ XUẤT ĐÁNH GIÁ VÀ GIẢI PHÁP HÀNH ĐỘNG KINH DOANH (BUSINESS RECOMMENDATIONS)

Dựa trên kết quả phân tích, nhóm đề xuất một số giải pháp nhằm cải thiện hiệu quả kinh doanh và nâng cao khả năng quản lý hoạt động trên TikTok Shop.

Thứ nhất, tối ưu hiệu quả các kênh bán hàng. Livestream và video ngắn gắn liên kết sản phẩm hiện đóng góp khoảng 81% tổng doanh thu, tương ứng lần lượt 863,21 triệu đồng và 694,37 triệu đồng. Vì vậy, doanh nghiệp nên tiếp tục ưu tiên đầu tư vào hai kênh này, nhưng cần đánh giá hiệu quả dựa trên doanh thu, số đơn hoàn thành, tỷ lệ chuyển đổi và chi phí triển khai thay vì chỉ dựa vào lượt xem. Đồng thời, cần duy trì hoạt động trên Showcase và TikTok Search để đa dạng hóa nguồn tiếp cận, hạn chế phụ thuộc quá lớn vào một số kênh chủ lực.

Thứ hai, xây dựng chiến lược chăm sóc khách hàng theo từng phân khúc. Với 481 khách hàng thuộc Cluster 1 có mức chi tiêu trung bình khoảng 1,79 triệu đồng và tần suất mua 7,2 đơn, doanh nghiệp nên ưu tiên các chương trình khách hàng thân thiết, ưu đãi dành riêng và giới thiệu sản phẩm phù hợp nhằm duy trì hoạt động mua lặp lại. Đối với 215 khách hàng thuộc Cluster 0, có thời gian chưa quay lại mua hàng trung bình 157,16 ngày, cần triển khai các chiến dịch nhắc nhớ thương hiệu, gửi mã ưu đãi tái kích hoạt và giới thiệu sản phẩm mới. Riêng Cluster 2 gồm 697 khách hàng có thể được khuyến khích tăng tần suất mua thông qua chương trình mua kèm, ưu đãi theo giá trị đơn hàng hoặc gợi ý sản phẩm liên quan.

Thứ ba, tăng cường kiểm soát chất lượng đơn hàng và chi phí vận hành. Tỷ lệ hủy đơn 6,14%, hoàn trả/khiếu nại 3,25% và giao không thành công 1,47% cho thấy doanh nghiệp cần theo dõi chặt chẽ toàn bộ quá trình xử lý đơn. Cụ thể, nên phân tích

nguyên nhân phát sinh theo khu vực giao hàng, phương thức thanh toán, giá trị đơn và nguồn bán; từ đó xác định nhóm giao dịch có rủi ro cao để ưu tiên xác minh thông tin, kiểm tra tồn kho và phối hợp với đơn vị vận chuyển. Bên cạnh đó, cần theo dõi chi phí chiết khấu 181,19 triệu đồng và phí vận chuyển 72,15 triệu đồng theo từng chiến dịch, nhằm đánh giá mức độ đóng góp của chương trình ưu đãi vào doanh thu và hạn chế tình trạng tăng đơn nhưng không cải thiện hiệu quả kinh doanh.

Thứ tư, khai thác Dashboard và Website như công cụ quản trị thường xuyên. Doanh nghiệp nên sử dụng Dashboard Power BI để theo dõi định kỳ doanh thu, nguồn truy cập, hiệu quả sản phẩm, phân khúc khách hàng và trạng thái đơn hàng. Website quản lý có thể hỗ trợ nhân viên tra cứu, lọc và xuất dữ liệu phục vụ báo cáo. Việc kết hợp hai công cụ sẽ giúp nhà quản lý phát hiện sớm biến động, theo dõi kết quả sau khi triển khai giải pháp và điều chỉnh kế hoạch kinh doanh dựa trên dữ liệu thay vì chỉ dựa vào đánh giá chủ quan.

7.3. ĐÁNH GIÁ HẠN CHẾ CỦA ĐỀ TÀI VÀ HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

Về hạn chế

Mặc dù đề tài đã hoàn thành các nội dung từ xử lý dữ liệu, phân tích kinh doanh đến xây dựng mô hình học máy và phát triển hệ thống quản lý, nghiên cứu vẫn còn một số hạn chế cần được xem xét.

Thứ nhất, hạn chế về nguồn dữ liệu và phạm vi nghiên cứu. Dữ liệu được sử dụng gồm 8.500 giao dịch với 27 thuộc tính, chủ yếu trong giai đoạn từ tháng 01 đến tháng 09/2024, được tổng hợp từ nguồn dữ liệu mở và dữ liệu bổ trợ. Vì vậy, dữ liệu chưa thể phản ánh đầy đủ quy mô, đặc điểm và những biến động mới nhất của hoạt động kinh doanh trên TikTok Shop. Bên cạnh đó, dữ liệu hiện có chưa cung cấp đầy đủ các thông tin chuyên sâu như chi phí quảng cáo, chi phí vận hành và lợi nhuận thực tế. Điều này khiến việc đánh giá hiệu quả kinh doanh mới tập trung nhiều vào doanh thu, số lượng đơn hàng và các chỉ số giao dịch, chưa thể phân tích toàn diện hiệu quả tài chính của từng sản phẩm hoặc kênh bán hàng.

Thứ hai, hạn chế trong quá trình phân tích và xây dựng mô hình học máy. Đối với phân cụm khách hàng, mô hình K-Means kết hợp RFM đã chia 1.393 khách hàng thành ba nhóm. Tuy nhiên, các cụm vẫn có sự chồng lấn nhất định, do đó kết quả

phân nhóm mới phản ánh đặc điểm mua sắm dựa trên ba chỉ số Recency, Frequency và Monetary, chưa thể hiện đầy đủ sở thích, mức độ tương tác hay sự thay đổi trong hành vi khách hàng.

Đối với mô hình dự đoán trạng thái đơn hàng Random Forest, mặc dù Accuracy đạt 87,94%, Recall của nhóm đơn hàng có vấn đề chỉ khoảng 0,02, tương ứng mô hình nhận diện đúng 4 trong 185 trường hợp thuộc nhóm này ở tập kiểm tra. Điều đó cho thấy mô hình chủ yếu nhận diện tốt nhóm đơn hàng thông thường nhưng còn hạn chế khi phát hiện các trường hợp hủy, hoàn trả hoặc giao hàng không thành công.

Thứ ba, hạn chế về khả năng ứng dụng thực tế của hệ thống. Dashboard và Website đã hỗ trợ các chức năng quản lý, tìm kiếm, lọc dữ liệu, trực quan hóa và xuất báo cáo. Tuy nhiên, hệ thống hiện chủ yếu hoạt động trên tập dữ liệu nghiên cứu, chưa được kết nối trực tiếp với nguồn giao dịch thời gian thực từ TikTok Shop. Do đó, người dùng vẫn cần cập nhật dữ liệu để có thể theo dõi tình hình kinh doanh mới.

Về hướng phát triển

Từ những hạn chế trên, đề tài có thể tiếp tục được mở rộng theo các định hướng sau:

Thứ nhất, mở rộng và nâng cao chất lượng dữ liệu. Nhóm có thể nghiên cứu tích hợp dữ liệu giao dịch thực tế từ các cửa hàng TikTok Shop, đồng thời xây dựng quy trình cập nhật dữ liệu định kỳ hoặc tự động. Bên cạnh các thông tin đơn hàng hiện có, cần bổ sung thêm dữ liệu về chi phí quảng cáo, chi phí vận chuyển, giá vốn sản phẩm, lợi nhuận và hiệu quả từng chiến dịch Marketing. Việc mở rộng các thuộc tính này sẽ giúp phân tích không chỉ dừng lại ở doanh thu mà còn đánh giá được khả năng sinh lời và hiệu quả sử dụng ngân sách của doanh nghiệp.

Thứ hai, cải thiện độ tin cậy và khả năng ứng dụng của các mô hình học máy. Đối với K-Means kết hợp RFM, nhóm có thể bổ sung các đặc trưng về sản phẩm thường mua, giá trị đơn hàng, tần suất tương tác và lịch sử mua sắm để phân nhóm khách hàng chi tiết hơn. Đồng thời, cần tiếp tục kiểm tra mức độ phù hợp của số cụm và đánh giá tính ổn định của kết quả phân nhóm trên những giai đoạn dữ liệu khác nhau.

Đối với Random Forest, có thể thử nghiệm thêm các phương pháp xử lý dữ liệu mất cân bằng, lựa chọn đặc trưng và điều chỉnh tham số nhằm cải thiện khả năng nhận diện nhóm

đơn hàng có vấn đề. Đối với Isolation Forest, cần tiếp tục kiểm tra các giao dịch được mô hình đánh dấu, rà soát cách hình thành nhãn bất thường ban đầu và thử nghiệm những thuật toán phát hiện bất thường khác. Trong quá trình đánh giá, cần chú trọng các chỉ số Precision, Recall và F1-score thay vì chỉ dựa vào Accuracy, đặc biệt khi mục tiêu là nhận diện những trường hợp chiếm tỷ lệ nhỏ trong tập dữ liệu.

Thứ ba, phát triển Dashboard và Website theo hướng tự động hóa. Hệ thống có thể được bổ sung khả năng kết nối với API hoặc nguồn dữ liệu được TikTok Shop hỗ trợ, từ đó tự động cập nhật giao dịch và làm mới các báo cáo. Bên cạnh những biểu đồ hiện có, nhóm có thể phát triển thêm các chức năng cảnh báo khi doanh thu biến động bất thường, tỷ lệ hủy đơn hoặc hoàn trả tăng cao, cũng như theo dõi hiệu quả kinh doanh theo từng chiến dịch, sản phẩm và nhóm khách hàng. Đặc biệt, việc bổ sung báo cáo lợi nhuận và hiệu quả Marketing sẽ giúp nhà quản lý có thêm cơ sở đánh giá chất lượng tăng trưởng thay vì chỉ tập trung vào doanh thu.

Thứ tư, kiểm chứng hệ thống trong môi trường kinh doanh thực tế. Sau khi hoàn thiện các chức năng, nhóm có thể triển khai thử nghiệm với dữ liệu của một hoặc nhiều cửa hàng TikTok Shop trong khoảng thời gian dài hơn. Qua đó, hệ thống sẽ được đánh giá về khả năng xử lý dữ liệu, tốc độ cập nhật, độ chính xác của báo cáo và mức độ hữu ích đối với người sử dụng. Những phản hồi trong quá trình thử nghiệm sẽ là cơ sở để điều chỉnh giao diện, tối ưu quy trình quản lý và hoàn thiện các chức năng phù hợp với nhu cầu vận hành thực tế.

Nhìn chung, đề tài đã xây dựng được quy trình tương đối hoàn chỉnh từ thu thập, xử lý, lưu trữ đến phân tích và trực quan hóa dữ liệu, đồng thời bước đầu ứng dụng học máy vào phân khúc khách hàng và nhận diện giao dịch bất thường. Tuy nhiên, để phát triển thành một giải pháp hỗ trợ quản trị kinh doanh có khả năng ứng dụng rộng rãi, hệ thống cần tiếp tục được hoàn thiện về chất lượng dữ liệu, hiệu quả mô hình và khả năng kết nối tự động. Đây sẽ là nền tảng để mở rộng đề tài theo hướng quản trị kinh doanh dựa trên dữ liệu, góp phần hỗ trợ doanh nghiệp theo dõi hoạt động bán hàng, kiểm soát rủi ro và đưa ra quyết định phù hợp trên TikTok Shop.